

1. 02-AGC Autonomous Yard Utility Vehicle - Golf Cart	2
1.1 02-AGC WP1 Project Initiation	4
1.1.1 02-AGC -WP1.1 Problem Definition	7
1.2 02-AGC WP2 Technology Research	8
1.2.1 02-AGC WP2 - Technology Research	11
1.3 02-AGC WP3 Business Research	17
1.3.1 02-AGC WP3 - Business Case Analysis and Stakeholder Management	21
1.3.2 02-AGC WP3 Project Charter	33
1.3.3 02-AGC WP3 Stakeholder Workshop	36
1.4 02-AGC WP4 Stakeholder Requirements	44
1.4.1 02-AGC WP4.1 Develop Stakeholder Requirements	47
1.4.2 02-AGC WP4.2 Analyze Stakeholder Requirements	51
1.4.3 02-AGC WP4.3 Import Stakeholder requirements in Jira	55
1.5 02-AGC WP5 System Requirements	63
1.5.1 02-AGC WP5.1 Analyze Stakeholder Requirements	67
1.5.2 02-AGC WP5.2 Develop System Requirements	77
1.5.3 02-AGC WP5.3 Ensure requirements traceability	81
1.5.4 02-AGC WP5.4 Review and Release System requirements	101
1.6 02-AGC WP6 System Architecture	104
1.6.1 02-AGC WP6.1 Static aspects of the system architecture specification	109
1.6.2 02-AGC WP6.2 Dynamic Aspects of the System Architecture Specification	114
1.6.3 02-AGC WP6.3 System architecture analysis	126
1.6.4 02-AGC WP6.4 Ensure consistency and establish bidirectional traceability	130
1.6.5 02-AGC WP6.5 Review and Communicate agreed system architecture	131
1.7 02-AGC WP7 Midterm 1 summary	132
1.7.1 02-AGC Article	135
1.7.2 02-AGC WP7.1: Midterm Project report development	138

02-AGC Autonomous Yard Utility Vehicle - Golf Cart

Nr	Team Member	Role	Competences
1	Denis Pleca	Team Lead	Python, Java, React,
2	tefan Istrati	Developer	Python - Flask framework. Database - SQLite.
3	Pavel Ciumacenco	Developer	
4	Vladlen Mitu	Developer	
5	Cristian Bulat	Developer	

User	Edits	Comments	Labels
Denis Pleca	58	0	0
tefan Istrati	15	0	0
Pavel Ciumacenco	13	0	0
Vladlen Mitu	1	0	0

Table of Contents:

- 02-AGC WP1 Project Initiation
 - 02-AGC -WP1.1 Problem Definition
- 02-AGC WP2 Technology Research
 - 02-AGC WP2 - Technology Research
- 02-AGC WP3 Business Research
 - 02-AGC WP3 - Business Case Analysis and Stakeholder Management
 - 02-AGC WP3 Project Charter
 - 02-AGC WP3 Stakeholder Workshop
- 02-AGC WP4 Stakeholder Requirements
 - 02-AGC WP4.1 Develop Stakeholder Requirements
 - 02-AGC WP4.2 Analyze Stakeholder Requirements
 - 02-AGC WP4.3 Import Stakeholder requirements in Jira
- 02-AGC WP5 System Requirements
 - 02-AGC WP5.1 Analyze Stakeholder Requirements
 - 02-AGC WP5.2 Develop System Requirements
 - 02-AGC WP5.3 Ensure requirements traceability
 - 02-AGC WP5.4 Review and Release System requirements
- 02-AGC WP6 System Architecture
 - 02-AGC WP6.1 Static aspects of the system architecture specification
 - 02-AGC WP6.2 Dynamic Aspects of the System Architecture Specification
 - 02-AGC WP6.3 System architecture analysis
 - 02-AGC WP6.4 Ensure consistency and establish bidirectional traceability
 - 02-AGC WP6.5 Review and Communicate agreed system architecture
- 02-AGC WP7 Midterm 1 summary
 - 02-AGC Article
 - 02-AGC WP7.1: Midterm Project report development

Contributors:

- Andrei Bragarencu
 - Denis Pleca
 - Pavel Ciumacenco
 - tefan Istrati

Project Overview

Project Definition

type	assignee	status	resolution	summary
------	----------	--------	------------	---------

 Jira server returned an error: [Ljava.lang.Object;
@75173abf
[View these issues in Jira](#)

Modern agricultural infrastructure in the farm environment faces increasing pressure to improve efficiency, reduce labor dependency, and integrate real-time monitoring across farm operations, as manpower capacity has seen a decline while production increases in the context of a competitive market. Many routine tasks, such as crop transport, equipment delivery, field inspection, and yard patrol, remain manual, time-consuming, and prone to inefficiencies, human error, and safety risks. Smaller and medium-sized farms often lack access to and knowledge of affordable autonomous solutions for daily logistics and utility work.

The Agricultural Yard Utility Vehicle addresses this inaccessibility by providing a smart, autonomous system capable of performing predefined missions within huge farm environments. By combining onboard sensors, edge computing, wireless connectivity, cloud services, and a web-based control system, the vehicle enables efficient logistics, continuous monitoring, and farm management.

Core Problems:

- **Operational Inefficiency:** Manual transportation and inspection processes decrease productivity.
- **Labor Shortages:** Human labor leads to increased costs and limits the scale of business.
- **Limited Farm Automation:** Expensive solutions for existing yard logistics and utility operations.
- **Safety Risks:** Manual intervention increases exposure to mechanical and environmental hazards.
- **Poor Data Integration:** Lack of real-time monitoring leads to inaccurate decisions.

02-AGC WP1 Poject Initiation

Work Product Status:

Legend	NEW - Backlog/Planned PROGRESS - Started DONE - Finished
Team Formation	 PBL26-7 - Jira server returned an error: [Ljava.lang.Object;@4868d913  PBL26-16 - Jira server returned an error: [Ljava.lang.Object;@22985c53  PBL26-17 - Jira server returned an error: [Ljava.lang.Object;@35ae8670  PBL26-18 - Jira server returned an error: [Ljava.lang.Object;@45a7ed3  PBL26-19 - Jira server returned an error: [Ljava.lang.Object;@1e72af14  PBL26-20 - Jira server returned an error: [Ljava.lang.Object;@13689c55

Problem Definition	 PBL26-37 - Jira server returned an error: [Ljava.lang.Object; @66537988
	 PBL26-46 - Jira server returned an error: [Ljava.lang.Object; @2bb086bd
	 PBL26-47 - Jira server returned an error: [Ljava.lang.Object; @5b43dba5
	 PBL26-48 - Jira server returned an error: [Ljava.lang.Object; @1a1c99e2
	 PBL26-49 - Jira server returned an error: [Ljava.lang.Object; @c1ba016
	 PBL26-50 - Jira server returned an error: [Ljava.lang.Object; @5fc7363d

- **Work Product Status:**
 - **Team Members**

Denis Pleca

Team Members

1	tefan Istrati	Developer
2	Pavel Ciumacenco	Developer
3	Vladlen Mitu	Developer
4	Cristian Bulat	Developer
5	Denis Pleca	Team Leader

02-AGC -WP1.1 Problem Definition

WP1.1 Problem Definition

Keywords: Transportation, logistics, agriculture, inspections, monitoring, autonomous driving, sensor, data processing

Problem Definition

Modern agricultural infrastructure in the farm environment faces increasing pressure to improve efficiency, reduce labor dependency, and integrate real-time monitoring across farm operations, as manpower capacity has seen a decline while production increases in the context of a competitive market. Many routine tasks, such as crop transport, equipment delivery, field inspection, and yard patrol, remain manual, time-consuming, and prone to inefficiencies, human error, and safety risks. Smaller and medium-sized farms often lack access to and knowledge of affordable autonomous solutions for daily logistics and utility work.

The Agricultural Yard Utility Vehicle addresses this inaccessibility by providing a smart, autonomous system capable of performing predefined missions within huge farm environments. By combining onboard sensors, edge computing, wireless connectivity, cloud services, and a web-based control system, the vehicle enables efficient logistics, continuous monitoring, and farm management.

Core Problems:

- **Operational Inefficiency:** Manual transportation and inspection processes decrease productivity.
- **Labor Shortages:** Human labor increases costs and limits business scale.
- **Limited Farm Automation:** Expensive solutions for existing yard logistics and utility operations.
- **Safety Risks:** Manual intervention increases exposure to mechanical and environmental hazards.
- **Poor Data Integration:** Lack of real-time monitoring leads to inaccurate decisions.

02-AGC WP2 Technology Research

Work Product Status

Legend	NEW - Backlog/Planned PROGRESS - Started DONE - Finished
Research proposed IoT technology for the agri-food sector.	 PBL26-73 - Jira server returned an error: [Ljava.lang.Object;@f8a12c0
	 PBL26-82 - Jira server returned an error: [Ljava.lang.Object;@4343f729
	 PBL26-83 - Jira server returned an error: [Ljava.lang.Object;@620cf816
	 PBL26-84 - Jira server returned an error: [Ljava.lang.Object;@1c1febab
	 PBL26-85 - Jira server returned an error: [Ljava.lang.Object;@4dd49d6c
	 PBL26-86 - Jira server returned an error: [Ljava.lang.Object;@4c3beaea

Team Members

1	tefan Istrati	Developer
2	Pavel Ciumacenco	Developer
3	Vladlen Mitu	Developer
4	Cristian Bulat	Developer
5	Denis Pleca	Team Leader

- [Denis Pleca](#) (91 days ago)

Research Disciplines

 [PBL26-103](#) - Jira server returned an error: [Ljava.lang.Object;@499dfcb5

 [PBL26-112](#) - Jira server returned an error: [Ljava.lang.Object;@3b5a3148

 [PBL26-113](#) - Jira server returned an error: [Ljava.lang.Object;@25e2082d

 [PBL26-114](#) - Jira server returned an error: [Ljava.lang.Object;@756221b4

 [PBL26-115](#) - Jira server returned an error: [Ljava.lang.Object;@5eaf9fe7

 [PBL26-116](#) - Jira server returned an error: [Ljava.lang.Object;@50a627ed

Document the research results

 [PBL26-133](#) - Jira server returned an error: [Ljava.lang.Object;@10ecea02

 [PBL26-142](#) - Jira server returned an error: [Ljava.lang.Object;@4a06931c

 [PBL26-143](#) - Jira server returned an error: [Ljava.lang.Object;@7b833f3a

 [PBL26-144](#) - Jira server returned an error: [Ljava.lang.Object;@5de80096

 [PBL26-145](#) - Jira server returned an error: [Ljava.lang.Object;@4febc169

 [PBL26-146](#) - Jira server returned an error: [Ljava.lang.Object;@560aeb8

02-AGC WP2 - Technology Research

Technology Research Perspective

The agri-food business sector represents one of the most vital components of the global economy, supporting food security, employment, and sustainable development. However, this sector faces increasing pressure from population growth, climate variability, resource constraints, labor shortages, and rising operational costs. Traditional, old-fashioned agricultural practices, largely dependent on manual monitoring and experience-based decision-making, are becoming insufficient to meet modern productivity and sustainability demands.

The digital transformation has emerged as a strategic priority across agricultural enterprises, and, as a response to all those challenges, the Internet of Things (IoT) has gained the main attention as an enabler of smart agriculture and data-driven farm management, allowing collection of data as well as operatively responding to triggers. IoT technologies support improved efficiency, transparency, and responsiveness across the agri-food value chain by enabling continuous data collection, connectivity of physical assets, and automated operational processes,

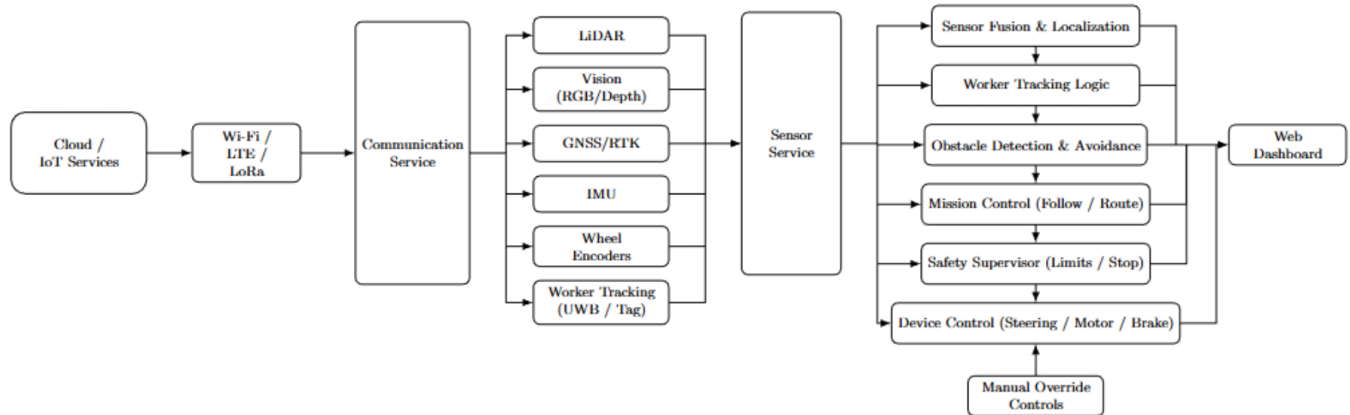
The growing adoption of IoT solutions reflects a shift towards a business model that is firstly more scalable and resilient and also has an increased precision rating compared to manual labor. The farms increasingly rely on real-time operational intelligence to optimize the usage of resources, reduce risks, and improve productivity. This evolution, besides affecting crop management, also affects logistics, asset coordination, quality assurance, and strategic planning functions.

Research Context and Motivation

The integration of IoT technologies into agri-food systems offers substantial potential for transforming traditional farming into intelligent, automated, and sustainable operations. By connecting physical infrastructure with digital platforms, IoT enables organizations to monitor conditions continuously, coordinate processes efficiently, and make evidence-based decisions in real time.

Despite growing, many agricultural enterprises continue to face barriers related to scalability, operational integration, cost efficiency, and system reliability. Understanding how IoT solutions can be strategically aligned with business objectives is essential for maximizing value creation and ensuring long-term competitiveness.

This research is motivated by the need to examine IoT technologies not solely as technical tools but as strategic business enablers capable of reshaping operational models within the agri-food sector.



Block diagram for the IoT-enabled autonomous utility cart

Role of IoT Technologies in the Agri-Food Business Sector

The agri-food business sector is currently undergoing a significant transformation driven by digitalization and the adoption of Internet of Things (IoT) technologies. Increasing global demand for food, rising operational costs, labor shortages, and the need for sustainable farming practices have encouraged agricultural enterprises to invest in smart, connected solutions. IoT-based systems enable data-driven decision-making by integrating sensors, communication networks, and intelligent platforms into agricultural operations, ultimately improving productivity, efficiency, and safety across the value chain [1][2].

From a business perspective, IoT solutions in agriculture are not merely technological upgrades but strategic tools that enhance operational efficiency and competitiveness. Smart agriculture systems allow continuous monitoring of environmental conditions, asset usage, and workforce activities, which reduces uncertainty and supports optimized resource allocation. Research published in Elsevier journals highlights that IoT-driven agricultural systems contribute to cost reduction by minimizing fuel consumption, optimizing transportation routes, and preventing equipment misuse through real-time tracking and analytics [3].

In agri-food enterprises, logistics and internal transportation represent a non-negligible share of operational costs. Autonomous or semi-autonomous vehicles equipped with IoT sensors can support tasks such as crop inspection, goods transportation, and perimeter patrolling. These systems generate actionable data that improves planning and reduces downtime, directly impacting profitability and service reliability [4].

Safety and risk management are increasingly important for business concerns in large-scale agricultural environments. According to global safety data published by the World Health Organization, vehicle-related incidents remain a major cause of injuries in both urban and rural contexts, including agricultural settings [5]. IoT-enabled vehicles can mitigate these risks through sensor-based obstacle detection, driver behavior monitoring, and automated alerts. For agri-food businesses, reducing accidents translates into lower insurance costs, fewer operational disruptions, and improved workforce well-being.

Additionally, continuous monitoring systems improve asset security by enabling patrol and surveillance functions without requiring constant human presence. This is particularly relevant for farms with large perimeters, where theft, vandalism, or animal intrusion can cause significant financial losses.

Recent research on robotics in agriculture emphasizes the growing role of intelligent machines in enhancing farming efficiency and consistency. Studies available on ResearchGate demonstrate that agricultural robots equipped with IoT connectivity can perform repetitive and monitoring-intensive tasks more reliably than manual labor, while simultaneously collecting valuable operational data [6]. This data supports predictive maintenance, performance benchmarking, and long-term strategic planning.

Elsevier-published studies further indicate that the integration of IoT, robotics, and decision-support systems leads to measurable improvements in yield management and resource efficiency. These outcomes are achieved by linking sensor data with analytics platforms that provide insights into soil conditions, crop health, and equipment utilization [7][8].

The agri-food sector increasingly values scalable and modular IoT solutions that can adapt to farms of different sizes and production models. Research shows that businesses adopting IoT-based mobility and monitoring solutions experience improved transparency, faster response times, and enhanced sustainability performance, all of which are critical factors for long-term competitiveness [2].

From this perspective, the proposed IoT-enabled golf cart solution aligns well with current industry trends. By serving as a multi-functional platform for transportation, inspection, and patrolling, the system addresses multiple operational needs simultaneously. The addition of sensors, communication modules, and data analytics capabilities transforms a conventional vehicle into a smart agricultural asset capable of generating business value beyond its primary function.

Based on the reviewed studies, the proposed IoT golf cart implementation demonstrates strong potential as a practical and cost-effective solution for agri-food businesses. Unlike fully autonomous heavy machinery, a sensor-enhanced electric cart represents a lower entry barrier for small and medium-sized farms while still offering measurable efficiency gains. The ability to integrate environmental sensors, GPS tracking, safety systems, and wireless communication enables real-time decision support and operational oversight.

Moreover, existing research confirms that incremental digitalization—rather than complete automation—is often more feasible and economically viable in agricultural contexts. This supports the relevance of the proposed solution as a transitional technology that bridges traditional farming practices with smart agriculture paradigms [1][3].

To summarize, verified academic research confirms that IoT solutions play a critical role in reshaping the agri-food business sector by improving efficiency, safety, and data-driven decision-making. The convergence of IoT, mobility, and sensor technologies has already produced measurable benefits in agricultural operations, logistics, and monitoring. Building on top of these established results, the proposed IoT-enabled golf cart system presents a promising application with strong business relevance. By enhancing transportation, inspection, and patrolling functions within farm environments, the solution aligns with current technological trends and addresses practical operational challenges faced by modern agri-food enterprises.

IoT Device and Edge Computing Domain

Modern smart agriculture systems integrate a variety of sensors and embedded electronics to observe environmental conditions and machine state. Critical sensors include Global Navigation Satellite System (GNSS) receivers for outdoor positioning, Inertial Measurement Units (IMUs) to measure motion and orientation, and wheel encoders to estimate traveled distance. Environmental sensors such as temperature, humidity, soil moisture, and light sensors support precision agriculture by providing real-time field data for irrigation and crop condition assessment.

1) Global Navigation Satellite System(GNSS) - provides global positioning for outdoor mobile robots.

- Necessary for waypoint navigation and route execution.
- Works best with augmentations like Real-time kinematic(RTK) for centimeter accuracy.[9]

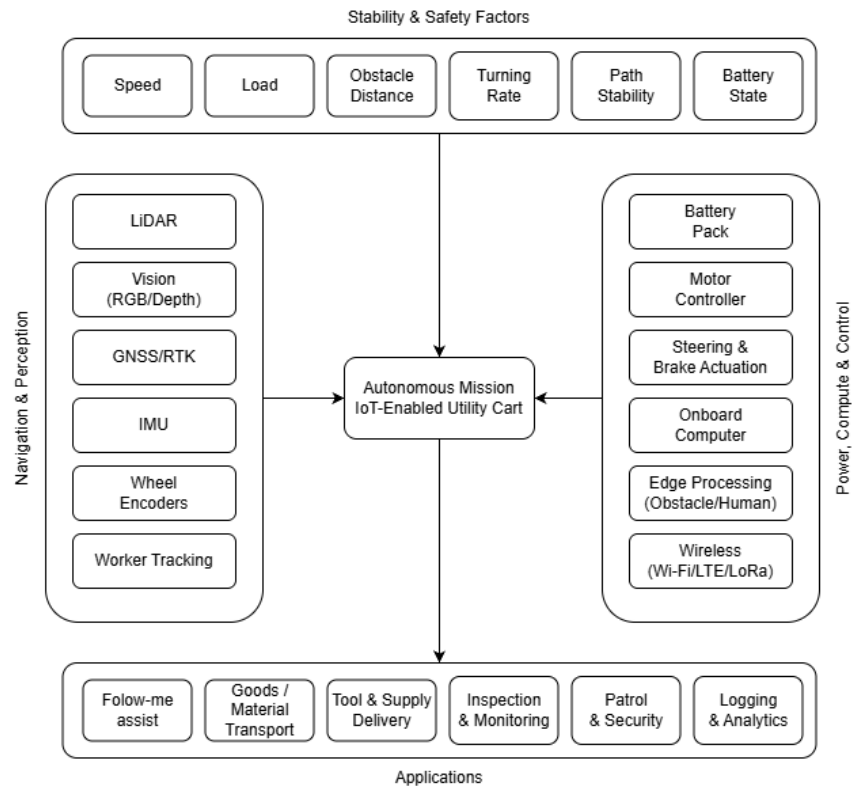
In intelligent agricultural vehicles, GNSS enables geofencing, route repetition, and autonomous coverage of large open fields with minimal infrastructure.

2) Inertial Measurement Unit(IMU) - IMUs (accelerometer + gyroscope) estimate motion when GNSS signals are weak or unavailable. Is essential for maintaining vehicle stability, estimating orientation[10][11]

3) Wheel Encoders - They are used to measure the wheels rotation and estimate the traveled distance. They provide odometry data that is used for short-term position estimation and speed calculation.[12][10]

4) LiDAR (Light Detection and Ranging) - this type of sensor is used to scan the environment to detect objects and create 3D maps. [13]

5)Ultrasonic / Proximity Sensor(Parktronic Sensors) - They are used to detect the presence of nearby objects without physical contact, based on principles such as infrared, ultrasonic, or inductive sensing. They provide distance or binary (object/no-object) data for obstacle detection, collision avoidance, and safe vehicle navigation in dynamic environments.[14]



Architecture for an IoT-enabled autonomous utility cart

Autonomous Agricultural Vehicles, Navigation Technologies, Safety Mechanisms, and Real-Time Processing

Autonomous agricultural vehicles combine high-accuracy positioning, perception, and control algorithms to perform navigation and logistics tasks without direct human control. Navigation technologies used in these vehicles include GNSS combined with RTK corrections to achieve centimeter-level positioning accuracy, particularly necessary for precise route following, docking, and geofenced operation. Sensor fusion algorithms combine GNSS, IMU, and odometry (wheel encoder) data through statistical frameworks such as Kalman filtering to improve localization accuracy in dynamic environments where satellite signals may be degraded. Perception systems - ultrasonic sensors paired with RGB and depth cameras — support obstacle detection and safe traversal through complex farm terrain.

Practical Applications in Modern Farming Environments

- **Autonomous Transport Platforms:** Agtonomy's autonomous vineyard robot demonstrates how IoT-enabled mobile platforms can follow predefined routes and transport loads through farmland without human drivers (Agtonomy Autonomous Robot Technology).[15]
- **Field Robots for Plant Monitoring and Logistics:** Systems like TerraSentia navigate crop rows autonomously for monitoring and can be extended to carry tools or samples.[16]
- **Precision Navigation in Agricultural Robots:** Research shows integration of GNSS, IMU, and wheel encoder fusion to allow reliable navigation in field robots operating in unstructured crop environments[10]

Communication Infrastructure, Data Management, and Cloud Computing

The effective deployment of Internet of Things (IoT) solutions in the agri-food business sector depends on a robust integration of communication infrastructure, data management mechanisms, and cloud computing platforms. These components enable distributed sensing, real-time data acquisition, scalable processing, and informed decision-making, which are essential for improving productivity and operational control in modern agriculture [17], [18].

Wireless Communication Technologies in Agricultural IoT Systems

Wireless communication technologies form the foundation of agricultural IoT systems by enabling connectivity between sensors, vehicles, and backend platforms across large and often remote farm environments. Existing research identifies several commonly used wireless technologies, including Wi-Fi, ZigBee, cellular networks (4G/5G), NB-IoT, and LoRa/LoRaWAN, each offering different trade-offs in terms of coverage range, bandwidth, power consumption, and deployment cost [17], [19].

Low-Power Wide-Area Network (LPWAN) technologies, particularly LoRaWAN and NB-IoT, are widely recognized as suitable solutions for agricultural scenarios due to their ability to support long-range communication with minimal energy consumption [17], [18]. These characteristics make LPWANs appropriate for battery-powered sensors deployed over large areas, such as soil moisture probes, environmental sensors, and asset-tracking devices. Studies indicate that LoRaWAN networks can achieve communication ranges of several kilometers in rural environments while maintaining low operational costs, making them attractive for agri-food businesses seeking scalable IoT deployments [17].

Data Transmission Reliability, Scalability, and Power Efficiency

Data transmission reliability and scalability are critical performance indicators in agricultural IoT systems, where sensor nodes are geographically dispersed and often operate under limited power constraints. Research shows that communication efficiency can be improved through architectural strategies such as edge data preprocessing, which reduces the volume of transmitted data by filtering or aggregating sensor readings before transmission to cloud servers [20]. This approach decreases network congestion, improves reliability, and extends device battery life.

Different communication technologies exhibit varying performance characteristics depending on the deployment context. While Wi-Fi and ZigBee are suitable for short-range, high-density environments such as greenhouses, LPWAN and cellular networks offer better scalability and coverage for open-field applications [17], [19]. Hybrid communication architectures are therefore frequently recommended, combining LPWAN technologies for low-data-rate sensing with cellular or broadband connections for higher-bandwidth applications such as video monitoring or centralized control systems [18].

Cloud Computing Platforms and Backend Architectures

Cloud computing plays a central role in agricultural IoT by providing scalable infrastructure for data storage, processing, and analytics. IoT-based agricultural systems generate large volumes of heterogeneous data, including time-series sensor readings, environmental measurements, and operational logs, which require flexible and scalable backend architectures [21]. Cloud platforms enable centralized data management, real-time processing, and remote access to monitoring dashboards, supporting both operational oversight and strategic decision-making.

Research highlights that cloud-based architectures facilitate the integration of advanced analytics and machine learning models for applications such as yield prediction, irrigation scheduling, and anomaly detection [22]. Additionally, multi-cloud and distributed cloud approaches have been proposed to improve system resilience, data availability, and computational efficiency while addressing data security and privacy concerns in large-scale agri-food operations [21].

Databases, Real-Time Monitoring, and Analytics

Efficient data management in agricultural IoT systems relies on database solutions capable of handling continuous data streams and long-term historical records. Cloud-based databases and data lakes are commonly employed to support scalable storage and efficient querying of IoT data [21]. These systems enable real-time monitoring platforms that visualize sensor data and generate alerts when predefined thresholds are exceeded, allowing timely intervention by farm operators [22].

Analytics solutions built on top of these data infrastructures transform raw sensor data into actionable insights. By applying statistical analysis and predictive models, agricultural decision support systems can recommend optimal actions related to irrigation, fertilization, and asset utilization, thereby improving resource efficiency and operational performance [22].

Impact of Data-Driven Management on Agricultural Productivity

The integration of wireless communication, cloud computing, and data analytics enables a data-driven approach to agricultural management. Studies demonstrate that such systems support precision agriculture practices by allowing farmers and agribusiness managers to monitor conditions continuously and adjust operations based on real-time and historical data [5]. This approach leads to improved productivity, reduced resource waste, and enhanced operational control, which are key objectives for competitive agri-food business enterprises.

User Experience and Interaction Design

The autonomous farm cart represents a type of mobile robotic platform designed to support transportation and logistics tasks in agricultural environments. The development of such a system requires the integration of both hardware components and software interfaces, including navigation algorithms and user interaction modules. These elements must be carefully designed to allow operators to easily monitor the system state and control its behavior. Developing intuitive user interfaces enables farmers and operators to supervise and manage the autonomous cart effectively. For example, a web-based dashboard can provide real-time visualization of vehicle position, sensor data, and mission status, allowing users to configure routes, monitor performance, and intervene when necessary. Such interfaces enhance user interaction and improve the overall usability and operational efficiency of the system.

User Interaction, System Integration, and Safety & Security

The successful deployment of an IoT-enabled golf cart in an agri-food environment depends on the seamless convergence of high-level processing, real-time control, and a dual-layered approach to risk management. This chapter analyzes the integration of the vehicle's specialized hardware, the interfaces facilitating human-machine synergy, and the critical distinction between operational safety and cybersecurity.

1. Human-Machine Interaction (HMI) in Farming Environments

In modern smart agriculture, HMI is the essential bridge between complex sensor data and operational decision-making [23]. For a vehicle equipped with an Industrial Computer, the interaction model shifts from simple remote monitoring to an active Edge-Computing interface.

Ruggedized On-Board Dashboards: Industrial computers support high-brightness displays (up to 1,600 nits) essential for outdoor visibility. These interfaces provide real-time telemetry from the drone controller, such as GPS health, battery status, and "First Person View" (FPV) streaming [24][25].

Context-Aware Control: The system must manage the transition between autonomous navigation (handled by the drone controller) and manual intervention via the Electrical Steering. This requires a "Trustworthy Interaction" design where the operator can instantly override automation [23][27].

1. System Integration: Hardware and Communication Layers

The proposed golf cart utilizes a two-layered control architecture to bridge the gap between aerial drone technology and ground-based agricultural machinery [28].

The Control Loop (Drone Controller & EPS): The drone controller serves as the "low-level" processor. It handles real-time sensor fusion from the IMU and GNSS/RTK to manage the Electrical Steering. Integration is achieved via PWM or CAN-bus protocols to ensure millisecond-level responsiveness [29][30].

The Intelligence Hub (Industrial Computer): The industrial computer acts as the "high-level" gateway. It processes heavy data loads from the Parktronic sensors and cameras, utilizing Edge AI to detect obstacles before syncing simplified data packets to the cloud [25][31].

1. Functional Safety and Risk Mitigation

While security focuses on malicious intent, Functional Safety ensures the vehicle does not cause harm due to technical malfunctions or environmental unpredictability [34]. In the agricultural context, this is governed by the ISO **25119** standard (Functional Safety for Agricultural Machinery) [35].

Proximity Safety (Parktronic Sensors): The system utilizes Parktronic (ultrasonic) sensors as a "fail-safe" layer. If these sensors detect an obstacle within a critical 1-meter radius, they trigger a hardware-level interrupt that bypasses the drone controller's navigation commands to apply the brakes immediately [35].

Operational Design Domain (ODD): Safety is maintained by defining strict operational boundaries. The Industrial Computer monitors "Geofencing" data; if the vehicle deviates from the predefined farm path, the system enters a "Safe State," neutralizing the Electronic Power Steering to prevent unintended movements into hazardous areas [28][34].

Fail-Operational Steering: The integration of Electrical Steering allows for software-defined safety limits, such as maximum speed-sensitive steering angles, preventing the cart from tipping over during sharp autonomous turns at higher speeds [29].

1. Cybersecurity and Data Security

As agricultural enterprises adopt connected machinery, the "attack surface" of the farm expands. Reliability is no longer just mechanical; it is digital [32].

Protocol Vulnerabilities: The use of drone-centric protocols like MAVLink introduces risks; without encryption, these signals can be intercepted or spoofed, leading to unauthorized steering commands [33].

Data Privacy and Sovereignty: High-resolution mapping data and crop analytics are sensitive business intelligence. Security measures include data encryption at rest on the Industrial Computer and secure TLS/SSL tunnels for cloud communication [32][33].

Bibliography

- [1] Elsevier ScienceDirect, "Information Systems and IoT Applications in Agriculture", Elsevier ScienceDirect, 2015. [Online]. Available: <https://www.sciencedirect.com/science/article/abs/pii/S1537511015301914>. [Accessed: Feb. 10, 2026].
- [2] Elsevier ScienceDirect, "IoT-Enabled Smart Farming Systems and Performance Optimization", Elsevier ScienceDirect, 2023. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S240584402305867X>. [Accessed: Feb. 11, 2026].
- [3] Elsevier ScienceDirect, "Operational Efficiency and Logistics Optimization in Agri-Food Systems", Elsevier ScienceDirect, 2025. [Online]. Available: <http://www.sciencedirect.com/science/article/pii/S0377221725009993>. [Accessed: Feb. 12, 2026].
- [4] Elsevier ScienceDirect, "Smart Mobility Solutions for Agricultural Environments", Elsevier ScienceDirect, 2025. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0040162525002586>. [Accessed: Feb. 12, 2026].
- [5] World Health Organization, "Global Health Observatory: Road Safety Data", WHO, 2023. [Online]. Available: <https://www.who.int/data/gho/data/themes/road-safety>. [Accessed: Feb. 13, 2026].
- [6] ResearchGate, "Robots in Agriculture: Revolutionizing Farming Practices", ResearchGate, 2024. [Online]. Available: https://www.researchgate.net/publication/381599567_Robots_in_Agriculture_Revolutionizing_Farming_Practices. [Accessed: Feb. 13, 2026].
- [7] Elsevier ScienceDirect, "Advanced Sensing and Analytics for Precision Agriculture", Elsevier ScienceDirect, 2024. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S2214317326000107>. [Accessed: Feb. 14, 2026].
- [8] Elsevier ScienceDirect, "Data-Driven Decision Support in Smart Agriculture", Elsevier ScienceDirect, 2025. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S2542660525002185>. [Accessed: Feb. 14, 2026].
- [9] Elsevier ScienceDirect, "Application of Navigation Technology in Agricultural Machinery: A Review and Prospects", Elsevier ScienceDirect, 2026. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S2589721725000832>. [Accessed: Feb. 15, 2026].
- [10] ResearchGate, "A Comprehensive Review of Agricultural Ground Automatic Navigation Systems Based on Multi-Sensor Fusion", ResearchGate, 2025. [Online]. Available: <https://www.researchgate.net/publication/395754429>. [Accessed: Feb. 15, 2026].
- [11] Elsevier ScienceDirect, "IMU-Based Motion Control for Agricultural Vehicles", Elsevier ScienceDirect, 2018. [Online]. Available: <https://www.sciencedirect.com/science/article/abs/pii/S0921889018300757>. [Accessed: Feb. 15, 2026].
- [12] Elsevier ScienceDirect, "Wheel Encoder Applications in Mobile Robotics", Elsevier ScienceDirect, 2024. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S2590123024005231>. [Accessed: Feb. 16, 2026].
- [13] MDPI, "LiDAR-Based Environmental Mapping for Precision Agriculture", MDPI Remote Sensing, 2023. [Online]. Available: <https://www.mdpi.com/2072-4292/15/22/5402>. [Accessed: Feb. 16, 2026].
- [14] Elsevier ScienceDirect, "Proximity Sensors for Safe Autonomous Operation", Elsevier ScienceDirect, 2020. [Online]. Available: <https://www.sciencedirect.com/science/article/abs/pii/S0959652620312282>. [Accessed: Feb. 16, 2026].
- [15] Agtonomy, "Autonomous Robot Technology for Agriculture", Agtonomy, 2025. [Online]. Available: <https://www.agtonomy.com/>. [Accessed: Feb. 17, 2026].
- [16] EarthSense, "TerraSentia Agricultural Field Robot Platform", EarthSense Inc., 2024. [Online]. Available: <https://www.earthsense.co/terrasentia>. [Accessed: Feb. 17, 2026].
- [17] P. Tang et al., "Application of IoT Wireless Communication Technology in Agricultural Irrigation Management", Sustainability, 2024. [Online]. Available: <https://www.mdpi.com/2071-1050/16/9/3575>. [Accessed: Feb. 17, 2026].

- [18] N. R. Gatkai et al., "Review of IoT and Electronics Enabled Smart Agriculture", IJABE, 2024. [Online]. Available: <https://www.ijabe.org/index.php/ijabe/article/view/8316>. [Accessed: Feb. 17, 2026].
- [19] Scientific African, "Systematic Review of IoT Technologies for Smart Agriculture", Scientific African, 2023. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S2468227623001577>. [Accessed: Feb. 18, 2026].
- [20] D. Krekovic et al., "Edge-Based Predictive Data Reduction for Smart Agriculture", arXiv, 2025. [Online]. Available: <https://arxiv.org/abs/2511.19103>. [Accessed: Feb. 18, 2026].
- [21] A. K. Sinha, "Multicloud-Powered Agriculture Through IoT and Analytics", IJSCSEIT, 2025. [Online]. Available: <https://www.cseit.org/index.php/CSEIT/article/view/CSEIT251112130>. [Accessed: Feb. 18, 2026].
- [22] A. K. Vedantham, "Cloud and IoT Technologies in Precision Agriculture", IJSCSEIT, 2024. [Online]. Available: <https://www.cseit.org/index.php/CSEIT/article/view/CSEIT241061140>. [Accessed: Feb. 18, 2026].
- [23] MDPI, "Human–Computer Interaction in Smart Greenhouses", Computers, 2025. [Online]. Available: <https://www.mdpi.com/2073-431X/14/12/553>. [Accessed: Feb. 18, 2026].
- [24] Bharat Skytech, "Agriculture Drone Components and Ground Stations", Bharat Skytech, 2026. [Online]. Available: <https://bharatskytech.com/>. [Accessed: Feb. 18, 2026].
- [25] RSIS International, "Edge AI and IoT for Real-Time Crop Disease Detection", RSIS International Journal, 2025. [Online]. Available: <https://rsisinternational.org/journals/ijrias/DigitalLibrary/volume-10-issue-7/1011-1027.pdf>. [Accessed: Feb. 18, 2026].
- [26] ResearchGate, "Human–Computer Interaction in Agricultural User Interfaces", ResearchGate, 2024. [Online]. Available: <https://www.researchgate.net/publication/378606995>. [Accessed: Feb. 18, 2026].
- [27] Promwad, "Universal Embedded HMI Platform for Machinery", Promwad Engineering, 2025. [Online]. Available: <https://promwad.com/case-studies/hmi-development>. [Accessed: Feb. 18, 2026].
- [28] PeerJ Computer Science, "Cloud-Integrated Drone Autonomy Architecture", PeerJ, 2025. [Online]. Available: <https://peerj.com/articles/cs-3238/>. [Accessed: Feb. 18, 2026].
- [29] UNL Digital Commons, "Embedded Control Architectures for Agricultural Robots", University of Nebraska, 2025. [Online]. Available: <https://digitalcommons.unl.edu/electengtheses/1122/>. [Accessed: Feb. 18, 2026].
- [30] ResearchGate, "Fuzzy Control for Agricultural Vehicle Depth", ResearchGate, 2025. [Online]. Available: <https://www.researchgate.net/publication/286570093>. [Accessed: Feb. 18, 2026].
- [31] DLR, "Self-Organizing Cyber-Physical Systems for Sustainable Agriculture", German Aerospace Center, 2025. [Online]. Available: <https://elib.dlr.de/217181/1/s43926-025-00205-6.pdf>. [Accessed: Feb. 18, 2026].
- [32] Farmonaut, "Security and Privacy in Smart Farming: 2026 Challenges", Farmonaut, 2026. [Online]. Available: <https://farmonaut.com/precision-farming/security-and-privacy-in-smart-farming-2026-challenges>. [Accessed: Feb. 18, 2026].
- [33] ResearchGate, "Cybersecurity in Precision Agriculture", ResearchGate, 2026. [Online]. Available: <https://www.researchgate.net/publication/388105508>. [Accessed: Feb. 18, 2026].
- [34] u-blox, "Autonomous Agriculture Safety and ISO 25119 Compliance", u-blox, 2026. [Online]. Available: <https://www.u-blox.com/en/autonomous-agriculture-functionally-safe-gnss>. [Accessed: Feb. 18, 2026].
- [35] SERMA Safety, "ISO 25119 Compliance for Agricultural Machinery", SERMA Safety & Security, 2026. [Online]. Available: <https://www.serma-safety-security.com/en/iso-25119-compliance-agricultural-machinery/>. [Accessed: Feb. 18, 2026].

02-AGC WP3 Business Research

Work Product Status

Legend	NEW - Backlog/Planned PROGRESS - Started DONE - Finished
Research business case analysis and stakeholder identification	 PBL26-206 - Jira server returned an error: [Ljava.lang.Object;@4f08f309  PBL26-242 - Jira server returned an error: [Ljava.lang.Object;@41b9da36  PBL26-243 - Jira server returned an error: [Ljava.lang.Object;@1eddf6ac  PBL26-244 - Jira server returned an error: [Ljava.lang.Object;@46cbfb58  PBL26-245 - Jira server returned an error: [Ljava.lang.Object;@3147522b

Team Members

1	Pavel Ciumacenco	Developer
2	Vladlen Mitu	Developer
3	tefan Istrati	Developer
4	Denis Pleca	Team Lead

- [Denis Pleca](#) (83 days ago)

Stakeholder interview

 [PBL26-207](#) - Jira server returned an error: [Ljava.lang.Object;@3d2bd85b

 [PBL26-247](#) - Jira server returned an error: [Ljava.lang.Object;@7cf44f4b

 [PBL26-248](#) - Jira server returned an error: [Ljava.lang.Object;@15deed30

 [PBL26-249](#) - Jira server returned an error: [Ljava.lang.Object;@6e863a1b

 [PBL26-250](#) - Jira server returned an error: [Ljava.lang.Object;@416cc44b

Business case survey

 [PBL26-208](#) - Jira server returned an error: [Ljava.lang.Object;@27894f43

 [PBL26-252](#) - Jira server returned an error: [Ljava.lang.Object;@433d7fb4

 [PBL26-253](#) - Jira server returned an error: [Ljava.lang.Object;@627aac63

 [PBL26-254](#) - Jira server returned an error: [Ljava.lang.Object;@1480599d

 [PBL26-255](#) - Jira server returned an error: [Ljava.lang.Object;@7dd81326

Project charter

 [PBL26-209](#) - Jira server returned an error: [Ljava.lang.Object;@d0ab95c

 [PBL26-257](#) - Jira server returned an error: [Ljava.lang.Object;@1a3222b8

 [PBL26-258](#) - Jira server returned an error: [Ljava.lang.Object;@559839c0

 [PBL26-259](#) - Jira server returned an error: [Ljava.lang.Object;@685c51a6

 [PBL26-260](#) - Jira server returned an error: [Ljava.lang.Object;@2e0b233a

02-AGC WP3 - Business Case Analysis and Stakeholder Management

Business Domain and Technology Application

Business Context Analysis

The proposed solution belongs to the domain of **precision agriculture**, where farms use sensor networks and digital monitoring platforms to support operational decisions. Modern agricultural enterprises operate under increasing pressure to improve yields while reducing resource consumption and labor costs. Traditional farm management practices rely heavily on periodic field inspections and farmer experience, which makes decision-making reactive rather than data-driven. According to the FAO, precision agriculture is defined as a management strategy that uses detailed, site-specific information to optimize productivity and resource efficiency [1]. This approach has become increasingly important as farms grow in scale and climate conditions become less predictable.

Current Workflows and Operational Challenges

In current agricultural workflows, most monitoring and decision-making tasks are performed manually. Soil conditions are typically evaluated by visual inspection or occasional measurements, while irrigation is often scheduled at fixed intervals regardless of actual soil moisture levels. Farmers may inspect fields once or twice per day, or even less frequently in large operations. This workflow introduces delays between environmental changes and operational responses. For example, soil moisture can drop below optimal levels hours or days before irrigation is applied, creating stress conditions that reduce yield. Similarly, excessive watering often occurs because farmers apply safety margins to avoid drought damage. Studies of traditional irrigation practices report that fixed irrigation schedules frequently lead to significant water waste and inconsistent plant growth [2].

Another limitation of traditional workflows is the lack of centralized data storage. Measurements are often recorded on paper or not recorded at all, making long-term analysis impossible. Without historical environmental data, farmers cannot identify patterns such as seasonal soil behavior or irrigation inefficiencies. This limits the ability to optimize farm operations and increases dependence on manual experience rather than measurable evidence [3]. In large farms, manual monitoring also requires significant labor, as workers must physically visit multiple fields to collect information. These workflows increase operational costs and reduce responsiveness to sudden environmental changes.

Potential Use Cases of the Proposed Solution

IoT-based agricultural systems address these limitations by introducing continuous monitoring and automated decision support. Instead of periodic measurements, wireless sensor networks collect environmental data such as soil moisture, temperature, humidity, and weather conditions at regular intervals. The collected data is transmitted to centralized platforms where it can be analyzed and visualized in real time. This changes the workflow from manual inspection to continuous observation. Farmers no longer need to rely on field visits to determine conditions; instead, they can observe environmental parameters remotely and respond immediately when abnormal conditions appear [3].

One of the most widely studied applications of agricultural IoT technology is smart irrigation. In these systems, soil moisture sensors continuously measure water availability in the soil and transmit readings to a control platform. Irrigation equipment is then activated only when moisture levels drop below defined thresholds. A wireless sensor irrigation system studied in Scientific Reports demonstrated that data-driven irrigation scheduling increased crop yield by **12.0 5% while reducing water consumption by 35.2%** compared to traditional irrigation methods [4]. These improvements occurred because irrigation was applied precisely when needed rather than according to fixed schedules. Crops maintained stable moisture levels without periods of water stress or excessive saturation.

Similar results were observed in a smart irrigation platform developed using IoT sensors and predictive algorithms. The system monitored environmental conditions and predicted irrigation requirements based on soil and weather data. Field tests showed approximately **30% reduction in water usage**, while irrigation predictions remained accurate within **5% deviation** from measured soil conditions [5]. This level of precision is difficult to achieve with manual monitoring because environmental variables change continuously throughout the day.

Real-time environmental monitoring also improves operational awareness by allowing farmers to detect problems earlier. A cloud-connected monitoring system described in IoT agriculture research transmitted soil and climate measurements to a web platform accessible through mobile devices. Farmers could observe field conditions remotely and receive alerts when values exceeded safe ranges [6]. Continuous monitoring reduces the time between problem occurrence and corrective action. Instead of discovering drought stress during the next field inspection, farmers can react immediately when sensor readings indicate abnormal conditions.

Data collected from sensors can also support advanced decision-making through analytics and machine learning models. Decision-support platforms analyze historical environmental data to identify patterns and optimize farm operations. Research on data-driven irrigation management showed that analytics-based systems reduced water waste by approximately **25% while increasing crop yields by about 20%** through optimized irrigation timing and improved resource allocation [7]. These improvements are achieved because data analysis identifies relationships between environmental conditions and plant growth that are not easily observable through manual monitoring.

Automation further enhances the benefits of IoT-based agricultural systems by reducing the need for manual intervention. In automated irrigation systems, sensor measurements directly control irrigation equipment through cloud-based controllers. A cloud-connected monitoring and irrigation system maintained optimal soil moisture conditions with approximately **92% control accuracy** and reduced water consumption by nearly **40%** compared to manual irrigation practices [2]. This type of automation ensures consistent operating conditions even when farm personnel are not present on site.

Expected Business and Operational Benefits

These technological improvements translate directly into business benefits. Increased yields improve revenue per hectare, while reduced water consumption lowers operating costs. For farms operating in regions with limited water resources, irrigation efficiency can determine long-term sustainability. Precision agriculture technologies enable farmers to apply exactly the required amount of water and fertilizer, reducing unnecessary input costs while maintaining optimal plant growth conditions [1].

Operational control is also significantly improved through centralized monitoring platforms. Farmers can observe environmental conditions across multiple fields from a single interface, eliminating the need for frequent physical inspections. Wireless sensor networks allow environmental data to be transmitted continuously to central servers where it can be stored and analyzed over long periods [3]. Historical datasets allow farmers to evaluate past decisions and refine management strategies over time. Instead of relying on intuition, farm management becomes evidence-based and measurable.

Existing Implementations and Business Cases

Case Study 1 – Large-Scale Precision Agriculture Farm (Iowa, USA)

A large-scale **5,000-acre corn and soybean farm in Iowa** implemented precision agriculture technologies including soil sensors, satellite crop monitoring, and farm analytics software. The system used variable-rate fertilization and real-time environmental monitoring to optimize planting density, irrigation, and nutrient application.

Measured results included:

- **28% increase in corn yield**
- **30% reduction in energy consumption**
- **20% reduction in fertilizer costs**
- **Approximately \$50,000 annual operational savings**

These improvements were achieved through targeted fertilizer application using Variable Rate Technology (VRT), combined with satellite-based crop monitoring and soil moisture sensing. The integration of sensor data with farm management software allowed operators to identify field zones requiring additional nutrients while avoiding over-application in other areas.

Technology	Impact on Yield (%)	Water Savings (%)
Soil Moisture Sensors	Up to 20	30
GPS Mapping	10-15	15
Drones for Crop Monitoring	10-12	20

According to the documented implementation, precision management enabled farmers to make **more accurate decisions regarding crop rotation and field treatment**, significantly improving productivity and reducing operational costs [9].

This case study demonstrates the economic value of data-driven agricultural management, particularly for large commercial farms where small efficiency improvements translate into significant financial savings.

Case Study 2 – Vineyard Monitoring System Using IoT Sensors (California, USA)

Commercial vineyards increasingly rely on IoT monitoring technologies to optimize irrigation and improve crop quality. A vineyard deployment in **Napa Valley, California** implemented wireless environmental sensors distributed across vine rows to monitor temperature and plant stress conditions in real time.

The sensor network provided continuous environmental data that was transmitted to a cloud-based platform where stress zones could be identified and irrigation schedules adjusted accordingly. The system enabled vineyard managers to make targeted interventions instead of applying uniform treatments across the entire field.

Field implementations of precision viticulture technologies have reported measurable improvements, including:

- **Approximately 15% increase in grape yield** through optimized growing conditions
- **Up to 25% reduction in water use** using sensor-based irrigation scheduling
- **Reduced chemical usage** through targeted treatments
- **Improved crop quality** through stable environmental conditions

Sensor-based irrigation systems and precision crop monitoring allow vineyard operators to maintain optimal moisture conditions and detect plant stress early. Continuous environmental monitoring enables more precise irrigation control compared to traditional periodic inspections [10][11].

Technology	Primary Use	Range	Accuracy	Power Efficiency	Ideal Environment/ Application
BLE	Useful for close-range plant and equipment monitoring within vine rows.	Short to medium	Moderate	High	Vine rows, equipment tagging, proximity-based monitoring
RFID	Ideal for tracking grape bins, equipment check-in/out, and storage workflows.	Short to long (passive/active)	High	Very high (passive), medium (active)	Storage barns, harvest stations, equipment control points
LoRaWAN	Excellent for long-range field monitoring across large vineyards and hilly terrain.	Very long	Low to moderate	Very high	Large vineyards, hilly or remote blocks, long-range soil/environment sensors
NB-IoT	Effective for soil probes and remote microclimate sensors in areas with deep penetration needs.	Very long	Moderate	High	Remote field sensors, buried probes, low-signal areas
Cellular IoT	Suitable for machinery tracking and wide-area monitoring.	Very long	Moderate	Medium to high	Tractors, sprayers, large vineyard fleets
GPS-IoT	Essential for tractor, ATV, and harvest crew tracking.	Global	Moderate	Medium	Moving vehicles, field crew tracking, harvest coordination
Wi-Fi HaLow	Strong for indoor winery operations with long-range wireless access.	Long (indoor)	Moderate	High	Indoor winery buildings, barrel rooms, processing facilities
Zigbee	Best for dense mesh networks covering compact vineyard clusters or processing zones.	Short to medium	Moderate	High	Close-range vine clusters, packing areas, indoor processing zones

This example demonstrates how IoT-based monitoring systems improve both productivity and product quality in commercial agriculture while reducing resource consumption.

Case Study 3 – IoT Agricultural Robotic Monitoring Platform

Mobile sensing platforms represent another important direction in agricultural IoT development. Autonomous agricultural robots equipped with sensors and cloud connectivity can perform continuous crop monitoring while collecting environmental and plant-condition data.

Recent research on robotic agricultural monitoring platforms describes systems integrating:

- Multimodal environmental sensors
- Real-time wireless data transmission
- Cloud-based storage and analysis
- Remote monitoring interfaces

Integrated robotic sensing platforms allow automated data collection in environments where fixed sensor networks may be difficult to deploy. Field studies show that robotic monitoring systems can collect high-resolution environmental data while maintaining reliable operation in real agricultural environments [12][13].

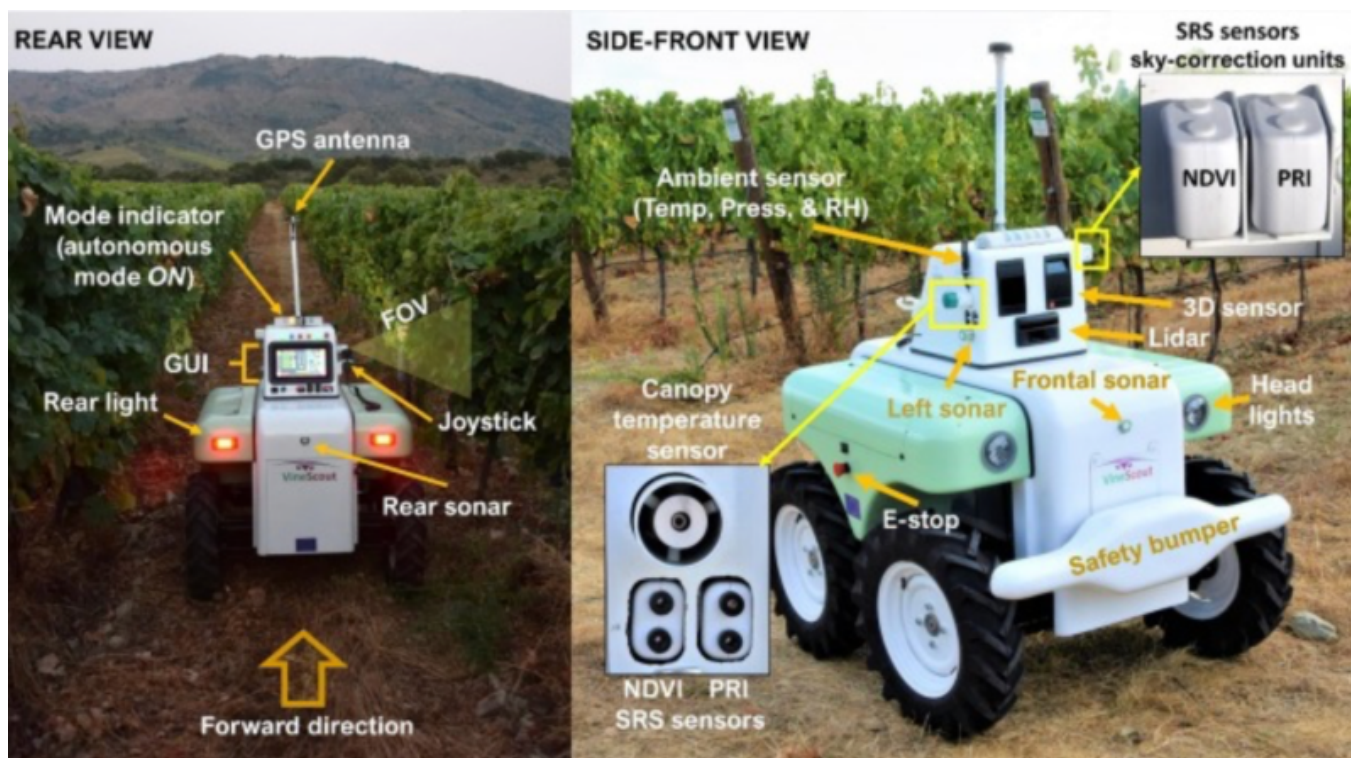


Fig.1 Image illustrating Main components of the Autonomous Ground Vehicle

Robotic systems have demonstrated measurable operational performance. For example as in Fig.1:

- Autonomous vineyard robots achieved approximately **87% operational accuracy** in field tasks such as pruning and plant detection [13]
- Robotic sensing platforms enabled high-resolution environmental mapping with strong correlation between sensor measurements and plant water conditions (R^2 up to **0.69–0.75**) [12]

These systems demonstrate the feasibility of mobile IoT sensing platforms capable of performing continuous agricultural monitoring.

The integration of mobile sensors with cloud platforms allows real-time data transmission and remote supervision, making robotic sensing platforms a practical extension of traditional IoT-based monitoring systems.

This implementation demonstrates that mobile IoT sensing platforms can complement stationary sensor networks and extend monitoring coverage across large agricultural areas.

Real-world implementations demonstrate that these benefits are achievable in practice. Field deployments of IoT irrigation systems consistently report water savings between **30% and 40%** while maintaining or improving crop yields [4][5][8]. These results confirm that IoT-based agricultural monitoring systems are not only technically feasible but also economically beneficial. The combination of continuous monitoring, automated control, and data analytics allows farms to operate more efficiently while reducing environmental impact.

Overall, agricultural IoT systems transform traditional farming workflows from manual and reactive processes into automated and data-driven operations. Continuous monitoring improves environmental visibility, automation reduces labor requirements, and data analytics enables more accurate decision-making. These improvements lead to measurable increases in productivity and resource efficiency while providing farmers with greater operational control.

To identify relevant technological solutions similar to the proposed Intelligent Farm Golf Cart, it is essential to define what constitutes a comparable system. For the purposes of this analysis, "similar systems" are interpreted broadly as autonomous or semi-autonomous mobile platforms used in agricultural or outdoor environments that incorporate key elements of mobility, autonomy, IoT integration, and cloud connectivity. This includes autonomous agricultural vehicles such as driverless tractors and field robots that perform tasks without direct human control; smart farm transport robots designed to carry produce, tools, or supplies across farmland; IoT-based farm logistics platforms that connect physical machines to cloud services for monitoring, data analytics, and remote control; and autonomous utility vehicles for outdoor environments.

The comparison in this section focuses on five main dimensions: mobility, referring to the vehicle's ability to traverse uneven terrain and execute transport tasks; autonomy, indicating the level of decision-making and path planning implemented; IoT integration, which encompasses the sensing, communication, data transmission components; cloud connectivity, enabling remote monitoring, analytics, and user interaction; use cases in smart agriculture, such as payload transport, route automation, and farm logistics optimization. By clarifying this scope, the analysis can more accurately position existing products and identify gaps that the Intelligent Farm Golf Cart aims to address.

Case Study 4 – Agtonomy Robot: Autonomous Electric Farm Vehicle

Agtonomy offers an autonomous electric agricultural robot designed to perform multiple tasks, including transport, mowing, and spraying by using GPS guidance and vision-based navigation. "Manage remotely with the TeleFarmer App" [14], cellular connectivity, and AI-driven autonomy to reduce the need for constant human supervision in specialty crops and field situations. The system integrates sensors such as GNSS and vision systems and connects to farm software ecosystems for remote control and data collection. Agtonomy's product is commercially available and positioned as a flexible autonomous solution for modern farms.[14]

Key Capabilities

- "Fully Electric: Operates on electric power, contributing to environmentally friendly farming practices and reducing emissions." [14]
- "Autonomous Navigation: Navigates autonomously using GPS guidance and vision-based navigation, minimizing the need for constant human supervision." [14]
- "TeleFarmer App: Allows farmers to remotely manage and monitor farm operations, providing real-time control and insights from anywhere." [14]
- "Connectivity: Integrated cellular and Starlink connectivity ensures reliable communication and data transfer, even in remote locations." [14]



Fig.1 Agtonomy Robot: Autonomous Electric Tractor



Fig.2 Agtonomy Robot: Autonomous Electric Tractor

Company: MOOCO ROBOTICS, INC.

Product: Agtonomy Robot

Core functionality: Autonomous navigation for multiple field tasks, including transport and spraying.

Technology used: GPS guidance, vision-based navigation, physical AI, cellular connectivity.

Deployment status: Commercial service available.

Case Study 5 — XAG R100 Agricultural Utility Vehicle

The XAG XAUV R100 series is a fully electric autonomous ground robot designed to adapt to plain fields and sloped terrains, carry payloads for transport, and support implements such as spraying systems or field sensors. It integrates RTK GNSS navigation, automated steering systems, and can operate with third-party payloads for diverse farm tasks. As part of XAG's smart agriculture ecosystem, it also connects with IoT tools for precise field operations and data collection. The R80 is already marketed and deployed in agriculture as a multifunctional autonomous platform.

Key Capabilities

- Autonomous Navigation: "The vehicle can operate semi-autonomously or fully autonomously depending on mission setup."[15]
- Path Tracking: "Vision algorithms detect and automatically correct deviations in real time, keeping the rover precisely on course."[15]
- AI Safety Assistance: "Automatically detect pedestrians ahead, alert at 4m and stop at 2m to ensure safety."[15]
- "Support Modular Add-ons for Endless Possibilities"[15]

There are three possible modular configurations for constructing the vehicle.



Fig. 3 XAG R100 Agricultural Utility Vehicle(1)



Fig. 4 XAG R100 Agricultural Utility Vehicle(2)



Fig. 5 XAG R100 Agricultural Utility Vehicle(3)

Case Study 6 — ASI Autonomous Farming Solutions (Mobius Platform)

Autonomous Solutions, Inc. (ASI) provides autonomy kits and robotic platforms that retrofit existing farm vehicles to operate autonomously. The Mobius platform enables remote control, obstacle detection, and fleet management, allowing tractors, sprayers, or mowers to be controlled from a central interface. It emphasizes scalable IoT connectivity and automation of traditional equipment across large farms. The solution is targeted at industrial and precision agriculture customers and is commercially supported.

Key Capabilities

- Remote Operation: operators can control vehicles from a centralized command center rather than sitting inside the tractor cab.
- Obstacle Detection and Safety: "The system integrates sensors (such as GNSS, LiDAR, radar, and vision systems, depending on configuration) to detect obstacles and ensure safe navigation."[16]
- Geofencing and Boundary Management: "Operations can be restricted to defined field boundaries to improve safety and precision."[16]

Company: Autonomous Solutions, Inc.

Product: Mobius Autonomous Agriculture Platform

Core functionality: Fleet autonomy and remote operation of farm vehicles.

Technology used: GNSS, obstacle detection, and central fleet control (IoT).

Deployment status: Commercially available retrofit kits and platforms.

Case Study 7 — AgriMOBY Multipurpose Autonomous Farm Vehicle (Prototype)

AgriMOBY is a research project developing a multipurpose, all-terrain autonomous vehicle capable of flexible use cases, including cargo transport, inspection, and field monitoring. It focuses on adaptability through an omnidirectional chassis design and aims to demonstrate autonomous logistics capabilities in horticulture and vineyard contexts. Though still in research and development, the project illustrates emerging trends toward autonomous transport robots in agriculture. Approximate dimensions 110 x 110 x 40 cm.

Key Capabilities

- Autonomous Navigation: Movement in multiple directions without conventional steering limitations
- Maneuverability: "Improved maneuverability in narrow vineyard rows."[17]
- Autonomy: "Increased the degree of energy optimisation and achieved much more efficient use of energy resources."[17]



Fig. 6 AgriMOBY Multipurpose Autonomous Farm Vehicle (Prototype)

Comparative Analysis: Intelligent Farm Golf Cart vs. Existing Solutions

Attribute	Intelligent Farm Golf Cart (Proposed)	Agtonomy Autonomous Farm Robot	XAG XAUV R80	ASI Mobius Autonomous Platform	AgriMOBY (Prototype)
Target Customers	Small & medium farms research labs	Commercial farms & specialty crop growers	Large, high-tech farms & enterprises	Industrial farms / large operations	Research institutions, prototypes
Level of Autonomy	Semi-autonomous to autonomous (configurable)	Autonomous with teleoperation override	High autonomy with RTK path following	Variable autonomy (retrofit)	Experimental autonomy
Core Functionality	Transport payloads, route scheduling, IoT telemetry	Multi-task field robot (transport, spraying)	All-terrain autonomous transport & payloads	Fleet autonomy & remote control	Experimental transport & inspection
Sensor Suite	GNSS/RTK, IMU, LiDAR/ ultrasonic, camera, bump & agri/environment sensors	GPS + vision (no public LiDAR)	RTK GNSS, depth sensors, and optional LiDAR	GNSS + obstacle sensors	Prototype sensors depend on configuration
Connectivity	Wi-Fi / BLE for config, 4G/5G or LoRaWAN for telemetry, MQTT/REST	GSM/4G teleoperation, cloud	Cellular, IoT ecosystem	On-site network, cloud management	Research communications
Cloud Integration	Full IoT + cloud dashboard + analytics	Cloud teleoperation & data logs	Integrated farm ecosystem	Central fleet control dashboard	Not focused on cloud
Safety Systems	LiDAR/Ultrasonic + bump sensors + emergency stop logic	Vision + obstacle avoidance	Integrated obstacle systems	Obstacle detection + safety overrides	Limited (prototype focus)
Payload Capacity	Designed for farm tools & medium loads	Medium (field dependent)	Higher capacity (all-terrain)	Varies (retrofitted vehicles)	Experimental
Scalability	Designed for scalable deployments across small/medium farms	Commercial but complex pricing	Enterprise scale	Retrofit integration across vehicle fleets	Research scalability, not commercial
Moldova Applicability (Ease of Adoption)	High (modular, affordable, customizable)	Medium (requires tech skills & network)	Low-Medium (high cost, large farms)	Medium (integration barriers)	Low (R&D stage)
Commercial Status	Development/prototype	Commercial	Commercial	Commercial	Research prototype

Gaps in Current Solutions vs. Proposed Intelligent Farm Golf Cart

Feature / Requirement	Agtonomy Robot	XAG XAUV R80	ASI Mobius Platform	AgriMOBY Prototype	Intelligent Farm Golf Cart (Proposed)	Gap Addressed
Affordable for Small Farms	Medium (high cost)	Low (expensive)	Medium (integration cost)	Low (prototype)	High (low-cost modular)	Accessible to SM farms
Flexible Autonomy Modes	High autonomy	High autonomy	Dependent on retrofit	Experimental	Assisted in full autonomy options	Adjustable for user needs
Low-power Connectivity (LoRaWAN)	x	x	x	x	LoRa + cellular	Works in low coverage areas
Cloud Analytics & Dashboard	Partial (telemetry)	Medium (ecosystem)	Fleet control only	None	Full telemetry + analytics	Better decision support
Modular Payload Support	Medium	High	Retrofit dependent	Research	Modular farm attachments	Farm-specific flexibility

Safe Short-range Perception	Vision + GPS	Integrated	Obstacle sensors	Limited	LiDAR + ultrasonic + bump	Multi-layer safety
Designed for Local / SMEs (e.g., Moldova)	Medium	Low	Medium	Low	Tailored for small/medium farms	Better fit for local markets
Simple User Interface	Medium	Medium	Medium	Experimental	User-friendly IoT dashboard	Easier adoption

Stakeholder Management Approach

Stakeholder Identification Process

The project focuses on the design and implementation of an Intelligent Farm Golf Cart, an IoT-based autonomous vehicle intended to support transportation and logistics operations in agricultural environments. The system is designed to carry tools, harvested crops, or supplies across farms, while integrating sensing, communication, and autonomous navigation technologies.

The solution targets small to medium-sized farms and aims to reduce manual labor, improve operational efficiency, and increase safety in routine transportation tasks.

Stakeholder Table

Stakeholder Group	Stakeholder	Description	Level of Interest	Level of Influence
Primary Users	Farmers (Small-scale/ Large-scale)	Farmers are the main end users. The system directly affects their productivity, labor efficiency, and operational costs.	High	Medium
Technical Team	Software Developers	Software developers design and implement the autonomy algorithms, cloud services, and user interfaces of the system. They are responsible for system logic, data processing, and overall digital functionality.	High	High
Technical Team	Hardware Engineers	Hardware engineers are responsible for integrating sensors, actuators, embedded systems, and power components. They ensure that the physical system operates safely and reliably in real farm conditions.	High	High
Institutional Bodies	Local Agricultural Agencies	They provide guidance, validation, or recommendations for adopting new technologies.	Medium	Medium
Business Partners	Agri-Tech Companies	Agri-tech companies may provide technology, expertise, or integration platforms for smart farming solutions. They are interested in commercial opportunities and technology transfer.	Medium	Medium
Business Partners	Cloud Service Providers	Cloud providers offer infrastructure for data storage, analytics, and system monitoring. They enable remote access and scalability of the platform.	Medium	Medium
Financial Stakeholders	Investors / Funding Organizations	Investors provide financial resources needed for development, testing, and potential commercialization. Their interest lies in return on investment and system scalability.	High	High
Research Institutions	Technical University of Moldova	The Technical University of Moldova supports the project through academic supervision, infrastructure, and research expertise. It ensures scientific validity and educational value.	High	High
Society	Consumers	Consumers benefit indirectly through more efficient food production and potentially lower costs. Their interest is mainly related to food availability and quality.	Medium	Low

Criteria for Interest and Influence Classification

Criteria for "Interest"

The level of interest is determined based on the following concrete factors:

Degree of dependency on the system - how much the stakeholder relies on the intelligent golf cart for daily operations or strategic objectives.

Economic impact - financial benefits, cost reductions, or risks associated with the system.

Operational integration - the extent to which the stakeholder integrates the system into their workflow.

Strategic alignment - whether the project supports their institutional, academic, or commercial goals.

Examples:

- Farmers have High interest because their productivity, labor efficiency, and operational costs directly depend on the system.
- Software and hardware engineers have High interest because system success reflects their technical work and evaluation.

Criteria for "Influence"

The level of influence is determined using measurable indicators such as:

Financial power – ability to fund or withdraw funding.

Regulatory authority – power to approve, regulate, or restrict deployment.

Technical control – responsibility for system architecture and performance.

Market power – ability to affect adoption or commercialization.

Blocking capacity – ability to delay, limit, or stop implementation.

Examples:

Investors have High influence because funding decisions determine scalability.

The Technical University of Moldova has a strong influence due to its academic control and evaluation authority.

Consumers have Low influence because they cannot directly affect system design or deployment decisions.

Why were these stakeholders selected?

Primary Users

- Farmers (Small-scale and Large-scale) were selected because they are the main end users whose productivity, operational efficiency, and economic performance depend directly on the intelligent golf cart system. Their level of adoption and satisfaction ultimately determines the practical success and commercial viability of the solution.

Technical Team

- Software Developers were selected because they are responsible for designing the system architecture, autonomy algorithms, cloud services, and user interfaces. Their technical expertise directly influences system functionality, performance, and reliability.

- Hardware Engineers were selected because they design and integrate the physical components of the system, including sensors, actuators, embedded platforms, and power systems. The safety, robustness, and real-world feasibility of the intelligent cart depend on its engineering decisions.

Institutional Bodies

- Local Agricultural Agencies were selected because they support the modernization of agricultural practices and can facilitate the adoption of smart technologies at the regional level. Their involvement helps align the project with agricultural policies and best practices.

Business Partners

- Agri-Tech Companies were selected because they provide technological solutions, expertise, and potential pathways for system integration and commercialization. Their participation increases the scalability and market relevance of the project.

- Cloud Service Providers were selected because they supply the infrastructure required for data storage, analytics, remote monitoring, and system scalability. Without cloud services, real-time data processing and system expansion would not be feasible.

Financial Stakeholders

- Investors and Funding Organizations were selected because they provide the financial resources required for development, testing, and potential deployment. Their investment decisions directly influence project scope, sustainability, and long-term impact.

Research Institutions

- The Technical University of Moldova was selected because it provides academic supervision, research infrastructure, and technical validation for the project. The university ensures scientific rigor, educational value, and compliance with engineering standards.

Society

- Consumers were selected because they are indirect beneficiaries of agricultural automation, as improvements in logistics and productivity can lead to better food quality, availability, and reduced production costs. Although they do not interact with the system directly, they are affected by its broader societal impact.

Stakeholder Mapping Methodology

High Influence – High Interest (Manage Closely)

The “Manage Closely” quadrant includes stakeholders who have both strong decision-making power and a high level of involvement in the project. This group consists of software developers, hardware engineers, investors, and the Technical University of Moldova. These stakeholders directly influence system architecture, funding, research direction, and overall project success.

High Influence – Low/Medium Interest (Keep Satisfied)

The “Keep Satisfied” quadrant includes stakeholders who have high power but lower direct involvement in daily project operations. Local agricultural agencies influence policy alignment and technology adoption at the regional level, while cloud service providers supply critical digital infrastructure for data processing and system scalability. Although they do not manage the system directly, their institutional and technical authority can significantly affect deployment conditions.

Low/Medium Influence – High Interest (Keep Informed)

The “Keep Informed” quadrant includes stakeholders who are highly interested in the project outcomes but have limited influence over strategic decisions. Farmers are strongly affected by the system, as it directly impacts their productivity and workflow. Agri-tech companies are also interested in technological collaboration and potential commercialization opportunities. These stakeholders should be regularly informed through demonstrations, feedback sessions, and performance reports to ensure their needs are reflected in system design.

Low Influence – Low/Medium Interest (Monitor)

The “Monitor” quadrant includes stakeholders who are indirectly affected by the project and have minimal influence over its development. Consumers fall into this category, as they benefit from agricultural automation through improved food production and efficiency. However, they do not interact with the system or influence its design. These stakeholders require minimal engagement, mainly through general communication about project outcomes and societal benefits.

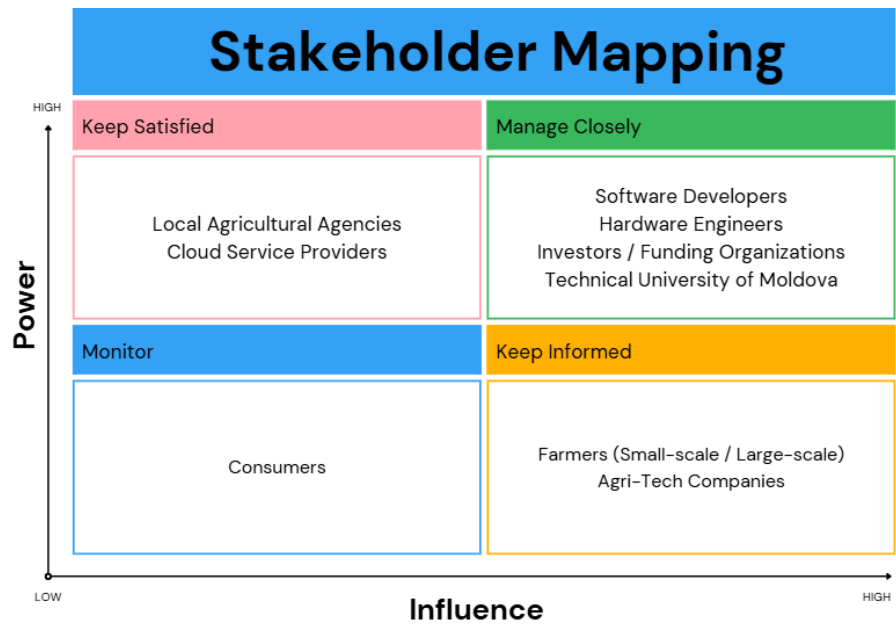


Fig.7 Stakeholder Mapping Methodology

References

[1] FAO, *Precision Agriculture for Smallholder Farmers*
<https://www.fao.org/family-farming/detail/en/c/1738176/>

[2] Hasib A., Akib A., *An IoT-Based Smart Plant Monitoring and Irrigation System*
<https://arxiv.org/abs/2601.15830>

[3] IJERT, *Smart Agriculture Monitoring System Based on IoT*
<https://www.ijert.org/smart-agriculture-monitoring-system-based-on-internet-of-things-iot>

[4] Scientific Reports, *Wireless Sensor Network for Irrigation Management*
<https://www.nature.com/articles/s41598-025-97303-w>

[5] ScienceDirect, *Smart Agriculture Using IoT for Automated Irrigation*
<https://doi.org/10.1016/j.atech.2025.101081>

[6] Rahman et al., *Soil Characterization through IoT*
<https://arxiv.org/abs/2411.17731>

[7] Springer, *IoT Sensors and Machine Learning for Precision Agriculture*
<https://link.springer.com/article/10.1007/s44279-025-00247-y>

[8] Frontiers in Water, *In-field IoT System for Precision Irrigation*
<https://www.frontiersin.org/articles/10.3389/frwa.2024.1353597>

[9] *Transforming Farm Operations with Big Data: Real-Life Success Stories*
<https://moldstud.com/articles/p-transforming-farm-operations-with-big-data-real-life-success-stories-and-insights>

[10] GAO RFID, *Cloud-Based Vineyard Asset and Yield Monitoring Systems*
<https://gaorfid.com/cloud-based-vineyard-asset-yield-monitoring-systems/>

[11] SIIT, *The Science Behind Precision Agriculture*
<https://siit.co/blog/index.php/how-to-revolutionize-crop-yields-the-science-behind-precision-agriculture/21072>

[12] *Robotics-Based Vineyard Water Potential Monitoring*, Computers and Electronics in Agriculture
<https://www.sciencedirect.com/science/article/pii/S0168169921003288>

[13] Silwal et al., *Autonomous Robotic Vine Pruning System*
<https://arxiv.org/abs/2112.00291>

[14] <https://agtechcr.com/en/robotics/agtonomy-robot/>

[15] <https://xa.com/en/r100#part-import-wrap>

[16] <https://asirobots.com/industries/agriculture>

[17] <https://cit.upc.edu/en/portfolio-item/agromoby-a-multipurpose-all-terrain-omnidirectional-autonomous-vehicle-for-agricultural-applications>

02-AGC WP3 Project Charter

Project Charter — IoT-Enabled Autonomous Agricultural Utility Vehicle

1. Project Purpose and Justification

Agricultural operations frequently experience inefficiencies in internal logistics, particularly in repetitive transport tasks between fields, storage areas, and operational zones. These tasks consume significant time, fuel, and labor resources while contributing minimal direct value to production. Field observations and stakeholder interviews indicate that a substantial portion of working time is spent on transporting materials, equipment, and harvested crops rather than performing productive agricultural activities.

Additionally, seasonal labor shortages increase workload pressure on farm operators, leading to operational delays, fatigue, and increased risk of accidents.

The purpose of this project is to design, develop, and evaluate an **IoT-enabled semi-autonomous utility vehicle** capable of assisting agricultural workers by performing repetitive transport, monitoring, and patrol tasks within controlled farm environments. The system integrates autonomous mobility, sensor-based perception, and cloud-enabled monitoring to improve operational efficiency and safety.

2. Problem Statement

Farm logistics are primarily dependent on manual coordination and human-operated vehicles, resulting in inefficiencies, delays, and increased operational costs. Transport activities such as moving fertilizers, equipment, and harvested crops are repetitive and time-consuming, often interrupting core agricultural processes.

Limited workforce availability further exacerbates the problem, forcing workers to handle multiple tasks simultaneously. This increases fatigue and the likelihood of operational errors or accidents, especially during peak agricultural seasons.

3. Project Objectives (SMART)

- Design and prototype a semi-autonomous utility vehicle within the project timeline (academic term).
- Achieve reliable autonomous navigation within predefined farm areas using waypoint navigation and geofencing.
- Implement obstacle detection and avoidance with safe stopping behavior in 95% of detected critical scenarios during testing.
- Enable real-time remote monitoring and control through an IoT-enabled dashboard interface.
- Demonstrate measurable improvement in internal logistics efficiency compared to manual transport workflows.
- Provide a functional human-machine interface allowing supervision and manual override.

4. Project Scope

In Scope

- Autonomous navigation in controlled farm environments
- IoT-based communication and monitoring
- Transport of materials, tools, and equipment
- Obstacle detection and safety mechanisms
- Manual override and remote control capabilities
- Human-machine interface design

Out of Scope

- Fully autonomous operation without human supervision
- Public road navigation
- Large-scale agricultural machinery automation
- Full commercial deployment

5. Key Deliverables

- High-level and detailed system architecture
- Functional prototype or simulation of the autonomous vehicle
- Sensor integration (GPS/RTK, ultrasonic, camera, IMU)
- IoT communication framework (edge + cloud)
- Web-based monitoring dashboard and control interface
- Navigation and obstacle avoidance algorithms
- System performance evaluation report
- Deployment and scalability recommendations

6. Stakeholders

Stakeholder	Role
Farmers / Farm Operators	Primary users and system beneficiaries
Agribusiness Managers	Decision-making and investment evaluation
IoT & IT Technicians	System deployment, maintenance, and integration
Project Team	Design, development, and validation
Academic Supervisors	Guidance, evaluation, and feedback

7. Functional Requirements

ID	Requirement
FR1	The system shall support autonomous navigation within predefined farm areas.
FR2	The system shall detect obstacles and perform safe stop or avoidance actions.
FR3	The system shall support a follow-me operational mode.
FR4	The system shall assist in transport of materials and equipment.
FR5	The system shall provide remote monitoring of status and location.
FR6	The system shall allow manual override and emergency stop.
FR7	The system shall record operational data and logs.

8. Non-Functional Requirements

ID	Requirement
NFR1	The system shall prioritize human safety at all times.
NFR2	The system shall operate reliably in agricultural environments (dust, uneven terrain, weather).
NFR3	The interface shall be intuitive and user-friendly.
NFR4	The system shall support wireless communication suitable for rural areas.
NFR5	The system shall maintain energy efficiency for extended operation.

9. Expected Benefits

- Reduction in time spent on repetitive transport tasks
- Improved coordination during harvesting and operational workflows

- Increased safety in shared farm environments
- Reduced workload pressure due to labor shortages
- Foundation for future automation in agriculture

10. Success Criteria

The project will be considered successful if the system demonstrates:

- Reliable autonomous navigation within controlled environments
- Safe interaction with humans, equipment, and terrain
- Improved logistical efficiency compared to manual processes
- Positive usability feedback from stakeholders
- Stable communication and monitoring via IoT platform

11. Risks and Mitigation Strategies

Risk ID	Description	Mitigation
R1	Sensor failure	Sensor fusion and redundancy
R2	GNSS signal degradation	Combine GNSS with IMU and odometry
R3	Connectivity loss	Enable offline operation and delayed sync
R4	Collision risks	Obstacle detection + emergency stop
R5	Difficult terrain	Robust hardware and adaptive navigation
R6	System failure during peak use	Allow manual fallback
R7	Low user trust	Provide supervised autonomy
R8	Improper system use	Training and simple interfaces

12. Assumptions and Constraints

Assumptions

- IoT and sensor technologies are reliable and scalable
- Farm environments allow controlled autonomous operation
- Stakeholders are willing to adopt semi-autonomous solutions

Constraints

- Limited budget and hardware resources
- Environmental variability (terrain, weather)
- Dependence on wireless connectivity availability
- Compliance with agricultural safety standards

02-AGC WP3 Stakeholder Workshop

1. Farmers (Small-scale / Large-scale)

1. What type of farm do you own, and what size is it?
 2. What are the most common daily transport or movement activities within the farm?
 3. Who usually takes care of these transport activities?
 4. During busy periods, which activities consume the most time?
 5. What difficulties do you encounter when transporting crops, tools, or materials?
 6. Have you had situations where delays occurred because of transport problems? Can you provide a short example?
 7. How does the lack of workforce affect the daily activity of the farm?
 8. What safety risks exist when vehicles or equipment move around the farm?
 9. What daily activities do you consider repetitive or inefficient?
 10. When a problem occurs with a vehicle or transport, how do you usually handle it?
 11. Have you improved any work processes in recent years? What did you change?
 12. What do you consider before investing in new equipment?
 13. How do you decide whether an investment in a new technology is worth making?
 14. Do you currently use smart technologies on the farm? If yes, which ones?
 15. Have you used GPS systems, sensors, or remote monitoring solutions?
 16. What concerns would you have regarding the use of an autonomous vehicle on the farm?
 17. Would you prefer a fully autonomous vehicle or one supervised by an operator? Why?
 18. What types of activities would you be willing to entrust to an autonomous vehicle?
 19. What would make you trust such a system?
 20. What concrete benefits would convince you to adopt this solution?
-

2. Software Developers (Students/Developers)

1. What types of complex software systems have you developed so far?
 2. What experience do you have with distributed architectures or IoT systems?
 3. How would you structure the software architecture for an autonomous vehicle connected to the cloud?
 4. What main components should such a system contain (vehicle, server, web application, etc.)?
 5. How would you separate responsibilities between the edge computing part and the cloud?
 6. What technologies or programming languages do you consider suitable for such a project?
 7. How would you ensure secure communication between the vehicle and the web platform?
 8. What communication protocols would you recommend for real-time data transmission?
 9. How would you handle the loss of internet connection during vehicle operation?
 10. What security measures do you consider essential in an autonomous system connected to the network?
 11. How would you design the system to allow remote software updates?
 12. What methods would you use for error monitoring and problem diagnostics?
 13. How would you ensure system scalability if it is implemented across multiple farms?
 14. What databases would be suitable for storing operational and monitoring data?
 15. How would you integrate data collected from sensors into the analysis and reporting system?
 16. What experience do you have with developing web interfaces for non-technical users?
 17. How would you make the interface easy to use for farmers?
 18. What testing strategies would you apply for a critical system such as an autonomous vehicle?
 19. What do you consider the biggest software challenge in such a project?
 20. From a maintenance perspective, what needs to be planned from the design phase?
-

3. Hardware Engineers

1. What experience do you have with integrating sensors in outdoor or agricultural environments?
 2. What challenges occur most frequently when using sensors in conditions of dust, humidity, or vibrations?
 3. What factors are essential when selecting sensors for an autonomous vehicle?
 4. What types of sensors do you consider necessary for the navigation of an autonomous vehicle on a farm?
 5. How do you ensure hardware reliability in a system that operates continuously outdoors?
 6. What methods do you use to protect electronic components against overheating or moisture?
 7. What are the main technical risks in an autonomous vehicle project from a hardware perspective?
 8. How do you test a hardware system before final implementation in the field?
 9. What solutions would you use for efficiently powering an autonomous vehicle (batteries, energy management)?
 10. How would you approach the problem of energy consumption in a system with multiple sensors and communication modules?
 11. What types of hardware communication do you consider suitable for an agricultural environment (Wi-Fi, LoRa, 4G/5G, etc.)?
 12. What experience do you have with integrating GPS and positioning systems?
 13. How would you handle interference or signal loss in an open field?
 14. What standards or practices do you follow to ensure the electrical safety of the system?
 15. How do you collaborate with the software team for complete hardware–software integration?
 16. What indicators would you use to evaluate the hardware performance of the vehicle?
 17. What hardware costs do you consider critical in such a project?
 18. From your perspective, what is the biggest hardware challenge in developing an autonomous vehicle for a farm?
-

4. Local Agricultural Agencies

1. What is the role of your institution in supporting farmers at the regional level?
 2. What types of agricultural technologies are currently supported or promoted?
 3. Are there specific regulations regarding the use of autonomous vehicles in agriculture?
 4. What procedures should be followed to implement an autonomous vehicle on a farm?
 5. What legal requirements exist regarding the safety of using autonomous equipment?
 6. How are systems based on sensors and remote monitoring treated from a legislative point of view?
 7. Are there funding programs or subsidies for the digitalization of agriculture?
 8. What criteria must be met for a farmer to obtain financial support for such a technology?
 9. What barriers do you observe in the adoption of innovative technologies by farmers?
 10. How can the adoption of autonomous solutions be encouraged at the regional level?
 11. What institutional concerns might arise regarding the use of autonomous vehicles?
 12. What standards or certifications should be respected for such a product to be approved?
 13. How do you evaluate the impact of a new technology on safety and the environment?
 14. What data or reports should be provided to authorities for monitoring?
 15. To what extent does the current rural infrastructure allow the implementation of autonomous solutions?
 16. Are there national or European initiatives that support automation in agriculture?
 17. How does your institution collaborate with developers of agricultural technologies?
 18. What factors influence the decision to officially support a new technology?
 19. What regional benefits could an autonomous vehicle bring to farms in Moldova?
 20. What recommendations would you offer for aligning the project with institutional requirements?
-

5. Investors / Funding Organizations

1. What types of technological or agricultural projects do you usually finance?
 2. What criteria are the most important when evaluating a new project?
 3. How do you evaluate the market potential of a technological solution in agriculture?
 4. What financial indicators do you follow before making an investment decision?
 5. What level of product maturity do you consider necessary for funding?
 6. How do you analyze risks in a project based on autonomous technologies?
 7. What risks do you consider the most critical in an autonomous vehicle project for agriculture?
 8. How do you evaluate the team's capacity to implement and develop the project?
 9. What monetization model do you consider suitable for such a solution (direct sale, subscription, leasing)?
 10. How important is validating the market need before investment?
 11. What evidence or data should be presented to demonstrate real demand?
 12. How do you analyze the potential for scaling at a regional or international level?
 13. What role do strategic partnerships play in the funding decision?
 14. What investment recovery period do you consider acceptable?
 15. What expectations do you have regarding the return on investment in such a project?
 16. How do you evaluate the competitive advantage of a new solution on the market?
 17. What common mistakes do you observe in agricultural technology startups?
 18. What level of involvement do you prefer after investment (advisory, strategic, operational)?
 19. What factors could determine the withdrawal or refusal of funding?
 20. What recommendations would you offer to increase the attractiveness of the project for investors?
-

Stakeholder 1 Interview:

1. What type of farm do you own and how big is it?
 2. What are the most common daily transport or travel activities on the farm?
 3. Who usually handles these transport activities?
 4. During busy periods, which activities are the most time-consuming?
 5. What difficulties do you encounter when transporting crops, tools, or materials?
 6. Have you ever experienced delays due to transportation issues? Can you give a brief example?
 7. How does labor shortage affect daily farm operations?
 8. What safety risks are there when moving vehicles or equipment on the farm?
 9. What daily activities do you consider repetitive or inefficient?
 10. When a problem arises with a vehicle or transport, how do you usually handle it?
 11. Have you improved any work processes in recent years? What have you changed?
 12. What do you consider before investing in new equipment?
 13. How do you decide whether an investment in new technology is worthwhile?
 14. Do you currently use smart technologies on your farm? If so, which ones?
 15. Have you used GPS systems, sensors, or remote monitoring solutions?
 16. What concerns would you have about using an autonomous vehicle on your farm?
 17. Would you prefer a fully autonomous vehicle or one supervised by an operator? Why?
 18. What types of activities would you be willing to entrust to an autonomous vehicle?
 19. What would make you trust such a system?
 20. What benefits
-

Workshop Documentation: Autonomous Yard Utility Vehicle Project

Name: Sergiu Goncearuc
Occupation: Farmer
Location: Republic of Moldova, r. Anenii Noi
Interviewer: Ciumacenco Pavel

Transcript

Interviewer: Thank you for agreeing to this interview. To start, could you tell us a few things about yourself and your farm?

Sergiu Goncearuc: Yes, of course... My name is Sergiu Goncearuc, I'm 44 years old, and I've been working in agriculture since I was a child. The farm is a family one — I took it over from my father. It's not very big, but not small either — we work about 120 hectares. Wheat, corn, sunflower... and a small plum orchard.

Most of the time it's just me and my brother, and during the busy season we bring in extra workers. That's how things work here.

Interviewer: How would you describe the daily structure of activities on the farm?

Sergiu: It depends on the season... but generally the day starts early. By 6 a.m. we're already in the field. If it's harvest season, the combine starts first. Then all the transport work begins.

In the afternoon we deal with organization, repairs, and inspections. Honestly, we spend almost half the time just moving things from one place to another.

Interviewer: What are the most common transport-related tasks?

Sergiu: We're always transporting something. Crops from the field to the storage, fertilizers, diesel... sometimes just a small part forgotten on the other side of the land.

And it may not seem like much, but these repeated trips consume time and fuel. Sometimes it feels like the tractors spend more time on the road than actually working.

Interviewer: Who organizes these tasks?

Sergiu: Mostly me. And my brother.

We don't have an automated system or anything digital. Everything is done from experience. Phone calls, quick discussions, decisions on the spot. If a problem appears, you have to react immediately.

Interviewer: What does a day during peak season look like?

Sergiu: It's intense... very intense.

The combine harvests continuously. If the trailer doesn't arrive on time, you have to stop. And when you stop, you lose time, money... and momentum.

Most time is lost in transportation and waiting. That's the reality.

Interviewer: What challenges do you face with transportation?

Sergiu: Field roads, especially after rain. Sometimes you enter with the tractor and realize you shouldn't have gone in...

Then there are breakdowns. If a trailer breaks down, the whole workflow stops. We don't have many spare units.

Interviewer: Can you describe a recent delay?

Sergiu: Yes... last year during the sunflower harvest. A trailer got a flat tire. The combine was full. We had to stop for almost two hours.

Maybe for someone outside agriculture two hours doesn't seem like much, but during peak season... it means a lot.

Interviewer: How does labor shortage affect your operations?

Sergiu: A lot. Very much.

Young people leave. Those who stay are few. When you don't have workers, you do everything yourself. You drive, inspect, repair. You get tired, and tiredness leads to mistakes.

Interviewer: What risks exist during transportation?

Sergiu: The farmyard becomes crowded during the season. Tractors, people, trucks. If someone isn't paying attention, an accident can happen.

Honestly, fatigue is the biggest risk.

Interviewer: Which tasks seem inefficient or repetitive to you?

Sergiu: Repetitive transport. Patrolling the yard. Manual checks.

They are necessary tasks, but they consume a lot of time and don't directly add value.

Interviewer: How do you solve logistical problems?

Sergiu: Quickly and practically. We redistribute equipment, call the mechanic, improvise. In agriculture you need to adapt quickly.

Interviewer: Have you ever improved a process on the farm?

Sergiu: Yes. We reorganized the farmyard. We moved the unloading area closer to the storage facility.

It seems simple, but it significantly reduced maneuvering time. Sometimes organization matters more than technology.

Interviewer: What technologies do you currently use?

Sergiu: We have GPS on the tractor. We use weather apps and a device for measuring grain moisture.

We're not fully digitalized, but we try to keep up.

Interviewer: What do you think about autonomous vehicles?

Sergiu: The idea is interesting... but it has to be safe.

The first concern is people's safety. Then reliability. If it breaks during the season, it becomes a big problem.

Interviewer: What level of autonomy would be acceptable?

Sergiu: At first, supervised. I should be able to monitor everything and stop it if needed.

Full autonomy... maybe in the future, but it needs to be tested well.

Interviewer: What tasks could such a vehicle take over?

Sergiu: Repetitive transport. Moving fertilizers. Patrolling the yard.

Basically the kind of work that doesn't require complicated decisions.

Interviewer: Thank you for your time.

Sergiu: You're welcome. Agriculture isn't easy, but if technology helps us in a practical way, not just theoretically, we're open to it. The important thing is that it's adapted to our reality here in Moldova.

Stakeholder 2 Interview:

Questions

1. What are the most frequent daily transport activities within the yard or workshop?
2. Who usually handles these transport activities?
3. During busy periods, which activities consume the most time?
4. What difficulties do you encounter when transporting wood or tools across the premises?
5. Have you experienced situations where delays occurred due to transport problems?
6. How does the lack of workforce influence daily operations?
7. What safety risks exist during wood transportation?
8. Which daily activities do you consider repetitive or inefficient?
9. When a problem occurs with a vehicle or transport equipment, how do you handle it?
10. Have you improved any process in recent years?
11. What do you consider before investing in new equipment?
12. How do you decide whether an investment is worth making?
13. Do you currently use modern technologies in your activity?
14. Have you used GPS systems or remote monitoring?
15. What concerns would you have about using an autonomous vehicle in the yard?
16. Would you prefer a fully autonomous vehicle or a supervised one?
17. What activities would you be willing to entrust to an autonomous vehicle?
18. What would make you trust such a system?
19. What concrete benefits would convince you to adopt this solution?

Workshop Documentation: Autonomous Yard Utility Vehicle Project

Name: Nicolae Paraschiv

Occupation: Entrepreneur

Location: Republic of Moldova, Streni District, Pneti Village

Interviewer: Istrati tefan

Transcript

Interviewer: Thank you for agreeing to this interview. To start, could you tell us a few things about yourself and the activity you carry out?

Nicolae Paraschiv: My name is Nicolae Paraschiv, I am 59 years old, and I work in wood processing. We operate with medium to large quantities. We have a fairly large yard where we store logs and boards, and inside the workshop we cut and prepare them for delivery to Chiinu.

Interviewer: What are the most frequent daily transport activities within the yard or workshop?

Nicolae Paraschiv: Every day we move logs to the cutting area, and then we transport the cut boards to the storage area or to the loading zone.

Interviewer: Who usually handles these transport activities?

Nicolae Paraschiv: Usually the workers. We have several people who mainly deal with handling and transporting the materials.

Interviewer: During busy periods, which activities consume the most time?

Nicolae Paraschiv: Transporting and rearranging the boards. When we have a large order, a lot of time is lost just moving them around.

Interviewer: What difficulties do you encounter when transporting wood or tools across the premises?

Nicolae Paraschiv: The heavy weight of the materials and the fact that sometimes multiple trips are required. When the ground is wet, it becomes even more complicated.

Interviewer: Have you experienced situations where delays occurred due to transport problems?

Nicolae Paraschiv: Yes. For example, when the forklift broke down, we had to move things manually and the delivery was delayed.

Interviewer: How does the lack of workforce influence daily operations?

Nicolae Paraschiv: If someone is absent, the others have to handle both transportation and the work on the machines. That slows down the entire process.

Interviewer: What safety risks exist during wood transportation?

Nicolae Paraschiv: There is a risk of injury when lifting, slipping, or if someone drops a heavy board.

Interviewer: Which daily activities do you consider repetitive or inefficient?

Nicolae Paraschiv: Moving boards from one place to another several times before they finally reach the loading area.

Interviewer: When a problem occurs with a vehicle or transport equipment, how do you handle it?

Nicolae Paraschiv: I call the mechanic or we improvise temporarily until it gets repaired.

Interviewer: Have you improved any process in recent years?

Nicolae Paraschiv: We reorganized the yard so that the route from the cutting area to the storage area is shorter.

Interviewer: What do you consider before investing in new equipment?

Nicolae Paraschiv: The price, usefulness, and how quickly I can recover the investment.

Interviewer: How do you decide whether an investment is worth making?

Nicolae Paraschiv: If it reduces costs or saves a significant amount of time.

Interviewer: Do you currently use modern technologies in your activity?

Nicolae Paraschiv: We have modern electric machines and a forklift, but nothing automated.

Interviewer: Have you used GPS systems or remote monitoring?

Nicolae Paraschiv: No, so far we have not used them.

Interviewer: What concerns would you have about using an autonomous vehicle in the yard?

Nicolae Paraschiv: That it might break down often or cause accidents or damage.

Interviewer: Would you prefer a fully autonomous vehicle or a supervised one?

Nicolae Paraschiv: At first I would prefer it to be supervised, until I gain trust in it.

Interviewer: What activities would you be willing to entrust to an autonomous vehicle?

Nicolae Paraschiv: Transporting the cut boards to storage and bringing tools where they are needed.

Interviewer: What would make you trust such a system?

Nicolae Paraschiv: If it works consistently without problems and has good safety systems.

Interviewer: What concrete benefits would convince you to adopt this solution?

Nicolae Paraschiv: Reduced transport costs, time savings, and fewer delivery delays.

Stakeholder 3 Interview:

Questions

1. Could you please do a short introduction and briefly explain your domain of experience and activity?
 2. (Comment rather than a full question, but part of the interaction) For the most part, you did very well. You covered most of the aspects.
 3. So it won't need continuous access to the cloud or other external navigation sources?
 4. What should we use for a cloud connection, and how should we manage it in the case where we have multiple devices?
 5. Could you tell us something about the final tests before the system is put into operation?
 6. Regarding the costs, approximately how much would the hardware cost?
-

Workshop Documentation: Autonomous Yard Utility Vehicle Project

Name: Andrei Bragarenco

Occupation: Hardware Engineer

Location: Republic of Moldova, Chiinu, Micro Lab

Interviewer: Istrati tefan and Ciumacenco Pavel

Transcript

Pavel: Okay, to start off, we will do a quick introduction.

Andrei: If it's easier for you in Romanian, it's better in Romanian.

Stefan: We also made the questions in English, so...

Pavel + Stefan: You can answer in Romanian.

Pavel: So, we are representatives of the Software Engineering Department, group FAF-232, and our project is Autonomous Yard Vehicle. Could you please do a short introduction and briefly explain your domain of experience and activity?

Andrei: My interests, as a stakeholder, are related to the fact that we could implement such solutions in rural households, and not only there, but in any enterprise where there is a controlled environment. Why? Because we have a problem with workforce, and there are situations where you won't hire a person just to move some things from point A to point B.

A particular use case: carrying a bag of cement from point A to point B and all you need is a button. It goes to the warehouse and sends a message telling them to put a bag of cement on it. Someone presses another button and it takes it back. This is a simple source-destination use case with a single button.

Another use case would be for fruit and vegetable pickers. It simply moves, and there is a button they press and the vehicle moves 5 meters forward. The simplest interface is a single button that you press and it does something depending on the circumstances. And you don't need a driver to move through a controlled environment, which is, for example, a row.

You can see on the left and right, for example with a camera, or in our case actually with GPS tracking, a precision GPS. You know exactly where you are going, you don't have navigation problems. If something appears in front of it, it moves at a very low speed, under 30 km/h or even under 5 km/h. That means it cannot harm anyone. We add emergency systems and we can implement some basic safety measures.

For example, it does not move if there is something in its ultrasonic barrier zone. We install ultrasonic sensors, regular parktronic sensors that are meant for low speed. At high speeds you need LiDAR or radar. At low speeds ultrasonic sensors are enough. On normal cars ultrasonic sensors disconnect above 5 km/h because their scanning speed is not good enough. Also at higher speeds you can get false noise from wind and other things.

So these are the circumstances, let's say, and my interests. To have a vehicle that can execute commands in three steps. For example, go to coordinate XY based on a simple command such as pressing a button. Or from the phone you give it coordinates. Then we can program commands like: go to coordinate XY, return to the destination, or go two steps backwards. These are basic commands.

It may seem trivial, but in a household there is usually a person who sits in a tractor and gets paid just to stay there all day. When someone buys this technology, for example this golf cart, it might cost 5,000 or 10,000 euros, but after that they don't have to pay the salary of a worker who earns around 20,000 lei per month. In three years, or even one or two years, you recover the full cost.

Also, you no longer have problems when the worker doesn't come to work, gets sick, or you have to pay insurance and so on. In the end it becomes cheaper. So this is where you gain an advantage.

Additionally, you can use it for several utilities. You can use it for inspection and other tasks. But for now we can focus on this use case: take something somewhere and bring it back. These three use cases: go from point A to B, go to coordinate XY, limp home (return home), or move forward for pickers. That's the classic version that would help.

For example, even to move cement from one place to another you still need a driver, which costs more than the tractor itself. If it only needs to go once or twice a day, the same person could do other tasks, or you have equipment that covers multiple tasks. On the other hand, your golf cart remains functional for manual driving if needed. So you can still use it normally, but you also have this automation that can give you a real advantage.

Okay, are there any other questions?

Pavel: For the most part, you did very well. You covered most of the aspects.

Andrei: That's why I say: in interviews you prepare questions and reference points, but it never goes exactly as in the book.

Stefan: We already talked about sensors. What should we use for a cloud connection and how should we manage it if we have multiple devices?

Andrei: Yes, of course. For the cloud part, I touched on it earlier, but it's important because you won't have just one vehicle, but maybe several. Then you need to send their status: what was done and where, how much battery they have, whether any incidents occurred.

For example, you send it to the field but something happens — maybe it hits something — and you need a notification. You connect to the camera and see what happened. So you need a camera, you need "eyes". Then you can take it out of that situation. You need remote control: you connect remotely and give commands such as forward, backward, or moving by coordinates, and create an exit or return scenario.

Maybe it entered an area where it cannot get out. Circumstances changed, someone placed something there, the ultrasonic sensor blocks it and it cannot move. If for 10 seconds it cannot get out, it sends you a notification and asks for control. You take control with a few clicks and move it forward and backward until you free it.

You can also write scenarios: driving in circles, patrolling, or other tasks. You will have a server where you can see it, track it, see where it moves and at what time. You can see scenarios, you can record a route. For example, someone drives it manually and you record the route, and after that it can follow the route automatically.

So you say you go from point A to point B, you drive it once and it records the route. The second time it will follow exactly the same path. This is called learning — the learning curve — meaning it learns a route. You drive it once and afterwards it repeats it through logs. You don't need extremely smart navigation; you can work with simple scenarios.

Pavel: So it doesn't need continuous access to the cloud or other external navigation sources.

Andrei: Yes, it doesn't need to see and avoid objects in complex ways — those things are too complicated. It just follows a predefined scenario assuming you know the environment completely. For example, in a field a person shouldn't randomly appear; if someone appears, the car stops, the obstacle disappears, and that's it.

If the obstacle stays there too long, you get a notification and you must intervene. Because if it starts navigating left and right trying to avoid things, it might damage plants. So it's better not to. Also, you're not in a hurry. It can even call your phone. That could be a feature: it calls and says "I have a problem." A robot voice says the vehicle is blocked. From your phone you connect, see what's wrong, connect to all cameras. The camera is just simple streaming. It's very easy to implement.

You will need a dashboard where you see everything like in a game. I send it somewhere and I have a map and I move it on the map. You can take pictures occasionally. These are "could have" features, depending on time. "Must have" is navigation. "Should have" is learning and scenarios. Navigation between points is mandatory. If I already explained what is must have and should have, what questions did I miss?

Stefan: Something about final testing before deployment.

Andrei: I'll tell you a use case. If you build it, make it drive one circle around Building 3. That's the validation test. The rest you can do yourselves. Someone sits in the vehicle with the stop button just in case, to disconnect if needed.

But you give the command "make a circle". If it completes the circle around the building, it means you validated that it can go anywhere you want.

You can also record the route. You drive and record the GPS track. We have GPS with RTK — Real-Time Kinematics. Normal GPS has an error of ± 3 to ± 6 meters. RTK has an error of about ± 7 centimeters. So it's very precise.

For tractors, if last year you made a furrow in a field, next year the tractor can place the seed exactly in the same coordinates. That's how precise it is. But it must be calibrated and there are problems.

What problems? Between buildings GPS does not work very well, even with RTK. We have signal reflections. Recording routes helps because you can have controlled deviations. On the map it may show one path while in reality you move slightly differently.

You can also create correction maps. When you are in a certain zone you apply certain coordinate corrections. Each point can have corrections to adjust the system. Even if the base precision is 7 cm, buildings block some satellites and you get measurement errors.

In open fields you have centimeter precision. Between buildings, because some satellites are blocked, the precision decreases. If the system sees only 3–4 satellites, the precision is low. If it sees 20–30 satellites, the precision becomes much higher. That is the main challenge.

So again, the validation test is a fully autonomous circle around Building 3. It's just asphalt around it. And add a sound and light signal. That's mandatory. It's a utility vehicle, so when it moves it must signal: beep-beep-beep.

Stefan: And one last question. Approximately, what are the hardware costs?

Andrei: The hardware equipment we already have. You will only need to list it and estimate the costs as existing equipment. I will give you the links.

Here is the steering system — it's already installed. There are cables connecting to it. I will also show you the GPS and introduce you to the guys so you can inspect everything and take pictures.

I want you, together with the team who previously worked on the project, to install the components and validate driving with this remote controller. You start the golf cart with this remote. Radu will help you after they show you how it works, and with him you'll move to the second phase to see how everything functions.

Also, this Pixhawk connects to a microcontroller and then to a computer. I gave you a computer, right?

Pavel: Yes, we just haven't picked it up yet.

Andrei: That computer is an industrial computer designed for the golf cart.

So this is a package of equipment. The Pixhawk connects to the vehicle — it's basically the "brain" of a drone. It was installed before and worked previously. From the computer you connect through USB and you have a Python API through which you control it.

You move one step forward, read GPS data, get your coordinates XYZ, and build your scenario logic. Hardware is not your biggest challenge — the important part is the software that controls the hardware.

So this device together with the computer forms the system. Computing happens here. From the cloud you send missions. A mission can be simply going from one point to another or a more complex scenario.

For real-time navigation you send short missions from point to point. At some point I'll leave the equipment here. You'll look at the Pixhawk 6C and search how to use the ArduPilot API, and for details you can ask Radu.

Stefan: And the parktronic sensors — are those all of them?

Andrei: Yes, they are there. You can connect them to the computer. They use a serial interface through a USB-UART converter. You'll see packets like FF and 8 bytes — two bytes per sensor representing distances — plus a checksum.

So you have 10-byte packets: one FF header, four pairs of bytes, and a checksum. That gives you the distance from each sensor.

In your model you can draw the vehicle, similar to car multimedia systems where you see the car and the parktronic sensors showing obstacles.

You'll have a UI where the data is displayed and you see a car model with obstacles shown. Four sensors in the back and four in the front. Those are our parktronic sensors.

We'll install the computer on the golf cart with bolts, route the cables, and connect everything. The previous team disconnected things since these are our vehicles, but we know where everything connects. The microcontroller also connects here, but they need to show you exactly how it works.

Okay?

I'll connect you with the guys who worked on this project, and that will be it from me.

Pavel: Yes, thank you very much.

Andrei: Did I clarify everything?

Pavel: Yes.

Andrei: With pleasure.

02-AGC WP4 Stakeholder Requirements

Work Product Status

Legend	NEW - Backlog/Planned PROGRESS - Started DONE - Finished
Develop Stakeholder Requirements	 PBL26-347 - Jira server returned an error: [Ljava.lang.Object;@4dbc10ff  PBL26-395 - Jira server returned an error: [Ljava.lang.Object;@1189f4c0  PBL26-396 - Jira server returned an error: [Ljava.lang.Object;@39862c52  PBL26-397 - Jira server returned an error: [Ljava.lang.Object;@c5992b1  PBL26-398 - Jira server returned an error: [Ljava.lang.Object;@7d613a49

Team Members

Nr	Team Member	Role
1	Denis Pleca	Team Lead
2	tefan Istrati	Developer
3	Pavel Ciumacenco	Developer
4	Vladlen Mitu	Developer

- Denis Pleca (68 days ago)

Analyze Stakeholder
Requirements

 [PBL26-348](#) - Jira server returned an error: [Ljava.
lang.Object;@172e0a02

 [PBL26-399](#) - Jira server returned an error: [Ljava.
lang.Object;@64d8bee6

 [PBL26-400](#) - Jira server returned an error: [Ljava.
lang.Object;@706d31c2

 [PBL26-401](#) - Jira server returned an error: [Ljava.
lang.Object;@50ebb5f8

 [PBL26-402](#) - Jira server returned an error: [Ljava.
lang.Object;@6f22af4b

Import Stakeholder requirements in Jira

 [PBL26-349](#) - Jira server returned an error: [Ljava.lang.Object;@5038a07b

 [PBL26-403](#) - Jira server returned an error: [Ljava.lang.Object;@1c46b794

 [PBL26-404](#) - Jira server returned an error: [Ljava.lang.Object;@417815ff

 [PBL26-405](#) - Jira server returned an error: [Ljava.lang.Object;@716cfbed

 [PBL26-406](#) - Jira server returned an error: [Ljava.lang.Object;@3ab44d5d

- [Work Product Status](#)
- [Team Members](#)

o

02-AGC WP4.1 Develop Stakeholder Requirements

Requirements

The stakeholder interviews conducted with a farmer (Sergiu Gonciaruc) and a wood processing entrepreneur (Nicolae Paraschiv) revealed several operational challenges related to repetitive transportation tasks, lack of workforce, safety concerns, and inefficiencies in logistics management. Both stakeholders described situations where a significant amount of time is spent moving materials across large working areas such as agricultural fields, farm yards, or industrial yards.

The interviews highlighted that transportation tasks are often repetitive and do not require complex decision-making, which makes them suitable for automation. At the same time, both interviewees emphasized that reliability and safety are critical factors for adopting autonomous systems. Stakeholders also expressed the need for supervision and monitoring capabilities, especially during the initial deployment of such technology.

Requirement ID	Stakeholder Type	Description	Stakeholder	Group	MoSCoW Classification	Rationale
R1	US	As a farmer, I want an autonomous vehicle to transport materials between field and storage so that I reduce time spent on repetitive transport tasks.	Farmers	Transport Automation	Must Have	This is the core functional objective of the proposed system. Without automated material transport, the system does not provide its primary value of reducing repetitive logistics work identified by farmers.
R2	US	As a farm operator, I want to monitor the autonomous vehicle remotely so that I can supervise its activity and intervene if needed.	Farmers	Monitoring	Should Have	Stakeholders emphasized the need for supervision during early adoption. While the vehicle could technically operate without it, monitoring significantly increases trust and operational control.
R3	JS	When harvest operations are running continuously, I want automated transport support so that harvesting machines do not stop due to lack of transport vehicles.	Farmers	Logistics Efficiency	Should Have	Continuous harvesting requires synchronized logistics. Automated support improves efficiency, but harvesting could still proceed manually if necessary.
R4	US	As a user, I want the vehicle to follow predefined routes so that transportation tasks can be executed consistently.	Farmers / Entrepreneurs	Navigation	Must Have	Route-based navigation is essential for predictable autonomous movement. Without this capability the vehicle cannot perform transport tasks reliably.
R5	JS	When working in environments with limited workforce, I want automation to perform repetitive tasks so that labor shortages do not affect productivity.	Farmers / Entrepreneurs	Workforce Optimization	Should Have	Labor shortages are a major challenge identified by stakeholders. Automation helps maintain productivity, but it is a strategic benefit rather than a minimum operational requirement.
R6	US	As a user, I want the vehicle to detect obstacles automatically so that accidents with people or equipment are avoided.	Farmers / Entrepreneurs	Safety	Must Have	Safety is critical for any autonomous system operating near people or equipment. Stakeholders clearly emphasized accident prevention as a primary concern.
R7	US	As a farm operator, I want a manual override or emergency stop so that I can immediately stop the system if something goes wrong.	Farmers	Safety	Must Have	Emergency stop functionality is a mandatory safety requirement in autonomous systems. It allows operators to immediately intervene if unexpected situations occur.
R8	JS	When the vehicle operates in farm yards or industrial spaces, I want reliable proximity detection so that collisions with equipment are prevented.	Farmers / Entrepreneurs	Safety	Must Have	Dense working environments require reliable collision prevention systems to avoid damage to equipment and injury to workers.
R9	US	As a user, I want the system to operate reliably during long working hours so that operations are not interrupted during peak season.	Farmers / Entrepreneurs	Reliability	Must Have	Agricultural work often involves long operational periods, especially during harvest. System reliability is essential to ensure the technology is practical and dependable.
R10	JS	When equipment breaks down or transport is delayed, I want an alternative automated transport solution so that workflow disruption is minimized.	Farmers	Logistics Efficiency	Could Have	This requirement provides additional operational flexibility but is not essential for the core system functionality.
R11	US	As a business owner, I want the system to reduce operational transport costs so that the investment becomes financially viable.	Entrepreneurs	Economic Efficiency	Should Have	Economic benefits are an important adoption driver. However, cost reduction is an outcome of the system rather than a technical requirement.
R12	US	As a user, I want the vehicle to transport tools and materials across the workspace so that workers can focus on higher-value tasks.	Entrepreneurs	Transport Automation	Must Have	Similar to R1, this requirement directly supports the main function of the system: reducing manual transportation tasks.
R13	JS	When operating in large yards or fields, I want automated logistics support so that materials can be moved efficiently between working zones.	Farmers / Entrepreneurs	Logistics Efficiency	Should Have	Improves operational efficiency in large workspaces but is not strictly required for basic autonomous transport functionality.

R14	US	As a user, I want the system to provide status information (location, battery level, operation status) so that I always know the vehicle condition.	Farmers / Entrepreneurs	Monitoring	Should Have	Provides transparency and better system management, but the system could technically operate without continuous status feedback.
R15	US	As a user, I want the system to be easy to use without complex configuration so that it can be adopted by non-technical operators.	Farmers / Entrepreneurs	User Experience	Could Have	Ease of use is important for adoption by non-technical operators, but the system could still function even if configuration requires some technical setup.
R16	US	As an operator, I want to send the vehicle to specific GPS coordinates so that it can perform transport tasks anywhere in the working area.	Farmers / Entrepreneurs	Navigation	Must Have	The interview highlighted the importance of coordinate-based commands such as "go to coordinate XY". This capability enables flexible operation across fields, yards, or warehouses.
R17	US	As a user, I want the vehicle to automatically return to its starting point so that transport cycles can be completed without manual retrieval.	Farmers / Entrepreneurs	Navigation	Must Have	The stakeholder emphasized "return home" or "limp home" functionality as a fundamental autonomous operation that ensures the vehicle can complete missions safely.
R18	JS	When workers are harvesting fruits or vegetables, I want the vehicle to move forward incrementally so that pickers can continue working without manually repositioning transport equipment.	Farmers	Logistics Efficiency	Should Have	The interview described a scenario where workers press a button and the vehicle advances a few meters. This supports harvesting efficiency but is not essential for basic logistics automation.
R19	US	As a user, I want the vehicle to be controlled using a simple interface such as a button or mobile command so that non-technical workers can operate it easily.	Farmers / Entrepreneurs	User Interface	Must Have	Stakeholders emphasized the importance of very simple controls. A minimal interface lowers the barrier for adoption and ensures the system can be used by workers without technical expertise.
R20	JS	When the vehicle encounters an obstacle and cannot continue moving, I want the system to notify the operator so that the issue can be resolved quickly.	Farmers / Entrepreneurs	Monitoring	Should Have	The interview suggested that if the vehicle remains blocked for a certain time, it should send a notification. This improves operational reliability but is not strictly required for basic movement.
R21	US	As an operator, I want to remotely control the vehicle through a dashboard or mobile interface so that I can resolve unexpected situations when autonomous operation is interrupted.	Farmers / Entrepreneurs	Remote Control	Should Have	Remote control functionality allows operators to intervene when obstacles or environmental changes prevent the vehicle from completing its route autonomously.
R22	US	As a system operator, I want a dashboard that displays vehicle location and operational data so that I can monitor its activity in real time.	Farmers / Entrepreneurs	Monitoring	Should Have	The interview mentioned a server or dashboard where the operator can observe vehicle movements and system status. This improves operational visibility and management.
R23	US	As an operator, I want the vehicle to record manually driven routes so that it can repeat the same routes autonomously in the future.	Farmers / Entrepreneurs	Navigation	Should Have	Route recording was suggested as a practical way to simplify navigation in controlled environments, allowing the system to repeat previously validated paths.
R24	JS	When multiple vehicles are deployed in the same environment, I want each vehicle to send operational data to a central system so that fleet activity can be monitored.	Farmers / Entrepreneurs	Logistics Efficiency	Could Have	The interview discussed scenarios with multiple vehicles sharing information through a server. While useful for scalability, it is not essential for a single-vehicle implementation.
R25	US	As a worker in the operational area, I want the vehicle to produce sound or light signals while moving so that people nearby are aware of its activity.	Farmers / Entrepreneurs	Safety	Must Have	The stakeholder explicitly mentioned that the vehicle should emit signals such as warning sounds while moving to alert nearby workers and prevent accidents.
R26	JS	When the system is deployed in a controlled environment, I want it to follow predefined scenarios rather than complex autonomous navigation so that the system remains reliable and predictable.	Farmers / Entrepreneurs	Navigation	Must Have	The interview emphasized using simple scenario-based navigation instead of complex obstacle avoidance logic to prevent unexpected behavior in agricultural environments.
R27	US	As an operator, I want to access video from onboard cameras so that I can inspect the vehicle surroundings when a problem occurs.	Farmers / Entrepreneurs	Monitoring	Could Have	The stakeholder suggested using cameras for remote inspection during incidents. This feature enhances situational awareness but is not required for basic functionality.
R28	JS	When validating the system before deployment, I want the vehicle to complete a predefined autonomous route so that the functionality of navigation can be confirmed.	Technical Team	System Validation	Must Have	The interview proposed a practical validation scenario where the vehicle completes an autonomous route around a building, confirming that the navigation system works correctly.

Classes

Transport Automation (R1, R12)

Stakeholder Requirement: The system should automate the transportation of tools and materials within agricultural or industrial environments.

User Stories:

As a farmer, I want an autonomous vehicle to transport materials between field and storage so that I reduce time spent on repetitive transport tasks.

As a user, I want the vehicle to transport tools and materials across the workspace so that workers can focus on higher-value tasks.

Job Stories:

When operating in large yards or fields, I want automated logistics support so that materials can be moved efficiently between working zones.

When equipment breaks down or transport is delayed, I want an alternative automated transport solution so that workflow disruption is minimized.

Navigation and Routing (R4, R16, R17, R23, R26, R28)

Stakeholder Requirement: The system should navigate autonomously using predefined routes to ensure reliable transportation.

User Stories:

As a user, I want the vehicle to follow predefined routes so that transportation tasks can be executed consistently.

Job Stories:

When the system operates in large environments, I want route-based navigation so that transport tasks can be performed reliably without manual guidance.

Safety (R6, R7, R8, R25)

Stakeholder Requirement: The system should ensure safe operation by preventing collisions and allowing emergency intervention.

User Stories:

As a user, I want the vehicle to detect obstacles automatically so that accidents with people or equipment are avoided.

As a farm operator, I want a manual override or emergency stop so that I can immediately stop the system if something goes wrong.

Job Stories:

When the vehicle operates in farm yards or industrial spaces, I want reliable proximity detection so that collisions with equipment are prevented.

Monitoring and Control (R2, R14, R20, R21, R22, R27)

Stakeholder Requirement: The system should provide monitoring capabilities so operators can supervise and manage the autonomous vehicle.

User Stories:

As a farm operator, I want to monitor the autonomous vehicle remotely so that I can supervise its activity and intervene if needed.

As a user, I want the system to provide status information (location, battery level, operation status) so that I always know the vehicle condition.

Job Stories:

When supervising automated operations, I want access to real-time system information so that I can respond quickly to potential issues.

Logistics Efficiency (R3, R10, R13,18, R24)

Stakeholder Requirement: The system should support continuous workflows by improving logistics coordination.

User Stories:

As a user, I want automated logistics support so that materials can be transported efficiently during work operations.

Job Stories:

When harvest operations are running continuously, I want automated transport support so that harvesting machines do not stop due to lack of transport vehicles.

When equipment breaks down or transport is delayed, I want automated support so that workflow disruption is minimized.

Workforce Optimization (R5)

Stakeholder Requirement: The system should reduce dependence on manual labor by automating repetitive tasks.

User Stories:

As a user, I want repetitive transport tasks to be automated so that workers can focus on more productive activities.

Job Stories:

When working in environments with limited workforce, I want automation to perform repetitive tasks so that labor shortages do not affect productivity.

Economic Efficiency (R11)

Stakeholder Requirement: The system should reduce operational transport costs and improve economic efficiency.

User Stories:

As a business owner, I want the system to reduce operational transport costs so that the investment becomes financially viable.

Job Stories:

When evaluating automation solutions, I want to ensure the system provides economic benefits so that the investment is justified.

System Reliability (R9)

Stakeholder Requirement: The system should operate reliably during long working periods without interruptions.

User Stories:

As a user, I want the system to operate reliably during long working hours so that operations are not interrupted during peak season.

Job Stories:

When operating during intensive agricultural periods, I want reliable autonomous transport so that logistics operations continue without failures.

User Experience (R15, R19)

Stakeholder Requirement: The system should be easy to use and accessible to non-technical operators.

User Stories:

As a user, I want the system to be easy to use without complex configuration so that it can be adopted by non-technical operators.

Job Stories:

When setting up the system, I want a simple configuration process so that I can start using the vehicle without technical expertise.

MoSCoW Analysis Groups**Must Have (Critical System Requirements)**

These requirements are essential for system functionality and safe operation:

- R1 – Autonomous transport capability
- R4 – Route-based navigation
- R6 – Obstacle detection
- R7 – Emergency stop / manual override
- R8 – Proximity detection
- R9 – Operational reliability
- R12 – Material transport functionality
- R16 – Coordinate-based navigation
- R17 – Automatic return to starting point
- R19 – Simple user control interface
- R25 – Safety signaling during movement
- R26 – Scenario-based navigation reliability
- R28 – System validation through autonomous route execution

Should Have (Important but Not Critical)

- R2 – Remote monitoring
- R3 – Harvest logistics support
- R5 – Workforce optimization
- R11 – Operational cost reduction
- R13 – Logistics support for large environments
- R14 – System status reporting
- R18 – Incremental vehicle movement for harvesting
- R20 – Operator notification when blocked
- R21 – Remote manual control
- R22 – Monitoring dashboard
- R23 – Route recording

Could Have (Optional Enhancements)

- R10 – Backup logistics during equipment failure
- R15 – Simplified user configuration
- R24 – Multi-device management
- R27 – Onboard camera remote access

Won't Have (For Current Version)

No explicit "Won't Have" requirements were identified at this stage, as most stakeholder needs remain relevant to the system's potential development roadmap.

02-AGC WP4.2 Analyze Stakeholder Requirements

Category	Stakeholder Requirements	US(User Stories)	JS(Job Stories)	Classification	Rationale
Affordability	The solution should be affordable.	As a business owner, I want the system to be cost-effective so that I can minimize operational expenses.	When evaluating automation solutions, I want to ensure the system is affordable so I can justify the investment.	Must Have	A key concern for business owners is to "minimize operational expenses" and "justify the investment"; this is crucial for business survival and ROI.
Task Prioritization	The solution should prioritize delicate tasks over processing speed.	As an operator, I want the system to handle delicate items with care so that fragile products do not get damaged.	When processing fragile items, I want the system to prioritize careful handling over speed so I can reduce product loss.	Must Have	Essential for maintaining product integrity; has a direct impact on product loss, which raises expenses and lowers customer satisfaction.
Performance Metrics	The system's success metric should be achieving at least half the throughput and accuracy of a human worker.	As a manager, I want the system to achieve at least half the throughput and accuracy of a human worker so that it remains a viable replacement.	When evaluating system performance, I want to compare throughput and accuracy against human workers so I can assess efficiency.	Must Have	Establishes the minimal feasible performance standard, which is necessary to assess if the system is functionally sufficient.
Telemetry & Predictive Maintenance	The system must collect telemetry data to improve efficiency and speed. Additionally, it should predict the robot's health status, potential failures, and maintenance needs.	As a maintenance technician, I want the system to collect telemetry data so that I can monitor its performance and predict potential failures.	When the system is operating, I want it to track efficiency and predict maintenance needs so I can prevent unexpected breakdowns.	Should Have	Essential for operational reliability and preventing costly downtime; necessary for long-term efficiency.
Adaptive Processing	An optional feature is the ability for the system to automatically detect and adjust processing speeds based on factors such as current workload, product size, and product type (e.g., eggs require more delicate handling, while cosmetics require less care).	As an operator, I want the system to adjust processing speeds automatically so that it can optimize performance based on workload and product type.	When workload or product type changes, I want the system to modify its speed so I can ensure efficiency and product safety.	Should Have	Important for optimizing performance across different products, but system can still function without this capability initially.
Simulation	The system could include a simulator for model testing.	As a developer, I want a simulator for model testing so that I can optimize the system before deployment.	When developing new models, I want to test them in a simulation so I can validate performance before real-world use.	Could Have	Although it might enhance implementation, it is not necessary for operation and is useful for testing but not for the system's main functionality.

Camera for System Monitoring	Cameras could be implemented for system monitoring to ensure the robot is classifying items correctly.	As a supervisor, I want cameras to monitor classification accuracy so that I can ensure the robot is sorting items correctly.	When the system is sorting products, I want real-time video monitoring so I can verify classification accuracy.	Should Have	Crucial for maintaining quality control and guaranteeing precise classification, but manual spot inspections could be employed first.
Camera for Safety	Cameras could be implemented for safety monitoring to detect if someone is too close to the robot and trigger an alarm while stopping operation if necessary.	As a safety officer, I want the system to detect if someone is too close so that it can stop operation and trigger an alarm.	When a person enters a restricted area, I want the system to pause and alert staff so I can prevent accidents.	Should Have	Essential for maintaining regulatory compliance and worker safety; avoids mishaps and possible liability problems
Camera for Security	Cameras could be implemented for security monitoring to detect tampering or theft attempts.	As a security manager, I want cameras to detect tampering or theft so that I can prevent unauthorized access.	When suspicious activity occurs near the robot, I want the system to alert security so I can take action.	Could Have	Adds benefit but is not necessary for basic functionality; at first, standard security measures may be adequate.
Conveyor Belt for Transportation	The system should have a conveyor belt to transport pallets once they are full, after the robot has placed the products inside.	As a logistics manager, I want a conveyor belt to transport full pallets so that I can streamline product movement.	When a pallet is full, I want the system to transfer it automatically so I can reduce manual handling.	Should Have	Vital for automating the entire process; without it, a large amount of manual intervention would be needed by the system.
Operation in Slightly Wet Environments	The system could support operation in slightly wet environments.	As an operations manager, I want the system to function in slightly wet environments so that it remains reliable in my facility.	When working in humid or damp conditions, I want the system to continue operating so I can avoid downtime.	Could Have	Would expand usability but not essential for core functionality in all environments
Conveyor Belt Synchronization	The system should synchronize with the conveyor belt.	As an engineer, I want the system to synchronize with the conveyor belt so that item movement is seamless.	When the conveyor belt speed changes, I want the robot to adjust accordingly so I can maintain efficiency.	Must Have	Essential for proper system functioning; without synchronization, the entire process would be inefficient or fail
Survey Extracted Stakeholder Requirements:					
Sorting Process Efficiency	The system should automate repetitive tasks to improve processing time.	As a sorting operator, I want to automate repetitive tasks so that I can reduce processing time.	When sorting materials, I want the system to dynamically adjust for material types to improve accuracy.	Must Have	Core purpose of the system; directly impacts ROI and operational performance

	The system should increase throughput by using high-speed robotic sorting.	As a facility owner, I want to implement high-speed robotic sorting systems to increase throughput.	When processing mixed recyclables, I want to use AI-driven systems to identify materials faster.	Must Have	Direct impact on business outcomes and system effectiveness ; fundamental value proposition
	The system should reduce manual labor costs.	As a manager, I want to reduce manual labor costs by introducing robotics into the sorting process.	When evaluating operational efficiency, I want data on automation reducing manual intervention so I can justify upgrades.	Must Have	Primary business justification for the system; essential for ROI
Cost Optimization	The system should minimize energy consumption during operations.	As an operations manager, I want to reduce energy consumption during sorting processes.	When planning upgrades, I want cost-benefit analyses for new technologies to justify investments.	Should Have	Important for ongoing operational costs but not critical for initial functionality
	Maintenance costs should be reduced by upgrading mechanical systems.	As a business owner, I want to minimize maintenance costs by upgrading mechanical systems.	When evaluating operational costs, I want detailed reports on labor vs automation expenses for decision-making.	Should Have	Important for long-term ROI but not critical for initial system deployment
	The system should provide ROI data before adopting advanced technologies.	As a stakeholder, I want proven ROI data before adopting advanced sorting technologies.	When evaluating automation solutions, I want financial projections so that investments are justified effectively.	Should Have	Important for business decision-making but not essential for the system's operation
Environmental Impact	The system should adopt sustainable methods that align with regulatory compliance and reduce environmental impact.	As an organization, I want to adopt sustainable sorting methods that reduce environmental impact.	When processing organic waste, I want systems that optimize composting conditions for better results.	Should Have	Important for regulatory compliance and corporate responsibility, but may not be critical for initial deployment
	Metrics should be provided on carbon footprint reductions from improved processes.	As a sustainability officer, I want metrics on carbon footprint reductions from improved processes.	When evaluating new systems, I want environmental benefits included in decision-making criteria for sustainability goals.	Could Have	Valuable for reporting and marketing but not essential for core functionality
	Metrics should be provided on carbon footprint reductions from improved processes.	As a sustainability officer, I want metrics on carbon footprint reductions from improved processes.	When evaluating new systems, I want environmental benefits included in decision-making criteria for sustainability goals.	Could Have	Valuable for reporting and marketing but not essential for core functionality
Automation and Technology Integration	The system should use AI-driven adaptive algorithms for efficient sorting based on input patterns and material types.	As an engineer, I want AI-driven adaptive sorting algorithms that adjust based on input patterns and material types.	When integrating new systems, I want seamless compatibility with existing infrastructure for smooth operations.	Must Have	Core to the system's ability to perform adaptive sorting effectively

	Multi-sensor detection systems should be implemented for improved material identification and classification accuracy.	As a facility manager, I want multi-sensor detection systems for improved material identification during operations.	When introducing automation solutions, I need minimal disruption to ongoing operations during implementation phases.	Should Have	Enhances accuracy but single-sensor systems could provide basic functionality initially
	Cloud-based optimization tools should enable real-time monitoring of sorting performance metrics remotely from any location.	As an operator, I want cloud-based optimization tools for real-time monitoring of sorting performance metrics remotely.	When monitoring performance remotely, I need dashboards that provide actionable insights in real-time for adjustments.	Could Have	Adds convenience but on-site monitoring could suffice initially
Embrace Advanced Technologies	Incorporate emerging technologies like AI, robotics, and sensor systems to enhance sorting operations. A	s a technology lead, I want the sorting system to leverage AI-driven analytics so that we can optimize performance, improve decision-making, and reduce operational costs.	When evaluating automation solutions, I require data on the performance and capabilities of systems based on AI, robotics, and sensor technologies to justify the investment.	Should Have	Important for system performance but specific technologies should be evaluated based on ROI

02-AGC WP4.3 Import Stakeholder requirements in Jira

Category	Stakeholder Requirements	Type	Classification	Rationale	Jira Link
Affordability	As a business owner, I want the system to be cost-effective so that I can minimize operational expenses.	US	Must Have	A key concern for business owners is to "minimize operational expenses" and "justify the investment"; this is crucial for business survival and ROI.	CD R-5
Task Prioritization	As a business owner, when processing fragile items, I want the system to prioritize careful handling over speed so I can reduce product loss.	US	Must Have	Essential for maintaining product integrity; has a direct impact on product loss, which raises expenses and lowers customer satisfaction.	CD R-15
Performance Metrics	As a manager, I want the system to achieve at least half the throughput and accuracy of a human worker so that it remains a viable replacement.	US	Must Have	Establishes the minimal feasible performance standard, which is necessary to assess if the system is functionally sufficient.	CD R-18
Telemetry & Predictive Maintenance	When the system is operating, it tracks efficiency and predicts maintenance needs so that it can prevent unexpected breakdowns.	JS	Should Have	Essential for operational reliability and preventing costly downtime; necessary for long-term efficiency.	CD R-22
Adaptive Processing	When workload or product type changes, the system modifies its speed so that it can ensure efficiency and product safety.	JS	Should Have	Important for optimizing performance across different products, but system can still function without this capability initially.	CD R-25
Simulation	As a developer, I want a simulator for model testing so that I can optimize the system before deployment.	US	Could Have	Although it might enhance implementation, it is not necessary for operation and is useful for testing but not for the system's main functionality.	CD R-27
Camera for System Monitoring	As a developer, I want cameras to monitor classification accuracy so that I can ensure the robot is sorting items correctly.	US	Should Have	Crucial for maintaining quality control and guaranteeing precise classification, but manual spot inspections could be employed first.	CD R-30
Camera for Safety	When a person enters a restricted area, the system pauses and alerts staff so I can prevent accidents.	JS	Should Have	Essential for maintaining regulatory compliance and worker safety; avoids mishaps and possible liability problems	CD R-34
Camera for Security	As a security manager, I want cameras to detect tampering or theft so that I can prevent unauthorized access.	US	Could Have	Adds benefit but is not necessary for basic functionality; at first, standard security measures may be adequate.	CD R-33
Conveyor Belt for Transportation	When a pallet is full, the system transfers it automatically so that it can reduce manual handling.	JS	Should Have	Vital for automating the entire process; without it, a large amount of manual intervention would be needed by the system.	CD R-38
Operation in Slightly Wet Environments	When working in humid or damp conditions, the system continues operating so that it can avoid downtime.	JS	Could Have	Would expand usability but not essential for core functionality in all environments	CD R-45
Conveyor Belt Synchronization	When the conveyor belt speed changes, the robot to adjust accordingly so that it can maintain efficiency.	JS	Must Have	Essential for proper system functioning; without synchronization, the entire process would be inefficient or fail	CD R-40
Survey Extracted Stakeholder Requirements:					
Sorting Process Efficiency	As a sorting operator, I want to automate repetitive tasks so that I can reduce processing time.		Must Have	Core purpose of the system; directly impacts ROI and operational performance	CD R-10 CD R-12
	As a facility owner, I want to implement high-speed robotic sorting systems to increase throughput.  CDR-50 - Jira server returned an error: [Ljava.lang.Object;@131eec6f		Must Have	Direct impact on business outcomes and system effectiveness; fundamental value proposition	

	As a manager, I want to reduce manual labor costs by introducing robotics into the sorting process.  CDR-52 - Jira server returned an error: [Ljava.lang.Object;@3aa0dcea		Must Have	Primary business justification for the system; essential for ROI	
Cost Optimization	As an operations manager, I want to reduce energy consumption during sorting processes.		Should Have	Important for ongoing operational costs but not critical for initial functionality	
	As a business owner, I want to minimize maintenance costs by upgrading mechanical systems.		Should Have	Important for long-term ROI but not critical for initial system deployment	
	As a stakeholder, I want proven ROI data before adopting advanced sorting technologies.		Should Have	Important for business decision-making but not essential for the system's operation	
Environmental Impact	As an organization, I want to adopt sustainable sorting methods that reduce environmental impact.		Should Have	Important for regulatory compliance and corporate responsibility, but may not be critical for initial deployment	CD R-42 CD R-44
	As a sustainability officer, I want metrics on carbon footprint reductions from improved processes.		Could Have	Valuable for reporting and marketing but not essential for core functionality	
	As a sustainability officer, I want metrics on carbon footprint reductions from improved processes.		Could Have	Valuable for reporting and marketing but not essential for core functionality	
Automation and Technology Integration	As an engineer, I want AI-driven adaptive sorting algorithms that adjust based on input patterns and material types.		Must Have	Core to the system's ability to perform adaptive sorting effectively	CD R-11 CD R-13
	As a facility manager, I want multi-sensor detection systems for improved material identification during operations.		Should Have	Enhances accuracy but single-sensor systems could provide basic functionality initially	
	As an operator, I want cloud-based optimization tools for real-time monitoring of sorting performance metrics remotely.		Could Have	Adds convenience but on-site monitoring could suffice initially	

Classes

 CDR-4 - Jira server returned an error: [Ljava.lang.Object;@d1232a8

- Stakeholder Requirement: The solution should be affordable and optimize operational costs.
- User Stories:

-  CDR-5 - Jira server returned an error: [Ljava.lang.Object;@54b1b814
-  CDR-6 - Jira server returned an error: [Ljava.lang.Object;@4a0b0561

- Job Stories:

-  CDR-7 - Jira server returned an error: [Ljava.lang.Object;
@cc306fc

-  CDR-8 - Jira server returned an error: [Ljava.lang.Object;
@61207366

 CDR-9 - Jira server returned an error: [Ljava.lang.Object;
@7ca7e3e1

- **Stakeholder Requirement:** The system should optimize sorting efficiency through automation.
- **User Stories:**

-  CDR-10 - Jira server returned an error: [Ljava.lang.Object;
@2a56b7d3

-  CDR-11 - Jira server returned an error: [Ljava.lang.Object;
@665e2e8

- **Job Stories:**

-  CDR-12 - Jira server returned an error: [Ljava.lang.Object;
@41fad9d0

-  CDR-13 - Jira server returned an error: [Ljava.lang.Object;
@6812acc7

 CDR-14 - Jira server returned an error: [Ljava.lang.Object;
@78f6b4af

- **Stakeholder Requirement:** The solution should prioritize delicate tasks over processing speed.
- **User Stories:**

-  CDR-15 - Jira server returned an error: [Ljava.lang.Object;
@6b8ef3c9

- **Job Stories:**

-  CDR-16 - Jira server returned an error: [Ljava.lang.Object;
@38da3f90

 CDR-17 - Jira server returned an error: [Ljava.lang.Object;
@7ed574b3

- **Stakeholder Requirement:** The system's success metric should be achieving at least half the throughput and accuracy of a human worker.
- **User Stories:**

-  CDR-18 - Jira server returned an error: [Ljava.lang.Object;
@6c7f23c5

- **Job Stories:**

-  CDR-19 - Jira server returned an error: [Ljava.lang.Object;
@1a7f2e8

 CDR-20 - Jira server returned an error: [Ljava.lang.Object;
@6227e739

- **Stakeholder Requirement:** The system must collect telemetry data to improve efficiency and predict failures.
- **User Stories:**

-  CDR-21 - Jira server returned an error: [Ljava.lang.Object;
@4f4e4416

- **Job Stories:**

-  CDR-22 - Jira server returned an error: [Ljava.lang.Object;
@462a843c

Material Handling Challenges & Worker Safety

- **Stakeholder Requirement:** The system should support proper material handling and ensure worker safety.
 - **User Stories:**
 - As a worker, I want training on handling diverse materials so that I can improve sorting accuracy.
 - As a worker, I want safety measures in place when operating near automated machinery.
 - **Job Stories:**
 - When handling heavy construction materials, I want automated lifting systems to reduce physical strain.
 - When training new employees, I want user-friendly interfaces for learning automated systems quickly.
-

 **CDR-23** - Jira server returned an error: [Ljava.lang.Object;
@64af5001

- **Stakeholder Requirement:** An optional feature is the ability for the system to automatically adjust processing speeds based on workload, product size, and type.
- **User Stories:**

◦  **CDR-24** - Jira server returned an error: [Ljava.lang.Object;
@641fb656

- **Job Stories:**

◦  **CDR-25** - Jira server returned an error: [Ljava.lang.Object;
@6d811d73

 **CDR-26** - Jira server returned an error: [Ljava.lang.Object;
@4db8749b

- **Stakeholder Requirement:** The system could include a simulator for model testing.
- **User Stories:**

◦  **CDR-27** - Jira server returned an error: [Ljava.lang.Object;
@64995247

- **Job Stories:**

◦  **CDR-28** - Jira server returned an error: [Ljava.lang.Object;
@dc370c3

 **CDR-29** - Jira server returned an error: [Ljava.lang.Object;
@48e3e5a1

System Monitoring

- **Stakeholder Requirement:** Cameras could be implemented for system monitoring.
- **User Stories:**

◦  **CDR-30** - Jira server returned an error: [Ljava.lang.Object;
@37d1e9f9

- **Job Stories:**

◦  **CDR-31** - Jira server returned an error: [Ljava.lang.Object;
@77120030

Safety Monitoring

- **Stakeholder Requirement:** Cameras could detect if someone is too close and stop operation if necessary.
- **User Stories:**

◦  **CDR-32** - Jira server returned an error: [Ljava.lang.Object;
@1239e478

- **Job Stories:**

◦  **CDR-34** - Jira server returned an error: [Ljava.lang.Object;
@6aa136f4

Security Monitoring

- **Stakeholder Requirement:** Cameras could detect tampering or theft attempts.
- **User Stories:**

◦  **CDR-33** - Jira server returned an error: [Ljava.lang.Object;
@8220370

- **Job Stories:**

◦  **CDR-35** - Jira server returned an error: [Ljava.lang.Object;
@4542a552

 **CDR-36** - Jira server returned an error: [Ljava.lang.Object;
@49a90047

Transportation

- **Stakeholder Requirement:** The system should have a conveyor belt to transport pallets once they are full.
- **User Stories:**

○

 **CDR-37** - Jira server returned an error: [Ljava.lang.Object;
@6739e0e8

- **Job Stories:**

○

 **CDR-38** - Jira server returned an error: [Ljava.lang.Object;
@889529a

Synchronization

- **Stakeholder Requirement:** The system should synchronize with the conveyor belt.
- **User Stories:**

○

 **CDR-39** - Jira server returned an error: [Ljava.lang.Object;
@18b9178b

- **Job Stories:**

○

 **CDR-40** - Jira server returned an error: [Ljava.lang.Object;
@14d4b2a3

 **CDR-41** - Jira server returned an error: [Ljava.lang.Object;
@743257fe

- **Stakeholder Requirement:** The system should support sustainable methods and adapt to certain environmental conditions.
- **User Stories:**

○

 **CDR-42** - Jira server returned an error: [Ljava.lang.Object;
@737dc0c7

-  CDR-43 - Jira server returned an error: [Ljava.lang.Object;
@be11311

- **Job Stories:**

-  CDR-44 - Jira server returned an error: [Ljava.lang.Object;
@41237468

-  CDR-45 - Jira server returned an error: [Ljava.lang.Object;
@3bdaa380

02-AGC WP5 System Requirements

Work Product Status

Legend	NEW - Backlog/Planned PROGRESS - Started DONE - Finished
Analyze Stakeholder Requirements	 PBL26-481 - Jira server returned an error: [Ljava.lang.Object:@5dd2e5ce  PBL26-529 - Jira server returned an error: [Ljava.lang.Object:@5825b123  PBL26-530 - Jira server returned an error: [Ljava.lang.Object:@718d2453  PBL26-531 - Jira server returned an error: [Ljava.lang.Object:@33a8653a  PBL26-532 - Jira server returned an error: [Ljava.lang.Object:@3beb643f

Team Members

Nr	Team Member	Role
1	Dmitrii Belih	Team Lead
2	Alexandra Bujor-Cobili	Developer
3	Alexandra Mihalevschi	Developer
4	Ivan Rudenco	Developer

- [Denis Pleca](#) (62 days ago)

Develop System
Requirements



[PBL26-482](#) - Jira server returned an error:
[Ljava.lang.Object;@620ed501



[PBL26-533](#) - Jira server returned an error:
[Ljava.lang.Object;@2c9c1688



[PBL26-534](#) - Jira server returned an error:
[Ljava.lang.Object;@6d371535



[PBL26-535](#) - Jira server returned an error:
[Ljava.lang.Object;@7edd3ef7



[PBL26-536](#) - Jira server returned an error:
[Ljava.lang.Object;@51b3410f

Ensure requirements
traceability



[PBL26-483](#) - Jira server returned an error:
[Ljava.lang.Object;@36421f25



[PBL26-537](#) - Jira server returned an error:
[Ljava.lang.Object;@1e38d1d7



[PBL26-538](#) - Jira server returned an error:
[Ljava.lang.Object;@3bca859d



[PBL26-539](#) - Jira server returned an error:
[Ljava.lang.Object;@452234ce



[PBL26-540](#) - Jira server returned an error:
[Ljava.lang.Object;@30a3d12d

Review and Release System requirements



[PBL26-484](#) - Jira server returned an error:
[Ljava.lang.Object;@58269382



[PBL26-541](#) - Jira server returned an error:
[Ljava.lang.Object;@3651b8e3



[PBL26-542](#) - Jira server returned an error:
[Ljava.lang.Object;@4eaf87a0



[PBL26-543](#) - Jira server returned an error:
[Ljava.lang.Object;@118f3634



[PBL26-544](#) - Jira server returned an error:
[Ljava.lang.Object;@77578308

- [Work Product Status](#)
- [Team Members](#)

o

02-AGC WP5.1 Analyze Stakeholder Requirements

ID	Requirement Type	Stakeholder	Description	Acceptance Criteria	Priority	Status	AGC-ID
SHR-01	User Story	Farmers	As a farmer, I want an autonomous vehicle to transport materials between field and storage so that I reduce time spent on repetitive transport tasks.	The vehicle transports materials autonomously between predefined points in at least 95% of tasks.	Must Have	In Work	 DRS -2 - Jira serv er retur ned an error : [Ljav a. lang. Obje ct; @f1 3f6d 2

SHR-02	User Story	Farmers	As a farm operator, I want to monitor the autonomous vehicle remotely so that I can supervise its activity and intervene if needed.	The system provides real-time monitoring via dashboard or mobile interface.	Should Have	Backlog	 DRS -3 - Jira serv er retur ned an error : [Ljav a. lang. Obje ct; @1d b9b7 45
SHR-03	Job Story	Farmers	When harvest operations are running continuously, I want automated transport support so that harvesting machines do not stop due to lack of transport vehicles.	Transport delays do not exceed a defined threshold (e.g., 2 minutes).	Should Have	Backlog	 DRS -4 - Jira serv er retur ned an error : [Ljav a. lang. Obje ct; @32 287f 29

SHR-04	User Story	Farmers / Entrepreneurs	As a user, I want the vehicle to follow predefined routes so that transportation tasks can be executed consistently.	The vehicle follows routes with minimal deviation (e.g., <1 meter).	Must Have	Backlog	 DRS -5 - Jira serv er retur ned an error : [Ljav a. lang. Obje ct; @75 0376 c1
SHR-05	Job Story	Farmers / Entrepreneurs	When working in environments with limited workforce, I want automation to perform repetitive tasks so that labor shortages do not affect productivity.	The system replaces at least one manual transport role.	Should Have	Backlog	 DRS -6 - Jira serv er retur ned an error : [Ljav a. lang. Obje ct; @69 3dad a1

SHR-06	User Story	Farmers / Entrepreneurs	As a user, I want the vehicle to detect obstacles automatically so that accidents with people or equipment are avoided.	Obstacles are detected and avoided in all test scenarios.	Must Have	Backlog	 DRS -7 - Jira serv er retur ned an error : [Ljav a. lang. Obje ct; @58 0889 8b
SHR-07	User Story	Farmers	As a farm operator, I want a manual override or emergency stop so that I can immediately stop the system if something goes wrong.	The system stops within <1 second after emergency input.	Must Have	Backlog	 DRS -8 - Jira serv er retur ned an error : [Ljav a. lang. Obje ct; @78 7e76 ad

SHR-08	Job Story	Farmers / Entrepreneurs	When the vehicle operates in farm yards or industrial spaces, I want reliable proximity detection so that collisions with equipment are prevented.	The system maintains a safe distance from obstacles at all times.	Must Have	Backlog	 DRS -9 - Jira serv er retur ned an error : [Ljav a. lang. Obje ct; @72 5e09 46
SHR-09	User Story	Farmers / Entrepreneurs	As a user, I want the system to operate reliably during long working hours so that operations are not interrupted during peak season.	The system operates continuously for 8–10 hours without failure.	Must Have	Backlog	 DRS -10 - Jira serv er retur ned an error : [Ljav a. lang. Obje ct; @66 2d4b 4d

SHR-10	Job Story	Farmers	When equipment breaks down or transport is delayed, I want an alternative automated transport solution so that workflow disruption is minimized.	The system resumes or reassigns tasks within a defined recovery time.	Could Have	Backlog	 DRS -11 - Jira serv er retur ned an error : [Ljav a. lang. Obje ct; @69 5964 b
SHR-11	User Story	Entrepreneurs	As a business owner, I want the system to reduce operational transport costs so that the investment becomes financially viable.	Cost reduction is measurable compared to manual transport.	Should Have	Backlog	 DRS -12 - Jira serv er retur ned an error : [Ljav a. lang. Obje ct; @77 f5ec 59

SHR-12	User Story	Entrepreneurs	As a user, I want the vehicle to transport tools and materials across the workspace so that workers can focus on higher-value tasks.	Transport tasks are completed without worker involvement.	Must Have	Backlog	 DRS -13 - Jira serv er retur ned an error : [Ljav a. lang. Obje ct; @45 a1b3 59
SHR-13	Job Story	Farmers / Entrepreneurs	When operating in large yards or fields, I want automated logistics support so that materials can be moved efficiently between working zones.	Transport time is reduced compared to manual processes.	Should Have	Backlog	 DRS -14 - Jira serv er retur ned an error : [Ljav a. lang. Obje ct; @3c 8497 13

SHR-14	User Story	Farmers / Entrepreneurs	As a user, I want the system to provide status information (location, battery level, operation status) so that I always know the vehicle condition.	Status data is updated in real time (<5 seconds delay).	Should Have	Backlog	 DRS -15 - Jira serv er retur ned an error : [Ljav a. lang. Obje ct; @7b b793 39
SHR-15	User Story	Farmers / Entrepreneurs	As a user, I want the system to be easy to use without complex configuration so that it can be adopted by non-technical operators.	System setup can be completed within <10 minutes by a new user.	Could Have	Backlog	 DRS -16 - Jira serv er retur ned an error : [Ljav a. lang. Obje ct; @10 b07b 92

SHR-16	User Story	Farmers / Entrepreneurs	As an operator, I want to send the vehicle to specific GPS coordinates so that it can perform transport tasks anywhere in the working area.	The vehicle reaches given coordinates with acceptable accuracy.	Must Have	Backlog	 DRS -17 - Jira serv er retur ned an error : [Ljav a. lang. Obje ct; @5e 34d1 8c
SHR-17	User Story	Farmers / Entrepreneurs	As a user, I want the vehicle to automatically return to its starting point so that transport cycles can be completed without manual retrieval.	The vehicle autonomously returns after completing tasks.	Must Have	Backlog	 DRS -18 - Jira serv er retur ned an error : [Ljav a. lang. Obje ct; @4d aee6 d8

SHR-18	Job Story	Farmers	When workers are harvesting fruits or vegetables, I want the vehicle to move forward incrementally so that pickers can continue working without manually repositioning transport equipment.	The vehicle moves forward in controlled increments on command.	Should Have	Backlog	 DRS -19 - Jira serv er retur ned an error : [Ljav a. lang. Obje ct; @74 52b4 81
SHR-19	User Story	Farmers / Entrepreneurs	As a user, I want the vehicle to be controlled using a simple interface such as a button or mobile command so that non-technical workers can operate it easily.	Users can operate the system with minimal training.	Must Have	Backlog	
SHR-20	Job Story	Farmers / Entrepreneurs	When the vehicle encounters an obstacle and cannot continue moving, I want the system to notify the operator so that the issue can be resolved quickly.	The system sends alerts after a defined blocking time.	Should Have	Backlog	
SHR-21	User Story	Farmers / Entrepreneurs	As an operator, I want to remotely control the vehicle through a dashboard or mobile interface so that I can resolve unexpected situations when autonomous operation is interrupted.	Remote control works with minimal latency.	Should Have	Backlog	
SHR-22	User Story	Farmers / Entrepreneurs	As a system operator, I want a dashboard that displays vehicle location and operational data so that I can monitor its activity in real time.	Dashboard reflects accurate real-time data.	Should Have	Backlog	
SHR-23	User Story	Farmers / Entrepreneurs	As an operator, I want the vehicle to record manually driven routes so that it can repeat the same routes autonomously in the future.	Routes can be recorded and replayed successfully.	Should Have	Backlog	
SHR-24	Job Story	Farmers / Entrepreneurs	When multiple vehicles are deployed in the same environment, I want each vehicle to send operational data to a central system so that fleet activity can be monitored.	Multiple vehicles transmit data to a shared system.	Could Have	Backlog	
SHR-25	User Story	Farmers / Entrepreneurs	As a worker in the operational area, I want the vehicle to produce sound or light signals while moving so that people nearby are aware of its activity.	Vehicle emits signals during movement.	Must Have	Backlog	
SHR-26	Job Story	Farmers / Entrepreneurs	When the system is deployed in a controlled environment, I want it to follow predefined scenarios rather than complex autonomous navigation so that the system remains reliable and predictable.	Predefined scenarios are executed consistently.	Must Have	Backlog	
SHR-27	User Story	Farmers / Entrepreneurs	As an operator, I want to access video from onboard cameras so that I can inspect the vehicle surroundings when a problem occurs.	Video feed is accessible when requested.	Could Have	Backlog	
SHR-28	Job Story	Technical Team	When validating the system before deployment, I want the vehicle to complete a predefined autonomous route so that the functionality of navigation can be confirmed.	Validation route is completed successfully without intervention.	Must Have	Backlog	

02-AGC WP5.2 Develop System Requirements

ID	Requirement Description	Requirement Type	Priority	Epic	Source (Stakeholder Requirement)	DRS-ID	Status
SR-01	The system shall autonomously transport materials between predefined locations without human intervention.	Functional	Must Have	Transport Automation	R1, R12, R3, R5	 DRS-2 - Jira server returned an error: [Ljava.lang.Object;@15e78032	Backlog
SR-02	The system shall support automated logistics operations across large working areas.	Functional	Should Have	Logistics Efficiency	R13	 DRS-4 - Jira server returned an error: [Ljava.lang.Object;@10f951c	Backlog
SR-03	The system shall follow predefined routes for navigation.	Functional	Must Have	Navigation and Routing	R4	 DRS-6 - Jira server returned an error: [Ljava.lang.Object;@74bb577f	Backlog
SR-04	The system shall navigate to specified GPS coordinates.	Functional	Must Have	Navigation and Routing	R16	 DRS-13 - Jira server returned an error: [Ljava.lang.Object;@2163b6ce	Backlog
SR-05	The system shall automatically return to a predefined home location after task completion.	Functional	Must Have	Navigation and Routing	R17	 DRS-3 - Jira server returned an error: [Ljava.lang.Object;@79901674	Backlog
SR-06	The system shall record and reuse manually defined routes for autonomous navigation.	Functional	Should Have	Navigation and Routing	R23	 DRS-9 - Jira server returned an error: [Ljava.lang.Object;@a336854	Backlog

SR-07	The system shall execute predefined navigation scenarios in controlled environments.	Functional	Must Have	Navigation and Routing	R26	 DRS-5 - Jira server returned an error: [Ljava.lang.Object;@51dd5e8f	Backlog
SR-08	The system shall detect obstacles in real time using onboard sensors.	Functional	Must Have	Safety	R6	 DRS-8 - Jira server returned an error: [Ljava.lang.Object;@7d34367	Backlog
SR-09	The system shall stop or avoid movement when obstacles are detected within a safety threshold.	Functional	Must Have	Safety	R6, R8	 DRS-11 - Jira server returned an error: [Ljava.lang.Object;@1f0521e2	Backlog
SR-10	The system shall include an emergency stop mechanism that halts all operations immediately.	Functional	Must Have	Safety	R7	 DRS-7 - Jira server returned an error: [Ljava.lang.Object;@2c963a02	Backlog
SR-11	The system shall emit visual or audio signals while moving to ensure safety awareness.	Functional	Must Have	Safety	R25	 DRS-12 - Jira server returned an error: [Ljava.lang.Object;@5326639f	Backlog
SR-12	The system shall provide real-time monitoring of vehicle status, including location and battery level.	Functional	Should Have	Monitoring and Control	R14	 DRS-14 - Jira server returned an error: [Ljava.lang.Object;@5c721db5	Backlog

SR-13	The system shall provide a remote monitoring interface accessible via web or mobile devices.	Functional	Should Have	Monitoring and Control	R2	 DRS-15 - Jira server returned an error: [Ljava.lang.Object;@e84bfc3	Backlog
SR-14	The system shall notify the operator when the vehicle is blocked or unable to proceed.	Functional	Should Have	Monitoring and Control	R20	 DRS-16 - Jira server returned an error: [Ljava.lang.Object;@1e160987	Backlog
SR-15	The system shall allow remote manual control of the vehicle.	Functional	Should Have	Monitoring and Control	R21	 DRS-17 - Jira server returned an error: [Ljava.lang.Object;@1f726108	Backlog
SR-16	The system shall provide a dashboard displaying real-time operational data.	Functional	Should Have	Monitoring and Control	R22, R27	 DRS-18 - Jira server returned an error: [Ljava.lang.Object;@2fab22ee	Backlog
SR-17	The system shall support incremental movement control for harvesting scenarios.	Functional	Should Have	Logistics Efficiency	R18	 DRS-19 - Jira server returned an error: [Ljava.lang.Object;@6219a64e	Backlog
SR-18	The system shall support communication between multiple vehicles and a central system.	Functional	Could Have	Logistics Efficiency	R24	 DRS-24 - Jira server returned an error: [Ljava.lang.Object;@53a059d9	Backlog

SR-19	The system shall operate continuously for extended periods without critical failure.	Non-functional	Must Have	System Reliability	R9	 DRS-28 - Jira server returned an error: [Ljava.lang.Object;@4124a7f7	Backlog
SR-20	The system shall provide a simple and intuitive user interface for non-technical users.	Non-functional	Must Have	User Experience	R15, R19	 DRS-29 - Jira server returned an error: [Ljava.lang.Object;@7cbb0c23	Backlog

02-AGC WP5.3 Ensure requirements traceability

ID	Requirement Description	SR-ID	Type Classification	Requirement Type	Priority	DRS-ID
	Performance & Efficiency	 DRS-30 - Jira server returned an error: [Ljava. lang.Object; @2d2596ae				
SR-01	The system shall minimize operational expenses by optimizing hosting, licensing, maintenance, and automation strategies.	 DRS-31 - Jira server returned an error: [Ljava. lang.Object; @6cc5034c	Property MASTER	Non-functional	Could Have	 DR S-2 - Jira serv er retu rne d an erro r: [Lja va. lang . Obj ect; @2 9b5 354 6

SR-02	The system shall achieve at least 50% of human worker performance with real-time monitoring.	 DRS-32 - Jira server returned an error: [Ljava. lang.Object; @7ad669f4	Property MASTER	Non-functional	Must Have	 DR S-4 - Jira serv er retu rne d an erro r: [Lja va. lang . Obj ect; @2 e96 7dd d
-------	--	---	--------------------	----------------	-----------	--

SR-03	The system shall automatically adjust processing speeds based on workload and product type.	 DRS-33 - Jira server returned an error: [Ljava.lang.Object;@31f26483	Functional MASTER	Functional	Should Have	 DRS-4 - Jira server returned an error: [Ljava.lang.Object;@1bf4d895
-------	---	---	-------------------	------------	-------------	--

SR-04	The system shall synchronize operation with conveyor belt speed such that it dynamically adjusts based on the conveyor.	 DRS-34 - Jira server returned an error: [Ljava. lang.Object; @5e66a9d8	Functional MASTER	Functional	Must Have	 DR S- 13 - Jira serv er retu rne d an erro r: [Lja va. lang . Obj ect; @1 70d 88b b
	Safety & Compliance	 DRS-35 - Jira server returned an error: [Ljava. lang.Object; @166fd0d				

SR-05	The system shall prioritize careful handling for fragile items over processing speed.	 DRS-36 - Jira server returned an error: [Ljava. lang.Object; @61608d92	Functional MASTER	Functional	Must Have	 DR S-3 - Jira serv er retu rne d an erro r: [Lja va. lang . Obj ect; @5 346 3cb 3
-------	---	---	----------------------	------------	-----------	--

SR-06	When a person enters a restricted area, the system shall halt operation and trigger an alarm.	 DRS-37 - Jira server returned an error: [Ljava.lang.Object;@4b17f7cc	Condition MASTER / Event MASTER	Functional	Should Have	 DR S-9 - Jira serv er retu rne d an erro r: [Lja va. lang . Obj ect; @5 6c7f 0e9
	Monitoring & Maintenance	 DRS-38 - Jira server returned an error: [Ljava.lang.Object;@6fcf0651				

SR-07	The system shall collect telemetry data and predict maintenance needs.	 DRS-39 - Jira server returned an error: [Ljava. lang.Object; @19d0b4ce	Functional MASTER	Functional	Should Have	 DR S-5 - Jira serv er retu rne d an erro r: [Lja va. lang . Obj ect; @2 b8c 7ff0
-------	--	---	----------------------	------------	----------------	---

SR-08	The system shall provide real-time video monitoring of sorting operations.	 DRS-40 - Jira server returned an error: [Ljava. lang.Object; @3e2cae87	Functional MASTER	Functional	Should Have	 DR S-8 - Jira serv er retu rne d an erro r: [Lja va. lang . Obj ect; @5 498 2cb 5
-------	--	---	----------------------	------------	----------------	--

SR-09	The system shall integrate with the conveyor belt for seamless pallet transfer.	 DRS-41 - Jira server returned an error: [Ljava. lang.Object; @312a23d2	Functional MASTER	Functional	Should Have	 DR S- 11 - Jira serv er retu rne d an erro r: [Lja va. lang . Obj ect; @2 7fa9 182
	Testing & Simulation & Environmental	 DRS-42 - Jira server returned an error: [Ljava. lang.Object; @6636f5fb				

SR-10	The system shall include a simulation module for pre-deployment performance validation.	 DRS-43 - Jira server returned an error: [Ljava. lang.Object; @66ae6fd9	Functional MASTER	Functional	Could Have	 DR S-7 - Jira serv er retu rne d an erro r: [Lja va. lang . Obj ect; @3 a4c 06b e
-------	---	---	----------------------	------------	---------------	--

SR-11	The system shall operate reliably in slightly wet environments.	 DRS-44 - Jira server returned an error: [Ljava. lang.Object; @4e3c95cc	Environment MASTER	Non-functional	Could Have	 DR S- 12 - Jira serv er retu rne d an erro r: [Lja va. lang . Obj ect; @3 0b8 439 9
	Development and Presentation	 DRS-45 - Jira server returned an error: [Ljava. lang.Object; @77ae63b5				

SR-12	The system shall include a local MVP that classifies and sorts objects (e.g., red/blue balls) before connecting to the cloud.	 DRS-46 - Jira server returned an error: [Ljava. lang.Object; @62eabde2	Functional MASTER	Functional	Must Have	 DR S- 14 - Jira serv er retu rne d an erro r: [Lja va. lang . Obj ect; @1 47c 087 0
-------	---	---	----------------------	------------	-----------	--

SR-13	When placing items into a box, the system shall use a grid-based algorithm to find the next available empty spot.	 DRS-47 - Jira server returned an error: [Ljava. lang.Object; @9ff412a	Functional MASTER	Functional	Must Have	 DR S- 15 - Jira serv er retu rne d an erro r: [Lja va. lang . Obj ect; @1 a54 c5d 3
	Cloud Integration	 DRS-50 - Jira server returned an error: [Ljava. lang.Object; @31f05291				

SR-14	The system shall connect to the cloud for remote monitoring and data aggregation from multiple robots.	 DRS-51 - Jira server returned an error: [Ljava. lang.Object; @3f06367	Functional MASTER	Functional	Should Have	 DR S- 16 - Jira serv er retu rne d an erro r: [Lja va. lang . Obj ect; @4 925 11f2
-------	--	--	----------------------	------------	----------------	---

SR-15	The system shall provide the option to send images to the cloud for analysis.	 DRS-53 - Jira server returned an error: [Ljava. lang.Object; @17203907	Functional MASTER	Functional	Could Have	 DR S- 17 - Jira serv er retu rne d an erro r: [Lja va. lang . Obj ect; @7 b98 174 6
-------	---	---	----------------------	------------	---------------	--

SR-16	The system shall allow operators to send basic start and stop commands from a web interface.	 DRS-54 - Jira server returned an error: [Ljava. lang.Object; @2b62ba51	Functional MASTER	Functional	Could Have	 DR S- 18 - Jira serv er retu rne d an erro r: [Lja va. lang . Obj ect; @6 bf1d c4b
-------	--	---	----------------------	------------	---------------	---

SR-17	The system shall include a remote manual mode for developers to send individual commands (e.g., "take picture", "go to coordinate").	 DRS-55 - Jira server returned an error: [Ljava. lang.Object; @76432abc	Functional MASTER	Functional	Should Have	 DR S- 19 - Jira serv er retu rne d an erro r: [Lja va. lang . Obj ect; @4 49ef c78
	Architecture	 DRS-56 - Jira server returned an error: [Ljava. lang.Object; @566ca5c1				

SR-18	For time-critical operations, all image analysis and robot control shall be performed locally on the robot's industrial PC.	 DRS-58 - Jira server returned an error: [Ljava.lang.Object;@4fb15120	Architecture MASTER	Non-functional	Must Have	 DRS-24 - Jira server returned an error: [Ljava.lang.Object;@1c9e7436
-------	---	---	---------------------	----------------	-----------	---

SR-19	The system workspace shall be physically isolated with a metal grid and emergency stop buttons.	 DRS-60 - Jira server returned an error: [Ljava. lang.Object; @7f84d80b	Passive Safety MASTER	Non-functional	Must Have	 DR S- 28 - Jira serv er retu rne d an erro r: [Lja va. lang . Obj ect; @1 ae4 83e
-------	---	---	--------------------------	----------------	-----------	--

SR-20	The system shall maintain a clear architectural distinction between physical safety and digital security.	 DRS-61 - Jira server returned an error: [Ljava. lang.Object; @35f737ac	Design Principle MASTER	Non-functional	Must Have	 DR S- 29 - Jira serv er retu rne d an erro r: [Lja va. lang . Obj ect; @1f a33 842
-------	---	---	----------------------------	----------------	-----------	---

02-AGC WP5.4 Review and Release System requirements

ID	System Requirement Jira Ticket	System Requirement	Actions	Assignee	Due Date
1.	 CDR-53 - Jira server returned an error: [Ljava.lang.Object;@2de6ca77	Performance & Efficiency  DRS-1 - Jira server returned an error: [Ljava.lang.Object;@15a0beff	Review all child requirements, track implementation progress	Vasile Ceban	2025-04-25
1.1	 CDR-54 - Jira server returned an error: [Ljava.lang.Object;@6fdb15f7	The system shall minimize operational expenses by optimizing hosting, licensing, maintenance, and automation strategies.	Evaluate current cost drivers and implement optimizations	Vasile Ceban	2025-05-10
1.2	 CDR-55 - Jira server returned an error: [Ljava.lang.Object;@1059411	Achieve at least 50% of human worker performance with real-time monitoring.	Define metrics, set up monitoring tools, simulate workloads	Vasile Ceban	2025-06-01
1.3	 CDR-56 - Jira server returned an error: [Ljava.lang.Object;@526f9742	Automatically adjust processing speeds based on workload and product type.	Develop control logic, test with various workloads	Vasile Ceban	2025-05-05
1.4	 CDR-57 - Jira server returned an error: [Ljava.lang.Object;@43ccfb3b	Synchronize operation with conveyor belt speed.	Integrate belt speed sensors, update system controller	vlad polisciuc	2025-06-01
2.	 CDR-58 - Jira server returned an error: [Ljava.lang.Object;@1ba3585	Safety & Compliance	Coordinate safety-related implementations	Arina Pereteatcu	2025-04-30
2.1	 CDR-59 - Jira server returned an error: [Ljava.lang.Object;@2c9c73b3	Prioritize careful handling for fragile items over speed.	Tag fragile items in DB, update handling algorithm	Arina Pereteatcu	2025-06-01

2.2	 CDR-60 - Jira server returned an error: [Ljava.lang.Object;@12ee247f	Halt operation and trigger an alarm when a person enters a restricted area.	Set up detection zones, integrate with control logic	Arina Pereteatcu	2025-05-15
3.	 CDR-61 - Jira server returned an error: [Ljava.lang.Object;@67c62359	Monitoring & Maintenance	Centralize telemetry and diagnostics tasks	Mihai Vieru	2025-05-01
3.1	 CDR-62 - Jira server returned an error: [Ljava.lang.Object;@c2178d9	Collect telemetry data and predict maintenance needs.	Implement data logging, build predictive model	Mihai Vieru	2025-05-20
3.2	 CDR-63 - Jira server returned an error: [Ljava.lang.Object;@3b01470b	Provide real-time video monitoring of sorting operations.	Install cameras, stream feed to control dashboard	Mihai Vieru	2025-05-12
4.	 CDR-64 - Jira server returned an error: [Ljava.lang.Object;@1d9fe07a	Automation & Integration	Ensure integration dependencies across systems	Arina Pereteatcu	2025-05-01
4.1	 CDR-65 - Jira server returned an error: [Ljava.lang.Object;@4ad32dde	Integrate with the conveyor belt for seamless pallet transfer.	Link control systems, validate timing and load distribution	Mihai Vieru	2025-05-22
5.	 CDR-67 - Jira server returned an error: [Ljava.lang.Object;@4175e6cf	Testing & Simulation	Oversee simulation modules for performance validation	vlad polisciuc	2025-05-08
5.1	 CDR-69 - Jira server returned an error: [Ljava.lang.Object;@6e2c7f5e	Include a simulation module for pre-deployment performance validation.	Design simulation logic, replicate realistic scenarios	vlad polisciuc	2025-05-25

6.	 CDR-68 - Jira server returned an error: [Ljava. lang.Object;@66a06c2a	Environmental Resilience	Oversee all environmental robustness tasks	vlad polisciuc	2025-05-01
6.1	 CDR-70 - Jira server returned an error: [Ljava. lang.Object;@306217df	Operate reliably in slightly wet environments.	Waterproof components, test in humid/wet settings	vlad polisciuc	2025-05-18

<https://youtube.com/shorts/ZU4WqCVVSfw?si=PiTvVXAWds5F9xvk>

02-AGC WP6 System Architecture

Work Product Status

Legend	NEW - Backlog/Planned PROGRESS - Started DONE - Finished
Static aspects of the system architecture	 PBL26-617 - Jira server returned an error: [Ljava.lang.Object; @2b799952  PBL26-677 - Jira server returned an error: [Ljava.lang.Object; @4a32b7e1  PBL26-678 - Jira server returned an error: [Ljava.lang.Object; @1bfd5ca  PBL26-679 - Jira server returned an error: [Ljava.lang.Object; @70d74543  PBL26-680 - Jira server returned an error: [Ljava.lang.Object; @3c4ba270

Dynamic aspects of the system architecture

 [PBL26-618](#) - Jira server returned an error: [Ljava.lang.Object;
@363b8740

 [PBL26-681](#) - Jira server returned an error: [Ljava.lang.Object;
@7191def9

 [PBL26-682](#) - Jira server returned an error: [Ljava.lang.Object;
@5c6691e

 [PBL26-683](#) - Jira server returned an error: [Ljava.lang.Object;
@21c5a643

 [PBL26-684](#) - Jira server returned an error: [Ljava.lang.Object;
@14f803d7

System architecture analyzis

 [PBL26-619](#) - Jira server returned an error: [Ljava.lang.Object;
@18fba42

 [PBL26-685](#) - Jira server returned an error: [Ljava.lang.Object;
@27885654

 [PBL26-686](#) - Jira server returned an error: [Ljava.lang.Object;
@42e4d7ae

 [PBL26-687](#) - Jira server returned an error: [Ljava.lang.Object;
@2f080833

 [PBL26-688](#) - Jira server returned an error: [Ljava.lang.Object;
@54aa3037

Ensure consistency and establish bidirectional traceability

 [PBL26-620](#) - Jira server returned an error: [Ljava.lang.Object;
@16a66a39

 [PBL26-689](#) - Jira server returned an error: [Ljava.lang.Object;
@e6dd8f

 [PBL26-690](#) - Jira server returned an error: [Ljava.lang.Object;
@912dfba

 [PBL26-691](#) - Jira server returned an error: [Ljava.lang.Object;
@205c1b2f

 [PBL26-692](#) - Jira server returned an error: [Ljava.lang.Object;
@1a15d03e

Review and Communicate agreed system architecture

 [PBL26-621](#) - Jira server returned an error: [Ljava.lang.Object;
@77eab72f

 [PBL26-693](#) - Jira server returned an error: [Ljava.lang.Object;
@2fd8c9d

 [PBL26-694](#) - Jira server returned an error: [Ljava.lang.Object;
@24aa0438

 [PBL26-695](#) - Jira server returned an error: [Ljava.lang.Object;
@1f1dfb34

 [PBL26-696](#) - Jira server returned an error: [Ljava.lang.Object;
@7cb47177

02-AGC WP6.1 Static aspects of the system architecture specification

Architectural Design

Key System Layers

We organize your components into **3 logical layers**:

1. User Layer

- Operator
- Web Dashboard (UI)

2. Cloud Layer

- Cloud Backend
- Database (Operational DB)

3. Edge / Vehicle Layer (IMPORTANT)

- Mini PC (Main Controller)
- Arduino (Low-level control)
- GPS / RTK Module
- Parktronic Sensors (Ultrasonic)
- Camera (Image Collector)
- Steering & Motor System
- (Optional) Human Interaction Layer

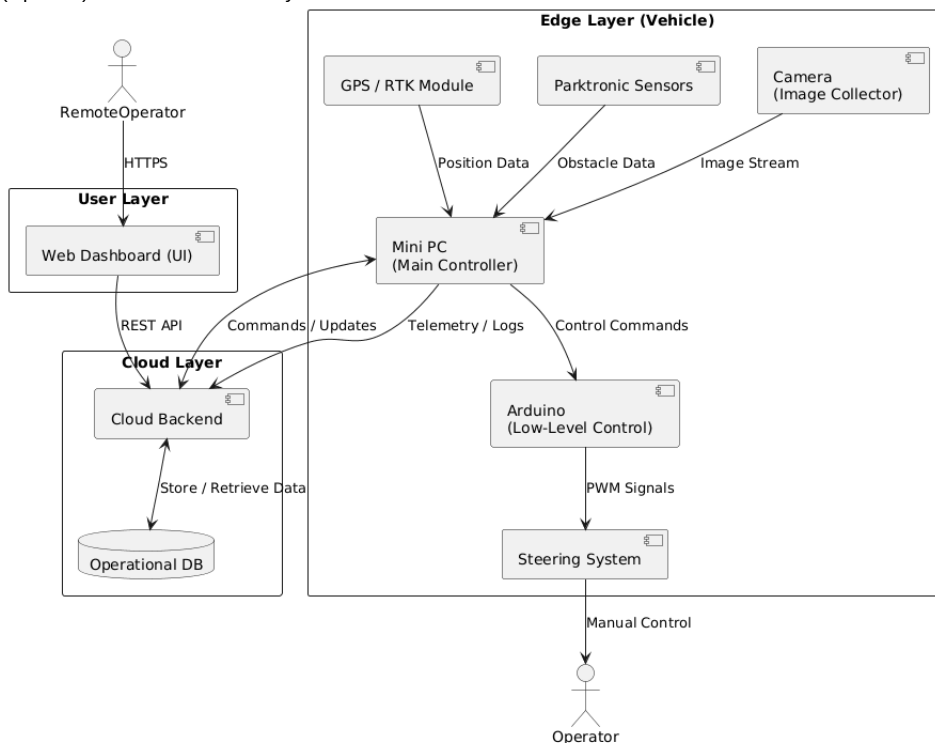


Figure 1: Main Components Diagram

Component Details and Interfaces

Mini PC System

The **Mini PC** represents the **central processing unit of the entire system** and acts as the main decision-making component. It is responsible for receiving data from all sensors, processing this information in real time, and generating the commands required to control the vehicle. In other words, it functions as the “brain” of the autonomous golf cart.

The MiniPC class coordinates the overall system operation. It initializes the system, processes incoming sensor data, executes navigation logic, and communicates with both the Arduino controller and the cloud backend. By placing the main intelligence in the Mini PC instead of the cloud, the system avoids latency issues and can react immediately to environmental changes such as obstacles or route deviations.

The internal structure of the Mini PC is divided into several functional modules. The NavigationEngine class is responsible for route planning and route execution. It contains functions such as planRoute() and followRoute() that allow the vehicle to move autonomously between predefined locations. The function returnToHome() ensures that the vehicle can automatically return to its starting position after completing a task.

The SensorFusion class plays a very important role in the system because it combines information coming from different sensors. Instead of treating GPS data, ultrasonic sensor data, and camera input separately, this module processes them together using functions such as mergeSensorData(). This allows the system to make more accurate decisions and increases the reliability of obstacle detection and navigation.

The DecisionEngine class is responsible for interpreting the processed sensor data and converting it into actions. For example, the function detectObstacle() determines whether the vehicle can continue moving safely, while calculateMovement() decides how the vehicle should move in the current situation. If a dangerous situation occurs, the function triggerEmergencyStop() immediately interrupts the navigation process.

Finally, the CommunicationManager class ensures the interaction between the Mini PC and the cloud system. It sends telemetry data such as position and battery level using sendTelemetry() and receives commands from the operator through receiveCommands(). This module makes remote monitoring and control possible without affecting the real-time performance of the vehicle.

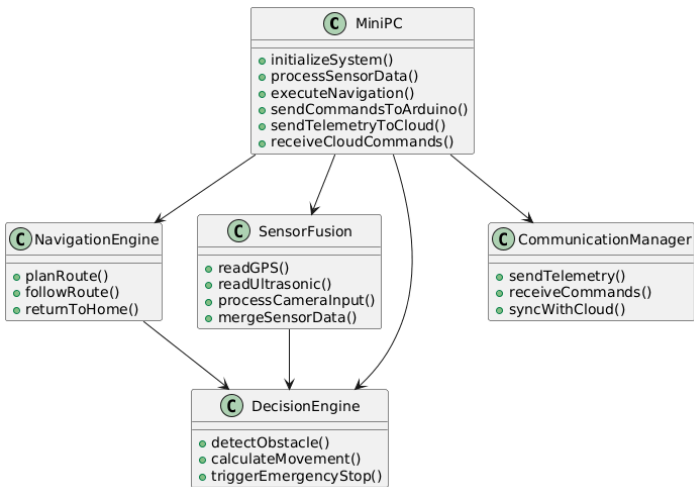


Figure 2: Mini PC Component Diagram

Arduino (Low-Level Controller) System

The Arduino controller represents the **low-level control layer** of the system. While the Mini PC makes high-level decisions, the Arduino is responsible for executing these decisions in a stable and predictable way. This separation between high-level and low-level control is very common in autonomous and robotic systems because it improves reliability and safety.

The ArduinoController class receives commands from the Mini PC using the function receiveCommand() and executes them using executeMovement(). It also includes the function emergencyStop(), which allows the system to stop the vehicle immediately if a dangerous situation is detected. This function is especially important in agricultural environments where people and machinery operate in the same space.

Inside the Arduino controller, the CommandParser class converts high-level commands into specific control signals. For example, a command such as “move forward” must be translated into a combination of speed and direction values. The function parseCommand() performs this translation, while validateCommand() ensures that the received command is correct before it is executed.

The MotorController class is responsible for the physical movement of the vehicle. It controls the speed and direction of the motors using functions such as setSpeed() and setDirection(). The function stopMotor() is used when the vehicle needs to stop safely or when an obstacle is detected.

The SafetyModule class ensures that the system operates safely even in unexpected situations. It continuously monitors the signals received from the Mini PC and triggers the triggerEmergencyStop() function if a problem is detected. This makes the Arduino controller a critical safety component of the entire system.

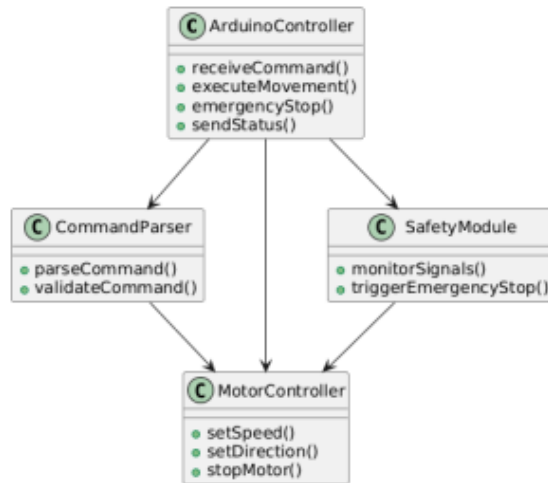


Figure 2: Arduino Component Diagram

Web Dashboard (UI) System

The Web Dashboard represents the **main interaction point between the user and the system**. It allows the operator to monitor the vehicle in real time and to send commands without directly interacting with the hardware. This is especially important in agricultural environments where the operator may need to supervise several processes at the same time.

The WebDashboard class coordinates the entire user interface. It provides functions such as `displayDashboard()`, `showVehicleStatus()`, `sendCommand()`, and `showAlerts()`. These functions allow the operator to observe the system state and react quickly if a problem occurs.

The DashboardView class is responsible for presenting the system information in a clear and understandable way. It includes functions such as `renderMetrics()` and `renderMap()`, which allow the user to see the vehicle position, speed, and operational status. The function `updateStatus()` ensures that the information displayed on the screen is updated in real time.

The ControlPanel class provides the functionality required to send commands to the vehicle. For example, the operator can start or stop the vehicle using the function `sendMoveCommand()` or activate manual control using `activateManualControl()`.

The AlertManager class is used to inform the operator about important events. Functions such as `displayAlert()` and `notifyUser()` are used when the vehicle detects an obstacle or when communication with the system is interrupted.

Finally, the APIClient class connects the Web Dashboard to the cloud backend. It sends requests using `sendRequest()` and receives updated information using `getUpdates()`. This component ensures that the user interface remains synchronized with the actual state of the vehicle.

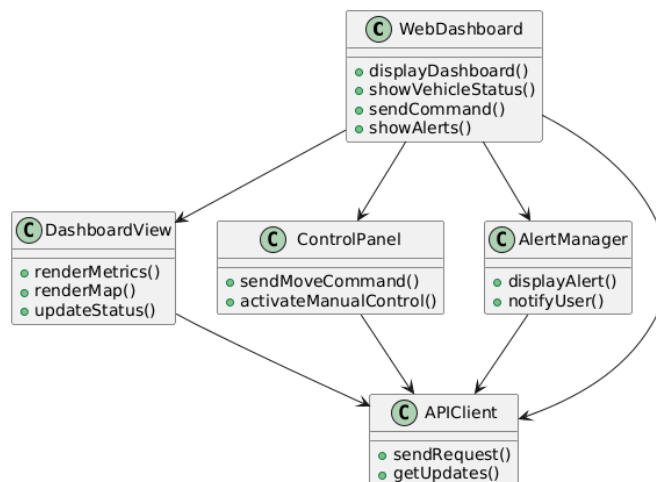


Figure 3: Web Dashboard Component Diagram

Cloud Backend System

The Cloud Backend represents the **data management and communication layer** of the system. It connects the Web Dashboard with the Mini PC and provides a centralized environment where system data can be stored and processed.

The CloudBackend class is responsible for handling user requests and managing system data. It includes functions such as processRequest() and storeTelemetry(), which allow the system to save information about the vehicle's position, activity, and performance. The function sendCommandToVehicle() allows the operator to control the vehicle remotely through the web interface.

The APIGateway class acts as an intermediary between the user interface and the backend services. It receives requests from the Web Dashboard and verifies them using the function validateUser(). The function handleRequest() ensures that each request is processed correctly and securely.

The DataManager class is responsible for storing and retrieving information from the database. It includes functions such as storeData() and retrieveData(), which allow the system to maintain a history of vehicle operations. The function saveLogs() is especially important for performance analysis and system testing.

The CommandDispatcher class ensures that commands received from the operator are transmitted to the Mini PC. Functions such as sendCommand() and queueCommand() are used to guarantee that the vehicle receives the correct instructions even when multiple commands are sent at the same time.

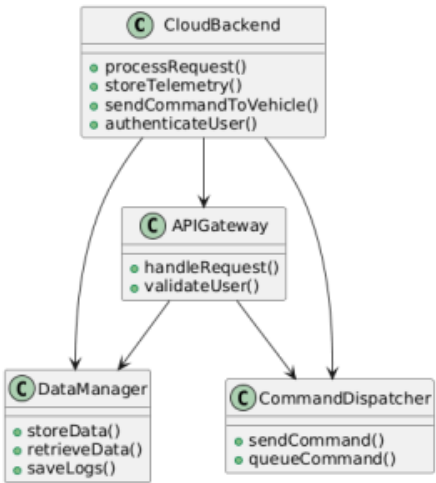


Figure 4: Cloud Backend Component Diagram

Sensor System

The Sensor System represents the **perception layer** of the autonomous vehicle. Its main purpose is to collect environmental data and transmit it to the Mini PC, where it can be analyzed and used for navigation and safety decisions.

The SensorSystem class coordinates the entire sensing process. It collects information from multiple sensors using the function collectData() and sends this information to the Mini PC using sendSensorData().

The GPSModule class provides the position of the vehicle. The function getCoordinates() returns the current geographic location, while getAccuracy() indicates the precision of the measurement. This information is essential for autonomous navigation and route tracking.

The UltrasonicSensors class is responsible for detecting obstacles. The function measureDistance() determines how far an object is from the vehicle, and detectObstacle() identifies whether the vehicle should stop or change direction.

The CameraModule class captures visual information about the environment. Functions such as captureImage() and streamVideo() allow the system to provide visual monitoring and support advanced features such as image recognition.

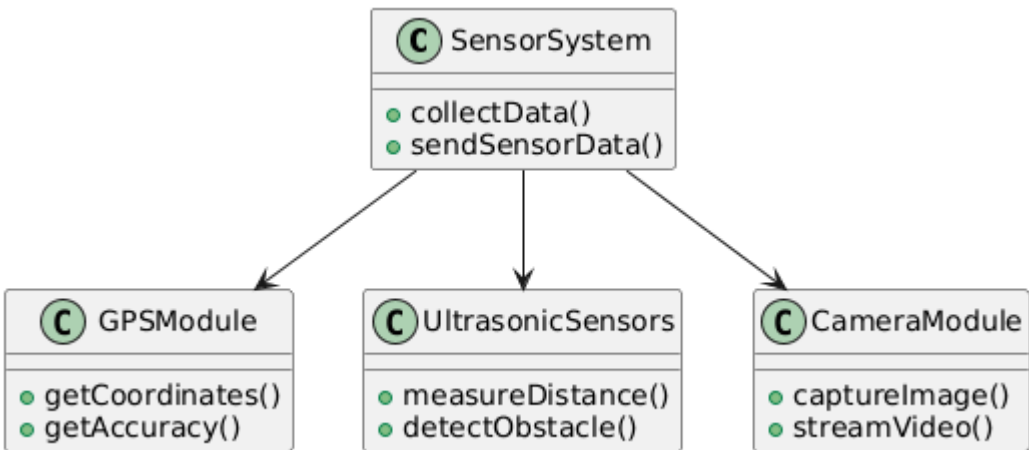


Figure 5: Sensor Component Diagram

Steering System

The Steering System represents the **mechanical actuation layer** of the vehicle. It converts electrical signals received from the Arduino controller into physical movement.

The SteeringSystem class defines the main movement functions of the vehicle, including turnLeft(), turnRight(), and stopSteering(). These functions allow the vehicle to follow a predefined route and avoid obstacles.

The MotorDriver class is responsible for generating the electrical signals required to control the steering mechanism. The function applyPWM() adjusts the motor speed, while controlDirection() determines the movement direction.

The SteeringMechanism class represents the physical structure that moves the wheels. The function rotateWheels() converts the electrical signals into mechanical motion.

The ManualControl class allows the operator to take control of the vehicle when necessary. Functions such as takeControl() and releaseControl() make it possible to switch between autonomous and manual operation safely.

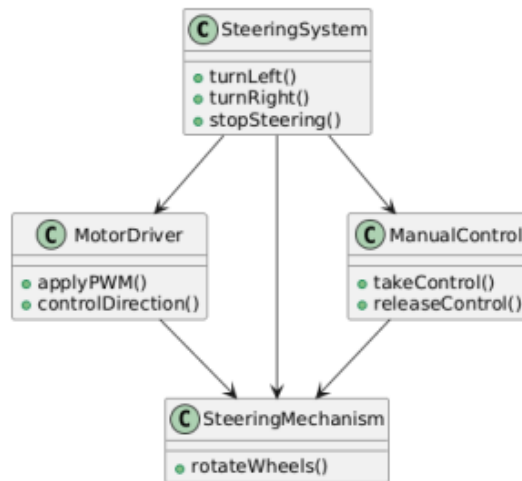


Figure 6: Steering Component Diagram

02-AGC WP6.2 Dynamic Aspects of the System Architecture Specification

The autonomous yard vehicle system operates in multiple dynamic modes to ensure safe operation, autonomous execution, and operator control while maintaining system integrity.

Mode	Purpose	Key Characteristics
Initialization	System startup and validation	Safety checks, sensor calibration, cloud connection
Autonomous Operation	Execution of transport tasks	Continuous navigation, telemetry logging
Manual Control	Operator-driven movement	Incremental movement, direct control
Safety Stop	Emergency handling	Immediate stop, system lock
Monitoring	System supervision	Movement tracking, alert generation

1. Core Processing System (Mini PC + Arduino)

The Core Processing System coordinates all subsystems of the autonomous yard vehicle. The Mini PC manages high-level logic and communication, while the Arduino executes real-time actuator control.

Architectural Element: Activity Diagram – System Initialization

Architectural Element	Description
-----------------------	-------------

Activity Diagram	System Initialization <pre>graph TD; Start(()) --> A1[Operator triggers Start System]; A1 --> A2[Web Dashboard sends Initialization Request]; A2 --> A3[Cloud Backend signals Mini PC to Start]; A3 --> subgraph Safety_Initialization [Safety Initialization]; A4[Mini PC initializes safety systems via Arduino] --> A5[Arduino performs Safety Check]; A5 --> A6{Safety OK?}; A6 -- yes --> A7[Safety Ready]; A6 -- no --> A8[Report Safety Error]; A8 --> End1((()); end; A7 --> subgraph Sensor_Navigation [Sensor & Navigation]; A9[Initialize and calibrate Sensors] --> A10[Load routes and get GPS position]; A10 --> A11[Calibration complete]; end; A11 --> subgraph Connectivity_Data [Connectivity & Data]; A12[Establish Cloud connection] --> A13[Initialize Data Services & Logging]; A13 --> A14[Sync with Operational DB]; end; A14 --> A15[Initialization Complete]; A15 --> A16[Web Dashboard displays Standby Mode]; A16 --> End2((());</pre> The flowchart illustrates the system initialization process. It begins with an operator triggering the start system, followed by a request from the web dashboard to the cloud backend, which then signals the mini PC to start. The process then enters a 'Safety Initialization' phase where the mini PC initializes safety systems via Arduino, and the Arduino performs a safety check. If the safety check is successful (Safety OK?), the system becomes 'Safety Ready'. If not, a 'Report Safety Error' is generated, leading to a failure state. Following safety initialization, the 'Sensor & Navigation' phase involves initializing and calibrating sensors, loading routes, and getting the GPS position. Once calibration is complete, the 'Connectivity & Data' phase involves establishing a cloud connection, initializing data services and logging, and syncing with the operational database. After these steps, the initialization is complete, and the web dashboard displays the standby mode, leading to the final end state.
Description	This diagram illustrates the initialization workflow of the autonomous yard vehicle. The operator triggers system startup through the dashboard, followed by coordinated initialization of safety systems, sensors, and communication modules. The system transitions to standby mode only after all subsystems are validated.

- Critical Interactions**
- Safety validation must complete before actuator enablement
 - Sensor calibration must complete before navigation
 - Cloud failure must not block startup

- Connection to Requirements**
- Supports safe startup and validation (SR-08)
 - Ensures subsystem readiness (SR-10)

Architectural Element: Activity Diagram – Autonomous Transport Execution

Architectural Element	Description
-----------------------	-------------

Activity Diagram	Autonomous Transport Execution <pre>graph TD; Start(()) --> A[Operator sends transport command]; A --> B[Cloud Backend sends task to Mini PC]; B --> C{ }; C --> D[Mini PC sends control commands to Arduino]; D --> E[Arduino sends PWM signals to Steering System]; E --> F[Arduino receives status feedback]; F --> G[Mini PC logs telemetry and execution status]; G --> H[Cloud Backend updates Web Dashboard]; H --> I{Destination Reached?}; I -- no --> J[Continue Route]; J --> C; I -- yes --> End((()));</pre>
Description	This diagram represents the autonomous transport execution process. The operator sends a command via the dashboard, which is transmitted through the cloud backend to the Mini PC. The Mini PC issues control commands to the Arduino, which executes movement. The process continues until the destination is reached.

Critical Interactions

- Continuous feedback loop between Arduino and Mini PC
- Real-time telemetry updates to cloud backend
- Destination validation controls loop termination

Connection to Requirements

- Supports autonomous task execution (SR-05)
- Ensures synchronization between control layers (SR-10)

Architectural Element: Activity Diagram – Incremental Movement

Architectural Element	Description
Activity Diagram	Incremental Movement <pre>graph TD; Start(()) --> A[Worker presses forward button]; A --> B[Control Interface sends Increment command]; B --> C[Mini PC commands small distance move]; C --> D[Arduino executes movement]; D --> E[Steering System completes step]; E --> F[Arduino reports "Movement Complete"]; F --> End((()));</pre>
Description	This diagram shows manual incremental movement control. A worker triggers a movement command, which is processed by the Mini PC and executed by the Arduino. The system confirms completion after each movement step.

Critical Interactions

- Immediate command-response cycle
- Accurate movement completion feedback

Connection to Requirements

- Supports manual override control

- Enables precise positioning operations

2. Safety and Security System

The Safety System ensures operator protection and system integrity through continuous monitoring, hazard detection, and rapid response.

Architectural Element: Activity Diagram – Emergency Stop

Architectural Element	Description
Activity Diagram	Emergency Stop <pre>graph TD; Start(()) --> A[Operator presses Emergency Button (Physical/UI)]; A --> B[Emergency Signal sent to Mini PC]; B --> C[Mini PC sends Immediate Stop command]; C --> D[Arduino cuts motor signal to Steering System]; D --> E[System enters Locked/Safety state]; E --> End((()));</pre>
Description	This diagram illustrates the emergency stop process. When triggered by the operator, the emergency signal is sent to the Mini PC, which immediately issues a stop command to the Arduino. The system transitions into a locked safety state.

Critical Interactions

- Emergency signal propagation must be immediate
- Motor shutdown must occur without delay

Connection to Requirements

- Implements emergency response (SR-08)
- Ensures fail-safe operation

Architectural Element: Activity Diagram – Obstacle Detection & Avoidance

Architectural Element	Description
-----------------------	-------------

Activity Diagram	Obstacle Detection & Avoidance <pre>graph TD; Start(()) --> A[Sensors send obstacle data to Mini PC]; A --> B[Mini PC & Safety Module analyze data]; B --> C{Obstacle Detected?}; C -- yes --> D[Decide to Stop or Change Direction]; D --> E[Mini PC sends command to Arduino]; E --> F[Arduino applies Stop/Turn to Steering System]; C -- no --> G[Mini PC sends "Continue" command]; G --> H[Arduino maintains motion]; F --> I{ }; H --> I; I --> End((()));</pre>
Description	This diagram illustrates how obstacle data from sensors is processed by the Mini PC. If an obstacle is detected, the system decides whether to stop or change direction. Otherwise, normal operation continues.

Critical Decision Points

- Obstacle detection condition
- Decision between stopping or avoidance

Critical Interactions

- Real-time sensor processing
- Immediate control command execution

Connection to Requirements

- Supports safe navigation (SR-05)
- Prevents collisions (SR-08)

Architectural Element: Activity Diagram – Blocked Vehicle Notification

Architectural Element	Description
Activity Diagram	Blocked Vehicle Notification <pre>graph TD; Start(()) --> A[Mini PC monitors movement]; A --> B{Movement detected?}; B -- yes --> C[Continue Monitoring]; B -- no --> D{Timeout reached?}; D -- yes --> E[Mini PC sends Alert to Cloud Backend]; E --> F[Cloud triggers Notification Service]; F --> G[Operator receives Alert Message]; D -- no --> H[Wait]; C --> I{ }; G --> I; H --> I; I --> End((()));</pre>
Description	This diagram shows how the system detects a blocked vehicle condition. The Mini PC monitors movement, and if no movement is detected within a defined timeout, an alert is sent to the cloud backend, triggering a notification to the operator.

Critical Decision Points

- Movement detection vs timeout threshold

Critical Interactions

- Communication between Mini PC and cloud backend
- Notification delivery to operator

Connection to Requirements

- Supports monitoring and fault detection
- Improves system responsiveness

3. System Coordination & Monitoring

Architectural Element: Sequence Diagram – System Initialization Coordination

Architectural Element	Description
Sequence Diagram	<pre>sequenceDiagram actor Operator participant WD as Web Dashboard participant CB as Cloud Backend participant MPC as Mini PC participant S as Sensors (GPS, Camera, Obstacle) participant A as Arduino participant SS as Steering System participant DB as Operational DB Note over MPC: System Start Operator->>WD: Start System WD->>CB: Initialization Request CB->>MPC: Start Initialization Note over MPC: Safety Initialization MPC->>A: Initialize safety systems A->>SS: Safety check SS-->>A: Safety OK A-->>MPC: Safety Ready Note over MPC: Sensor Initialization MPC->>S: Initialize sensors S->>S: Calibrate sensors S-->>MPC: Sensors Ready Note over MPC: Control Initialization MPC->>A: Initialize control system A->>SS: System check SS-->>A: OK A-->>MPC: Control Ready Note over MPC: Navigation Initialization MPC->>S: Load routes S->>MPC: Get GPS position S-->>MPC: Position data S->>MPC: Calibration complete Note over MPC: Communication Initialization MPC->>CB: Establish connection CB-->>MPC: Connection OK Note over MPC: Data Initialization MPC->>DB: Initialize data services DB->>DB: Setup logging DB-->>MPC: Ready DB->>MPC: Data Ready Note over MPC: System Ready MPC->>CB: Initialization complete CB->>WD: System Ready WD->>Operator: Display Standby Mode</pre>
Description	The Operator represents the user who initiates the system startup. The Web Dashboard is the user interface used to send the initialization request and display system status. The Cloud Backend manages communication between the interface and the vehicle, coordinating initialization steps and data flow. The Mini PC acts as the main onboard controller, orchestrating the initialization of all subsystems. The Sensors (GPS, Camera, Obstacle) provide environmental perception and positioning capabilities. The Arduino handles low-level control and communication with hardware components. The Steering System represents the physical actuation layer used for movement validation and control checks. The Operational DB stores system data, logs, and initialization information. The process begins when the Operator starts the system through the Web Dashboard, which sends an initialization request to the Cloud Backend. The Cloud Backend forwards the request to the Mini PC, triggering the system initialization sequence. First, safety systems are initialized and verified through the Arduino and Steering System to ensure safe operation. Next, the sensors are initialized and calibrated, enabling accurate perception and positioning. The control system is then initialized, confirming that hardware components respond correctly. After that, navigation is configured by loading routes and obtaining GPS position data. The system then establishes communication with the Cloud Backend and initializes data services, including logging to the Operational DB. Once all subsystems report readiness, the Mini PC signals completion to the Cloud Backend, which updates the Web Dashboard. Finally, the Operator is informed that the system is ready and enters standby mode, prepared for operation.

Critical Interactions

- Synchronized startup across subsystems
- Dependency between safety and control systems

Connection to Requirements

- Ensures system readiness before operation

- Supports diagnostics and configuration

Architectural Element: Sequence Diagram – Autonomous Transport Execution

Architectural Element	Description
Sequence Diagram	<pre>sequenceDiagram actor Operator participant WD as Web Dashboard participant CB as Cloud Backend participant MPC as Mini PC participant AR as Arduino participant SS as Steering System Operator->>WD: Send transport command WD->>CB: REST API (task request) CB->>MPC: Commands / Updates MPC->>AR: Control commands AR->>SS: PWM signals SS-->>AR: Status feedback AR-->>MPC: Execution status MPC->>CB: Telemetry / Logs CB-->>WD: Status update WD-->>Operator: </pre>
Description	The Operator represents the human user who initiates transport tasks. The Web Dashboard is the user interface through which commands are sent and system status is visualized. The Cloud Backend acts as an intermediary that processes requests, manages communication, and stores operational data. The Mini PC is the main controller located on the vehicle, responsible for executing high-level logic and coordinating subsystems. The Arduino handles low-level control by converting commands into hardware actions. The Steering System represents the physical actuation layer that executes movement through PWM signals and provides feedback on execution status. The process begins when the Operator sends a transport command through the Web Dashboard. This request is transmitted to the Cloud Backend via a REST API, which then forwards the command to the Mini PC as control instructions. The Mini PC processes the request and sends control commands to the Arduino, which translates them into PWM signals for the Steering System to perform the required movement. The Steering System executes the action and returns status feedback to the Arduino, which propagates execution status back to the Mini PC. The Mini PC then sends telemetry and logs to the Cloud Backend, allowing the system to track performance and state. Finally, the Cloud Backend updates the Web Dashboard, providing the Operator with real-time status information about the transport operation.

Connection to Requirements: SR-01, SR-05, SR-07

Architectural Element: Sequence Diagram – Obstacle Detection & Avoidance

Architectural Element	Description
-----------------------	-------------

Sequence Diagram	<pre> sequenceDiagram participant Sensors participant Mini PC participant Safety Module participant Arduino participant Steering System Sensors->>Mini PC: Obstacle data Mini PC->>Safety Module: Analyze data alt [Obstacle detected] Safety Module->>Mini PC: Stop/Avoid decision Mini PC->>Arduino: Stop or change direction Arduino->>Steering System: Apply stop/turn else [No obstacle] Safety Module->>Mini PC: Continue movement Mini PC->>Steering System: Maintain motion end </pre>
Description	The Sensors represent the perception layer responsible for detecting obstacles and sending environmental data. The Mini PC analyzes incoming sensor data and coordinates system responses. The Safety Module evaluates risks and determines whether the vehicle should stop or change direction. The Arduino translates decisions into control signals, and the Steering System executes movement commands. The process begins when sensors detect an obstacle and send data to the Mini PC. The system evaluates the situation and either stops, changes direction, or continues movement.

Connection to Requirements: SR-09

Architectural Element: Sequence Diagram – Blocked Vehicle Notification

Architectural Element	Description
Sequence Diagram	<pre> sequenceDiagram participant Mini PC participant Cloud Backend participant Notification Service/ UI participant Operator Mini PC->>Mini PC: Detect no movement (timeout) Mini PC->>Cloud Backend: Send alert Cloud Backend->>Notification Service/ UI: Trigger notification Notification Service/ UI->>Operator: Alert message </pre>
Description	The Mini PC monitors vehicle movement and detects abnormal situations such as lack of motion. When a timeout condition is reached, it sends an alert to the Cloud Backend. The Cloud processes the alert and triggers a notification through the Notification Service, which informs the Operator.

Connection to Requirements: SR-14

Architectural Element: Sequence Diagram – Emergency Stop

Architectural Element	Description
-----------------------	-------------

Sequence Diagram	<pre> sequenceDiagram actor Operator participant UI as Emergency Button / UI participant PC as Mini PC participant Arduino participant SS as Steering System Operator->>UI: Press stop activate UI UI->>PC: Emergency signal deactivate UI activate PC PC->>Arduino: Immediate stop command deactivate PC activate Arduino Arduino->>SS: Cut motor signal deactivate Arduino deactivate SS </pre>
Description	The Operator triggers an emergency stop through a physical button or UI. The signal is sent to the Mini PC, which immediately issues a stop command to the Arduino. The Arduino cuts motor signals, and the Steering System halts the vehicle instantly.

Connection to Requirements: SR-10

Architectural Element: Sequence Diagram – Incremental Movement

Architectural Element	Description
Sequence Diagram	<pre> sequenceDiagram actor Worker participant CI as Control Interface participant PC as Mini PC participant Arduino participant SS as Steering System Worker->>CI: Press forward button activate CI CI->>PC: Increment command deactivate CI activate PC PC->>Arduino: Move forward small distance deactivate PC activate Arduino Arduino->>SS: Execute movement deactivate Arduino activate SS SS-->>Arduino: Movement complete deactivate SS deactivate Arduino </pre>
Description	The Worker initiates movement using a control interface. The Mini PC interprets the command and sends movement instructions to the Arduino. The Arduino converts the command into control signals, and the Steering System executes a small movement step. A confirmation is returned after execution.

4. System State Models

The state diagrams describe the dynamic behavior of each major system component, showing transitions between operational states and responses to events

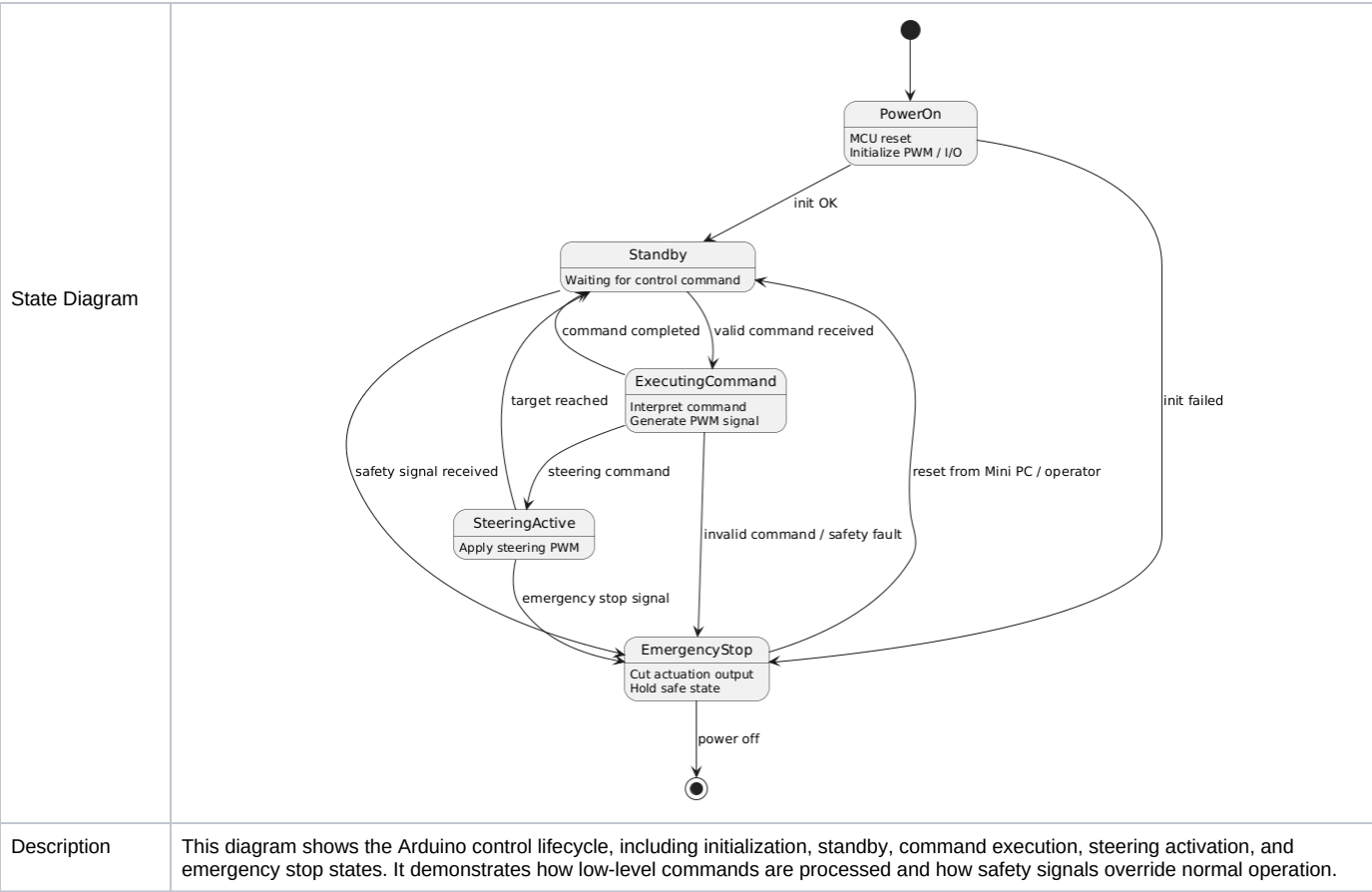
Architectural Element: State Diagram – Mini PC (Main Controller)

Architectural Element	Description
-----------------------	-------------

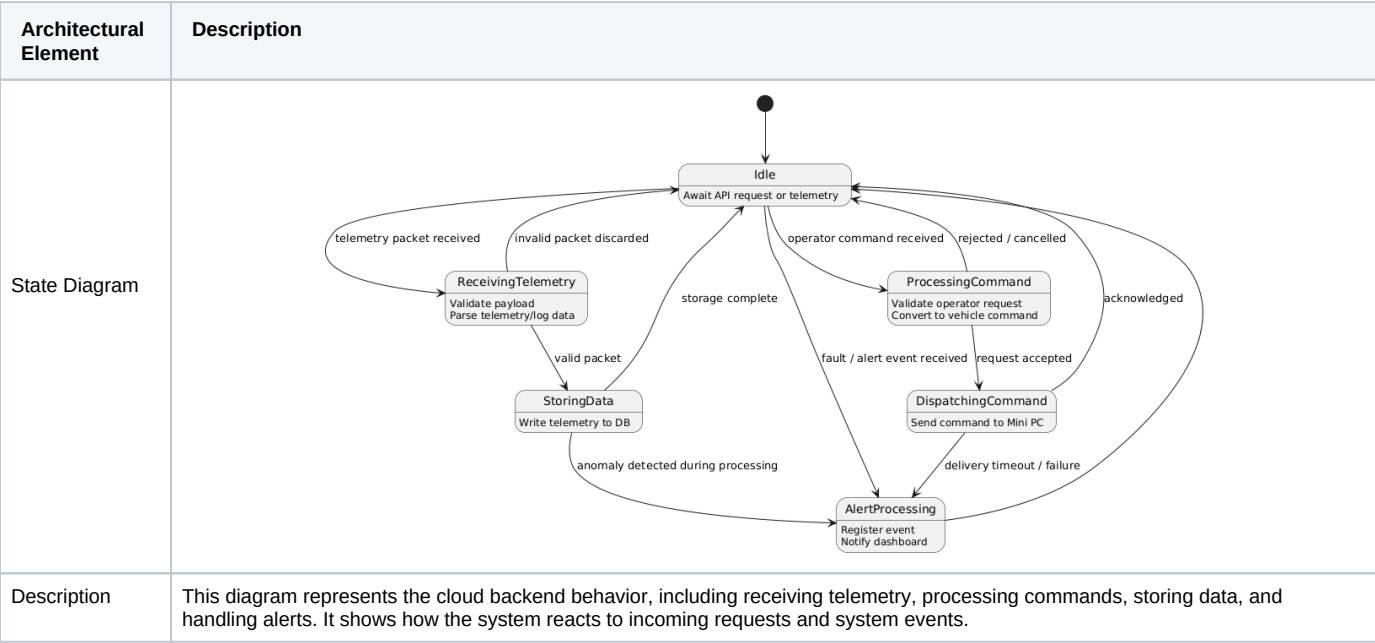
State Diagram	<pre>stateDiagram-v2 [*] --> Booting Booting: Load OS and services\nInitialize communication\nInitialize sensor interfaces Booting --> SelfCheck: startup complete SelfCheck: Verify GPS, sensors, camera\nVerify Arduino link SelfCheck --> Ready: all checks passed SelfCheck --> Fault: check failed Ready: Waiting for route/task Ready --> MissionPlanning: command received from Cloud Ready --> Ready: task cancelled MissionPlanning: Validate task\nBuild/confirm route MissionPlanning --> Ready: route valid MissionPlanning --> Ready: task aborted MissionPlanning --> Ready: component failure detected MissionPlanning --> Ready: route invalid / no localization MissionPlanning --> ObstacleHandling: obstacle detected ObstacleHandling: Stop or slow vehicle\nEvaluate safe continuation ObstacleHandling --> Ready: path clear ObstacleHandling --> RemoteOverride: operator intervention required ObstacleHandling --> Fault: unresolved safety issue RemoteOverride: Accept remote/manual command RemoteOverride --> Ready: autonomy resumed RemoteOverride --> Fault: control conflict / failure AutonomousDriving: Process GPS/RTK\nProcess obstacle data\nSend control commands to Arduino AutonomousDriving --> Ready: destination reached AutonomousDriving --> Ready: critical error AutonomousDriving --> Fault: control conflict / failure MissionComplete: Send completion status\nUpload final telemetry MissionComplete --> Ready: reset successful Fault: Stop command issued\nPublish alert to cloud Fault --> [*]: shutdown</pre>
Description	This diagram illustrates the lifecycle of the Mini PC controller, including booting, self-check, ready state, mission planning, autonomous driving, obstacle handling, remote override, and fault states. It shows transitions based on events such as command reception, obstacle detection, and system failures.

Architectural Element: State Diagram – Arduino (Low-Level Control Unit)

Architectural Element	Description
-----------------------	-------------



Architectural Element: State Diagram – Cloud Backend



Architectural Element: State Diagram – Web Dashboard (UI)

Architectural Element	Description
State Diagram	<pre>stateDiagram-v2 [*] --> Disconnected state Disconnected { No active backend session } state Authenticating { Validate credentials/session } state Monitoring { Display status, telemetry, map } state ViewingAlert { Show warning / fault / event } state SendingCommand { Build and submit command } Disconnected --> Authenticating : user opens dashboard / login failed Authenticating --> Disconnected : session lost / logout Authenticating --> Monitoring : login successful Monitoring --> ViewingAlert : alert received ViewingAlert --> Monitoring : alert acknowledged Monitoring --> SendingCommand : operator issues command SendingCommand --> Monitoring : command acknowledged SendingCommand --> ViewingAlert : command failed / rejected ViewingAlert --> SendingCommand : operator responds with action</pre>
Description	This diagram illustrates the user interface behavior, including authentication, monitoring, alert viewing, and command submission. It shows how user interactions drive system operations.

Architectural Element: State Diagram – Steering System

Architectural Element	Description
State Diagram	<pre>stateDiagram-v2 [*] --> Neutral state Neutral { No active steering motion } state AutoSteering { Actuator follows Arduino signal } state ManualSteering { Operator directly controls steering } state LockedSafe { Steering held in safe state } Neutral --> AutoSteering : PWM command from Arduino AutoSteering --> Neutral : steering target reached AutoSteering --> ManualSteering : manual override engaged ManualSteering --> AutoSteering : autonomous mode restored ManualSteering --> LockedSafe : emergency stop / fault LockedSafe --> ManualSteering : emergency stop / fault ManualSteering --> Neutral : operator takes control Neutral --> ManualSteering : manual input released LockedSafe --> Neutral : reset / recovery</pre>
Description	This diagram shows the steering system states, including neutral, auto steering, manual steering, and locked safe state. It illustrates transitions based on control commands, manual overrides, and safety events.

02-AGC WP6.3 System architecture analysis

1. Component Special Characteristics Overview

This document presents the special characteristics analysis of the Autonomous Golf Cart System, organized by its major components. Each component contains critical characteristics that require particular attention during design, implementation, and operation. The analysis follows the system's layered architecture: User Layer, Cloud Layer, and Edge/Vehicle Layer.

2. Mini PC System (Software Systems Architect)

The Mini PC functions as the central intelligence and main decision-making component of the autonomous golf cart. As the brain of the vehicle, it receives data from all sensors, processes information in real time, and generates commands to control the vehicle. The following special characteristics are critical to its operation:

Special Characteristic	Description
Decision Logic Timing	Must maintain consistent sub-100ms response time to ensure real-time synchronization with physical components and immediate reaction to environmental changes such as obstacles or route deviations
System State Management	Critical for maintaining operational integrity during mode transitions between normal autonomous operation, manual control, and emergency stop modes
Sensor Fusion Accuracy	SensorFusion module must correctly merge GPS, ultrasonic, and camera data to support reliable obstacle detection and navigation decisions
Fault Tolerance Logic	Essential for handling partial system failures gracefully, allowing continued operation in degraded mode when appropriate, particularly when one sensor source becomes unavailable

3. Arduino Low-Level Controller (Robotics / Embedded Systems Engineer)

The Arduino controller represents the low-level control layer of the system. It receives high-level commands from the Mini PC and executes them in a stable and predictable way, forming a critical safety boundary between software decisions and physical actuation.

Special Characteristic	Description
Command Execution Reliability	Must translate high-level movement commands into correct motor control signals with zero ambiguity; the CommandParser must validate all commands before execution
Emergency Stop Response Time	Critical for immediate vehicle halt when safety conditions are triggered, preventing equipment damage or human injury in agricultural environments where people and machinery share space
Motor Control Precision	MotorController must maintain accurate speed and direction signals to enable consistent autonomous navigation and safe deceleration
Safety Module Continuity	SafetyModule must continuously monitor signals from the Mini PC and autonomously trigger emergency stop even if higher-level communication is interrupted

4. Web Dashboard — User Interface System (UI/UX Team Lead)

The Web Dashboard is the primary interaction point between the operator and the system, enabling real-time monitoring and remote command dispatch. In agricultural environments where operators may supervise multiple simultaneous processes, the following characteristics are critical:

Special Characteristic	Description
Alert Prioritization Algorithm	Critical for ensuring that high-priority safety and operational alerts are prominently displayed to operators, preventing them from missing crucial information such as obstacle detections or communication failures
Role-Based Access Control	Essential for maintaining system security and preventing unauthorized configuration changes that could compromise vehicle performance or safety features

Real-Time Status Synchronization	DashboardView must update vehicle position, speed, and operational status in real time to ensure operators always act on current system state
Command Transmission Reliability	ControlPanel must guarantee that operator commands (move, stop, manual override) reach the cloud backend and are confirmed received before UI feedback is displayed

5. Cloud Backend System (Data / Systems Engineer)

The Cloud Backend serves as the data management and communication layer, connecting the Web Dashboard with the Mini PC and providing centralized storage and processing. Its special characteristics govern system-wide data integrity and command reliability:

Special Characteristic	Description
Real-Time Data Pipeline Latency	Must maintain sub-second telemetry transfer for position, battery, and operational metrics to support timely operator decision-making and accurate dashboarding
Command Dispatch Integrity	CommandDispatcher must guarantee ordered, non-duplicate delivery of operator commands to the vehicle even when multiple commands are submitted concurrently
Data Consistency Validation	DataManager must ensure reliable storage and retrieval of operational logs, supporting compliance verification, performance analysis, and system diagnostics
Authentication Security	APIGateway must validate all incoming requests to prevent unauthorized system access that could compromise vehicle operations or safety features

6. Sensor System (Computer Vision / Perception Engineer)

The Sensor System constitutes the perception layer of the autonomous vehicle, collecting environmental data and transmitting it to the Mini PC. The accuracy and reliability of this layer directly determines navigation quality and safety outcomes.

Special Characteristic	Description
GPS Positioning Accuracy	GPSModule must provide high-accuracy coordinates with measurable precision metrics; critical for autonomous route following and geofencing
Obstacle Detection Reliability	UltrasonicSensors must detect obstacles with no acceptable false negatives; missed detections create direct collision risk in dynamic agricultural environments
Camera Processing Latency	CameraModule must capture and stream visual data within strict time constraints to support real-time environment monitoring and potential image-recognition features
Sensor Data Validation	SensorSystem must filter out false or corrupted readings from any sensor module before forwarding data to the Mini PC, preventing erroneous navigation or unnecessary emergency stops

7. Steering & Motor System (Mechanical Systems Engineer)

The Steering System converts electrical signals from the Arduino controller into physical vehicle movement, forming the final actuation layer of the system. Its characteristics directly affect vehicle maneuverability, path accuracy, and safe transitions between control modes.

Special Characteristic	Description
Movement Precision	SteeringMechanism must maintain accurate wheel rotation in response to control signals to ensure reliable path following across varied terrain conditions
PWM Signal Stability	MotorDriver must apply consistent PWM signals to prevent speed fluctuations that could cause unsafe vehicle behavior or navigation deviations
Manual-Autonomous Handoff Safety	ManualControl must enable smooth, safe transitions between autonomous and manual operation modes without abrupt movement or loss of control
Emergency Braking Response	System must achieve consistent sub-second stopping capability when emergency stop is triggered, preventing accidents in shared human-vehicle spaces

8. Rationale and Failure Mode Analysis

8.1 Mini PC — Decision Logic Timing

The Mini PC's decision logic timing is identified as a special characteristic due to its fundamental role in coordinating the entire autonomous operation. The system architecture requires consistent sub-100ms decision-making for safe navigation. Processing delays would cause the vehicle to miss obstacle avoidance windows or issue commands that are no longer valid for the vehicle's current position. This failure mode presents high severity risk.

Mitigation: Implement real-time processing prioritization, dedicated CPU resource allocation for the navigation and decision engines, and automated alerting when execution times approach the 80ms threshold.

8.2 Sensor System — Obstacle Detection Reliability

Failure Mode	Risk	Severity	Mitigation Strategy
Sensor blind spots or false negatives in ultrasonic detection	A human, animal, or obstacle in the vehicle's path goes undetected, resulting in collision	Critical — direct potential for injury or property damage	Implement redundant, overlapping sensor coverage using multiple detection technologies (ultrasonic + vision-based); require positive safety zone clearance before resuming motion; implement periodic automatic self-testing of sensor health

8.3 Arduino — Emergency Stop Response Time

The Arduino's emergency stop capability is a special characteristic because it operates as the last hardware-enforced safety boundary. If the SafetyModule fails to trigger immediately when a hazardous condition is detected, the vehicle's momentum could cause injury or equipment damage. This is especially critical in environments where human operators may walk near the vehicle.

Mitigation: Implement emergency stop as a hardware-level interrupt independent of software execution state. Regular functional testing of the stop circuit should be part of pre-operation checklists. Response time must not exceed 200ms from trigger to full motor halt.

8.4 Web Dashboard — Alert Prioritization Algorithm

Failure Mode	Risk	Severity	Mitigation Strategy
Critical safety alerts buried beneath routine operational notifications	Operator misses a proximity alert, communication failure, or emergency stop event	High — delayed response to safety events could cause harm	Implement tiered alert classification (critical/warning/informational); critical alerts must interrupt all other UI activity with persistent visual and audible signals; alerts must require explicit operator acknowledgment before dismissal

8.5 Cloud Backend — Command Dispatch Integrity

The CommandDispatcher must guarantee that vehicle commands are delivered in the correct order and are not duplicated. A command sent twice (e.g., 'start' followed by an erroneous duplicate) or delivered out of sequence could cause unpredictable vehicle behavior. This is especially important during high-latency network conditions.

Mitigation: Implement sequence-numbered command queuing with acknowledgment confirmation. Commands without confirmed delivery should be retried with exponential backoff, and conflicting or duplicate commands should be resolved by the APIGateway before forwarding.

9. System Requirements Refinement

Based on the analysis of failure modes and mitigation strategies, the following additions and modifications to system requirements are recommended:

9.1 New Requirements

Requirement ID	Description
SR-11 (Decision Logic Performance)	The Mini PC shall maintain decision logic execution times below 100ms for 99.99% of all navigation cycles, with automated monitoring and alerting when execution times exceed 80ms. Dedicated processing allocation shall prevent resource contention from impacting navigation timing.

SR-12 (Sensor Redundancy)	The Sensor System shall implement overlapping detection coverage using at least two independent sensing technologies for obstacle detection. The system shall perform automatic self-tests of all sensor modules at startup and at four-hour intervals during operation.
SR-13 (Emergency Stop Verification)	The Arduino Controller shall verify emergency stop circuit integrity at system initialization and log response time for each triggered stop event. Stop response time shall not exceed 200ms from trigger signal to full motor halt.
SR-14 (Command Integrity)	The Cloud Backend shall implement sequence-numbered command queuing with delivery confirmation. Duplicate or out-of-sequence commands shall be rejected, and unconfirmed commands shall be retried with a maximum of three attempts before operator notification.
SR-15 (Alert Classification)	The Web Dashboard shall implement a three-tier alert classification system (Critical, Warning, Informational). Critical alerts shall interrupt all other dashboard activity and require explicit operator acknowledgment. Alert logs shall be stored with timestamps for audit review.
SR-16 (System Diagnostics)	All critical special characteristics shall be subject to automated diagnostic verification during system initialization and at scheduled intervals during operation. Diagnostic results shall be logged with timestamp and verification status.

9.2 Modified Requirements

Req uire ment ID	Original	Modified
SR-08	Safety system shall respond to detected obstacle or human presence within 1 second	The safety system shall respond to detected human or obstacle presence within 200ms and include blind spot detection verification protocols to be executed at startup and every four hours. The system shall maintain safety response time logs for compliance verification.
SR-09	Vehicle shall follow predefined routes autonomously	Vehicle shall follow predefined routes autonomously with GPS positional accuracy verified to within $\pm 0.5\text{m}$. Route deviation exceeding 1m shall trigger an operator alert and automatic speed reduction until re-alignment is confirmed.
SR-10	Web dashboard shall display vehicle status	The Web Dashboard shall display vehicle position, speed, battery level, and active alerts in real time with maximum display latency of 2 seconds. Critical alerts shall persist until explicitly acknowledged by the operator.

10. Conclusion

The detailed analysis of special characteristics across all system components of the Autonomous Golf Cart reveals several critical areas requiring careful design attention and ongoing monitoring. The time-sensitive aspects of the system — particularly Mini PC decision logic timing, sensor data processing, and command dispatch latency — form the operational backbone of reliable autonomous behavior.

Safety features, especially obstacle detection and emergency stop response, require redundancy and rigorous validation protocols beyond basic functional requirements. The refined requirements address these concerns by adding specific performance targets, redundancy mandates, and verification procedures that will enhance overall system reliability.

The identification of potential failure modes highlights the need for robust self-monitoring, calibration verification, and automated diagnostics throughout all system layers. By implementing the refined requirements and focusing development effort on the identified special characteristics, the Autonomous Golf Cart system will achieve the reliability, safety, and operational performance required for deployment in real agricultural environments.

02-AGC WP6.4 Ensure consistency and establish bidirectional traceability

02-AGC WP6.5 Review and Communicate agreed system architecture

02-AGC WP7 Midterm 1 summary

Work Product Status

Legend	NEW - Backlog/Planned PROGRESS - Started DONE - Finished
Midterm Project report development	 PBL26-783 - Jira server returned an error: [Ljava.lang.Object; @391f8e4b  PBL26-843 - Jira server returned an error: [Ljava.lang.Object; @66c61d64  PBL26-844 - Jira server returned an error: [Ljava.lang.Object; @1785f10d  PBL26-845 - Jira server returned an error: [Ljava.lang.Object; @660d847f  PBL26-846 - Jira server returned an error: [Ljava.lang.Object; @49609ab1

Project presentation preparation

 [PBL26-784](#) - Jira server returned an error: [Ljava.lang.Object;
@20305fee

 [PBL26-847](#) - Jira server returned an error: [Ljava.lang.Object;
@3b5c70f6

 [PBL26-848](#) - Jira server returned an error: [Ljava.lang.Object;
@184b5d2d

 [PBL26-849](#) - Jira server returned an error: [Ljava.lang.Object;
@461e6447

 [PBL26-850](#) - Jira server returned an error: [Ljava.lang.Object;
@59103ab6

Project Paper Creation

 [PBL26-785](#) - Jira server returned an error: [Ljava.lang.Object;
@11223475

 [PBL26-851](#) - Jira server returned an error: [Ljava.lang.Object;
@70f27f9b

 [PBL26-852](#) - Jira server returned an error: [Ljava.lang.Object;
@1b78647

 [PBL26-853](#) - Jira server returned an error: [Ljava.lang.Object;
@93a2b4b

 [PBL26-854](#) - Jira server returned an error: [Ljava.lang.Object;
@71043983

02-AGC Article

Autonomous Yard Utility Vehicle - Golf Cart

Denis PLECA,
tefan ISTRATI,
Pavel CIUMACENCO,
Vladlen MIU

Department of Software Engineering and Automation, group FAF-232, Faculty of Computers, Informatics, and
Microelectronics, Technical University of Moldova, Chiinu, Republic of Moldova

*Corresponding Author: Ciumacenco Pavel, pavel.ciumacenco@isa.utm.md

Tutor/scientific coordonator Andrei Bragarenco, university professor,
Department of Software Engineering and Automation, Faculty of Computers, Informatics, and
Microelectronics, Technical University of Moldova, Chiinu, Republic of Moldova

Abstract. The rapid development of the Internet of Things (IoT), edge computing, and sensor-based systems has significantly influenced modern agricultural technologies. This paper presents the design and analysis of an intelligent monitoring and control system for a small autonomous vehicle intended for agricultural environments. The proposed system integrates positioning sensors, obstacle detection sensors, image acquisition modules, and a cloud-based monitoring platform. The system architecture is based on three main layers: the user layer, the cloud layer, and the edge layer, where the vehicle operates as a mobile IoT sensing platform. The paper describes the overall system architecture, the interaction between hardware and software components, and the operational principles of the system. In addition, the proposed solution is analyzed in the context of existing research in precision agriculture and IoT-based farm management systems. The results from previous scientific studies show that IoT-based monitoring systems can significantly improve productivity, reduce resource consumption, and support data-driven decision-making in agricultural environments. The proposed project demonstrates the feasibility of a modular and scalable architecture that can be further extended for autonomous monitoring, transport, and inspection tasks in smart farming systems.

Key-words: Internet of Things, smart agriculture, autonomous vehicle, edge computing, precision farming, sensor-based monitoring

Introduction

The agricultural sector is currently undergoing a major transformation driven by digital technologies such as the Internet of Things (IoT), artificial intelligence, and cloud computing. These technologies enable the development of smart farming systems that can collect, process, and analyze large amounts of data directly from the field. As a result, farmers are able to make more precise decisions, reduce operational costs, and improve crop productivity.

Recent scientific studies show that IoT-based precision agriculture systems are capable of significantly improving agricultural efficiency by integrating sensors, wireless communication technologies, and cloud platforms into a single architecture [1], [2]. These systems are typically based on continuous data collection from soil, environmental, and operational sensors, followed by real-time analysis and automated decision-making. According to a recent review published in *Agronomy*, IoT-based precision agriculture can reduce resource usage while maintaining or even increasing crop yield through data-driven decision support [1].

Another important trend in modern agriculture is the use of autonomous or semi-autonomous vehicles equipped with sensors and communication modules. These vehicles are used for monitoring fields, transporting materials, and performing inspection tasks. Research published in *Scientific Reports* shows that smart sensor systems and real-time data processing can significantly improve agricultural productivity by enabling precise monitoring of soil conditions, irrigation, and crop health [2].

The objective of this paper is to design and analyze an intelligent monitoring and control system for a small autonomous vehicle based on IoT technologies. The proposed system combines sensors, edge computing, cloud data processing, and a web-based user interface. The paper focuses on the system architecture, the interaction between system components, and the relevance of the proposed solution in the context of modern smart agriculture technologies.

Background and Related Research

The development of smart agriculture systems is strongly connected to the evolution of IoT-based monitoring platforms. Modern agricultural systems are no longer based only on traditional farming practices but increasingly rely on real-time data collected from sensors and processed using cloud platforms. A recent review published in *Sensors* highlights that IoT-based monitoring systems are now widely used for soil moisture monitoring, environmental data collection, irrigation control, and crop health analysis [3]. These systems typically include sensor nodes, wireless communication networks, data processing platforms, and decision-support tools. The study also shows that the use of real-time data can significantly reduce fertilizer and water consumption while improving crop productivity.

Another study published in *Scientific Reports* analyzes the practical applications of IoT sensors in precision agriculture and viticulture. The research confirms that sensor-based monitoring allows continuous measurement of soil parameters such as humidity, temperature, and nutrient concentration, which enables more efficient agricultural management [2]. The authors emphasize that real-time monitoring systems reduce the need for manual inspection and allow farmers to react faster to environmental changes.

IoT-based monitoring systems are also strongly connected to cloud computing and big-data analysis. According to a recent study published in *Agronomy*, the integration of IoT sensors with cloud-based data processing platforms enables predictive decision-making in agriculture, such as yield prediction, irrigation scheduling, and crop health monitoring [1]. This demonstrates that data-driven agriculture is becoming one of the most important research directions in the field of smart farming.

In addition to theoretical research, several experimental studies have confirmed the practical benefits of IoT-based agricultural systems. For example, an experimental precision irrigation system described in a study published in *Frontiers in Water* demonstrates that real-time monitoring combined with automated decision-making can significantly improve water management and reduce waste [4]. Similar results were obtained in a study published in the *Indian Journal of Horticulture*, where an IoT-based irrigation system increased crop yield by more than 12% while reducing water usage by more than 30% [5]. These research results clearly show that IoT-based monitoring systems are not only theoretical concepts but practical solutions that are already being implemented in modern agriculture.

Proposed System Architecture

The proposed system is based on a three-layer architecture: the user layer, the cloud layer, and the edge layer. This structure allows a clear separation of functions and improves the scalability of the system.

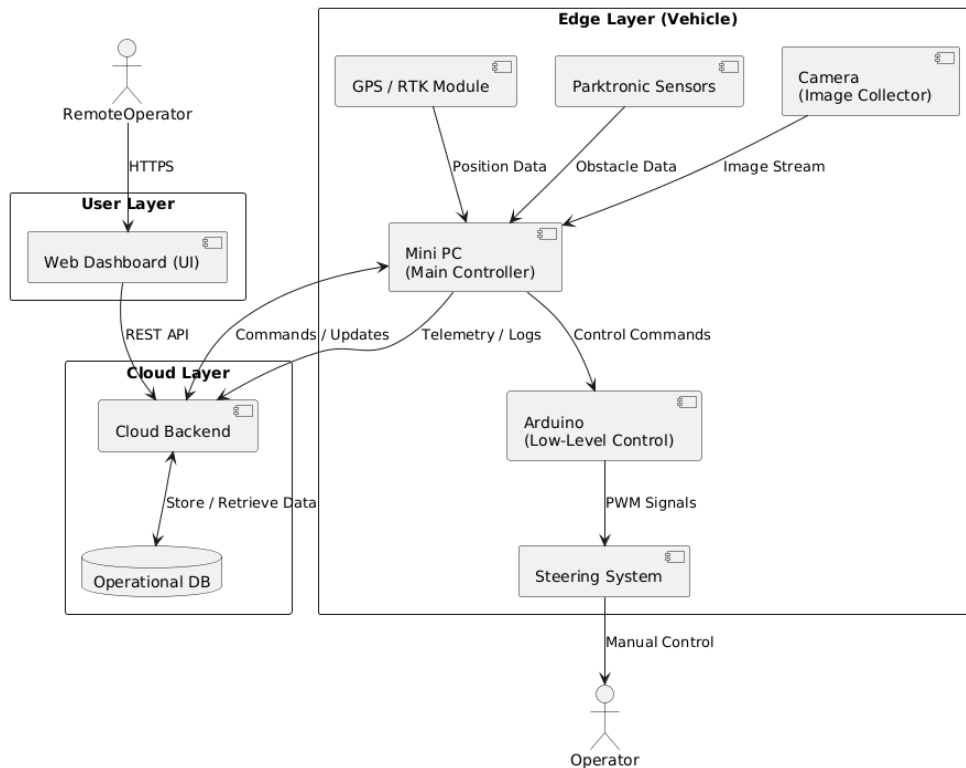


Figure 1. Architecture Diagram for Autonomous Yard Utility Vehicle

The user layer consists of a web-based dashboard that allows the operator to monitor the vehicle and visualize the collected data in real time. The dashboard provides information about the position of the vehicle, sensor data, and the operational status of the system. It also allows the operator to send control commands to the vehicle.

The cloud layer is responsible for data storage, communication management, and interaction between the web interface and the vehicle. The cloud backend processes incoming requests from the dashboard and sends commands to the vehicle. At the same time, it stores telemetry data, images, and sensor measurements in a cloud database. Cloud-based architectures are widely used in smart agriculture systems because they allow large-scale data storage and real-time data analysis [1], [3].

The edge layer represents the physical autonomous vehicle. The vehicle is equipped with a mini-PC that acts as the main processing unit. The mini-PC receives data from sensors, processes the information in real time, and sends relevant data to the cloud system. A microcontroller (Arduino) is used for low-level control of the steering system and other hardware components.

The vehicle is equipped with several sensors, including a GPS/RTK module for accurate positioning, proximity sensors for obstacle detection, and a camera for image acquisition. These sensors transform the vehicle into a mobile IoT sensing platform that can be used for monitoring agricultural environments.

System Operation and Data Flow

The system operates by continuously collecting, processing, and transmitting data. During operation, the sensors installed on the vehicle send real-time data to the mini-PC. The GPS module provides position data, proximity sensors detect obstacles, and the camera captures images from the surrounding environment.

The mini-PC processes these data and determines the necessary actions. If the vehicle needs to change direction or avoid an obstacle, the mini-PC sends control commands to the Arduino microcontroller. The microcontroller then controls the steering system using PWM signals. This architecture ensures a clear separation between high-level decision-making and low-level hardware control.

At the same time, the collected data are transmitted to the cloud backend and stored in the database. According to recent research, the use of cloud-based monitoring systems significantly improves the reliability and scalability of agricultural IoT platforms because data can be stored and analyzed remotely [1], [3].

The operator can access the web dashboard to monitor the vehicle in real time and send commands if necessary. This means that the system supports both autonomous operation and remote control.

Relevance of the Proposed System in Smart Agriculture

The proposed system is strongly related to modern precision agriculture technologies. Research studies show that IoT-based systems can significantly improve productivity by providing accurate real-time data from the field. According to a study published in *Agronomy*, precision agriculture systems based on IoT sensors can increase crop yield while reducing the use of fertilizers and water [1].

Another study published in *Scientific Reports* confirms that sensor-based monitoring systems improve the efficiency of irrigation and fertilization processes because farmers can make decisions based on real-time environmental data instead of manual estimation [2]. This demonstrates that sensor-based mobile platforms, such as the proposed autonomous vehicle, can play an important role in modern agriculture.

Experimental results from an IoT-based irrigation system show that crop yield increased by more than 12% while water consumption was reduced by more than 30% [5]. These results clearly demonstrate the advantages of data-driven agricultural management. The proposed system can support similar improvements by providing continuous monitoring and real-time data collection.

Advantages of the Proposed Solution

The main advantage of the proposed system is its modular architecture. Each component has a clearly defined role, which allows the system to be easily extended with additional sensors or new functionalities. For example, image processing algorithms or autonomous navigation functions can be added in future versions.

Another important advantage is the use of edge computing. Data are processed directly on the vehicle, which reduces response time and allows the system to operate even when the internet connection is unstable. At the same time, cloud-based data storage ensures long-term data availability and remote monitoring.

The system is also scalable. The proposed architecture can be extended from a single vehicle to multiple vehicles connected to the same cloud platform. This makes the solution suitable for future smart agriculture applications such as field monitoring, automated inspection, and precision transport.

Conclusion

This paper presented the design and analysis of an intelligent IoT-based monitoring and control system for a small autonomous vehicle intended for agricultural environments. The proposed system integrates sensors, edge computing, cloud communication, and a web-based monitoring platform. The research presented in recent scientific publications confirms that IoT-based monitoring systems can significantly improve agricultural productivity, reduce resource consumption, and support data-driven decision-making. The proposed system follows the same principles and demonstrates the feasibility of a modular and scalable architecture for smart agriculture applications.

Future work will focus on improving the autonomous navigation capabilities of the vehicle, integrating additional sensors, and developing more advanced data analysis algorithms.

References

1. T. Alahmad, M. Neményi, A. Nyéki. Titlul lucrrii. *Numele revistei Anul, Volumul*, pp. 154–196, doi:

Cri sau capitole:

2. A. Authors, "A technical survey on practical applications and guidelines for IoT sensors in precision agriculture and viticulture," *Scientific Reports*, 2024.

3. Authors, "Smart Sensors and Smart Data for Precision Agriculture: A Review," *Sensors*, 2024.

Lucrriile conferinelor:

4. Y. Dong et al., "Implementation of an in-field IoT system for precision irrigation management," *Frontiers in Water*, 2024.

5. M. Salimath et al., "Optimizing tomato production with IoT-enabled precision irrigation," *Indian Journal of Horticulture*, 2025.

6. N. Vatin et al., "Precision Agriculture and Sustainable Yields: Insights from IoT-Driven Farming," *BIO Web of Conferences*, 2024.

7. S. Garg et al., "Towards a Multimodal System for Precision Agriculture using IoT and Machine Learning," 2021.

8. G. Stamatescu et al., "IoT-Enabled Distributed Data Processing for Precision Agriculture," 2019.

9. A. Z. Babar and O. Akan, "Sustainable and Precision Agriculture with the Internet of Everything (IoE)," 2024.

02-AGC WP7.1: Midterm Project report development

Work Product Status:

Legend	NEW - Backlog/Planned PROGRESS - Started DONE - Finished
Team Formation	 PBL26-7 - Jira server returned an error: [Ljava.lang.Object;@245b8d09  PBL26-16 - Jira server returned an error: [Ljava.lang.Object;@320ab91c  PBL26-17 - Jira server returned an error: [Ljava.lang.Object;@7d498679  PBL26-18 - Jira server returned an error: [Ljava.lang.Object;@2115dd6  PBL26-19 - Jira server returned an error: [Ljava.lang.Object;@e8eeae  PBL26-20 - Jira server returned an error: [Ljava.lang.Object;@61b644b9

Problem Definition	 PBL26-37 - Jira server returned an error: [Ljava.lang.Object; @7630d8dd
	 PBL26-46 - Jira server returned an error: [Ljava.lang.Object; @3981776a
	 PBL26-47 - Jira server returned an error: [Ljava.lang.Object; @68d55be8
	 PBL26-48 - Jira server returned an error: [Ljava.lang.Object; @6f467e25
	 PBL26-49 - Jira server returned an error: [Ljava.lang.Object; @594be8b0
	 PBL26-50 - Jira server returned an error: [Ljava.lang.Object; @2bf2bad3

- **Work Product Status:**
 - **Team Members**

Denis Pleca

Team Members

1	tefan Istrati	Developer
2	Pavel Ciumacenco	Developer
3	Vladlen Mitu	Developer
4	Cristian Bulat	Developer
5	Denis Pleca	Team Leader

Technology Research Perspective

The agri-food business sector represents one of the most vital components of the global economy, supporting food security, employment, and sustainable development. However, this sector faces increasing pressure from population growth, climate variability, resource constraints, labor shortages, and rising operational costs. Traditional, old-fashioned agricultural practices, largely dependent on manual monitoring and experience-based decision-making, are becoming insufficient to meet modern productivity and sustainability demands.

The digital transformation has emerged as a strategic priority across agricultural enterprises, and, as a response to all those challenges, the Internet of Things (IoT) has gained the main attention as an enabler of smart agriculture and data-driven farm management, allowing collection of data as well as operatively responding to triggers. IoT technologies support improved efficiency, transparency, and responsiveness across the agri-food value chain by enabling continuous data collection, connectivity of physical assets, and automated operational processes,

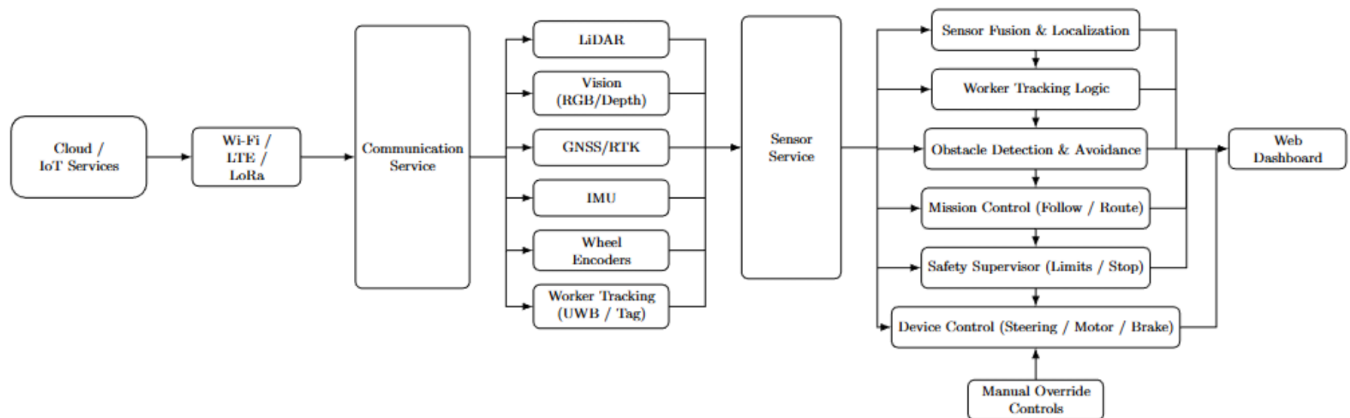
The growing adoption of IoT solutions reflects a shift towards a business model that is firstly more scalable and resilient and also has an increased precision rating compared to manual labor. The farms increasingly rely on real-time operational intelligence to optimize the usage of resources, reduce risks, and improve productivity. This evolution, besides affecting crop management, also affects logistics, asset coordination, quality assurance, and strategic planning functions.

Research Context and Motivation

The integration of IoT technologies into agri-food systems offers substantial potential for transforming traditional farming into intelligent, automated, and sustainable operations. By connecting physical infrastructure with digital platforms, IoT enables organizations to monitor conditions continuously, coordinate processes efficiently, and make evidence-based decisions in real time.

Despite growing, many agricultural enterprises continue to face barriers related to scalability, operational integration, cost efficiency, and system reliability. Understanding how IoT solutions can be strategically aligned with business objectives is essential for maximizing value creation and ensuring long-term competitiveness.

This research is motivated by the need to examine IoT technologies not solely as technical tools but as strategic business enablers capable of reshaping operational models within the agri-food sector.



Block diagram for the IoT-enabled autonomous utility cart

Role of IoT Technologies in the Agri-Food Business Sector

The agri-food business sector is currently undergoing a significant transformation driven by digitalization and the adoption of Internet of Things (IoT) technologies. Increasing global demand for food, rising operational costs, labor shortages, and the need for sustainable farming practices have encouraged agricultural enterprises to invest in smart, connected solutions. IoT-based systems enable data-driven decision-making by integrating sensors, communication networks, and intelligent platforms into agricultural operations, ultimately improving productivity, efficiency, and safety across the value chain [1][2].

From a business perspective, IoT solutions in agriculture are not merely technological upgrades but strategic tools that enhance operational efficiency and competitiveness. Smart agriculture systems allow continuous monitoring of environmental conditions, asset usage, and workforce activities, which reduces uncertainty and supports optimized resource allocation. Research published in Elsevier journals highlights that IoT-driven agricultural systems contribute to cost reduction by minimizing fuel consumption, optimizing transportation routes, and preventing equipment misuse through real-time tracking and analytics [3].

In agri-food enterprises, logistics and internal transportation represent a non-negligible share of operational costs. Autonomous or semi-autonomous vehicles equipped with IoT sensors can support tasks such as crop inspection, goods transportation, and perimeter patrolling. These systems generate actionable data that improves planning and reduces downtime, directly impacting profitability and service reliability [4].

Safety and risk management are increasingly important for business concerns in large-scale agricultural environments. According to global safety data published by the World Health Organization, vehicle-related incidents remain a major cause of injuries in both urban and rural contexts, including agricultural settings [5]. IoT-enabled vehicles can mitigate these risks through sensor-based obstacle detection, driver behavior monitoring, and automated alerts. For agri-food businesses, reducing accidents translates into lower insurance costs, fewer operational disruptions, and improved workforce well-being.

Additionally, continuous monitoring systems improve asset security by enabling patrol and surveillance functions without requiring constant human presence. This is particularly relevant for farms with large perimeters, where theft, vandalism, or animal intrusion can cause significant financial losses.

Recent research on robotics in agriculture emphasizes the growing role of intelligent machines in enhancing farming efficiency and consistency. Studies available on ResearchGate demonstrate that agricultural robots equipped with IoT connectivity can perform repetitive and monitoring-intensive tasks more reliably than manual labor, while simultaneously collecting valuable operational data [6]. This data supports predictive maintenance, performance benchmarking, and long-term strategic planning.

Elsevier-published studies further indicate that the integration of IoT, robotics, and decision-support systems leads to measurable improvements in yield management and resource efficiency. These outcomes are achieved by linking sensor data with analytics platforms that provide insights into soil conditions, crop health, and equipment utilization [7][8].

The agri-food sector increasingly values scalable and modular IoT solutions that can adapt to farms of different sizes and production models. Research shows that businesses adopting IoT-based mobility and monitoring solutions experience improved transparency, faster response times, and enhanced sustainability performance, all of which are critical factors for long-term competitiveness [2].

From this perspective, the proposed IoT-enabled golf cart solution aligns well with current industry trends. By serving as a multi-functional platform for transportation, inspection, and patrolling, the system addresses multiple operational needs simultaneously. The addition of sensors, communication modules, and data analytics capabilities transforms a conventional vehicle into a smart agricultural asset capable of generating business value beyond its primary function.

Based on the reviewed studies, the proposed IoT golf cart implementation demonstrates strong potential as a practical and cost-effective solution for agri-food businesses. Unlike fully autonomous heavy machinery, a sensor-enhanced electric cart represents a lower entry barrier for small and medium-sized farms while still offering measurable efficiency gains. The ability to integrate environmental sensors, GPS tracking, safety systems, and wireless communication enables real-time decision support and operational oversight.

Moreover, existing research confirms that incremental digitalization—rather than complete automation—is often more feasible and economically viable in agricultural contexts. This supports the relevance of the proposed solution as a transitional technology that bridges traditional farming practices with smart agriculture paradigms [1][3].

To summarize, verified academic research confirms that IoT solutions play a critical role in reshaping the agri-food business sector by improving efficiency, safety, and data-driven decision-making. The convergence of IoT, mobility, and sensor technologies has already produced measurable benefits in agricultural operations, logistics, and monitoring. Building on top of these established results, the proposed IoT-enabled golf cart system presents a promising application with strong business relevance. By enhancing transportation, inspection, and patrolling functions within farm environments, the solution aligns with current technological trends and addresses practical operational challenges faced by modern agri-food enterprises.

IoT Device and Edge Computing Domain

Modern smart agriculture systems integrate a variety of sensors and embedded electronics to observe environmental conditions and machine state. Critical sensors include Global Navigation Satellite System (GNSS) receivers for outdoor positioning, Inertial Measurement Units (IMUs) to measure motion and orientation, and wheel encoders to estimate traveled distance. Environmental sensors such as temperature, humidity, soil moisture, and light sensors support precision agriculture by providing real-time field data for irrigation and crop condition assessment.

1) Global Navigation Satellite System(GNSS) - provides global positioning for outdoor mobile robots.

- Necessary for waypoint navigation and route execution.
- Works best with augmentations like Real-time kinematic(RTK) for centimeter accuracy.[9]

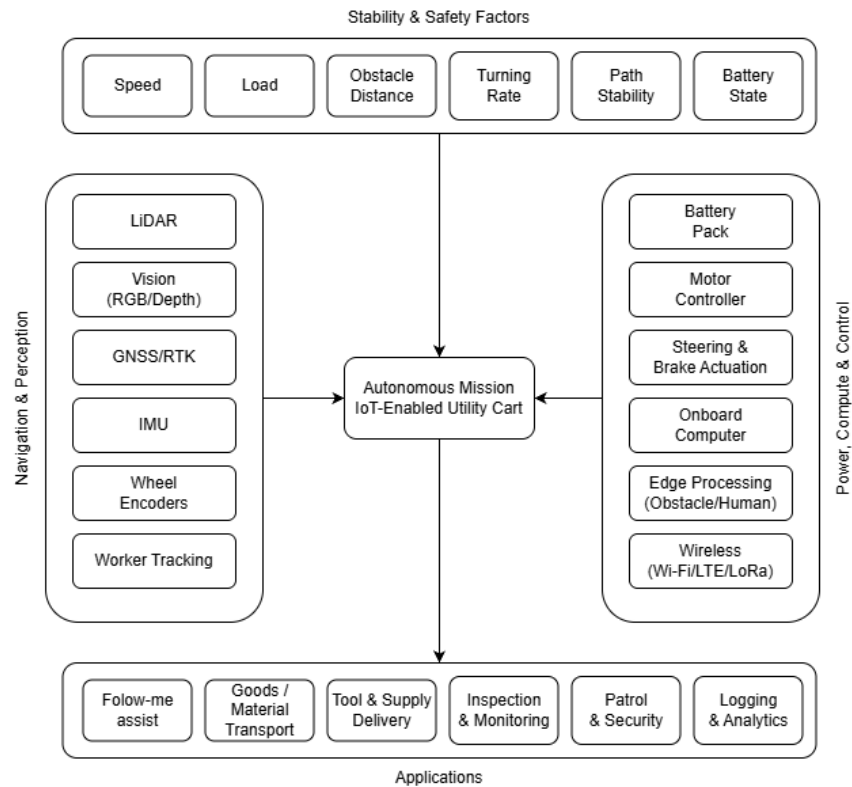
In intelligent agricultural vehicles, GNSS enables geofencing, route repetition, and autonomous coverage of large open fields with minimal infrastructure.

2) Inertial Measurement Unit(IMU) - IMUs (accelerometer + gyroscope) estimate motion when GNSS signals are weak or unavailable. Is essential for maintaining vehicle stability, estimating orientation[10][11]

3) Wheel Encoders - They are used to measure the wheels rotation and estimate the traveled distance. They provide odometry data that is used for short-term position estimation and speed calculation.[12][10]

4) LiDAR (Light Detection and Ranging) - this type of sensor is used to scan the environment to detect objects and create 3D maps. [13]

5)Ultrasonic / Proximity Sensor(Parktronic Sensors) - They are used to detect the presence of nearby objects without physical contact, based on principles such as infrared, ultrasonic, or inductive sensing. They provide distance or binary (object/no-object) data for obstacle detection, collision avoidance, and safe vehicle navigation in dynamic environments.[14]



Architecture for an IoT-enabled autonomous utility cart

Autonomous Agricultural Vehicles, Navigation Technologies, Safety Mechanisms, and Real-Time Processing

Autonomous agricultural vehicles combine high-accuracy positioning, perception, and control algorithms to perform navigation and logistics tasks without direct human control. Navigation technologies used in these vehicles include GNSS combined with RTK corrections to achieve centimeter-level positioning accuracy, particularly necessary for precise route following, docking, and geofenced operation. Sensor fusion algorithms combine GNSS, IMU, and odometry (wheel encoder) data through statistical frameworks such as Kalman filtering to improve localization accuracy in dynamic environments where satellite signals may be degraded. Perception systems - ultrasonic sensors paired with RGB and depth cameras — support obstacle detection and safe traversal through complex farm terrain.

Practical Applications in Modern Farming Environments

- **Autonomous Transport Platforms:** Agtonomy's autonomous vineyard robot demonstrates how IoT-enabled mobile platforms can follow predefined routes and transport loads through farmland without human drivers (Agtonomy Autonomous Robot Technology).[15]
- **Field Robots for Plant Monitoring and Logistics:** Systems like TerraSentia navigate crop rows autonomously for monitoring and can be extended to carry tools or samples.[16]
- **Precision Navigation in Agricultural Robots:** Research shows integration of GNSS, IMU, and wheel encoder fusion to allow reliable navigation in field robots operating in unstructured crop environments[10]

Communication Infrastructure, Data Management, and Cloud Computing

The effective deployment of Internet of Things (IoT) solutions in the agri-food business sector depends on a robust integration of communication infrastructure, data management mechanisms, and cloud computing platforms. These components enable distributed sensing, real-time data acquisition, scalable processing, and informed decision-making, which are essential for improving productivity and operational control in modern agriculture [17], [18].

Wireless Communication Technologies in Agricultural IoT Systems

Wireless communication technologies form the foundation of agricultural IoT systems by enabling connectivity between sensors, vehicles, and backend platforms across large and often remote farm environments. Existing research identifies several commonly used wireless technologies, including Wi-Fi, ZigBee, cellular networks (4G/5G), NB-IoT, and LoRa/LoRaWAN, each offering different trade-offs in terms of coverage range, bandwidth, power consumption, and deployment cost [17], [19].

Low-Power Wide-Area Network (LPWAN) technologies, particularly LoRaWAN and NB-IoT, are widely recognized as suitable solutions for agricultural scenarios due to their ability to support long-range communication with minimal energy consumption [17], [18]. These characteristics make LPWANs appropriate for battery-powered sensors deployed over large areas, such as soil moisture probes, environmental sensors, and asset-tracking devices. Studies indicate that LoRaWAN networks can achieve communication ranges of several kilometers in rural environments while maintaining low operational costs, making them attractive for agri-food businesses seeking scalable IoT deployments [17].

Data Transmission Reliability, Scalability, and Power Efficiency

Data transmission reliability and scalability are critical performance indicators in agricultural IoT systems, where sensor nodes are geographically dispersed and often operate under limited power constraints. Research shows that communication efficiency can be improved through architectural strategies such as edge data preprocessing, which reduces the volume of transmitted data by filtering or aggregating sensor readings before transmission to cloud servers [20]. This approach decreases network congestion, improves reliability, and extends device battery life.

Different communication technologies exhibit varying performance characteristics depending on the deployment context. While Wi-Fi and ZigBee are suitable for short-range, high-density environments such as greenhouses, LPWAN and cellular networks offer better scalability and coverage for open-field applications [17], [19]. Hybrid communication architectures are therefore frequently recommended, combining LPWAN technologies for low-data-rate sensing with cellular or broadband connections for higher-bandwidth applications such as video monitoring or centralized control systems [18].

Cloud Computing Platforms and Backend Architectures

Cloud computing plays a central role in agricultural IoT by providing scalable infrastructure for data storage, processing, and analytics. IoT-based agricultural systems generate large volumes of heterogeneous data, including time-series sensor readings, environmental measurements, and operational logs, which require flexible and scalable backend architectures [21]. Cloud platforms enable centralized data management, real-time processing, and remote access to monitoring dashboards, supporting both operational oversight and strategic decision-making.

Research highlights that cloud-based architectures facilitate the integration of advanced analytics and machine learning models for applications such as yield prediction, irrigation scheduling, and anomaly detection [22]. Additionally, multi-cloud and distributed cloud approaches have been proposed to improve system resilience, data availability, and computational efficiency while addressing data security and privacy concerns in large-scale agri-food operations [21].

Databases, Real-Time Monitoring, and Analytics

Efficient data management in agricultural IoT systems relies on database solutions capable of handling continuous data streams and long-term historical records. Cloud-based databases and data lakes are commonly employed to support scalable storage and efficient querying of IoT data [21]. These systems enable real-time monitoring platforms that visualize sensor data and generate alerts when predefined thresholds are exceeded, allowing timely intervention by farm operators [22].

Analytics solutions built on top of these data infrastructures transform raw sensor data into actionable insights. By applying statistical analysis and predictive models, agricultural decision support systems can recommend optimal actions related to irrigation, fertilization, and asset utilization, thereby improving resource efficiency and operational performance [22].

Impact of Data-Driven Management on Agricultural Productivity

The integration of wireless communication, cloud computing, and data analytics enables a data-driven approach to agricultural management. Studies demonstrate that such systems support precision agriculture practices by allowing farmers and agribusiness managers to monitor conditions continuously and adjust operations based on real-time and historical data [5]. This approach leads to improved productivity, reduced resource waste, and enhanced operational control, which are key objectives for competitive agri-food business enterprises.

User Experience and Interaction Design

The autonomous farm cart represents a type of mobile robotic platform designed to support transportation and logistics tasks in agricultural environments. The development of such a system requires the integration of both hardware components and software interfaces, including navigation algorithms and user interaction modules. These elements must be carefully designed to allow operators to easily monitor the system state and control its behavior. Developing intuitive user interfaces enables farmers and operators to supervise and manage the autonomous cart effectively. For example, a web-based dashboard can provide real-time visualization of vehicle position, sensor data, and mission status, allowing users to configure routes, monitor performance, and intervene when necessary. Such interfaces enhance user interaction and improve the overall usability and operational efficiency of the system.

User Interaction, System Integration, and Safety & Security

The successful deployment of an IoT-enabled golf cart in an agri-food environment depends on the seamless convergence of high-level processing, real-time control, and a dual-layered approach to risk management. This chapter analyzes the integration of the vehicle's specialized hardware, the interfaces facilitating human-machine synergy, and the critical distinction between operational safety and cybersecurity.

1. Human-Machine Interaction (HMI) in Farming Environments

In modern smart agriculture, HMI is the essential bridge between complex sensor data and operational decision-making [23]. For a vehicle equipped with an Industrial Computer, the interaction model shifts from simple remote monitoring to an active Edge-Computing interface.

Ruggedized On-Board Dashboards: Industrial computers support high-brightness displays (up to 1,600 nits) essential for outdoor visibility. These interfaces provide real-time telemetry from the drone controller, such as GPS health, battery status, and "First Person View" (FPV) streaming [24][25].

Context-Aware Control: The system must manage the transition between autonomous navigation (handled by the drone controller) and manual intervention via the Electrical Steering. This requires a "Trustworthy Interaction" design where the operator can instantly override automation [23][27].

1. System Integration: Hardware and Communication Layers

The proposed golf cart utilizes a two-layered control architecture to bridge the gap between aerial drone technology and ground-based agricultural machinery [28].

The Control Loop (Drone Controller & EPS): The drone controller serves as the "low-level" processor. It handles real-time sensor fusion from the IMU and GNSS/RTK to manage the Electrical Steering. Integration is achieved via PWM or CAN-bus protocols to ensure millisecond-level responsiveness [29][30].

The Intelligence Hub (Industrial Computer): The industrial computer acts as the "high-level" gateway. It processes heavy data loads from the Parktronic sensors and cameras, utilizing Edge AI to detect obstacles before syncing simplified data packets to the cloud [25][31].

1. Functional Safety and Risk Mitigation

While security focuses on malicious intent, Functional Safety ensures the vehicle does not cause harm due to technical malfunctions or environmental unpredictability [34]. In the agricultural context, this is governed by the ISO **25119** standard (Functional Safety for Agricultural Machinery) [35].

Proximity Safety (Parktronic Sensors): The system utilizes Parktronic (ultrasonic) sensors as a "fail-safe" layer. If these sensors detect an obstacle within a critical 1-meter radius, they trigger a hardware-level interrupt that bypasses the drone controller's navigation commands to apply the brakes immediately [35].

Operational Design Domain (ODD): Safety is maintained by defining strict operational boundaries. The Industrial Computer monitors "Geofencing" data; if the vehicle deviates from the predefined farm path, the system enters a "Safe State," neutralizing the Electronic Power Steering to prevent unintended movements into hazardous areas [28][34].

Fail-Operational Steering: The integration of Electrical Steering allows for software-defined safety limits, such as maximum speed-sensitive steering angles, preventing the cart from tipping over during sharp autonomous turns at higher speeds [29].

1. Cybersecurity and Data Security

As agricultural enterprises adopt connected machinery, the "attack surface" of the farm expands. Reliability is no longer just mechanical; it is digital [32].

Protocol Vulnerabilities: The use of drone-centric protocols like MAVLink introduces risks; without encryption, these signals can be intercepted or spoofed, leading to unauthorized steering commands [33].

Data Privacy and Sovereignty: High-resolution mapping data and crop analytics are sensitive business intelligence. Security measures include data encryption at rest on the Industrial Computer and secure TLS/SSL tunnels for cloud communication [32][33].

Bibliography

- [1] Elsevier ScienceDirect, "Information Systems and IoT Applications in Agriculture", Elsevier ScienceDirect, 2015. [Online]. Available: <https://www.sciencedirect.com/science/article/abs/pii/S1537511015301914>. [Accessed: Feb. 10, 2026].
- [2] Elsevier ScienceDirect, "IoT-Enabled Smart Farming Systems and Performance Optimization", Elsevier ScienceDirect, 2023. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S240584402305867X>. [Accessed: Feb. 11, 2026].
- [3] Elsevier ScienceDirect, "Operational Efficiency and Logistics Optimization in Agri-Food Systems", Elsevier ScienceDirect, 2025. [Online]. Available: <http://www.sciencedirect.com/science/article/pii/S0377221725009993>. [Accessed: Feb. 12, 2026].
- [4] Elsevier ScienceDirect, "Smart Mobility Solutions for Agricultural Environments", Elsevier ScienceDirect, 2025. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0040162525002586>. [Accessed: Feb. 12, 2026].
- [5] World Health Organization, "Global Health Observatory: Road Safety Data", WHO, 2023. [Online]. Available: <https://www.who.int/data/gho/data/themes/road-safety>. [Accessed: Feb. 13, 2026].
- [6] ResearchGate, "Robots in Agriculture: Revolutionizing Farming Practices", ResearchGate, 2024. [Online]. Available: https://www.researchgate.net/publication/381599567_Robots_in_Agriculture_Revolutionizing_Farming_Practices. [Accessed: Feb. 13, 2026].
- [7] Elsevier ScienceDirect, "Advanced Sensing and Analytics for Precision Agriculture", Elsevier ScienceDirect, 2024. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S2214317326000107>. [Accessed: Feb. 14, 2026].
- [8] Elsevier ScienceDirect, "Data-Driven Decision Support in Smart Agriculture", Elsevier ScienceDirect, 2025. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S2542660525002185>. [Accessed: Feb. 14, 2026].
- [9] Elsevier ScienceDirect, "Application of Navigation Technology in Agricultural Machinery: A Review and Prospects", Elsevier ScienceDirect, 2026. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S2589721725000832>. [Accessed: Feb. 15, 2026].
- [10] ResearchGate, "A Comprehensive Review of Agricultural Ground Automatic Navigation Systems Based on Multi-Sensor Fusion", ResearchGate, 2025. [Online]. Available: <https://www.researchgate.net/publication/395754429>. [Accessed: Feb. 15, 2026].
- [11] Elsevier ScienceDirect, "IMU-Based Motion Control for Agricultural Vehicles", Elsevier ScienceDirect, 2018. [Online]. Available: <https://www.sciencedirect.com/science/article/abs/pii/S0921889018300757>. [Accessed: Feb. 15, 2026].
- [12] Elsevier ScienceDirect, "Wheel Encoder Applications in Mobile Robotics", Elsevier ScienceDirect, 2024. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S2590123024005231>. [Accessed: Feb. 16, 2026].
- [13] MDPI, "LiDAR-Based Environmental Mapping for Precision Agriculture", MDPI Remote Sensing, 2023. [Online]. Available: <https://www.mdpi.com/2072-4292/15/22/5402>. [Accessed: Feb. 16, 2026].
- [14] Elsevier ScienceDirect, "Proximity Sensors for Safe Autonomous Operation", Elsevier ScienceDirect, 2020. [Online]. Available: <https://www.sciencedirect.com/science/article/abs/pii/S0959652620312282>. [Accessed: Feb. 16, 2026].
- [15] Agtonomy, "Autonomous Robot Technology for Agriculture", Agtonomy, 2025. [Online]. Available: <https://www.agtonomy.com/>. [Accessed: Feb. 17, 2026].
- [16] EarthSense, "TerraSentia Agricultural Field Robot Platform", EarthSense Inc., 2024. [Online]. Available: <https://www.earthsense.co/terrasentia>. [Accessed: Feb. 17, 2026].
- [17] P. Tang et al., "Application of IoT Wireless Communication Technology in Agricultural Irrigation Management", Sustainability, 2024. [Online]. Available: <https://www.mdpi.com/2071-1050/16/9/3575>. [Accessed: Feb. 17, 2026].

- [18] N. R. Gatkai et al., "Review of IoT and Electronics Enabled Smart Agriculture", IJABE, 2024. [Online]. Available: <https://www.ijabe.org/index.php/ijabe/article/view/8316>. [Accessed: Feb. 17, 2026].
- [19] Scientific African, "Systematic Review of IoT Technologies for Smart Agriculture", Scientific African, 2023. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S2468227623001577>. [Accessed: Feb. 18, 2026].
- [20] D. Krekovic et al., "Edge-Based Predictive Data Reduction for Smart Agriculture", arXiv, 2025. [Online]. Available: <https://arxiv.org/abs/2511.19103>. [Accessed: Feb. 18, 2026].
- [21] A. K. Sinha, "Multicloud-Powered Agriculture Through IoT and Analytics", IJSCSEIT, 2025. [Online]. Available: <https://www.cseit.org/index.php/CSEIT/article/view/CSEIT251112130>. [Accessed: Feb. 18, 2026].
- [22] A. K. Vedantham, "Cloud and IoT Technologies in Precision Agriculture", IJSCSEIT, 2024. [Online]. Available: <https://www.cseit.org/index.php/CSEIT/article/view/CSEIT241061140>. [Accessed: Feb. 18, 2026].
- [23] MDPI, "Human–Computer Interaction in Smart Greenhouses", Computers, 2025. [Online]. Available: <https://www.mdpi.com/2073-431X/14/12/553>. [Accessed: Feb. 18, 2026].
- [24] Bharat Skytech, "Agriculture Drone Components and Ground Stations", Bharat Skytech, 2026. [Online]. Available: <https://bharatskytech.com/>. [Accessed: Feb. 18, 2026].
- [25] RSIS International, "Edge AI and IoT for Real-Time Crop Disease Detection", RSIS International Journal, 2025. [Online]. Available: <https://rsisinternational.org/journals/ijrias/DigitalLibrary/volume-10-issue-7/1011-1027.pdf>. [Accessed: Feb. 18, 2026].
- [26] ResearchGate, "Human–Computer Interaction in Agricultural User Interfaces", ResearchGate, 2024. [Online]. Available: <https://www.researchgate.net/publication/378606995>. [Accessed: Feb. 18, 2026].
- [27] Promwad, "Universal Embedded HMI Platform for Machinery", Promwad Engineering, 2025. [Online]. Available: <https://promwad.com/case-studies/hmi-development>. [Accessed: Feb. 18, 2026].
- [28] PeerJ Computer Science, "Cloud-Integrated Drone Autonomy Architecture", PeerJ, 2025. [Online]. Available: <https://peerj.com/articles/cs-3238/>. [Accessed: Feb. 18, 2026].
- [29] UNL Digital Commons, "Embedded Control Architectures for Agricultural Robots", University of Nebraska, 2025. [Online]. Available: <https://digitalcommons.unl.edu/electengtheses/1122/>. [Accessed: Feb. 18, 2026].
- [30] ResearchGate, "Fuzzy Control for Agricultural Vehicle Depth", ResearchGate, 2025. [Online]. Available: <https://www.researchgate.net/publication/286570093>. [Accessed: Feb. 18, 2026].
- [31] DLR, "Self-Organizing Cyber-Physical Systems for Sustainable Agriculture", German Aerospace Center, 2025. [Online]. Available: <https://elib.dlr.de/217181/1/s43926-025-00205-6.pdf>. [Accessed: Feb. 18, 2026].
- [32] Farmonaut, "Security and Privacy in Smart Farming: 2026 Challenges", Farmonaut, 2026. [Online]. Available: <https://farmonaut.com/precision-farming/security-and-privacy-in-smart-farming-2026-challenges>. [Accessed: Feb. 18, 2026].
- [33] ResearchGate, "Cybersecurity in Precision Agriculture", ResearchGate, 2026. [Online]. Available: <https://www.researchgate.net/publication/388105508>. [Accessed: Feb. 18, 2026].
- [34] u-blox, "Autonomous Agriculture Safety and ISO 25119 Compliance", u-blox, 2026. [Online]. Available: <https://www.u-blox.com/en/autonomous-agriculture-functionally-safe-gnss>. [Accessed: Feb. 18, 2026].
- [35] SERMA Safety, "ISO 25119 Compliance for Agricultural Machinery", SERMA Safety & Security, 2026. [Online]. Available: <https://www.serma-safety-security.com/en/iso-25119-compliance-agricultural-machinery/>. [Accessed: Feb. 18, 2026].

Business Domain and Technology Application

Business Context Analysis

The proposed solution belongs to the domain of **precision agriculture**, where farms use sensor networks and digital monitoring platforms to support operational decisions. Modern agricultural enterprises operate under increasing pressure to improve yields while reducing resource consumption and labor costs. Traditional farm management practices rely heavily on periodic field inspections and farmer experience, which makes decision-making reactive rather than data-driven. According to the FAO, precision agriculture is defined as a management strategy that uses detailed, site-specific information to optimize productivity and resource efficiency [1]. This approach has become increasingly important as farms grow in scale and climate conditions become less predictable.

Current Workflows and Operational Challenges

In current agricultural workflows, most monitoring and decision-making tasks are performed manually. Soil conditions are typically evaluated by visual inspection or occasional measurements, while irrigation is often scheduled at fixed intervals regardless of actual soil moisture levels. Farmers may inspect fields once or twice per day, or even less frequently in large operations. This workflow introduces delays between environmental changes and operational responses. For example, soil moisture can drop below optimal levels hours or days before irrigation is applied, creating stress conditions that reduce yield. Similarly, excessive watering often occurs because farmers apply safety margins to avoid drought damage. Studies of traditional irrigation practices report that fixed irrigation schedules frequently lead to significant water waste and inconsistent plant growth [2].

Another limitation of traditional workflows is the lack of centralized data storage. Measurements are often recorded on paper or not recorded at all, making long-term analysis impossible. Without historical environmental data, farmers cannot identify patterns such as seasonal soil behavior or irrigation inefficiencies. This limits the ability to optimize farm operations and increases dependence on manual experience rather than measurable evidence [3]. In large farms, manual monitoring also requires significant labor, as workers must physically visit multiple fields to collect information. These workflows increase operational costs and reduce responsiveness to sudden environmental changes.

Potential Use Cases of the Proposed Solution

IoT-based agricultural systems address these limitations by introducing continuous monitoring and automated decision support. Instead of periodic measurements, wireless sensor networks collect environmental data such as soil moisture, temperature, humidity, and weather conditions at regular intervals. The collected data is transmitted to centralized platforms where it can be analyzed and visualized in real time. This changes the workflow from manual inspection to continuous observation. Farmers no longer need to rely on field visits to determine conditions; instead, they can observe environmental parameters remotely and respond immediately when abnormal conditions appear [3].

One of the most widely studied applications of agricultural IoT technology is smart irrigation. In these systems, soil moisture sensors continuously measure water availability in the soil and transmit readings to a control platform. Irrigation equipment is then activated only when moisture levels drop below defined thresholds. A wireless sensor irrigation system studied in Scientific Reports demonstrated that data-driven irrigation scheduling increased crop yield by **12.0 5% while reducing water consumption by 35.2%** compared to traditional irrigation methods [4]. These improvements occurred because irrigation was applied precisely when needed rather than according to fixed schedules. Crops maintained stable moisture levels without periods of water stress or excessive saturation.

Similar results were observed in a smart irrigation platform developed using IoT sensors and predictive algorithms. The system monitored environmental conditions and predicted irrigation requirements based on soil and weather data. Field tests showed approximately **30% reduction in water usage**, while irrigation predictions remained accurate within **5% deviation** from measured soil conditions [5]. This level of precision is difficult to achieve with manual monitoring because environmental variables change continuously throughout the day.

Real-time environmental monitoring also improves operational awareness by allowing farmers to detect problems earlier. A cloud-connected monitoring system described in IoT agriculture research transmitted soil and climate measurements to a web platform accessible through mobile devices. Farmers could observe field conditions remotely and receive alerts when values exceeded safe ranges [6]. Continuous monitoring reduces the time between problem occurrence and corrective action. Instead of discovering drought stress during the next field inspection, farmers can react immediately when sensor readings indicate abnormal conditions.

Data collected from sensors can also support advanced decision-making through analytics and machine learning models. Decision-support platforms analyze historical environmental data to identify patterns and optimize farm operations. Research on data-driven irrigation management showed that analytics-based systems reduced water waste by approximately **25% while increasing crop yields by about 20%** through optimized irrigation timing and improved resource allocation [7]. These improvements are achieved because data analysis identifies relationships between environmental conditions and plant growth that are not easily observable through manual monitoring.

Automation further enhances the benefits of IoT-based agricultural systems by reducing the need for manual intervention. In automated irrigation systems, sensor measurements directly control irrigation equipment through cloud-based controllers. A cloud-connected monitoring and irrigation system maintained optimal soil moisture conditions with approximately **92% control accuracy** and reduced water consumption by nearly **40%** compared to manual irrigation practices [2]. This type of automation ensures consistent operating conditions even when farm personnel are not present on site.

Expected Business and Operational Benefits

These technological improvements translate directly into business benefits. Increased yields improve revenue per hectare, while reduced water consumption lowers operating costs. For farms operating in regions with limited water resources, irrigation efficiency can determine long-term sustainability. Precision agriculture technologies enable farmers to apply exactly the required amount of water and fertilizer, reducing unnecessary input costs while maintaining optimal plant growth conditions [1].

Operational control is also significantly improved through centralized monitoring platforms. Farmers can observe environmental conditions across multiple fields from a single interface, eliminating the need for frequent physical inspections. Wireless sensor networks allow environmental data to be transmitted continuously to central servers where it can be stored and analyzed over long periods [3]. Historical datasets allow farmers to evaluate past decisions and refine management strategies over time. Instead of relying on intuition, farm management becomes evidence-based and measurable.

Existing Implementations and Business Cases

Case Study 1 – Large-Scale Precision Agriculture Farm (Iowa, USA)

A large-scale **5,000-acre corn and soybean farm in Iowa** implemented precision agriculture technologies including soil sensors, satellite crop monitoring, and farm analytics software. The system used variable-rate fertilization and real-time environmental monitoring to optimize planting density, irrigation, and nutrient application.

Measured results included:

- **28% increase in corn yield**
- **30% reduction in energy consumption**
- **20% reduction in fertilizer costs**
- **Approximately \$50,000 annual operational savings**

These improvements were achieved through targeted fertilizer application using Variable Rate Technology (VRT), combined with satellite-based crop monitoring and soil moisture sensing. The integration of sensor data with farm management software allowed operators to identify field zones requiring additional nutrients while avoiding over-application in other areas.

Technology	Impact on Yield (%)	Water Savings (%)
Soil Moisture Sensors	Up to 20	30
GPS Mapping	10-15	15
Drones for Crop Monitoring	10-12	20

According to the documented implementation, precision management enabled farmers to make **more accurate decisions regarding crop rotation and field treatment**, significantly improving productivity and reducing operational costs [9].

This case study demonstrates the economic value of data-driven agricultural management, particularly for large commercial farms where small efficiency improvements translate into significant financial savings.

Case Study 2 – Vineyard Monitoring System Using IoT Sensors (California, USA)

Commercial vineyards increasingly rely on IoT monitoring technologies to optimize irrigation and improve crop quality. A vineyard deployment in **Napa Valley, California** implemented wireless environmental sensors distributed across vine rows to monitor temperature and plant stress conditions in real time.

The sensor network provided continuous environmental data that was transmitted to a cloud-based platform where stress zones could be identified and irrigation schedules adjusted accordingly. The system enabled vineyard managers to make targeted interventions instead of applying uniform treatments across the entire field.

Field implementations of precision viticulture technologies have reported measurable improvements, including:

- Approximately **15% increase in grape yield** through optimized growing conditions
- Up to **25% reduction in water use** using sensor-based irrigation scheduling
- Reduced chemical usage through targeted treatments
- Improved crop quality through stable environmental conditions

Sensor-based irrigation systems and precision crop monitoring allow vineyard operators to maintain optimal moisture conditions and detect plant stress early. Continuous environmental monitoring enables more precise irrigation control compared to traditional periodic inspections [10][11].

Technology	Primary Use	Range	Accuracy	Power Efficiency	Ideal Environment/ Application
BLE	Useful for close-range plant and equipment monitoring within vine rows.	Short to medium	Moderate	High	Vine rows, equipment tagging, proximity-based monitoring
RFID	Ideal for tracking grape bins, equipment check-in/out, and storage workflows.	Short to long (passive/active)	High	Very high (passive), medium (active)	Storage barns, harvest stations, equipment control points
LoRaWAN	Excellent for long-range field monitoring across large vineyards and hilly terrain.	Very long	Low to moderate	Very high	Large vineyards, hilly or remote blocks, long-range soil/environment sensors
NB-IoT	Effective for soil probes and remote microclimate sensors in areas with deep penetration needs.	Very long	Moderate	High	Remote field sensors, buried probes, low-signal areas
Cellular IoT	Suitable for machinery tracking and wide-area monitoring.	Very long	Moderate	Medium to high	Tractors, sprayers, large vineyard fleets
GPS-IoT	Essential for tractor, ATV, and harvest crew tracking.	Global	Moderate	Medium	Moving vehicles, field crew tracking, harvest coordination
Wi-Fi HaLow	Strong for indoor winery operations with long-range wireless access.	Long (indoor)	Moderate	High	Indoor winery buildings, barrel rooms, processing facilities
Zigbee	Best for dense mesh networks covering compact vineyard clusters or processing zones.	Short to medium	Moderate	High	Close-range vine clusters, packing areas, indoor processing zones

This example demonstrates how IoT-based monitoring systems improve both productivity and product quality in commercial agriculture while reducing resource consumption.

Case Study 3 – IoT Agricultural Robotic Monitoring Platform

Mobile sensing platforms represent another important direction in agricultural IoT development. Autonomous agricultural robots equipped with sensors and cloud connectivity can perform continuous crop monitoring while collecting environmental and plant-condition data.

Recent research on robotic agricultural monitoring platforms describes systems integrating:

- Multimodal environmental sensors
- Real-time wireless data transmission
- Cloud-based storage and analysis
- Remote monitoring interfaces

Integrated robotic sensing platforms allow automated data collection in environments where fixed sensor networks may be difficult to deploy. Field studies show that robotic monitoring systems can collect high-resolution environmental data while maintaining reliable operation in real agricultural environments [12][13].

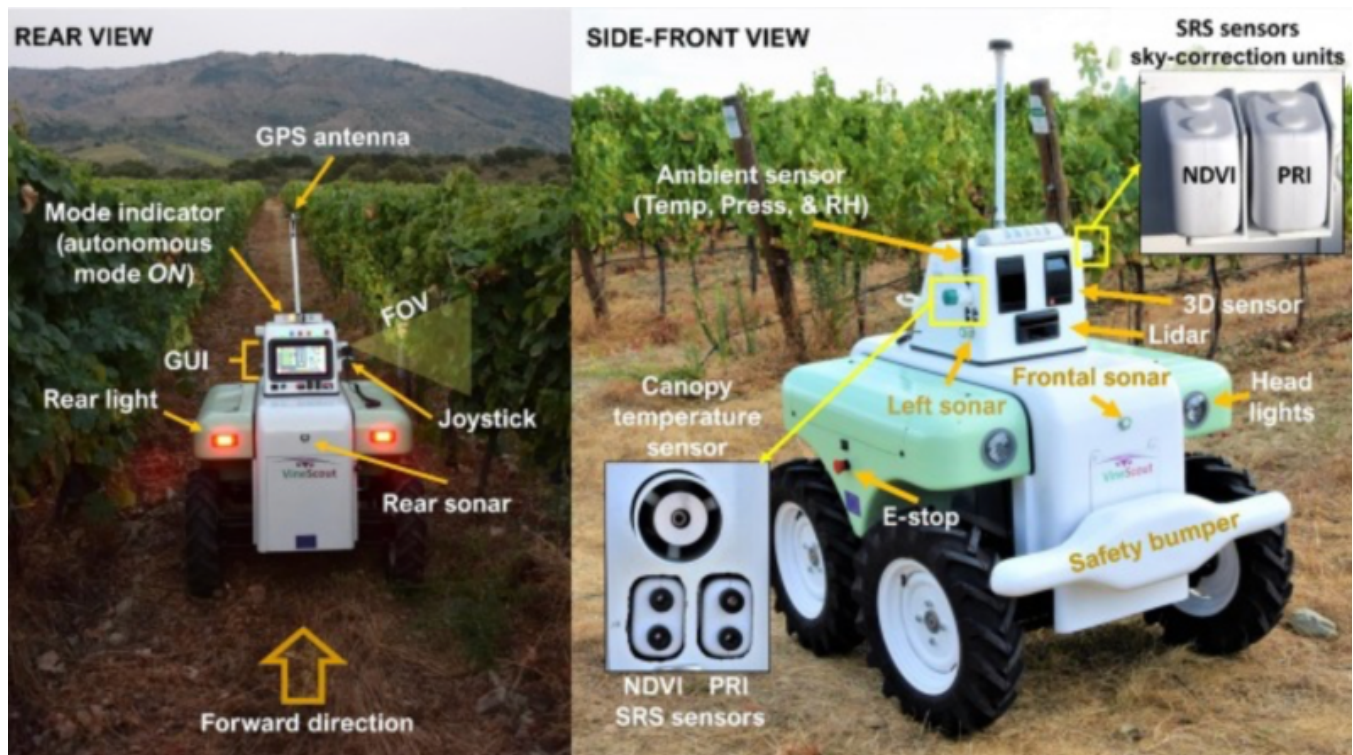


Fig.1 Image illustrating Main components of the Autonomous Ground Vehicle

Robotic systems have demonstrated measurable operational performance. For example as in Fig.1:

- Autonomous vineyard robots achieved approximately **87% operational accuracy** in field tasks such as pruning and plant detection [13]
- Robotic sensing platforms enabled high-resolution environmental mapping with strong correlation between sensor measurements and plant water conditions (R^2 up to **0.69–0.75**) [12]

These systems demonstrate the feasibility of mobile IoT sensing platforms capable of performing continuous agricultural monitoring.

The integration of mobile sensors with cloud platforms allows real-time data transmission and remote supervision, making robotic sensing platforms a practical extension of traditional IoT-based monitoring systems.

This implementation demonstrates that mobile IoT sensing platforms can complement stationary sensor networks and extend monitoring coverage across large agricultural areas.

Real-world implementations demonstrate that these benefits are achievable in practice. Field deployments of IoT irrigation systems consistently report water savings between **30% and 40%** while maintaining or improving crop yields [4][5][8]. These results confirm that IoT-based agricultural monitoring systems are not only technically feasible but also economically beneficial. The combination of continuous monitoring, automated control, and data analytics allows farms to operate more efficiently while reducing environmental impact.

Overall, agricultural IoT systems transform traditional farming workflows from manual and reactive processes into automated and data-driven operations. Continuous monitoring improves environmental visibility, automation reduces labor requirements, and data analytics enables more accurate decision-making. These improvements lead to measurable increases in productivity and resource efficiency while providing farmers with greater operational control.

To identify relevant technological solutions similar to the proposed Intelligent Farm Golf Cart, it is essential to define what constitutes a comparable system. For the purposes of this analysis, “similar systems” are interpreted broadly as autonomous or semi-autonomous mobile platforms used in agricultural or outdoor environments that incorporate key elements of mobility, autonomy, IoT integration, and cloud connectivity. This includes autonomous agricultural vehicles such as driverless tractors and field robots that perform tasks without direct human control; smart farm transport robots designed to carry produce, tools, or supplies across farmland; IoT-based farm logistics platforms that connect physical machines to cloud services for monitoring, data analytics, and remote control; and autonomous utility vehicles for outdoor environments.

The comparison in this section focuses on five main dimensions: mobility, referring to the vehicle's ability to traverse uneven terrain and execute transport tasks; autonomy, indicating the level of decision-making and path planning implemented; IoT integration, which encompasses the sensing, communication, data transmission components; cloud connectivity, enabling remote monitoring, analytics, and user interaction; use cases in smart agriculture, such as payload transport, route automation, and farm logistics optimization. By clarifying this scope, the analysis can more accurately position existing products and identify gaps that the Intelligent Farm Golf Cart aims to address.

Case Study 4 – Agtonomy Robot: Autonomous Electric Farm Vehicle

Agtonomy offers an autonomous electric agricultural robot designed to perform multiple tasks, including transport, mowing, and spraying by using GPS guidance and vision-based navigation. "Manage remotely with the TeleFarmer App" [14], cellular connectivity, and AI-driven autonomy to reduce the need for constant human supervision in specialty crops and field situations. The system integrates sensors such as GNSS and vision systems and connects to farm software ecosystems for remote control and data collection. Agtonomy's product is commercially available and positioned as a flexible autonomous solution for modern farms.[14]

Key Capabilities

- "Fully Electric: Operates on electric power, contributing to environmentally friendly farming practices and reducing emissions." [14]
- "Autonomous Navigation: Navigates autonomously using GPS guidance and vision-based navigation, minimizing the need for constant human supervision." [14]
- "TeleFarmer App: Allows farmers to remotely manage and monitor farm operations, providing real-time control and insights from anywhere." [14]
- "Connectivity: Integrated cellular and Starlink connectivity ensures reliable communication and data transfer, even in remote locations." [14]



Fig.1 Agtonomy Robot: Autonomous Electric Tractor



Fig.2 Agtonomy Robot: Autonomous Electric Tractor

Company: MOOCO ROBOTICS, INC.

Product: Agtonomy Robot

Core functionality: Autonomous navigation for multiple field tasks, including transport and spraying.

Technology used: GPS guidance, vision-based navigation, physical AI, cellular connectivity.

Deployment status: Commercial service available.

Case Study 5 — XAG R100 Agricultural Utility Vehicle

The XAG XAUV R100 series is a fully electric autonomous ground robot designed to adapt to plain fields and sloped terrains, carry payloads for transport, and support implements such as spraying systems or field sensors. It integrates RTK GNSS navigation, automated steering systems, and can operate with third-party payloads for diverse farm tasks. As part of XAG's smart agriculture ecosystem, it also connects with IoT tools for precise field operations and data collection. The R80 is already marketed and deployed in agriculture as a multifunctional autonomous platform.

Key Capabilities

- Autonomous Navigation: "The vehicle can operate semi-autonomously or fully autonomously depending on mission setup." [15]
- Path Tracking: "Vision algorithms detect and automatically correct deviations in real time, keeping the rover precisely on course." [15]
- AI Safety Assistance: "Automatically detect pedestrians ahead, alert at 4m and stop at 2m to ensure safety." [15]
- "Support Modular Add-ons for Endless Possibilities" [15]

There are three possible modular configurations for constructing the vehicle.



Fig. 3 XAG R100 Agricultural Utility Vehicle(1)



Fig. 4 XAG R100 Agricultural Utility Vehicle(2)



Fig. 5 XAG R100 Agricultural Utility Vehicle(3)

Case Study 6 — ASI Autonomous Farming Solutions (Mobius Platform)

Autonomous Solutions, Inc. (ASI) provides autonomy kits and robotic platforms that retrofit existing farm vehicles to operate autonomously. The Mobius platform enables remote control, obstacle detection, and fleet management, allowing tractors, sprayers, or mowers to be controlled from a central interface. It emphasizes scalable IoT connectivity and automation of traditional equipment across large farms. The solution is targeted at industrial and precision agriculture customers and is commercially supported.

Key Capabilities

- Remote Operation: operators can control vehicles from a centralized command center rather than sitting inside the tractor cab.
- Obstacle Detection and Safety: "The system integrates sensors (such as GNSS, LiDAR, radar, and vision systems, depending on configuration) to detect obstacles and ensure safe navigation."[16]
- Geofencing and Boundary Management: "Operations can be restricted to defined field boundaries to improve safety and precision."[16]

Company: Autonomous Solutions, Inc.

Product: Mobius Autonomous Agriculture Platform

Core functionality: Fleet autonomy and remote operation of farm vehicles.

Technology used: GNSS, obstacle detection, and central fleet control (IoT).

Deployment status: Commercially available retrofit kits and platforms.

Case Study 7 — AgriMOBY Multipurpose Autonomous Farm Vehicle (Prototype)

AgriMOBY is a research project developing a multipurpose, all-terrain autonomous vehicle capable of flexible use cases, including cargo transport, inspection, and field monitoring. It focuses on adaptability through an omnidirectional chassis design and aims to demonstrate autonomous logistics capabilities in horticulture and vineyard contexts. Though still in research and development, the project illustrates emerging trends toward autonomous transport robots in agriculture. Approximate dimensions 110 x 110 x 40 cm.

Key Capabilities

- Autonomous Navigation: Movement in multiple directions without conventional steering limitations
- Maneuverability: "Improved maneuverability in narrow vineyard rows."[17]
- Autonomy: "Increased the degree of energy optimisation and achieved much more efficient use of energy resources."[17]



Fig. 6 AgriMOBY Multipurpose Autonomous Farm Vehicle (Prototype)

Comparative Analysis: Intelligent Farm Golf Cart vs. Existing Solutions

Attribute	Intelligent Farm Golf Cart (Proposed)	Agtonomy Autonomous Farm Robot	XAG XAUV R80	ASI Mobius Autonomous Platform	AgriMOBY (Prototype)
Target Customers	Small & medium farms research labs	Commercial farms & specialty crop growers	Large, high-tech farms & enterprises	Industrial farms / large operations	Research institutions, prototypes
Level of Autonomy	Semi-autonomous to autonomous (configurable)	Autonomous with teleoperation override	High autonomy with RTK path following	Variable autonomy (retrofit)	Experimental autonomy
Core Functionality	Transport payloads, route scheduling, IoT telemetry	Multi-task field robot (transport, spraying)	All-terrain autonomous transport & payloads	Fleet autonomy & remote control	Experimental transport & inspection
Sensor Suite	GNSS/RTK, IMU, LiDAR/ ultrasonic, camera, bump & agri/environment sensors	GPS + vision (no public LiDAR)	RTK GNSS, depth sensors, and optional LiDAR	GNSS + obstacle sensors	Prototype sensors depend on configuration
Connectivity	Wi-Fi / BLE for config, 4G/5G or LoRaWAN for telemetry, MQTT/REST	GSM/4G teleoperation, cloud	Cellular, IoT ecosystem	On-site network, cloud management	Research communications
Cloud Integration	Full IoT + cloud dashboard + analytics	Cloud teleoperation & data logs	Integrated farm ecosystem	Central fleet control dashboard	Not focused on cloud
Safety Systems	LiDAR/Ultrasonic + bump sensors + emergency stop logic	Vision + obstacle avoidance	Integrated obstacle systems	Obstacle detection + safety overrides	Limited (prototype focus)
Payload Capacity	Designed for farm tools & medium loads	Medium (field dependent)	Higher capacity (all-terrain)	Varies (retrofitted vehicles)	Experimental
Scalability	Designed for scalable deployments across small/medium farms	Commercial but complex pricing	Enterprise scale	Retrofit integration across vehicle fleets	Research scalability, not commercial
Moldova Applicability (Ease of Adoption)	High (modular, affordable, customizable)	Medium (requires tech skills & network)	Low-Medium (high cost, large farms)	Medium (integration barriers)	Low (R&D stage)
Commercial Status	Development/prototype	Commercial	Commercial	Commercial	Research prototype

Gaps in Current Solutions vs. Proposed Intelligent Farm Golf Cart

Feature / Requirement	Agtonomy Robot	XAG XAUV R80	ASI Mobius Platform	AgriMOBY Prototype	Intelligent Farm Golf Cart (Proposed)	Gap Addressed
Affordable for Small Farms	Medium (high cost)	Low (expensive)	Medium (integration cost)	Low (prototype)	High (low-cost modular)	Accessible to SM farms
Flexible Autonomy Modes	High autonomy	High autonomy	Dependent on retrofit	Experimental	Assisted in full autonomy options	Adjustable for user needs
Low-power Connectivity (LoRaWAN)	x	x	x	x	LoRa + cellular	Works in low coverage areas
Cloud Analytics & Dashboard	Partial (telemetry)	Medium (ecosystem)	Fleet control only	None	Full telemetry + analytics	Better decision support
Modular Payload Support	Medium	High	Retrofit dependent	Research	Modular farm attachments	Farm-specific flexibility

Safe Short-range Perception	Vision + GPS	Integrated	Obstacle sensors	Limited	LiDAR + ultrasonic + bump	Multi-layer safety
Designed for Local / SMEs (e.g., Moldova)	Medium	Low	Medium	Low	Tailored for small/medium farms	Better fit for local markets
Simple User Interface	Medium	Medium	Medium	Experimental	User-friendly IoT dashboard	Easier adoption

Stakeholder Management Approach

Stakeholder Identification Process

The project focuses on the design and implementation of an Intelligent Farm Golf Cart, an IoT-based autonomous vehicle intended to support transportation and logistics operations in agricultural environments. The system is designed to carry tools, harvested crops, or supplies across farms, while integrating sensing, communication, and autonomous navigation technologies.

The solution targets small to medium-sized farms and aims to reduce manual labor, improve operational efficiency, and increase safety in routine transportation tasks.

Stakeholder Table

Stakeholder Group	Stakeholder	Description	Level of Interest	Level of Influence
Primary Users	Farmers (Small-scale/ Large-scale)	Farmers are the main end users. The system directly affects their productivity, labor efficiency, and operational costs.	High	Medium
Technical Team	Software Developers	Software developers design and implement the autonomy algorithms, cloud services, and user interfaces of the system. They are responsible for system logic, data processing, and overall digital functionality.	High	High
Technical Team	Hardware Engineers	Hardware engineers are responsible for integrating sensors, actuators, embedded systems, and power components. They ensure that the physical system operates safely and reliably in real farm conditions.	High	High
Institutional Bodies	Local Agricultural Agencies	They provide guidance, validation, or recommendations for adopting new technologies.	Medium	Medium
Business Partners	Agri-Tech Companies	Agri-tech companies may provide technology, expertise, or integration platforms for smart farming solutions. They are interested in commercial opportunities and technology transfer.	Medium	Medium
Business Partners	Cloud Service Providers	Cloud providers offer infrastructure for data storage, analytics, and system monitoring. They enable remote access and scalability of the platform.	Medium	Medium
Financial Stakeholders	Investors / Funding Organizations	Investors provide financial resources needed for development, testing, and potential commercialization. Their interest lies in return on investment and system scalability.	High	High
Research Institutions	Technical University of Moldova	The Technical University of Moldova supports the project through academic supervision, infrastructure, and research expertise. It ensures scientific validity and educational value.	High	High
Society	Consumers	Consumers benefit indirectly through more efficient food production and potentially lower costs. Their interest is mainly related to food availability and quality.	Medium	Low

Criteria for Interest and Influence Classification

Criteria for "Interest"

The level of interest is determined based on the following concrete factors:

Degree of dependency on the system - how much the stakeholder relies on the intelligent golf cart for daily operations or strategic objectives.

Economic impact - financial benefits, cost reductions, or risks associated with the system.

Operational integration - the extent to which the stakeholder integrates the system into their workflow.

Strategic alignment - whether the project supports their institutional, academic, or commercial goals.

Examples:

- Farmers have High interest because their productivity, labor efficiency, and operational costs directly depend on the system.
- Software and hardware engineers have High interest because system success reflects their technical work and evaluation.

Criteria for "Influence"

The level of influence is determined using measurable indicators such as:

Financial power – ability to fund or withdraw funding.

Regulatory authority – power to approve, regulate, or restrict deployment.

Technical control – responsibility for system architecture and performance.

Market power – ability to affect adoption or commercialization.

Blocking capacity – ability to delay, limit, or stop implementation.

Examples:

Investors have High influence because funding decisions determine scalability.

The Technical University of Moldova has a strong influence due to its academic control and evaluation authority.

Consumers have Low influence because they cannot directly affect system design or deployment decisions.

Why were these stakeholders selected?

Primary Users

- Farmers (Small-scale and Large-scale) were selected because they are the main end users whose productivity, operational efficiency, and economic performance depend directly on the intelligent golf cart system. Their level of adoption and satisfaction ultimately determines the practical success and commercial viability of the solution.

Technical Team

- Software Developers were selected because they are responsible for designing the system architecture, autonomy algorithms, cloud services, and user interfaces. Their technical expertise directly influences system functionality, performance, and reliability.

- Hardware Engineers were selected because they design and integrate the physical components of the system, including sensors, actuators, embedded platforms, and power systems. The safety, robustness, and real-world feasibility of the intelligent cart depend on its engineering decisions.

Institutional Bodies

- Local Agricultural Agencies were selected because they support the modernization of agricultural practices and can facilitate the adoption of smart technologies at the regional level. Their involvement helps align the project with agricultural policies and best practices.

Business Partners

- Agri-Tech Companies were selected because they provide technological solutions, expertise, and potential pathways for system integration and commercialization. Their participation increases the scalability and market relevance of the project.

- Cloud Service Providers were selected because they supply the infrastructure required for data storage, analytics, remote monitoring, and system scalability. Without cloud services, real-time data processing and system expansion would not be feasible.

Financial Stakeholders

- Investors and Funding Organizations were selected because they provide the financial resources required for development, testing, and potential deployment. Their investment decisions directly influence project scope, sustainability, and long-term impact.

Research Institutions

- The Technical University of Moldova was selected because it provides academic supervision, research infrastructure, and technical validation for the project. The university ensures scientific rigor, educational value, and compliance with engineering standards.

Society

- Consumers were selected because they are indirect beneficiaries of agricultural automation, as improvements in logistics and productivity can lead to better food quality, availability, and reduced production costs. Although they do not interact with the system directly, they are affected by its broader societal impact.

Stakeholder Mapping Methodology

High Influence – High Interest (Manage Closely)

The “Manage Closely” quadrant includes stakeholders who have both strong decision-making power and a high level of involvement in the project. This group consists of software developers, hardware engineers, investors, and the Technical University of Moldova. These stakeholders directly influence system architecture, funding, research direction, and overall project success.

High Influence – Low/Medium Interest (Keep Satisfied)

The “Keep Satisfied” quadrant includes stakeholders who have high power but lower direct involvement in daily project operations. Local agricultural agencies influence policy alignment and technology adoption at the regional level, while cloud service providers supply critical digital infrastructure for data processing and system scalability. Although they do not manage the system directly, their institutional and technical authority can significantly affect deployment conditions.

Low/Medium Influence – High Interest (Keep Informed)

The “Keep Informed” quadrant includes stakeholders who are highly interested in the project outcomes but have limited influence over strategic decisions. Farmers are strongly affected by the system, as it directly impacts their productivity and workflow. Agri-tech companies are also interested in technological collaboration and potential commercialization opportunities. These stakeholders should be regularly informed through demonstrations, feedback sessions, and performance reports to ensure their needs are reflected in system design.

Low Influence – Low/Medium Interest (Monitor)

The “Monitor” quadrant includes stakeholders who are indirectly affected by the project and have minimal influence over its development. Consumers fall into this category, as they benefit from agricultural automation through improved food production and efficiency. However, they do not interact with the system or influence its design. These stakeholders require minimal engagement, mainly through general communication about project outcomes and societal benefits.

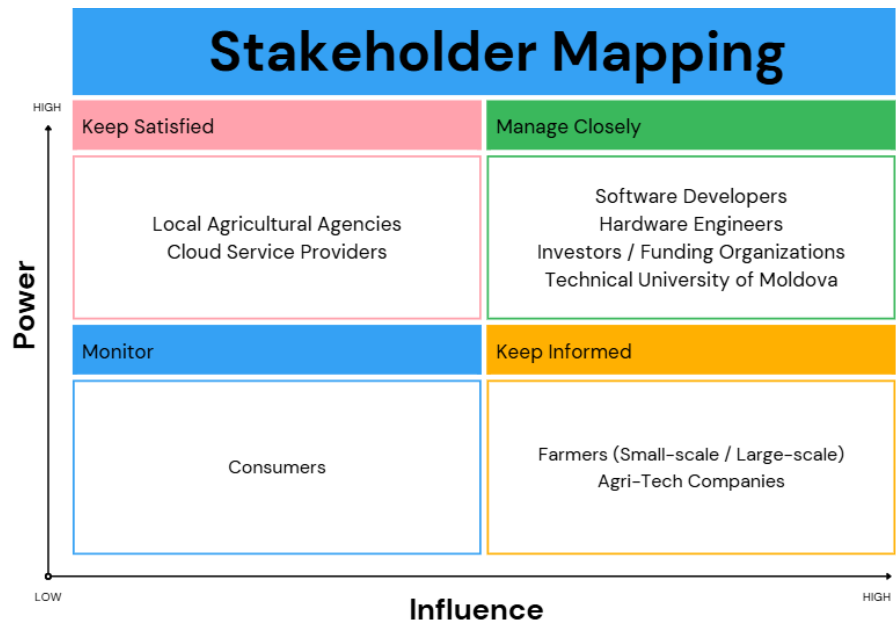


Fig.7 Stakeholder Mapping Methodology

References

[1] FAO, *Precision Agriculture for Smallholder Farmers*
<https://www.fao.org/family-farming/detail/en/c/1738176/>

[2] Hasib A., Akib A., *An IoT-Based Smart Plant Monitoring and Irrigation System*
<https://arxiv.org/abs/2601.15830>

[3] IJERT, *Smart Agriculture Monitoring System Based on IoT*
<https://www.ijert.org/smart-agriculture-monitoring-system-based-on-internet-of-things-iot>

[4] Scientific Reports, *Wireless Sensor Network for Irrigation Management*
<https://www.nature.com/articles/s41598-025-97303-w>

[5] ScienceDirect, *Smart Agriculture Using IoT for Automated Irrigation*
<https://doi.org/10.1016/j.atech.2025.101081>

[6] Rahman et al., *Soil Characterization through IoT*
<https://arxiv.org/abs/2411.17731>

[7] Springer, *IoT Sensors and Machine Learning for Precision Agriculture*
<https://link.springer.com/article/10.1007/s44279-025-00247-y>

[8] Frontiers in Water, *In-field IoT System for Precision Irrigation*
<https://www.frontiersin.org/articles/10.3389/frwa.2024.1353597>

[9] *Transforming Farm Operations with Big Data: Real-Life Success Stories*
<https://moldstud.com/articles/p-transforming-farm-operations-with-big-data-real-life-success-stories-and-insights>

[10] GAO RFID, *Cloud-Based Vineyard Asset and Yield Monitoring Systems*
<https://gaorfid.com/cloud-based-vineyard-asset-yield-monitoring-systems/>

[11] SIIT, *The Science Behind Precision Agriculture*
<https://siit.co/blog/index.php/how-to-revolutionize-crop-yields-the-science-behind-precision-agriculture/21072>

[12] *Robotics-Based Vineyard Water Potential Monitoring*, Computers and Electronics in Agriculture
<https://www.sciencedirect.com/science/article/pii/S0168169921003288>

[13] Silwal et al., *Autonomous Robotic Vine Pruning System*
<https://arxiv.org/abs/2112.00291>

[14] <https://agtecher.com/en/robotics/agtonomy-robot/>

[15] <https://xa.com/en/r100#part-import-wrap>

[16] <https://asirobots.com/industries/agriculture>

[17] <https://cit.upc.edu/en/portfolio-item/agromoby-a-multipurpose-all-terrain-omnidirectional-autonomous-vehicle-for-agricultural-applications>

1. Farmers (Small-scale / Large-scale)

1. What type of farm do you own, and what size is it?
 2. What are the most common daily transport or movement activities within the farm?
 3. Who usually takes care of these transport activities?
 4. During busy periods, which activities consume the most time?
 5. What difficulties do you encounter when transporting crops, tools, or materials?
 6. Have you had situations where delays occurred because of transport problems? Can you provide a short example?
 7. How does the lack of workforce affect the daily activity of the farm?
 8. What safety risks exist when vehicles or equipment move around the farm?
 9. What daily activities do you consider repetitive or inefficient?
 10. When a problem occurs with a vehicle or transport, how do you usually handle it?
 11. Have you improved any work processes in recent years? What did you change?
 12. What do you consider before investing in new equipment?
 13. How do you decide whether an investment in a new technology is worth making?
 14. Do you currently use smart technologies on the farm? If yes, which ones?
 15. Have you used GPS systems, sensors, or remote monitoring solutions?
 16. What concerns would you have regarding the use of an autonomous vehicle on the farm?
 17. Would you prefer a fully autonomous vehicle or one supervised by an operator? Why?
 18. What types of activities would you be willing to entrust to an autonomous vehicle?
 19. What would make you trust such a system?
 20. What concrete benefits would convince you to adopt this solution?
-

2. Software Developers (Students/Developers)

1. What types of complex software systems have you developed so far?
 2. What experience do you have with distributed architectures or IoT systems?
 3. How would you structure the software architecture for an autonomous vehicle connected to the cloud?
 4. What main components should such a system contain (vehicle, server, web application, etc.)?
 5. How would you separate responsibilities between the edge computing part and the cloud?
 6. What technologies or programming languages do you consider suitable for such a project?
 7. How would you ensure secure communication between the vehicle and the web platform?
 8. What communication protocols would you recommend for real-time data transmission?
 9. How would you handle the loss of internet connection during vehicle operation?
 10. What security measures do you consider essential in an autonomous system connected to the network?
 11. How would you design the system to allow remote software updates?
 12. What methods would you use for error monitoring and problem diagnostics?
 13. How would you ensure system scalability if it is implemented across multiple farms?
 14. What databases would be suitable for storing operational and monitoring data?
 15. How would you integrate data collected from sensors into the analysis and reporting system?
 16. What experience do you have with developing web interfaces for non-technical users?
 17. How would you make the interface easy to use for farmers?
 18. What testing strategies would you apply for a critical system such as an autonomous vehicle?
 19. What do you consider the biggest software challenge in such a project?
 20. From a maintenance perspective, what needs to be planned from the design phase?
-

3. Hardware Engineers

1. What experience do you have with integrating sensors in outdoor or agricultural environments?
 2. What challenges occur most frequently when using sensors in conditions of dust, humidity, or vibrations?
 3. What factors are essential when selecting sensors for an autonomous vehicle?
 4. What types of sensors do you consider necessary for the navigation of an autonomous vehicle on a farm?
 5. How do you ensure hardware reliability in a system that operates continuously outdoors?
 6. What methods do you use to protect electronic components against overheating or moisture?
 7. What are the main technical risks in an autonomous vehicle project from a hardware perspective?
 8. How do you test a hardware system before final implementation in the field?
 9. What solutions would you use for efficiently powering an autonomous vehicle (batteries, energy management)?
 10. How would you approach the problem of energy consumption in a system with multiple sensors and communication modules?
 11. What types of hardware communication do you consider suitable for an agricultural environment (Wi-Fi, LoRa, 4G/5G, etc.)?
 12. What experience do you have with integrating GPS and positioning systems?
 13. How would you handle interference or signal loss in an open field?
 14. What standards or practices do you follow to ensure the electrical safety of the system?
 15. How do you collaborate with the software team for complete hardware–software integration?
 16. What indicators would you use to evaluate the hardware performance of the vehicle?
 17. What hardware costs do you consider critical in such a project?
 18. From your perspective, what is the biggest hardware challenge in developing an autonomous vehicle for a farm?
-

4. Local Agricultural Agencies

1. What is the role of your institution in supporting farmers at the regional level?
 2. What types of agricultural technologies are currently supported or promoted?
 3. Are there specific regulations regarding the use of autonomous vehicles in agriculture?
 4. What procedures should be followed to implement an autonomous vehicle on a farm?
 5. What legal requirements exist regarding the safety of using autonomous equipment?
 6. How are systems based on sensors and remote monitoring treated from a legislative point of view?
 7. Are there funding programs or subsidies for the digitalization of agriculture?
 8. What criteria must be met for a farmer to obtain financial support for such a technology?
 9. What barriers do you observe in the adoption of innovative technologies by farmers?
 10. How can the adoption of autonomous solutions be encouraged at the regional level?
 11. What institutional concerns might arise regarding the use of autonomous vehicles?
 12. What standards or certifications should be respected for such a product to be approved?
 13. How do you evaluate the impact of a new technology on safety and the environment?
 14. What data or reports should be provided to authorities for monitoring?
 15. To what extent does the current rural infrastructure allow the implementation of autonomous solutions?
 16. Are there national or European initiatives that support automation in agriculture?
 17. How does your institution collaborate with developers of agricultural technologies?
 18. What factors influence the decision to officially support a new technology?
 19. What regional benefits could an autonomous vehicle bring to farms in Moldova?
 20. What recommendations would you offer for aligning the project with institutional requirements?
-

5. Investors / Funding Organizations

1. What types of technological or agricultural projects do you usually finance?
 2. What criteria are the most important when evaluating a new project?
 3. How do you evaluate the market potential of a technological solution in agriculture?
 4. What financial indicators do you follow before making an investment decision?
 5. What level of product maturity do you consider necessary for funding?
 6. How do you analyze risks in a project based on autonomous technologies?
 7. What risks do you consider the most critical in an autonomous vehicle project for agriculture?
 8. How do you evaluate the team's capacity to implement and develop the project?
 9. What monetization model do you consider suitable for such a solution (direct sale, subscription, leasing)?
 10. How important is validating the market need before investment?
 11. What evidence or data should be presented to demonstrate real demand?
 12. How do you analyze the potential for scaling at a regional or international level?
 13. What role do strategic partnerships play in the funding decision?
 14. What investment recovery period do you consider acceptable?
 15. What expectations do you have regarding the return on investment in such a project?
 16. How do you evaluate the competitive advantage of a new solution on the market?
 17. What common mistakes do you observe in agricultural technology startups?
 18. What level of involvement do you prefer after investment (advisory, strategic, operational)?
 19. What factors could determine the withdrawal or refusal of funding?
 20. What recommendations would you offer to increase the attractiveness of the project for investors?
-

Stakeholder 1 Interview:

1. What type of farm do you own and how big is it?
 2. What are the most common daily transport or travel activities on the farm?
 3. Who usually handles these transport activities?
 4. During busy periods, which activities are the most time-consuming?
 5. What difficulties do you encounter when transporting crops, tools, or materials?
 6. Have you ever experienced delays due to transportation issues? Can you give a brief example?
 7. How does labor shortage affect daily farm operations?
 8. What safety risks are there when moving vehicles or equipment on the farm?
 9. What daily activities do you consider repetitive or inefficient?
 10. When a problem arises with a vehicle or transport, how do you usually handle it?
 11. Have you improved any work processes in recent years? What have you changed?
 12. What do you consider before investing in new equipment?
 13. How do you decide whether an investment in new technology is worthwhile?
 14. Do you currently use smart technologies on your farm? If so, which ones?
 15. Have you used GPS systems, sensors, or remote monitoring solutions?
 16. What concerns would you have about using an autonomous vehicle on your farm?
 17. Would you prefer a fully autonomous vehicle or one supervised by an operator? Why?
 18. What types of activities would you be willing to entrust to an autonomous vehicle?
 19. What would make you trust such a system?
 20. What benefits
-

Name: Sergiu Goncearuc
Occupation: Farmer
Location: Republic of Moldova, r. Anenii Noi
Interviewer: Ciumacenco Pavel

Transcript

Interviewer: Thank you for agreeing to this interview. To start, could you tell us a few things about yourself and your farm?

Sergiu Goncearuc: Yes, of course... My name is Sergiu Goncearuc, I'm 44 years old, and I've been working in agriculture since I was a child. The farm is a family one — I took it over from my father. It's not very big, but not small either — we work about 120 hectares. Wheat, corn, sunflower... and a small plum orchard.

Most of the time it's just me and my brother, and during the busy season we bring in extra workers. That's how things work here.

Interviewer: How would you describe the daily structure of activities on the farm?

Sergiu: It depends on the season... but generally the day starts early. By 6 a.m. we're already in the field. If it's harvest season, the combine starts first. Then all the transport work begins.

In the afternoon we deal with organization, repairs, and inspections. Honestly, we spend almost half the time just moving things from one place to another.

Interviewer: What are the most common transport-related tasks?

Sergiu: We're always transporting something. Crops from the field to the storage, fertilizers, diesel... sometimes just a small part forgotten on the other side of the land.

And it may not seem like much, but these repeated trips consume time and fuel. Sometimes it feels like the tractors spend more time on the road than actually working.

Interviewer: Who organizes these tasks?

Sergiu: Mostly me. And my brother.

We don't have an automated system or anything digital. Everything is done from experience. Phone calls, quick discussions, decisions on the spot. If a problem appears, you have to react immediately.

Interviewer: What does a day during peak season look like?

Sergiu: It's intense... very intense.

The combine harvests continuously. If the trailer doesn't arrive on time, you have to stop. And when you stop, you lose time, money... and momentum.

Most time is lost in transportation and waiting. That's the reality.

Interviewer: What challenges do you face with transportation?

Sergiu: Field roads, especially after rain. Sometimes you enter with the tractor and realize you shouldn't have gone in...

Then there are breakdowns. If a trailer breaks down, the whole workflow stops. We don't have many spare units.

Interviewer: Can you describe a recent delay?

Sergiu: Yes... last year during the sunflower harvest. A trailer got a flat tire. The combine was full. We had to stop for almost two hours.

Maybe for someone outside agriculture two hours doesn't seem like much, but during peak season... it means a lot.

Interviewer: How does labor shortage affect your operations?

Sergiu: A lot. Very much.

Young people leave. Those who stay are few. When you don't have workers, you do everything yourself. You drive, inspect, repair. You get tired, and tiredness leads to mistakes.

Interviewer: What risks exist during transportation?

Sergiu: The farmyard becomes crowded during the season. Tractors, people, trucks. If someone isn't paying attention, an accident can happen.

Honestly, fatigue is the biggest risk.

Interviewer: Which tasks seem inefficient or repetitive to you?

Sergiu: Repetitive transport. Patrolling the yard. Manual checks.

They are necessary tasks, but they consume a lot of time and don't directly add value.

Interviewer: How do you solve logistical problems?

Sergiu: Quickly and practically. We redistribute equipment, call the mechanic, improvise. In agriculture you need to adapt quickly.

Interviewer: Have you ever improved a process on the farm?

Sergiu: Yes. We reorganized the farmyard. We moved the unloading area closer to the storage facility.

It seems simple, but it significantly reduced maneuvering time. Sometimes organization matters more than technology.

Interviewer: What technologies do you currently use?

Sergiu: We have GPS on the tractor. We use weather apps and a device for measuring grain moisture.

We're not fully digitalized, but we try to keep up.

Interviewer: What do you think about autonomous vehicles?

Sergiu: The idea is interesting... but it has to be safe.

The first concern is people's safety. Then reliability. If it breaks during the season, it becomes a big problem.

Interviewer: What level of autonomy would be acceptable?

Sergiu: At first, supervised. I should be able to monitor everything and stop it if needed.

Full autonomy... maybe in the future, but it needs to be tested well.

Interviewer: What tasks could such a vehicle take over?

Sergiu: Repetitive transport. Moving fertilizers. Patrolling the yard.

Basically the kind of work that doesn't require complicated decisions.

Interviewer: Thank you for your time.

Sergiu: You're welcome. Agriculture isn't easy, but if technology helps us in a practical way, not just theoretically, we're open to it. The important thing is that it's adapted to our reality here in Moldova.

Stakeholder 2 Interview:

Questions

1. What are the most frequent daily transport activities within the yard or workshop?
2. Who usually handles these transport activities?
3. During busy periods, which activities consume the most time?
4. What difficulties do you encounter when transporting wood or tools across the premises?
5. Have you experienced situations where delays occurred due to transport problems?
6. How does the lack of workforce influence daily operations?
7. What safety risks exist during wood transportation?
8. Which daily activities do you consider repetitive or inefficient?
9. When a problem occurs with a vehicle or transport equipment, how do you handle it?
10. Have you improved any process in recent years?
11. What do you consider before investing in new equipment?
12. How do you decide whether an investment is worth making?
13. Do you currently use modern technologies in your activity?
14. Have you used GPS systems or remote monitoring?
15. What concerns would you have about using an autonomous vehicle in the yard?
16. Would you prefer a fully autonomous vehicle or a supervised one?
17. What activities would you be willing to entrust to an autonomous vehicle?
18. What would make you trust such a system?
19. What concrete benefits would convince you to adopt this solution?

Workshop Documentation: Autonomous Yard Utility Vehicle Project

Name: Nicolae Paraschiv

Occupation: Entrepreneur

Location: Republic of Moldova, Streni District, Pneti Village

Interviewer: Istrati tefan

Transcript

Interviewer: Thank you for agreeing to this interview. To start, could you tell us a few things about yourself and the activity you carry out?

Nicolae Paraschiv: My name is Nicolae Paraschiv, I am 59 years old, and I work in wood processing. We operate with medium to large quantities. We have a fairly large yard where we store logs and boards, and inside the workshop we cut and prepare them for delivery to Chiinu.

Interviewer: What are the most frequent daily transport activities within the yard or workshop?

Nicolae Paraschiv: Every day we move logs to the cutting area, and then we transport the cut boards to the storage area or to the loading zone.

Interviewer: Who usually handles these transport activities?

Nicolae Paraschiv: Usually the workers. We have several people who mainly deal with handling and transporting the materials.

Interviewer: During busy periods, which activities consume the most time?

Nicolae Paraschiv: Transporting and rearranging the boards. When we have a large order, a lot of time is lost just moving them around.

Interviewer: What difficulties do you encounter when transporting wood or tools across the premises?

Nicolae Paraschiv: The heavy weight of the materials and the fact that sometimes multiple trips are required. When the ground is wet, it becomes even more complicated.

Interviewer: Have you experienced situations where delays occurred due to transport problems?

Nicolae Paraschiv: Yes. For example, when the forklift broke down, we had to move things manually and the delivery was delayed.

Interviewer: How does the lack of workforce influence daily operations?

Nicolae Paraschiv: If someone is absent, the others have to handle both transportation and the work on the machines. That slows down the entire process.

Interviewer: What safety risks exist during wood transportation?

Nicolae Paraschiv: There is a risk of injury when lifting, slipping, or if someone drops a heavy board.

Interviewer: Which daily activities do you consider repetitive or inefficient?

Nicolae Paraschiv: Moving boards from one place to another several times before they finally reach the loading area.

Interviewer: When a problem occurs with a vehicle or transport equipment, how do you handle it?

Nicolae Paraschiv: I call the mechanic or we improvise temporarily until it gets repaired.

Interviewer: Have you improved any process in recent years?

Nicolae Paraschiv: We reorganized the yard so that the route from the cutting area to the storage area is shorter.

Interviewer: What do you consider before investing in new equipment?

Nicolae Paraschiv: The price, usefulness, and how quickly I can recover the investment.

Interviewer: How do you decide whether an investment is worth making?

Nicolae Paraschiv: If it reduces costs or saves a significant amount of time.

Interviewer: Do you currently use modern technologies in your activity?

Nicolae Paraschiv: We have modern electric machines and a forklift, but nothing automated.

Interviewer: Have you used GPS systems or remote monitoring?

Nicolae Paraschiv: No, so far we have not used them.

Interviewer: What concerns would you have about using an autonomous vehicle in the yard?

Nicolae Paraschiv: That it might break down often or cause accidents or damage.

Interviewer: Would you prefer a fully autonomous vehicle or a supervised one?

Nicolae Paraschiv: At first I would prefer it to be supervised, until I gain trust in it.

Interviewer: What activities would you be willing to entrust to an autonomous vehicle?

Nicolae Paraschiv: Transporting the cut boards to storage and bringing tools where they are needed.

Interviewer: What would make you trust such a system?

Nicolae Paraschiv: If it works consistently without problems and has good safety systems.

Interviewer: What concrete benefits would convince you to adopt this solution?

Nicolae Paraschiv: Reduced transport costs, time savings, and fewer delivery delays.

Stakeholder 3 Interview:

Questions

1. Could you please do a short introduction and briefly explain your domain of experience and activity?
 2. (Comment rather than a full question, but part of the interaction) For the most part, you did very well. You covered most of the aspects.
 3. So it won't need continuous access to the cloud or other external navigation sources?
 4. What should we use for a cloud connection, and how should we manage it in the case where we have multiple devices?
 5. Could you tell us something about the final tests before the system is put into operation?
 6. Regarding the costs, approximately how much would the hardware cost?
-

Workshop Documentation: Autonomous Yard Utility Vehicle Project

Name: Andrei Bragarenco

Occupation: Hardware Engineer

Location: Republic of Moldova, Chiinu, Micro Lab

Interviewer: Istrati tefan and Ciumacenco Pavel

Transcript

Pavel: Okay, to start off, we will do a quick introduction.

Andrei: If it's easier for you in Romanian, it's better in Romanian.

Stefan: We also made the questions in English, so...

Pavel + Stefan: You can answer in Romanian.

Pavel: So, we are representatives of the Software Engineering Department, group FAF-232, and our project is Autonomous Yard Vehicle. Could you please do a short introduction and briefly explain your domain of experience and activity?

Andrei: My interests, as a stakeholder, are related to the fact that we could implement such solutions in rural households, and not only there, but in any enterprise where there is a controlled environment. Why? Because we have a problem with workforce, and there are situations where you won't hire a person just to move some things from point A to point B.

A particular use case: carrying a bag of cement from point A to point B and all you need is a button. It goes to the warehouse and sends a message telling them to put a bag of cement on it. Someone presses another button and it takes it back. This is a simple source-destination use case with a single button.

Another use case would be for fruit and vegetable pickers. It simply moves, and there is a button they press and the vehicle moves 5 meters forward. The simplest interface is a single button that you press and it does something depending on the circumstances. And you don't need a driver to move through a controlled environment, which is, for example, a row.

You can see on the left and right, for example with a camera, or in our case actually with GPS tracking, a precision GPS. You know exactly where you are going, you don't have navigation problems. If something appears in front of it, it moves at a very low speed, under 30 km/h or even under 5 km/h. That means it cannot harm anyone. We add emergency systems and we can implement some basic safety measures.

For example, it does not move if there is something in its ultrasonic barrier zone. We install ultrasonic sensors, regular parktronic sensors that are meant for low speed. At high speeds you need LiDAR or radar. At low speeds ultrasonic sensors are enough. On normal cars ultrasonic sensors disconnect above 5 km/h because their scanning speed is not good enough. Also at higher speeds you can get false noise from wind and other things.

So these are the circumstances, let's say, and my interests. To have a vehicle that can execute commands in three steps. For example, go to coordinate XY based on a simple command such as pressing a button. Or from the phone you give it coordinates. Then we can program commands like: go to coordinate XY, return to the destination, or go two steps backwards. These are basic commands.

It may seem trivial, but in a household there is usually a person who sits in a tractor and gets paid just to stay there all day. When someone buys this technology, for example this golf cart, it might cost 5,000 or 10,000 euros, but after that they don't have to pay the salary of a worker who earns around 20,000 lei per month. In three years, or even one or two years, you recover the full cost.

Also, you no longer have problems when the worker doesn't come to work, gets sick, or you have to pay insurance and so on. In the end it becomes cheaper. So this is where you gain an advantage.

Additionally, you can use it for several utilities. You can use it for inspection and other tasks. But for now we can focus on this use case: take something somewhere and bring it back. These three use cases: go from point A to B, go to coordinate XY, limp home (return home), or move forward for pickers. That's the classic version that would help.

For example, even to move cement from one place to another you still need a driver, which costs more than the tractor itself. If it only needs to go once or twice a day, the same person could do other tasks, or you have equipment that covers multiple tasks. On the other hand, your golf cart remains functional for manual driving if needed. So you can still use it normally, but you also have this automation that can give you a real advantage.

Okay, are there any other questions?

Pavel: For the most part, you did very well. You covered most of the aspects.

Andrei: That's why I say: in interviews you prepare questions and reference points, but it never goes exactly as in the book.

Stefan: We already talked about sensors. What should we use for a cloud connection and how should we manage it if we have multiple devices?

Andrei: Yes, of course. For the cloud part, I touched on it earlier, but it's important because you won't have just one vehicle, but maybe several. Then you need to send their status: what was done and where, how much battery they have, whether any incidents occurred.

For example, you send it to the field but something happens — maybe it hits something — and you need a notification. You connect to the camera and see what happened. So you need a camera, you need "eyes". Then you can take it out of that situation. You need remote control: you connect remotely and give commands such as forward, backward, or moving by coordinates, and create an exit or return scenario.

Maybe it entered an area where it cannot get out. Circumstances changed, someone placed something there, the ultrasonic sensor blocks it and it cannot move. If for 10 seconds it cannot get out, it sends you a notification and asks for control. You take control with a few clicks and move it forward and backward until you free it.

You can also write scenarios: driving in circles, patrolling, or other tasks. You will have a server where you can see it, track it, see where it moves and at what time. You can see scenarios, you can record a route. For example, someone drives it manually and you record the route, and after that it can follow the route automatically.

So you say you go from point A to point B, you drive it once and it records the route. The second time it will follow exactly the same path. This is called learning — the learning curve — meaning it learns a route. You drive it once and afterwards it repeats it through logs. You don't need extremely smart navigation; you can work with simple scenarios.

Pavel: So it doesn't need continuous access to the cloud or other external navigation sources.

Andrei: Yes, it doesn't need to see and avoid objects in complex ways — those things are too complicated. It just follows a predefined scenario assuming you know the environment completely. For example, in a field a person shouldn't randomly appear; if someone appears, the car stops, the obstacle disappears, and that's it.

If the obstacle stays there too long, you get a notification and you must intervene. Because if it starts navigating left and right trying to avoid things, it might damage plants. So it's better not to. Also, you're not in a hurry. It can even call your phone. That could be a feature: it calls and says "I have a problem." A robot voice says the vehicle is blocked. From your phone you connect, see what's wrong, connect to all cameras. The camera is just simple streaming. It's very easy to implement.

You will need a dashboard where you see everything like in a game. I send it somewhere and I have a map and I move it on the map. You can take pictures occasionally. These are "could have" features, depending on time. "Must have" is navigation. "Should have" is learning and scenarios. Navigation between points is mandatory. If I already explained what is must have and should have, what questions did I miss?

Stefan: Something about final testing before deployment.

Andrei: I'll tell you a use case. If you build it, make it drive one circle around Building 3. That's the validation test. The rest you can do yourselves. Someone sits in the vehicle with the stop button just in case, to disconnect if needed.

But you give the command "make a circle". If it completes the circle around the building, it means you validated that it can go anywhere you want.

You can also record the route. You drive and record the GPS track. We have GPS with RTK — Real-Time Kinematics. Normal GPS has an error of ± 3 to ± 6 meters. RTK has an error of about ± 7 centimeters. So it's very precise.

For tractors, if last year you made a furrow in a field, next year the tractor can place the seed exactly in the same coordinates. That's how precise it is. But it must be calibrated and there are problems.

What problems? Between buildings GPS does not work very well, even with RTK. We have signal reflections. Recording routes helps because you can have controlled deviations. On the map it may show one path while in reality you move slightly differently.

You can also create correction maps. When you are in a certain zone you apply certain coordinate corrections. Each point can have corrections to adjust the system. Even if the base precision is 7 cm, buildings block some satellites and you get measurement errors.

In open fields you have centimeter precision. Between buildings, because some satellites are blocked, the precision decreases. If the system sees only 3–4 satellites, the precision is low. If it sees 20–30 satellites, the precision becomes much higher. That is the main challenge.

So again, the validation test is a fully autonomous circle around Building 3. It's just asphalt around it. And add a sound and light signal. That's mandatory. It's a utility vehicle, so when it moves it must signal: beep-beep-beep.

Stefan: And one last question. Approximately, what are the hardware costs?

Andrei: The hardware equipment we already have. You will only need to list it and estimate the costs as existing equipment. I will give you the links.

Here is the steering system — it's already installed. There are cables connecting to it. I will also show you the GPS and introduce you to the guys so you can inspect everything and take pictures.

I want you, together with the team who previously worked on the project, to install the components and validate driving with this remote controller. You start the golf cart with this remote. Radu will help you after they show you how it works, and with him you'll move to the second phase to see how everything functions.

Also, this Pixhawk connects to a microcontroller and then to a computer. I gave you a computer, right?

Pavel: Yes, we just haven't picked it up yet.

Andrei: That computer is an industrial computer designed for the golf cart.

So this is a package of equipment. The Pixhawk connects to the vehicle — it's basically the "brain" of a drone. It was installed before and worked previously. From the computer you connect through USB and you have a Python API through which you control it.

You move one step forward, read GPS data, get your coordinates XYZ, and build your scenario logic. Hardware is not your biggest challenge — the important part is the software that controls the hardware.

So this device together with the computer forms the system. Computing happens here. From the cloud you send missions. A mission can be simply going from one point to another or a more complex scenario.

For real-time navigation you send short missions from point to point. At some point I'll leave the equipment here. You'll look at the Pixhawk 6C and search how to use the ArduPilot API, and for details you can ask Radu.

Stefan: And the parktronic sensors — are those all of them?

Andrei: Yes, they are there. You can connect them to the computer. They use a serial interface through a USB-UART converter. You'll see packets like FF and 8 bytes — two bytes per sensor representing distances — plus a checksum.

So you have 10-byte packets: one FF header, four pairs of bytes, and a checksum. That gives you the distance from each sensor.

In your model you can draw the vehicle, similar to car multimedia systems where you see the car and the parktronic sensors showing obstacles.

You'll have a UI where the data is displayed and you see a car model with obstacles shown. Four sensors in the back and four in the front. Those are our parktronic sensors.

We'll install the computer on the golf cart with bolts, route the cables, and connect everything. The previous team disconnected things since these are our vehicles, but we know where everything connects. The microcontroller also connects here, but they need to show you exactly how it works.

Okay?

I'll connect you with the guys who worked on this project, and that will be it from me.

Pavel: Yes, thank you very much.

Andrei: Did I clarify everything?

Pavel: Yes.

Andrei: With pleasure.

Project Charter — IoT-Enabled Autonomous Agricultural Utility Vehicle

1. Project Purpose and Justification

Agricultural operations frequently experience inefficiencies in internal logistics, particularly in repetitive transport tasks between fields, storage areas, and operational zones. These tasks consume significant time, fuel, and labor resources while contributing minimal direct value to production. Field observations and stakeholder interviews indicate that a substantial portion of working time is spent on transporting materials, equipment, and harvested crops rather than performing productive agricultural activities.

Additionally, seasonal labor shortages increase workload pressure on farm operators, leading to operational delays, fatigue, and increased risk of accidents.

The purpose of this project is to design, develop, and evaluate an **IoT-enabled semi-autonomous utility vehicle** capable of assisting agricultural workers by performing repetitive transport, monitoring, and patrol tasks within controlled farm environments. The system integrates autonomous mobility, sensor-based perception, and cloud-enabled monitoring to improve operational efficiency and safety.

2. Problem Statement

Farm logistics are primarily dependent on manual coordination and human-operated vehicles, resulting in inefficiencies, delays, and increased operational costs. Transport activities such as moving fertilizers, equipment, and harvested crops are repetitive and time-consuming, often interrupting core agricultural processes.

Limited workforce availability further exacerbates the problem, forcing workers to handle multiple tasks simultaneously. This increases fatigue and the likelihood of operational errors or accidents, especially during peak agricultural seasons.

3. Project Objectives (SMART)

- Design and prototype a semi-autonomous utility vehicle within the project timeline (academic term).
- Achieve reliable autonomous navigation within predefined farm areas using waypoint navigation and geofencing.
- Implement obstacle detection and avoidance with safe stopping behavior in 95% of detected critical scenarios during testing.
- Enable real-time remote monitoring and control through an IoT-enabled dashboard interface.
- Demonstrate measurable improvement in internal logistics efficiency compared to manual transport workflows.
- Provide a functional human-machine interface allowing supervision and manual override.

4. Project Scope

In Scope

- Autonomous navigation in controlled farm environments
- IoT-based communication and monitoring
- Transport of materials, tools, and equipment
- Obstacle detection and safety mechanisms
- Manual override and remote control capabilities
- Human-machine interface design

Out of Scope

- Fully autonomous operation without human supervision
- Public road navigation
- Large-scale agricultural machinery automation
- Full commercial deployment

5. Key Deliverables

- High-level and detailed system architecture
- Functional prototype or simulation of the autonomous vehicle
- Sensor integration (GPS/RTK, ultrasonic, camera, IMU)
- IoT communication framework (edge + cloud)
- Web-based monitoring dashboard and control interface
- Navigation and obstacle avoidance algorithms
- System performance evaluation report
- Deployment and scalability recommendations

6. Stakeholders

Stakeholder	Role
Farmers / Farm Operators	Primary users and system beneficiaries

Agribusiness Managers	Decision-making and investment evaluation
IoT & IT Technicians	System deployment, maintenance, and integration
Project Team	Design, development, and validation
Academic Supervisors	Guidance, evaluation, and feedback

7. Functional Requirements

ID	Requirement
FR1	The system shall support autonomous navigation within predefined farm areas.
FR2	The system shall detect obstacles and perform safe stop or avoidance actions.
FR3	The system shall support a follow-me operational mode.
FR4	The system shall assist in transport of materials and equipment.
FR5	The system shall provide remote monitoring of status and location.
FR6	The system shall allow manual override and emergency stop.
FR7	The system shall record operational data and logs.

8. Non-Functional Requirements

ID	Requirement
NFR1	The system shall prioritize human safety at all times.
NFR2	The system shall operate reliably in agricultural environments (dust, uneven terrain, weather).
NFR3	The interface shall be intuitive and user-friendly.
NFR4	The system shall support wireless communication suitable for rural areas.
NFR5	The system shall maintain energy efficiency for extended operation.

9. Expected Benefits

- Reduction in time spent on repetitive transport tasks
- Improved coordination during harvesting and operational workflows
- Increased safety in shared farm environments
- Reduced workload pressure due to labor shortages
- Foundation for future automation in agriculture

10. Success Criteria

The project will be considered successful if the system demonstrates:

- Reliable autonomous navigation within controlled environments
- Safe interaction with humans, equipment, and terrain
- Improved logistical efficiency compared to manual processes
- Positive usability feedback from stakeholders
- Stable communication and monitoring via IoT platform

11. Risks and Mitigation Strategies

Risk ID	Description	Mitigation
R1	Sensor failure	Sensor fusion and redundancy
R2	GNSS signal degradation	Combine GNSS with IMU and odometry
R3	Connectivity loss	Enable offline operation and delayed sync
R4	Collision risks	Obstacle detection + emergency stop
R5	Difficult terrain	Robust hardware and adaptive navigation
R6	System failure during peak use	Allow manual fallback
R7	Low user trust	Provide supervised autonomy
R8	Improper system use	Training and simple interfaces

12. Assumptions and Constraints

Assumptions

- IoT and sensor technologies are reliable and scalable
- Farm environments allow controlled autonomous operation
- Stakeholders are willing to adopt semi-autonomous solutions

Constraints

- Limited budget and hardware resources
- Environmental variability (terrain, weather)
- Dependence on wireless connectivity availability
- Compliance with agricultural safety standards

Requirements

The stakeholder interviews conducted with a farmer (Sergiu Gonciaruc) and a wood processing entrepreneur (Nicolae Paraschiv) revealed several operational challenges related to repetitive transportation tasks, lack of workforce, safety concerns, and inefficiencies in logistics management. Both stakeholders described situations where a significant amount of time is spent moving materials across large working areas such as agricultural fields, farm yards, or industrial yards.

The interviews highlighted that transportation tasks are often repetitive and do not require complex decision-making, which makes them suitable for automation. At the same time, both interviewees emphasized that reliability and safety are critical factors for adopting autonomous systems. Stakeholders also expressed the need for supervision and monitoring capabilities, especially during the initial deployment of such technology.

Re qui re me nt ID	St or y T y pe	Description	Stake holder	Group	MoSC oW Classi fication	Rationale
R1	US	As a farmer, I want an autonomous vehicle to transport materials between field and storage so that I reduce time spent on repetitive transport tasks.	Farme rs	Trans port Auto mation	Must Have	This is the core functional objective of the proposed system. Without automated material transport, the system does not provide its primary value of reducing repetitive logistics work identified by farmers.
R2	US	As a farm operator, I want to monitor the autonomous vehicle remotely so that I can supervise its activity and intervene if needed.	Farme rs	Monit oring	Shoul d Have	Stakeholders emphasized the need for supervision during early adoption. While the vehicle could technically operate without it, monitoring significantly increases trust and operational control.
R3	JS	When harvest operations are running continuously, I want automated transport support so that harvesting machines do not stop due to lack of transport vehicles.	Farme rs	Logist ics Efficie ncy	Shoul d Have	Continuous harvesting requires synchronized logistics. Automated support improves efficiency, but harvesting could still proceed manually if necessary.

R4	US	As a user, I want the vehicle to follow predefined routes so that transportation tasks can be executed consistently.	Farmers / Entrepreneurs	Navigation	Must Have	Route-based navigation is essential for predictable autonomous movement. Without this capability the vehicle cannot perform transport tasks reliably.
R5	JS	When working in environments with limited workforce, I want automation to perform repetitive tasks so that labor shortages do not affect productivity.	Farmers / Entrepreneurs	Workforce Optimization	Should Have	Labor shortages are a major challenge identified by stakeholders. Automation helps maintain productivity, but it is a strategic benefit rather than a minimum operational requirement.
R6	US	As a user, I want the vehicle to detect obstacles automatically so that accidents with people or equipment are avoided.	Farmers / Entrepreneurs	Safety	Must Have	Safety is critical for any autonomous system operating near people or equipment. Stakeholders clearly emphasized accident prevention as a primary concern.
R7	US	As a farm operator, I want a manual override or emergency stop so that I can immediately stop the system if something goes wrong.	Farmers	Safety	Must Have	Emergency stop functionality is a mandatory safety requirement in autonomous systems. It allows operators to immediately intervene if unexpected situations occur.
R8	JS	When the vehicle operates in farm yards or industrial spaces, I want reliable proximity detection so that collisions with equipment are prevented.	Farmers / Entrepreneurs	Safety	Must Have	Dense working environments require reliable collision prevention systems to avoid damage to equipment and injury to workers.
R9	US	As a user, I want the system to operate reliably during long working hours so that operations are not interrupted during peak season.	Farmers / Entrepreneurs	Reliability	Must Have	Agricultural work often involves long operational periods, especially during harvest. System reliability is essential to ensure the technology is practical and dependable.
R10	JS	When equipment breaks down or transport is delayed, I want an alternative automated transport solution so that workflow disruption is minimized.	Farmers	Logistics Efficiency	Could Have	This requirement provides additional operational flexibility but is not essential for the core system functionality.
R11	US	As a business owner, I want the system to reduce operational transport costs so that the investment becomes financially viable.	Entrepreneurs	Economic Efficiency	Should Have	Economic benefits are an important adoption driver. However, cost reduction is an outcome of the system rather than a technical requirement.
R12	US	As a user, I want the vehicle to transport tools and materials across the workspace so that workers can focus on higher-value tasks.	Entrepreneurs	Transport Automation	Must Have	Similar to R1, this requirement directly supports the main function of the system: reducing manual transportation tasks.
R13	JS	When operating in large yards or fields, I want automated logistics support so that materials can be moved efficiently between working zones.	Farmers / Entrepreneurs	Logistics Efficiency	Should Have	Improves operational efficiency in large workspaces but is not strictly required for basic autonomous transport functionality.
R14	US	As a user, I want the system to provide status information (location, battery level, operation status) so that I always know the vehicle condition.	Farmers / Entrepreneurs	Monitoring	Should Have	Provides transparency and better system management, but the system could technically operate without continuous status feedback.
R15	US	As a user, I want the system to be easy to use without complex configuration so that it can be adopted by non-technical operators.	Farmers / Entrepreneurs	User Experience	Could Have	Ease of use is important for adoption by non-technical operators, but the system could still function even if configuration requires some technical setup.
R16	US	As an operator, I want to send the vehicle to specific GPS coordinates so that it can perform transport tasks anywhere in the working area.	Farmers / Entrepreneurs	Navigation	Must Have	The interview highlighted the importance of coordinate-based commands such as "go to coordinate XY". This capability enables flexible operation across fields, yards, or warehouses.
R17	US	As a user, I want the vehicle to automatically return to its starting point so that transport cycles can be completed without manual retrieval.	Farmers / Entrepreneurs	Navigation	Must Have	The stakeholder emphasized "return home" or "limp home" functionality as a fundamental autonomous operation that ensures the vehicle can complete missions safely.
R18	JS	When workers are harvesting fruits or vegetables, I want the vehicle to move forward incrementally so that pickers can continue working without manually repositioning transport equipment.	Farmers	Logistics Efficiency	Should Have	The interview described a scenario where workers press a button and the vehicle advances a few meters. This supports harvesting efficiency but is not essential for basic logistics automation.
R19	US	As a user, I want the vehicle to be controlled using a simple interface such as a button or mobile command so that non-technical workers can operate it easily.	Farmers / Entrepreneurs	User Interface	Must Have	Stakeholders emphasized the importance of very simple controls. A minimal interface lowers the barrier for adoption and ensures the system can be used by workers without technical expertise.
R20	JS	When the vehicle encounters an obstacle and cannot continue moving, I want the system to notify the operator so that the issue can be resolved quickly.	Farmers / Entrepreneurs	Monitoring	Should Have	The interview suggested that if the vehicle remains blocked for a certain time, it should send a notification. This improves operational reliability but is not strictly required for basic movement.
R21	US	As an operator, I want to remotely control the vehicle through a dashboard or mobile interface so that I can resolve unexpected situations when autonomous operation is interrupted.	Farmers / Entrepreneurs	Remote Control	Should Have	Remote control functionality allows operators to intervene when obstacles or environmental changes prevent the vehicle from completing its route autonomously.

R22	US	As a system operator, I want a dashboard that displays vehicle location and operational data so that I can monitor its activity in real time.	Farmers / Entrepreneurs	Monitoring	Should Have	The interview mentioned a server or dashboard where the operator can observe vehicle movements and system status. This improves operational visibility and management.
R23	US	As an operator, I want the vehicle to record manually driven routes so that it can repeat the same routes autonomously in the future.	Farmers / Entrepreneurs	Navigation	Should Have	Route recording was suggested as a practical way to simplify navigation in controlled environments, allowing the system to repeat previously validated paths.
R24	JS	When multiple vehicles are deployed in the same environment, I want each vehicle to send operational data to a central system so that fleet activity can be monitored.	Farmers / Entrepreneurs	Logistics Efficiency	Could Have	The interview discussed scenarios with multiple vehicles sharing information through a server. While useful for scalability, it is not essential for a single-vehicle implementation.
R25	US	As a worker in the operational area, I want the vehicle to produce sound or light signals while moving so that people nearby are aware of its activity.	Farmers / Entrepreneurs	Safety	Must Have	The stakeholder explicitly mentioned that the vehicle should emit signals such as warning sounds while moving to alert nearby workers and prevent accidents.
R26	JS	When the system is deployed in a controlled environment, I want it to follow predefined scenarios rather than complex autonomous navigation so that the system remains reliable and predictable.	Farmers / Entrepreneurs	Navigation	Must Have	The interview emphasized using simple scenario-based navigation instead of complex obstacle avoidance logic to prevent unexpected behavior in agricultural environments.
R27	US	As an operator, I want to access video from onboard cameras so that I can inspect the vehicle surroundings when a problem occurs.	Farmers / Entrepreneurs	Monitoring	Could Have	The stakeholder suggested using cameras for remote inspection during incidents. This feature enhances situational awareness but is not required for basic functionality.
R28	JS	When validating the system before deployment, I want the vehicle to complete a predefined autonomous route so that the functionality of navigation can be confirmed.	Technical Team	System Validation	Must Have	The interview proposed a practical validation scenario where the vehicle completes an autonomous route around a building, confirming that the navigation system works correctly.

Classes

Transport Automation (R1, R12)

Stakeholder Requirement: The system should automate the transportation of tools and materials within agricultural or industrial environments.

User Stories:

As a farmer, I want an autonomous vehicle to transport materials between field and storage so that I reduce time spent on repetitive transport tasks.

As a user, I want the vehicle to transport tools and materials across the workspace so that workers can focus on higher-value tasks.

Job Stories:

When operating in large yards or fields, I want automated logistics support so that materials can be moved efficiently between working zones.

When equipment breaks down or transport is delayed, I want an alternative automated transport solution so that workflow disruption is minimized.

Navigation and Routing (R4, R16, R17, R23, R26, R28)

Stakeholder Requirement: The system should navigate autonomously using predefined routes to ensure reliable transportation.

User Stories:

As a user, I want the vehicle to follow predefined routes so that transportation tasks can be executed consistently.

Job Stories:

When the system operates in large environments, I want route-based navigation so that transport tasks can be performed reliably without manual guidance.

Safety (R6, R7, R8, R25)

Stakeholder Requirement: The system should ensure safe operation by preventing collisions and allowing emergency intervention.

User Stories:

As a user, I want the vehicle to detect obstacles automatically so that accidents with people or equipment are avoided.

As a farm operator, I want a manual override or emergency stop so that I can immediately stop the system if something goes wrong.

Job Stories:

When the vehicle operates in farm yards or industrial spaces, I want reliable proximity detection so that collisions with equipment are prevented.

Monitoring and Control (R2, R14, R20, R21, R22, R27)

Stakeholder Requirement: The system should provide monitoring capabilities so operators can supervise and manage the autonomous vehicle.

User Stories:

As a farm operator, I want to monitor the autonomous vehicle remotely so that I can supervise its activity and intervene if needed.

As a user, I want the system to provide status information (location, battery level, operation status) so that I always know the vehicle condition.

Job Stories:

When supervising automated operations, I want access to real-time system information so that I can respond quickly to potential issues.

Logistics Efficiency (R3, R10, R13,18, R24)

Stakeholder Requirement: The system should support continuous workflows by improving logistics coordination.

User Stories:

As a user, I want automated logistics support so that materials can be transported efficiently during work operations.

Job Stories:

When harvest operations are running continuously, I want automated transport support so that harvesting machines do not stop due to lack of transport vehicles.

When equipment breaks down or transport is delayed, I want automated support so that workflow disruption is minimized.

Workforce Optimization (R5)

Stakeholder Requirement: The system should reduce dependence on manual labor by automating repetitive tasks.

User Stories:

As a user, I want repetitive transport tasks to be automated so that workers can focus on more productive activities.

Job Stories:

When working in environments with limited workforce, I want automation to perform repetitive tasks so that labor shortages do not affect productivity.

Economic Efficiency (R11)

Stakeholder Requirement: The system should reduce operational transport costs and improve economic efficiency.

User Stories:

As a business owner, I want the system to reduce operational transport costs so that the investment becomes financially viable.

Job Stories:

When evaluating automation solutions, I want to ensure the system provides economic benefits so that the investment is justified.

System Reliability (R9)

Stakeholder Requirement: The system should operate reliably during long working periods without interruptions.

User Stories:

As a user, I want the system to operate reliably during long working hours so that operations are not interrupted during peak season.

Job Stories:

When operating during intensive agricultural periods, I want reliable autonomous transport so that logistics operations continue without failures.

User Experience (R15, R19)

Stakeholder Requirement: The system should be easy to use and accessible to non-technical operators.

User Stories:

As a user, I want the system to be easy to use without complex configuration so that it can be adopted by non-technical operators.

Job Stories:

When setting up the system, I want a simple configuration process so that I can start using the vehicle without technical expertise.

MoSCoW Analysis Groups

Must Have (Critical System Requirements)

These requirements are essential for system functionality and safe operation:

- R1 – Autonomous transport capability
- R4 – Route-based navigation
- R6 – Obstacle detection
- R7 – Emergency stop / manual override
- R8 – Proximity detection
- R9 – Operational reliability
- R12 – Material transport functionality
- R16 – Coordinate-based navigation
- R17 – Automatic return to starting point
- R19 – Simple user control interface
- R25 – Safety signaling during movement
- R26 – Scenario-based navigation reliability
- R28 – System validation through autonomous route execution

Should Have (Important but Not Critical)

- R2 – Remote monitoring
- R3 – Harvest logistics support
- R5 – Workforce optimization
- R11 – Operational cost reduction
- R13 – Logistics support for large environments
- R14 – System status reporting
- R18 – Incremental vehicle movement for harvesting
- R20 – Operator notification when blocked
- R21 – Remote manual control
- R22 – Monitoring dashboard
- R23 – Route recording

Could Have (Optional Enhancements)

- R10 – Backup logistics during equipment failure
- R15 – Simplified user configuration
- R24 – Multi-device management
- R27 – Onboard camera remote access

Won't Have (For Current Version)

No explicit "Won't Have" requirements were identified at this stage, as most stakeholder needs remain relevant to the system's potential development roadmap.

Category	Stakeholder Requirements	US(User Stories)	JS(Job Stories)	Classification	Rationale
Affordability	The solution should be affordable.	As a business owner, I want the system to be cost-effective so that I can minimize operational expenses.	When evaluating automation solutions, I want to ensure the system is affordable so I can justify the investment.	Must Have	A key concern for business owners is to "minimize operational expenses" and "justify the investment"; this is crucial for business survival and ROI.
Task Prioritization	The solution should prioritize delicate tasks over processing speed.	As an operator, I want the system to handle delicate items with care so that fragile products do not get damaged.	When processing fragile items, I want the system to prioritize careful handling over speed so I can reduce product loss.	Must Have	Essential for maintaining product integrity; has a direct impact on product loss, which raises expenses and lowers customer satisfaction.

Performance Metrics	The system's success metric should be achieving at least half the throughput and accuracy of a human worker.	As a manager, I want the system to achieve at least half the throughput and accuracy of a human worker so that it remains a viable replacement.	When evaluating system performance, I want to compare throughput and accuracy against human workers so I can assess efficiency.	Must Have	Establishes the minimal feasible performance standard, which is necessary to assess if the system is functionally sufficient.
Telemetry & Predictive Maintenance	The system must collect telemetry data to improve efficiency and speed. Additionally, it should predict the robot's health status, potential failures, and maintenance needs.	As a maintenance technician, I want the system to collect telemetry data so that I can monitor its performance and predict potential failures.	When the system is operating, I want it to track efficiency and predict maintenance needs so I can prevent unexpected breakdowns.	Should Have	Essential for operational reliability and preventing costly downtime; necessary for long-term efficiency.
Adaptive Processing	An optional feature is the ability for the system to automatically detect and adjust processing speeds based on factors such as current workload, product size, and product type (e.g., eggs require more delicate handling, while cosmetics require less care).	As an operator, I want the system to adjust processing speeds automatically so that it can optimize performance based on workload and product type.	When workload or product type changes, I want the system to modify its speed so I can ensure efficiency and product safety.	Should Have	Important for optimizing performance across different products, but system can still function without this capability initially.
Simulation	The system could include a simulator for model testing.	As a developer, I want a simulator for model testing so that I can optimize the system before deployment.	When developing new models, I want to test them in a simulation so I can validate performance before real-world use.	Could Have	Although it might enhance implementation, it is not necessary for operation and is useful for testing but not for the system's main functionality.
Camera for System Monitoring	Cameras could be implemented for system monitoring to ensure the robot is classifying items correctly.	As a supervisor, I want cameras to monitor classification accuracy so that I can ensure the robot is sorting items correctly.	When the system is sorting products, I want real-time video monitoring so I can verify classification accuracy.	Should Have	Crucial for maintaining quality control and guaranteeing precise classification, but manual spot inspections could be employed first.
Camera for Safety	Cameras could be implemented for safety monitoring to detect if someone is too close to the robot and trigger an alarm while stopping operation if necessary.	As a safety officer, I want the system to detect if someone is too close so that it can stop operation and trigger an alarm.	When a person enters a restricted area, I want the system to pause and alert staff so I can prevent accidents.	Should Have	Essential for maintaining regulatory compliance and worker safety; avoids mishaps and possible liability problems
Camera for Security	Cameras could be implemented for security monitoring to detect tampering or theft attempts.	As a security manager, I want cameras to detect tampering or theft so that I can prevent unauthorized access.	When suspicious activity occurs near the robot, I want the system to alert security so I can take action.	Could Have	Adds benefit but is not necessary for basic functionality; at first, standard security measures may be adequate.

Conveyor Belt for Transportation	The system should have a conveyor belt to transport pallets once they are full, after the robot has placed the products inside.	As a logistics manager, I want a conveyor belt to transport full pallets so that I can streamline product movement.	When a pallet is full, I want the system to transfer it automatically so I can reduce manual handling.	Should Have	Vital for automating the entire process; without it, a large amount of manual intervention would be needed by the system.
Operation in Slightly Wet Environments	The system could support operation in slightly wet environments.	As an operations manager, I want the system to function in slightly wet environments so that it remains reliable in my facility.	When working in humid or damp conditions, I want the system to continue operating so I can avoid downtime.	Could Have	Would expand usability but not essential for core functionality in all environments
Conveyor Belt Synchronization	The system should synchronize with the conveyor belt.	As an engineer, I want the system to synchronize with the conveyor belt so that item movement is seamless.	When the conveyor belt speed changes, I want the robot to adjust accordingly so I can maintain efficiency.	Must Have	Essential for proper system functioning; without synchronization, the entire process would be inefficient or fail
Survey Extracted Stakeholder Requirements:					
Sorting Process Efficiency	The system should automate repetitive tasks to improve processing time.	As a sorting operator, I want to automate repetitive tasks so that I can reduce processing time.	When sorting materials, I want the system to dynamically adjust for material types to improve accuracy.	Must Have	Core purpose of the system; directly impacts ROI and operational performance
	The system should increase throughput by using high-speed robotic sorting.	As a facility owner, I want to implement high-speed robotic sorting systems to increase throughput.	When processing mixed recyclables, I want to use AI-driven systems to identify materials faster.	Must Have	Direct impact on business outcomes and system effectiveness ; fundamental value proposition
	The system should reduce manual labor costs.	As a manager, I want to reduce manual labor costs by introducing robotics into the sorting process.	When evaluating operational efficiency, I want data on automation reducing manual intervention so I can justify upgrades.	Must Have	Primary business justification for the system; essential for ROI
Cost Optimization	The system should minimize energy consumption during operations.	As an operations manager, I want to reduce energy consumption during sorting processes.	When planning upgrades, I want cost-benefit analyses for new technologies to justify investments.	Should Have	Important for ongoing operational costs but not critical for initial functionality
	Maintenance costs should be reduced by upgrading mechanical systems.	As a business owner, I want to minimize maintenance costs by upgrading mechanical systems.	When evaluating operational costs, I want detailed reports on labor vs automation expenses for decision-making.	Should Have	Important for long-term ROI but not critical for initial system deployment
	The system should provide ROI data before adopting advanced technologies.	As a stakeholder, I want proven ROI data before adopting advanced sorting technologies.	When evaluating automation solutions, I want financial projections so that investments are justified effectively.	Should Have	Important for business decision-making but not essential for the system's operation

Environmental Impact	The system should adopt sustainable methods that align with regulatory compliance and reduce environmental impact.	As an organization, I want to adopt sustainable sorting methods that reduce environmental impact.	When processing organic waste, I want systems that optimize composting conditions for better results.	Should Have	Important for regulatory compliance and corporate responsibility, but may not be critical for initial deployment
	Metrics should be provided on carbon footprint reductions from improved processes.	As a sustainability officer, I want metrics on carbon footprint reductions from improved processes.	When evaluating new systems, I want environmental benefits included in decision-making criteria for sustainability goals.	Could Have	Valuable for reporting and marketing but not essential for core functionality
	Metrics should be provided on carbon footprint reductions from improved processes.	As a sustainability officer, I want metrics on carbon footprint reductions from improved processes.	When evaluating new systems, I want environmental benefits included in decision-making criteria for sustainability goals.	Could Have	Valuable for reporting and marketing but not essential for core functionality
Automation and Technology Integrations	The system should use AI-driven adaptive algorithms for efficient sorting based on input patterns and material types.	As an engineer, I want AI-driven adaptive sorting algorithms that adjust based on input patterns and material types.	When integrating new systems, I want seamless compatibility with existing infrastructure for smooth operations.	Must Have	Core to the system's ability to perform adaptive sorting effectively
	Multi-sensor detection systems should be implemented for improved material identification and classification accuracy.	As a facility manager, I want multi-sensor detection systems for improved material identification during operations.	When introducing automation solutions, I need minimal disruption to ongoing operations during implementation phases.	Should Have	Enhances accuracy but single-sensor systems could provide basic functionality initially
	Cloud-based optimization tools should enable real-time monitoring of sorting performance metrics remotely from any location.	As an operator, I want cloud-based optimization tools for real-time monitoring of sorting performance metrics remotely.	When monitoring performance remotely, I need dashboards that provide actionable insights in real-time for adjustments.	Could Have	Adds convenience but on-site monitoring could suffice initially
Embrace Advanced Technologies	Incorporate emerging technologies like AI, robotics, and sensor systems to enhance sorting operations. A	As a technology lead, I want the sorting system to leverage AI-driven analytics so that we can optimize performance, improve decision-making, and reduce operational costs.	When evaluating automation solutions, I require data on the performance and capabilities of systems based on AI, robotics, and sensor technologies to justify the investment.	Should Have	Important for system performance but specific technologies should be evaluated based on ROI

Category	Stakeholder Requirements	Type	Classification	Rationale	Jira Link
Affordability	As a business owner, I want the system to be cost-effective so that I can minimize operational expenses.	US	Must Have	A key concern for business owners is to "minimize operational expenses" and "justify the investment"; this is crucial for business survival and ROI.	CD R-5
Task Prioritization	As a business owner, when processing fragile items, I want the system to prioritize careful handling over speed so I can reduce product loss.	US	Must Have	Essential for maintaining product integrity; has a direct impact on product loss, which raises expenses and lowers customer satisfaction.	CD R-15
Performance Metrics	As a manager, I want the system to achieve at least half the throughput and accuracy of a human worker so that it remains a viable replacement.	US	Must Have	Establishes the minimal feasible performance standard, which is necessary to assess if the system is functionally sufficient.	CD R-18
Telemetry & Predictive Maintenance	When the system is operating, it tracks efficiency and predicts maintenance needs so that it can prevent unexpected breakdowns.	JS	Should Have	Essential for operational reliability and preventing costly downtime; necessary for long-term efficiency.	CD R-22

Adaptive Processing	When workload or product type changes, the system modifies its speed so that it can ensure efficiency and product safety.	JS	Should Have	Important for optimizing performance across different products, but system can still function without this capability initially.	CD R-25
Simulation	As a developer, I want a simulator for model testing so that I can optimize the system before deployment.	US	Could Have	Although it might enhance implementation, it is not necessary for operation and is useful for testing but not for the system's main functionality.	CD R-27
Camera for System Monitoring	As a developer, I want cameras to monitor classification accuracy so that I can ensure the robot is sorting items correctly.	US	Should Have	Crucial for maintaining quality control and guaranteeing precise classification, but manual spot inspections could be employed first.	CD R-30
Camera for Safety	When a person enters a restricted area, the system pauses and alerts staff so I can prevent accidents.	JS	Should Have	Essential for maintaining regulatory compliance and worker safety; avoids mishaps and possible liability problems	CD R-34
Camera for Security	As a security manager, I want cameras to detect tampering or theft so that I can prevent unauthorized access.	US	Could Have	Adds benefit but is not necessary for basic functionality; at first, standard security measures may be adequate.	CD R-33
Conveyor Belt for Transportation	When a pallet is full, the system transfers it automatically so that it can reduce manual handling.	JS	Should Have	Vital for automating the entire process; without it, a large amount of manual intervention would be needed by the system.	CD R-38
Operation in Slightly Wet Environments	When working in humid or damp conditions, the system continues operating so that it can avoid downtime.	JS	Could Have	Would expand usability but not essential for core functionality in all environments	CD R-45
Conveyor Belt Synchronization	When the conveyor belt speed changes, the robot to adjust accordingly so that it can maintain efficiency.	JS	Must Have	Essential for proper system functioning; without synchronization, the entire process would be inefficient or fail	CD R-40
Survey Extracted Stakeholder Requirements:					
Sorting Process Efficiency	As a sorting operator, I want to automate repetitive tasks so that I can reduce processing time.		Must Have	Core purpose of the system; directly impacts ROI and operational performance	CD R-10 CD R-12
	As a facility owner, I want to implement high-speed robotic sorting systems to increase throughput.  CDR-50 - Jira server returned an error: [Ljava.lang.Object;@56cd9cba		Must Have	Direct impact on business outcomes and system effectiveness; fundamental value proposition	
	As a manager, I want to reduce manual labor costs by introducing robotics into the sorting process.  CDR-52 - Jira server returned an error: [Ljava.lang.Object;@159c3591		Must Have	Primary business justification for the system; essential for ROI	
Cost Optimization	As an operations manager, I want to reduce energy consumption during sorting processes.		Should Have	Important for ongoing operational costs but not critical for initial functionality	
	As a business owner, I want to minimize maintenance costs by upgrading mechanical systems.		Should Have	Important for long-term ROI but not critical for initial system deployment	
	As a stakeholder, I want proven ROI data before adopting advanced sorting technologies.		Should Have	Important for business decision-making but not essential for the system's operation	
Environmental Impact	As an organization, I want to adopt sustainable sorting methods that reduce environmental impact.		Should Have	Important for regulatory compliance and corporate responsibility, but may not be critical for initial deployment	CD R-42 CD R-44
	As a sustainability officer, I want metrics on carbon footprint reductions from improved processes.		Could Have	Valuable for reporting and marketing but not essential for core functionality	
	As a sustainability officer, I want metrics on carbon footprint reductions from improved processes.		Could Have	Valuable for reporting and marketing but not essential for core functionality	

Automation and Technology Integration	As an engineer, I want AI-driven adaptive sorting algorithms that adjust based on input patterns and material types.		Must Have	Core to the system's ability to perform adaptive sorting effectively	CD R-11 CD R-13
	As a facility manager, I want multi-sensor detection systems for improved material identification during operations.		Should Have	Enhances accuracy but single-sensor systems could provide basic functionality initially	
	As an operator, I want cloud-based optimization tools for real-time monitoring of sorting performance metrics remotely.		Could Have	Adds convenience but on-site monitoring could suffice initially	

Classes

 **CDR-4 - Jira server returned an error: [Ljava.lang.Object; @78e929ba**

- **Stakeholder Requirement:** The solution should be affordable and optimize operational costs.
- **User Stories:**

○

 **CDR-5 - Jira server returned an error: [Ljava.lang.Object; @e11d29b**

○

 **CDR-6 - Jira server returned an error: [Ljava.lang.Object; @29cb0fd1**

- **Job Stories:**

○

 **CDR-7 - Jira server returned an error: [Ljava.lang.Object; @63b494e9**

○

 **CDR-8 - Jira server returned an error: [Ljava.lang.Object; @1d1a5373**

 **CDR-9 - Jira server returned an error: [Ljava.lang.Object; @7c49f08f**

- **Stakeholder Requirement:** The system should optimize sorting efficiency through automation.
- **User Stories:**

○

 **CDR-10 - Jira server returned an error: [Ljava.lang.Object; @67bb6025**

-  CDR-11 - Jira server returned an error: [Ljava.lang.Object;
@19e00589

- **Job Stories:**

-  CDR-12 - Jira server returned an error: [Ljava.lang.Object;
@36bfe925

-  CDR-13 - Jira server returned an error: [Ljava.lang.Object;
@414a7d96

 CDR-14 - Jira server returned an error: [Ljava.lang.Object;
@43f76e3e

- **Stakeholder Requirement:** The solution should prioritize delicate tasks over processing speed.
- **User Stories:**

-  CDR-15 - Jira server returned an error: [Ljava.lang.Object;
@323b9b4d

- **Job Stories:**

-  CDR-16 - Jira server returned an error: [Ljava.lang.Object;
@6f5c05de

 CDR-17 - Jira server returned an error: [Ljava.lang.Object;
@793b0adb

- **Stakeholder Requirement:** The system's success metric should be achieving at least half the throughput and accuracy of a human worker.
- **User Stories:**

-  CDR-18 - Jira server returned an error: [Ljava.lang.Object;
@5d01f2f5

- **Job Stories:**

-  **CDR-19** - Jira server returned an error: [Ljava.lang.Object;
@7a9652e8

 **CDR-20** - Jira server returned an error: [Ljava.lang.Object;
@663d10fe

- **Stakeholder Requirement:** The system must collect telemetry data to improve efficiency and predict failures.
- **User Stories:**

-  **CDR-21** - Jira server returned an error: [Ljava.lang.Object;
@4dea38a5

- **Job Stories:**

-  **CDR-22** - Jira server returned an error: [Ljava.lang.Object;
@731f4cf4

Material Handling Challenges & Worker Safety

- **Stakeholder Requirement:** The system should support proper material handling and ensure worker safety.
- **User Stories:**
 - As a worker, I want training on handling diverse materials so that I can improve sorting accuracy.
 - As a worker, I want safety measures in place when operating near automated machinery.
- **Job Stories:**
 - When handling heavy construction materials, I want automated lifting systems to reduce physical strain.
 - When training new employees, I want user-friendly interfaces for learning automated systems quickly.

 **CDR-23** - Jira server returned an error: [Ljava.lang.Object;
@c4ccf23

- **Stakeholder Requirement:** An optional feature is the ability for the system to automatically adjust processing speeds based on workload, product size, and type.
- **User Stories:**

-  **CDR-24** - Jira server returned an error: [Ljava.lang.Object;
@28eba63a

- **Job Stories:**

-  **CDR-25** - Jira server returned an error: [Ljava.lang.Object;
@520c8ac4

 **CDR-26** - Jira server returned an error: [Ljava.lang.Object;
@6430b936

- **Stakeholder Requirement:** The system could include a simulator for model testing.
- **User Stories:**

-  **CDR-27** - Jira server returned an error: [Ljava.lang.Object;
@4d08e53c

- **Job Stories:**

-  **CDR-28** - Jira server returned an error: [Ljava.lang.Object;
@30f81fe0

 **CDR-29** - Jira server returned an error: [Ljava.lang.Object;
@59370890

System Monitoring

- **Stakeholder Requirement:** Cameras could be implemented for system monitoring.
- **User Stories:**

-  **CDR-30** - Jira server returned an error: [Ljava.lang.Object;
@439e9730

- **Job Stories:**

-  **CDR-31** - Jira server returned an error: [Ljava.lang.Object;
@42d2233e

Safety Monitoring

- **Stakeholder Requirement:** Cameras could detect if someone is too close and stop operation if necessary.
- **User Stories:**

-  CDR-32 - Jira server returned an error: [Ljava.lang.Object;
@6147f469

- **Job Stories:**

-  CDR-34 - Jira server returned an error: [Ljava.lang.Object;
@58e2fe2d

Security Monitoring

- **Stakeholder Requirement:** Cameras could detect tampering or theft attempts.
- **User Stories:**

-  CDR-33 - Jira server returned an error: [Ljava.lang.Object;
@433ff5f2

- **Job Stories:**

-  CDR-35 - Jira server returned an error: [Ljava.lang.Object;
@12dd9506

 **CDR-36 - Jira server returned an error: [Ljava.lang.Object;
@41574b5**

Transportation

- **Stakeholder Requirement:** The system should have a conveyor belt to transport pallets once they are full.
- **User Stories:**

-  CDR-37 - Jira server returned an error: [Ljava.lang.Object;
@54be0132

- **Job Stories:**

-  CDR-38 - Jira server returned an error: [Ljava.lang.Object;
@33372472

Synchronization

- **Stakeholder Requirement:** The system should synchronize with the conveyor belt.
- **User Stories:**

-  CDR-39 - Jira server returned an error: [Ljava.lang.Object;
@41cdc118

- **Job Stories:**

-  CDR-40 - Jira server returned an error: [Ljava.lang.Object;
@f04d719

 CDR-41 - Jira server returned an error: [Ljava.lang.Object;
@1264f035

- **Stakeholder Requirement:** The system should support sustainable methods and adapt to certain environmental conditions.
- **User Stories:**

-  CDR-42 - Jira server returned an error: [Ljava.lang.Object;
@7915f3e0

-  CDR-43 - Jira server returned an error: [Ljava.lang.Object;
@66f2a9bd

- **Job Stories:**

-  CDR-44 - Jira server returned an error: [Ljava.lang.Object;
@11003682

-  CDR-45 - Jira server returned an error: [Ljava.lang.Object;
@36409cd2

ID	Requirement Type	Stakeholder	Description	Acceptance Criteria	Priority	Status	AGC-ID
----	------------------	-------------	-------------	---------------------	----------	--------	--------

SHR-01	User Story	Farmers	As a farmer, I want an autonomous vehicle to transport materials between field and storage so that I reduce time spent on repetitive transport tasks.	The vehicle transports materials autonomously between predefined points in at least 95% of tasks.	Must Have	In Work	 DRS -2 - Jira serv er retur ned an error : [Ljav a. lang. Obje ct; @a1 124b
SHR-02	User Story	Farmers	As a farm operator, I want to monitor the autonomous vehicle remotely so that I can supervise its activity and intervene if needed.	The system provides real-time monitoring via dashboard or mobile interface.	Should Have	Backlog	 DRS -3 - Jira serv er retur ned an error : [Ljav a. lang. Obje ct; @13 f317 59

SHR-03	Job Story	Farmers	When harvest operations are running continuously, I want automated transport support so that harvesting machines do not stop due to lack of transport vehicles.	Transport delays do not exceed a defined threshold (e.g., 2 minutes).	Should Have	Backlog	 DRS -4 - Jira serv er retur ned an error : [Ljav a. lang. Obje ct; @10 d976 f1
SHR-04	User Story	Farmers / Entrepreneurs	As a user, I want the vehicle to follow predefined routes so that transportation tasks can be executed consistently.	The vehicle follows routes with minimal deviation (e.g., <1 meter).	Must Have	Backlog	 DRS -5 - Jira serv er retur ned an error : [Ljav a. lang. Obje ct; @bc 0c60 5

SHR-05	Job Story	Farmers / Entrepreneurs	When working in environments with limited workforce, I want automation to perform repetitive tasks so that labor shortages do not affect productivity.	The system replaces at least one manual transport role.	Should Have	Backlog	 DRS -6 - Jira serv er retur ned an error : [Ljav a. lang. Obje ct; @12 002c 55
SHR-06	User Story	Farmers / Entrepreneurs	As a user, I want the vehicle to detect obstacles automatically so that accidents with people or equipment are avoided.	Obstacles are detected and avoided in all test scenarios.	Must Have	Backlog	 DRS -7 - Jira serv er retur ned an error : [Ljav a. lang. Obje ct; @79 514b 19

SHR-07	User Story	Farmers	As a farm operator, I want a manual override or emergency stop so that I can immediately stop the system if something goes wrong.	The system stops within <1 second after emergency input.	Must Have	Backlog	 DRS -8 - Jira serv er retur ned an error : [Ljav a. lang. Obje ct; @72 ea2f 03
SHR-08	Job Story	Farmers / Entrepreneurs	When the vehicle operates in farm yards or industrial spaces, I want reliable proximity detection so that collisions with equipment are prevented.	The system maintains a safe distance from obstacles at all times.	Must Have	Backlog	 DRS -9 - Jira serv er retur ned an error : [Ljav a. lang. Obje ct; @ee 350c 8

SHR-09	User Story	Farmers / Entrepreneurs	As a user, I want the system to operate reliably during long working hours so that operations are not interrupted during peak season.	The system operates continuously for 8–10 hours without failure.	Must Have	Backlog	 DRS -10 - Jira serv er retur ned an error : [Ljav a. lang. Obj ct; @37 ca2b b1
SHR-10	Job Story	Farmers	When equipment breaks down or transport is delayed, I want an alternative automated transport solution so that workflow disruption is minimized.	The system resumes or reassigns tasks within a defined recovery time.	Could Have	Backlog	 DRS -11 - Jira serv er retur ned an error : [Ljav a. lang. Obj ct; @2b 42ca 44

SHR-11	User Story	Entrepreneurs	As a business owner, I want the system to reduce operational transport costs so that the investment becomes financially viable.	Cost reduction is measurable compared to manual transport.	Should Have	Backlog	 DRS -12 - Jira serv er retur ned an error : [Ljav a. lang. Obje ct; @5d 2422 cc
SHR-12	User Story	Entrepreneurs	As a user, I want the vehicle to transport tools and materials across the workspace so that workers can focus on higher-value tasks.	Transport tasks are completed without worker involvement.	Must Have	Backlog	 DRS -13 - Jira serv er retur ned an error : [Ljav a. lang. Obje ct; @43 8dbe 76

SHR-13	Job Story	Farmers / Entrepreneurs	When operating in large yards or fields, I want automated logistics support so that materials can be moved efficiently between working zones.	Transport time is reduced compared to manual processes.	Should Have	Backlog	 DRS -14 - Jira serv er retur ned an error : [Ljav a. lang. Obje ct; @55 7cfe 3d
SHR-14	User Story	Farmers / Entrepreneurs	As a user, I want the system to provide status information (location, battery level, operation status) so that I always know the vehicle condition.	Status data is updated in real time (<5 seconds delay).	Should Have	Backlog	 DRS -15 - Jira serv er retur ned an error : [Ljav a. lang. Obje ct; @75 51ab 90

SHR-15	User Story	Farmers / Entrepreneurs	As a user, I want the system to be easy to use without complex configuration so that it can be adopted by non-technical operators.	System setup can be completed within <10 minutes by a new user.	Could Have	Backlog	 DRS -16 - Jira serv er retur ned an error : [Ljav a. lang. Obje ct; @28 d02a 2e
SHR-16	User Story	Farmers / Entrepreneurs	As an operator, I want to send the vehicle to specific GPS coordinates so that it can perform transport tasks anywhere in the working area.	The vehicle reaches given coordinates with acceptable accuracy.	Must Have	Backlog	 DRS -17 - Jira serv er retur ned an error : [Ljav a. lang. Obje ct; @e8 510c 2

SHR-17	User Story	Farmers / Entrepreneurs	As a user, I want the vehicle to automatically return to its starting point so that transport cycles can be completed without manual retrieval.	The vehicle autonomously returns after completing tasks.	Must Have	Backlog	 DRS -18 - Jira serv er retur ned an error : [Ljav a. lang. Obje ct; @9b d5f1 c
SHR-18	Job Story	Farmers	When workers are harvesting fruits or vegetables, I want the vehicle to move forward incrementally so that pickers can continue working without manually repositioning transport equipment.	The vehicle moves forward in controlled increments on command.	Should Have	Backlog	 DRS -19 - Jira serv er retur ned an error : [Ljav a. lang. Obje ct; @44 1af9 19

SHR-19	User Story	Farmers / Entrepreneurs	As a user, I want the vehicle to be controlled using a simple interface such as a button or mobile command so that non-technical workers can operate it easily.	Users can operate the system with minimal training.	Must Have	Backlog	
SHR-20	Job Story	Farmers / Entrepreneurs	When the vehicle encounters an obstacle and cannot continue moving, I want the system to notify the operator so that the issue can be resolved quickly.	The system sends alerts after a defined blocking time.	Should Have	Backlog	
SHR-21	User Story	Farmers / Entrepreneurs	As an operator, I want to remotely control the vehicle through a dashboard or mobile interface so that I can resolve unexpected situations when autonomous operation is interrupted.	Remote control works with minimal latency.	Should Have	Backlog	
SHR-22	User Story	Farmers / Entrepreneurs	As a system operator, I want a dashboard that displays vehicle location and operational data so that I can monitor its activity in real time.	Dashboard reflects accurate real-time data.	Should Have	Backlog	
SHR-23	User Story	Farmers / Entrepreneurs	As an operator, I want the vehicle to record manually driven routes so that it can repeat the same routes autonomously in the future.	Routes can be recorded and replayed successfully.	Should Have	Backlog	
SHR-24	Job Story	Farmers / Entrepreneurs	When multiple vehicles are deployed in the same environment, I want each vehicle to send operational data to a central system so that fleet activity can be monitored.	Multiple vehicles transmit data to a shared system.	Could Have	Backlog	
SHR-25	User Story	Farmers / Entrepreneurs	As a worker in the operational area, I want the vehicle to produce sound or light signals while moving so that people nearby are aware of its activity.	Vehicle emits signals during movement.	Must Have	Backlog	
SHR-26	Job Story	Farmers / Entrepreneurs	When the system is deployed in a controlled environment, I want it to follow predefined scenarios rather than complex autonomous navigation so that the system remains reliable and predictable.	Predefined scenarios are executed consistently.	Must Have	Backlog	
SHR-27	User Story	Farmers / Entrepreneurs	As an operator, I want to access video from onboard cameras so that I can inspect the vehicle surroundings when a problem occurs.	Video feed is accessible when requested.	Could Have	Backlog	
SHR-28	Job Story	Technical Team	When validating the system before deployment, I want the vehicle to complete a predefined autonomous route so that the functionality of navigation can be confirmed.	Validation route is completed successfully without intervention.	Must Have	Backlog	

ID	Requirement Description	Requirement Type	Priority	Epic	Source (Stakeholder Requirement)	DRS-ID	Status
SR-01	The system shall autonomously transport materials between predefined locations without human intervention.	Functional	Must Have	Transport Automation	R1, R12, R3, R5	 DRS-2 - Jira server returned an error: [Ljava.lang.Object;@64b8011	Backlog
SR-02	The system shall support automated logistics operations across large working areas.	Functional	Should Have	Logistics Efficiency	R13	 DRS-4 - Jira server returned an error: [Ljava.lang.Object;@7f41b929	Backlog
SR-03	The system shall follow predefined routes for navigation.	Functional	Must Have	Navigation and Routing	R4	 DRS-6 - Jira server returned an error: [Ljava.lang.Object;@26776ffa	Backlog

SR-04	The system shall navigate to specified GPS coordinates.	Functional	Must Have	Navigation and Routing	R16	 DRS-13 - Jira server returned an error: [Ljava.lang.Object;@43db3b4c	Backlog
SR-05	The system shall automatically return to a predefined home location after task completion.	Functional	Must Have	Navigation and Routing	R17	 DRS-3 - Jira server returned an error: [Ljava.lang.Object;@584d67ce	Backlog
SR-06	The system shall record and reuse manually defined routes for autonomous navigation.	Functional	Should Have	Navigation and Routing	R23	 DRS-9 - Jira server returned an error: [Ljava.lang.Object;@50b73323	Backlog
SR-07	The system shall execute predefined navigation scenarios in controlled environments.	Functional	Must Have	Navigation and Routing	R26	 DRS-5 - Jira server returned an error: [Ljava.lang.Object;@25c16c8d	Backlog
SR-08	The system shall detect obstacles in real time using onboard sensors.	Functional	Must Have	Safety	R6	 DRS-8 - Jira server returned an error: [Ljava.lang.Object;@75a6fbb0	Backlog
SR-09	The system shall stop or avoid movement when obstacles are detected within a safety threshold.	Functional	Must Have	Safety	R6, R8	 DRS-11 - Jira server returned an error: [Ljava.lang.Object;@551ffa1c	Backlog

SR-10	The system shall include an emergency stop mechanism that halts all operations immediately.	Functional	Must Have	Safety	R7	 DRS-7 - Jira server returned an error: [Ljava.lang. Object;@1ac0c8c1	Backlog
SR-11	The system shall emit visual or audio signals while moving to ensure safety awareness.	Functional	Must Have	Safety	R25	 DRS-12 - Jira server returned an error: [Ljava.lang. Object;@e227f02	Backlog
SR-12	The system shall provide real-time monitoring of vehicle status, including location and battery level.	Functional	Should Have	Monitoring and Control	R14	 DRS-14 - Jira server returned an error: [Ljava.lang. Object;@70d20a70	Backlog
SR-13	The system shall provide a remote monitoring interface accessible via web or mobile devices.	Functional	Should Have	Monitoring and Control	R2	 DRS-15 - Jira server returned an error: [Ljava.lang. Object;@3a2a9308	Backlog
SR-14	The system shall notify the operator when the vehicle is blocked or unable to proceed.	Functional	Should Have	Monitoring and Control	R20	 DRS-16 - Jira server returned an error: [Ljava.lang. Object;@751ca1aa	Backlog
SR-15	The system shall allow remote manual control of the vehicle.	Functional	Should Have	Monitoring and Control	R21	 DRS-17 - Jira server returned an error: [Ljava.lang. Object;@6f263c72	Backlog

SR-16	The system shall provide a dashboard displaying real-time operational data.	Functional	Should Have	Monitoring and Control	R22, R27	 DRS-18 - Jira server returned an error: [Ljava.lang.Object;@39c6d2f1	Backlog
SR-17	The system shall support incremental movement control for harvesting scenarios.	Functional	Should Have	Logistics Efficiency	R18	 DRS-19 - Jira server returned an error: [Ljava.lang.Object;@4bc9a682	Backlog
SR-18	The system shall support communication between multiple vehicles and a central system.	Functional	Could Have	Logistics Efficiency	R24	 DRS-24 - Jira server returned an error: [Ljava.lang.Object;@70cb80ee	Backlog
SR-19	The system shall operate continuously for extended periods without critical failure.	Non-functional	Must Have	System Reliability	R9	 DRS-28 - Jira server returned an error: [Ljava.lang.Object;@2e20849c	Backlog
SR-20	The system shall provide a simple and intuitive user interface for non-technical users.	Non-functional	Must Have	User Experience	R15, R19	 DRS-29 - Jira server returned an error: [Ljava.lang.Object;@129ea8e0	Backlog

Architectural Design

Key System Layers

We organize your components into **3 logical layers**:

1. User Layer

- Operator
- Web Dashboard (UI)

2. Cloud Layer

- Cloud Backend
- Database (Operational DB)

3. Edge / Vehicle Layer (IMPORTANT)

- Mini PC (Main Controller)
- Arduino (Low-level control)
- GPS / RTK Module
- Parktronic Sensors (Ultrasonic)
- Camera (Image Collector)
- Steering & Motor System
- (Optional) Human Interaction Layer

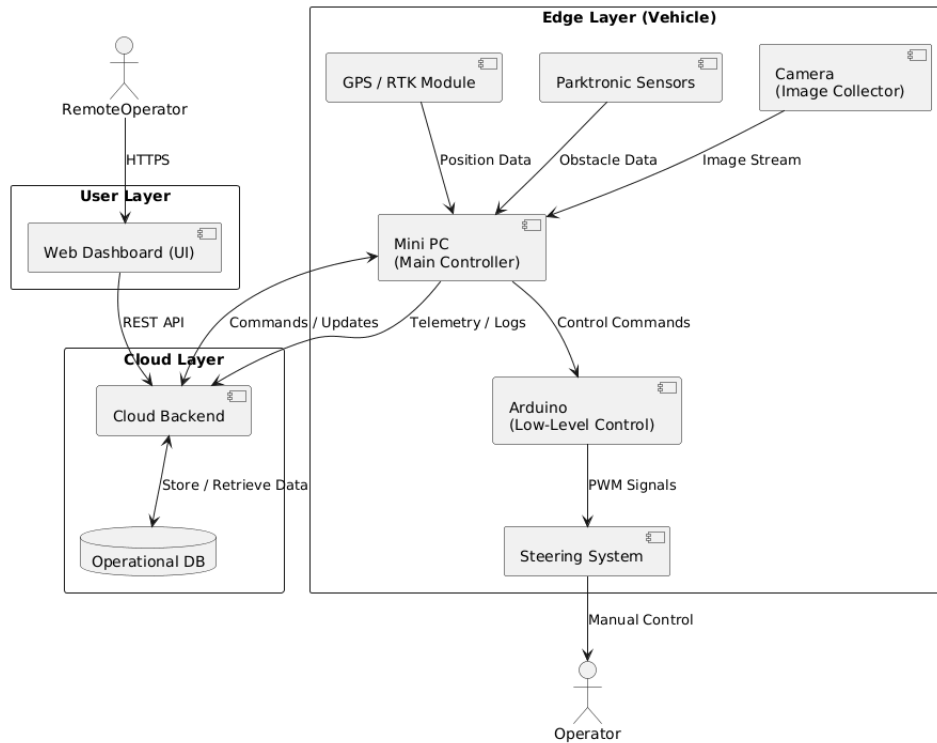


Figure 1: Main Components Diagram

Component Details and Interfaces

Mini PC System

The **Mini PC** represents the central processing unit of the entire system and acts as the main decision-making component. It is responsible for receiving data from all sensors, processing this information in real time, and generating the commands required to control the vehicle. In other words, it functions as the "brain" of the autonomous golf cart.

The MiniPC class coordinates the overall system operation. It initializes the system, processes incoming sensor data, executes navigation logic, and communicates with both the Arduino controller and the cloud backend. By placing the main intelligence in the Mini PC instead of the cloud, the system avoids latency issues and can react immediately to environmental changes such as obstacles or route deviations.

The internal structure of the Mini PC is divided into several functional modules. The NavigationEngine class is responsible for route planning and route execution. It contains functions such as `planRoute()` and `followRoute()` that allow the vehicle to move autonomously between predefined locations. The function `returnToHome()` ensures that the vehicle can automatically return to its starting position after completing a task.

The SensorFusion class plays a very important role in the system because it combines information coming from different sensors. Instead of treating GPS data, ultrasonic sensor data, and camera input separately, this module processes them together using functions such as `mergeSensorData()`. This allows the system to make more accurate decisions and increases the reliability of obstacle detection and navigation.

The DecisionEngine class is responsible for interpreting the processed sensor data and converting it into actions. For example, the function `detectObstacle()` determines whether the vehicle can continue moving safely, while `calculateMovement()` decides how the vehicle should move in the current situation. If a dangerous situation occurs, the function `triggerEmergencyStop()` immediately interrupts the navigation process.

Finally, the CommunicationManager class ensures the interaction between the Mini PC and the cloud system. It sends telemetry data such as position and battery level using `sendTelemetry()` and receives commands from the operator through `receiveCommands()`. This module makes remote monitoring and control possible without affecting the real-time performance of the vehicle.

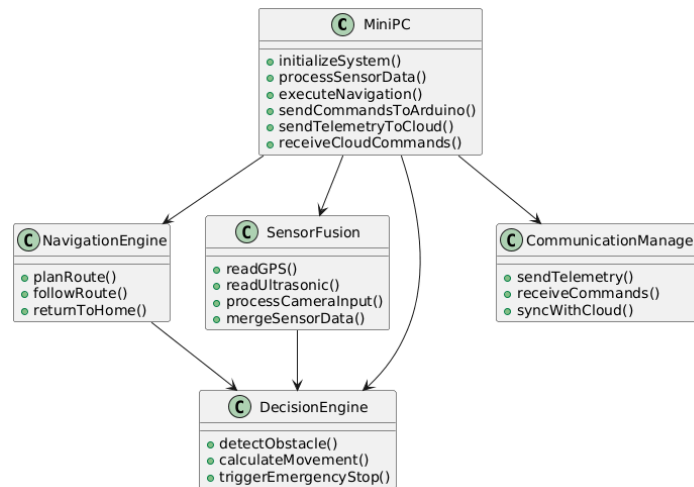


Figure 2: Mini PC Component Diagram

Arduino (Low-Level Controller) System

The Arduino controller represents the **low-level control layer** of the system. While the Mini PC makes high-level decisions, the Arduino is responsible for executing these decisions in a stable and predictable way. This separation between high-level and low-level control is very common in autonomous and robotic systems because it improves reliability and safety.

The ArduinoController class receives commands from the Mini PC using the function `receiveCommand()` and executes them using `executeMovement()`. It also includes the function `emergencyStop()`, which allows the system to stop the vehicle immediately if a dangerous situation is detected. This function is especially important in agricultural environments where people and machinery operate in the same space.

Inside the Arduino controller, the CommandParser class converts high-level commands into specific control signals. For example, a command such as "move forward" must be translated into a combination of speed and direction values. The function `parseCommand()` performs this translation, while `validateCommand()` ensures that the received command is correct before it is executed.

The MotorController class is responsible for the physical movement of the vehicle. It controls the speed and direction of the motors using functions such as `setSpeed()` and `setDirection()`. The function `stopMotor()` is used when the vehicle needs to stop safely or when an obstacle is detected.

The SafetyModule class ensures that the system operates safely even in unexpected situations. It continuously monitors the signals received from the Mini PC and triggers the `triggerEmergencyStop()` function if a problem is detected. This makes the Arduino controller a critical safety component of the entire system.

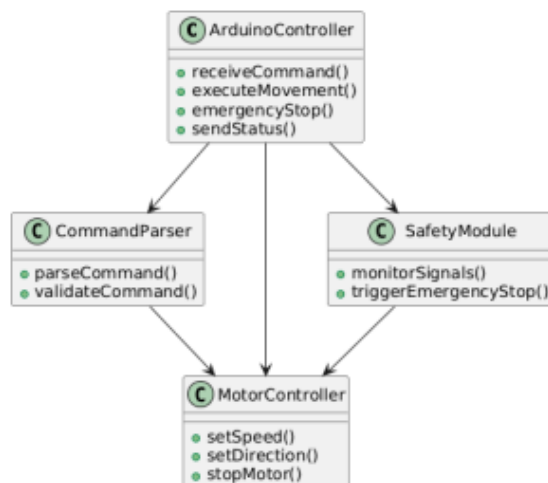


Figure 2: Arduino Component Diagram

Web Dashboard (UI) System

The Web Dashboard represents the **main interaction point between the user and the system**. It allows the operator to monitor the vehicle in real time and to send commands without directly interacting with the hardware. This is especially important in agricultural environments where the operator may need to supervise several processes at the same time.

The WebDashboard class coordinates the entire user interface. It provides functions such as displayDashboard(), showVehicleStatus(), sendCommand(), and showAlerts(). These functions allow the operator to observe the system state and react quickly if a problem occurs.

The DashboardView class is responsible for presenting the system information in a clear and understandable way. It includes functions such as renderMetrics() and renderMap(), which allow the user to see the vehicle position, speed, and operational status. The function updateStatus() ensures that the information displayed on the screen is updated in real time.

The ControlPanel class provides the functionality required to send commands to the vehicle. For example, the operator can start or stop the vehicle using the function sendMoveCommand() or activate manual control using activateManualControl().

The AlertManager class is used to inform the operator about important events. Functions such as displayAlert() and notifyUser() are used when the vehicle detects an obstacle or when communication with the system is interrupted.

Finally, the APIClient class connects the Web Dashboard to the cloud backend. It sends requests using sendRequest() and receives updated information using getUpdates(). This component ensures that the user interface remains synchronized with the actual state of the vehicle.

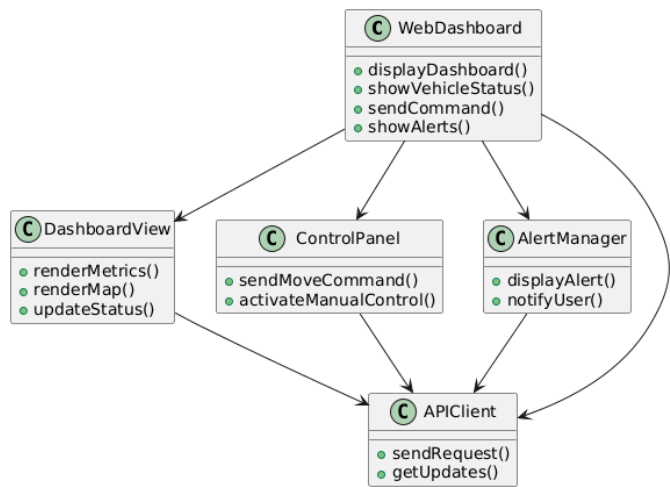


Figure 3: Web Dashboard Component Diagram

Cloud Backend System

The Cloud Backend represents the **data management and communication layer** of the system. It connects the Web Dashboard with the Mini PC and provides a centralized environment where system data can be stored and processed.

The CloudBackend class is responsible for handling user requests and managing system data. It includes functions such as processRequest() and store telemetry(), which allow the system to save information about the vehicle's position, activity, and performance. The function sendCommandToVehicle() allows the operator to control the vehicle remotely through the web interface.

The APIGateway class acts as an intermediary between the user interface and the backend services. It receives requests from the Web Dashboard and verifies them using the function validateUser(). The function handleRequest() ensures that each request is processed correctly and securely.

The DataManager class is responsible for storing and retrieving information from the database. It includes functions such as storeData() and retrieveData(), which allow the system to maintain a history of vehicle operations. The function saveLogs() is especially important for performance analysis and system testing.

The CommandDispatcher class ensures that commands received from the operator are transmitted to the Mini PC. Functions such as sendCommand() and queueCommand() are used to guarantee that the vehicle receives the correct instructions even when multiple commands are sent at the same time.

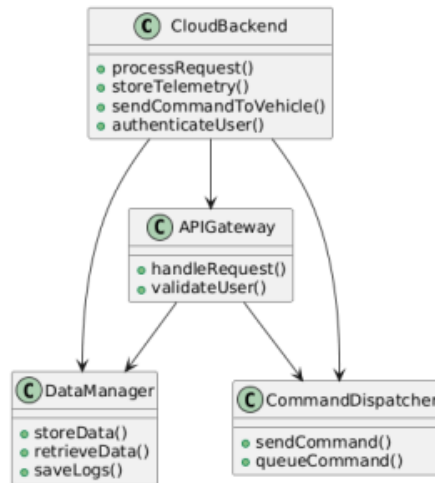


Figure 4: Cloud Backend Component Diagram

Sensor System

The Sensor System represents the **perception layer** of the autonomous vehicle. Its main purpose is to collect environmental data and transmit it to the Mini PC, where it can be analyzed and used for navigation and safety decisions.

The **SensorSystem** class coordinates the entire sensing process. It collects information from multiple sensors using the function `collectData()` and sends this information to the Mini PC using `sendSensorData()`.

The **GPSSystem** class provides the position of the vehicle. The function `getCoordinates()` returns the current geographic location, while `getAccuracy()` indicates the precision of the measurement. This information is essential for autonomous navigation and route tracking.

The **UltrasonicSensors** class is responsible for detecting obstacles. The function `measureDistance()` determines how far an object is from the vehicle, and `detectObstacle()` identifies whether the vehicle should stop or change direction.

The **CameraModule** class captures visual information about the environment. Functions such as `captureImage()` and `streamVideo()` allow the system to provide visual monitoring and support advanced features such as image recognition.

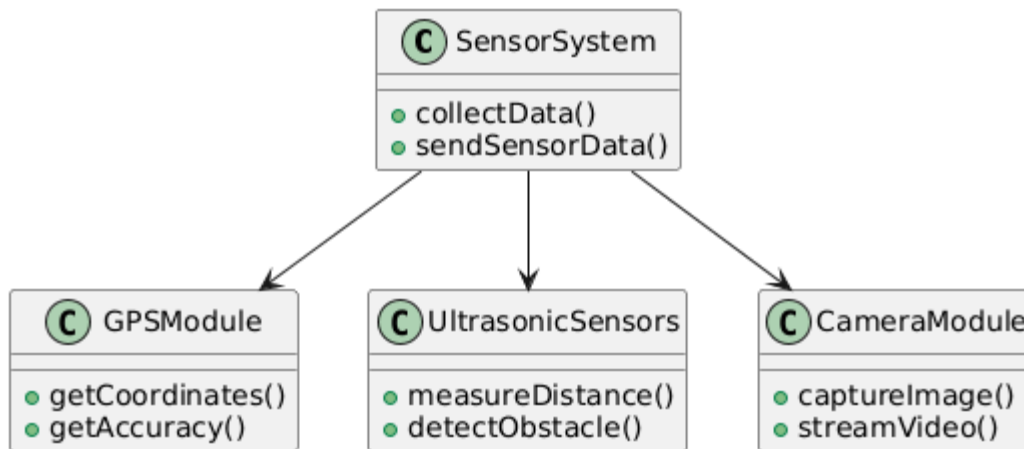


Figure 5: Sensor Component Diagram

Steering System

The Steering System represents the **mechanical actuation layer** of the vehicle. It converts electrical signals received from the Arduino controller into physical movement.

The **SteeringSystem** class defines the main movement functions of the vehicle, including `turnLeft()`, `turnRight()`, and `stopSteering()`. These functions allow the vehicle to follow a predefined route and avoid obstacles.

The **MotorDriver** class is responsible for generating the electrical signals required to control the steering mechanism. The function `applyPWM()` adjusts the motor speed, while `controlDirection()` determines the movement direction.

The SteeringMechanism class represents the physical structure that moves the wheels. The function rotateWheels() converts the electrical signals into mechanical motion.

The ManualControl class allows the operator to take control of the vehicle when necessary. Functions such as takeControl()and releaseControl() make it possible to switch between autonomous and manual operation safely.

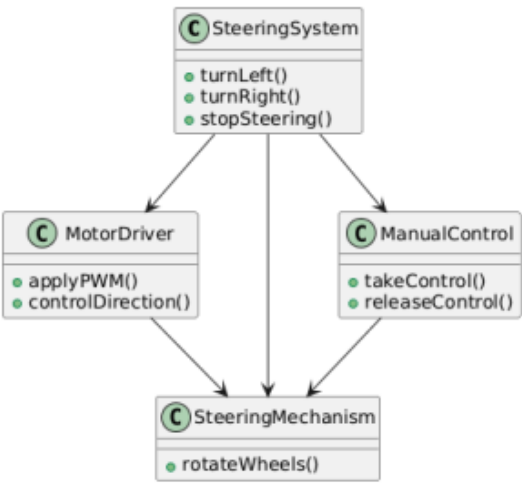


Figure 6: Steering Component Diagram

The autonomous yard vehicle system operates in multiple dynamic modes to ensure safe operation, autonomous execution, and operator control while maintaining system integrity.

Mode	Purpose	Key Characteristics
Initialization	System startup and validation	Safety checks, sensor calibration, cloud connection
Autonomous Operation	Execution of transport tasks	Continuous navigation, telemetry logging
Manual Control	Operator-driven movement	Incremental movement, direct control
Safety Stop	Emergency handling	Immediate stop, system lock
Monitoring	System supervision	Movement tracking, alert generation

1. Core Processing System (Mini PC + Arduino)

The Core Processing System coordinates all subsystems of the autonomous yard vehicle. The Mini PC manages high-level logic and communication, while the Arduino executes real-time actuator control.

Architectural Element: Activity Diagram – System Initialization

Architectural Element	Description
-----------------------	-------------

Activity Diagram	System Initialization <pre>graph TD; Start(()) --> A1[Operator triggers Start System]; A1 --> A2[Web Dashboard sends Initialization Request]; A2 --> A3[Cloud Backend signals Mini PC to Start]; A3 --> subgraph Safety_Initialization [Safety Initialization]; A4[Mini PC initializes safety systems via Arduino] --> A5[Arduino performs Safety Check]; A5 --> A6{Safety OK?}; A6 -- yes --> A7[Safety Ready]; A6 -- no --> A8[Report Safety Error]; A8 --> End1((()); end; A7 --> subgraph Sensor_Navigation [Sensor & Navigation]; A9[Initialize and calibrate Sensors] --> A10[Load routes and get GPS position]; A10 --> A11[Calibration complete]; end; A11 --> subgraph Connectivity_Data [Connectivity & Data]; A12[Establish Cloud connection] --> A13[Initialize Data Services & Logging]; A13 --> A14[Sync with Operational DB]; end; A14 --> A15[Initialization Complete]; A15 --> A16[Web Dashboard displays Standby Mode]; A16 --> End2((());</pre> The flowchart illustrates the system initialization process. It begins with an operator triggering the start system, followed by a request from the web dashboard to the cloud backend, which then signals the mini PC to start. The process then enters a 'Safety Initialization' phase where the mini PC initializes safety systems via an Arduino. The Arduino performs a safety check, and if it fails, an error is reported. If successful, the system moves to 'Sensor & Navigation' to initialize and calibrate sensors and load routes. This is followed by 'Connectivity & Data' where a cloud connection is established, data services are initialized, and the system syncs with the operational database. Finally, the system completes initialization and the web dashboard displays standby mode.
Description	This diagram illustrates the initialization workflow of the autonomous yard vehicle. The operator triggers system startup through the dashboard, followed by coordinated initialization of safety systems, sensors, and communication modules. The system transitions to standby mode only after all subsystems are validated.

- Critical Interactions**
- Safety validation must complete before actuator enablement
 - Sensor calibration must complete before navigation
 - Cloud failure must not block startup

- Connection to Requirements**
- Supports safe startup and validation (SR-08)
 - Ensures subsystem readiness (SR-10)

Architectural Element: Activity Diagram – Autonomous Transport Execution

Architectural Element	Description
-----------------------	-------------

Activity Diagram	Autonomous Transport Execution <pre>graph TD; Start(()) --> A1[Operator sends transport command]; A1 --> A2[Cloud Backend sends task to Mini PC]; A2 --> A3{ }; A3 --> A4[Mini PC sends control commands to Arduino]; A4 --> A5[Arduino sends PWM signals to Steering System]; A5 --> A6[Arduino receives status feedback]; A6 --> A7[Mini PC logs telemetry and execution status]; A7 --> A8[Cloud Backend updates Web Dashboard]; A8 --> A9{Destination Reached?}; A9 -- yes --> End((())) ; A9 -- no --> A3 ; A3 --> A10[Continue Route]; A10 --> A3 ;</pre> The flowchart illustrates the autonomous transport execution process. It begins with an operator sending a transport command, which is then processed by the cloud backend and sent to the Mini PC. The Mini PC sends control commands to the Arduino, which sends PWM signals to the steering system. The Arduino receives status feedback, which is logged by the Mini PC and updated on the cloud backend. A decision point checks if the destination is reached. If yes, the process ends. If no, it loops back to the Mini PC sending control commands. A 'Continue Route' path also loops back to the Mini PC sending control commands.
Description	This diagram represents the autonomous transport execution process. The operator sends a command via the dashboard, which is transmitted through the cloud backend to the Mini PC. The Mini PC issues control commands to the Arduino, which executes movement. The process continues until the destination is reached.

Critical Interactions

- Continuous feedback loop between Arduino and Mini PC
- Real-time telemetry updates to cloud backend
- Destination validation controls loop termination

Connection to Requirements

- Supports autonomous task execution (SR-05)
- Ensures synchronization between control layers (SR-10)

Architectural Element: Activity Diagram – Incremental Movement

Architectural Element	Description
Activity Diagram	Incremental Movement <pre>graph TD; Start(()) --> A1[Worker presses forward button]; A1 --> A2[Control Interface sends Increment command]; A2 --> A3[Mini PC commands small distance move]; A3 --> A4[Arduino executes movement]; A4 --> A5[Steering System completes step]; A5 --> A6[Arduino reports "Movement Complete"]; A6 --> End((())) ;</pre> The flowchart illustrates the incremental movement process. It begins with a worker pressing a forward button, which triggers a command from the control interface to the Mini PC. The Mini PC commands a small distance move, which the Arduino executes. The steering system completes the step, and the Arduino reports "Movement Complete".
Description	This diagram shows manual incremental movement control. A worker triggers a movement command, which is processed by the Mini PC and executed by the Arduino. The system confirms completion after each movement step.

Critical Interactions

- Immediate command-response cycle
- Accurate movement completion feedback

Connection to Requirements

- Supports manual override control

- Enables precise positioning operations

2. Safety and Security System

The Safety System ensures operator protection and system integrity through continuous monitoring, hazard detection, and rapid response.

Architectural Element: Activity Diagram – Emergency Stop

Architectural Element	Description
Activity Diagram	Emergency Stop <pre>graph TD; Start(()) --> A[Operator presses Emergency Button (Physical/UI)]; A --> B[Emergency Signal sent to Mini PC]; B --> C[Mini PC sends Immediate Stop command]; C --> D[Arduino cuts motor signal to Steering System]; D --> E[System enters Locked/Safety state]; E --> End((()))</pre>
Description	This diagram illustrates the emergency stop process. When triggered by the operator, the emergency signal is sent to the Mini PC, which immediately issues a stop command to the Arduino. The system transitions into a locked safety state.

Critical Interactions

- Emergency signal propagation must be immediate
- Motor shutdown must occur without delay

Connection to Requirements

- Implements emergency response (SR-08)
- Ensures fail-safe operation

Architectural Element: Activity Diagram – Obstacle Detection & Avoidance

Architectural Element	Description
-----------------------	-------------

Activity Diagram	Obstacle Detection & Avoidance <pre>graph TD; Start(()) --> A[Sensors send obstacle data to Mini PC]; A --> B[Mini PC & Safety Module analyze data]; B --> C{Obstacle Detected?}; C -- yes --> D[Decide to Stop or Change Direction]; C -- no --> E[Mini PC sends "Continue" command]; D --> F[Mini PC sends command to Arduino]; F --> G[Arduino applies Stop/Turn to Steering System]; E --> H[Arduino maintains motion]; G --> I{ }; H --> I; I --> End((()));</pre>
Description	This diagram illustrates how obstacle data from sensors is processed by the Mini PC. If an obstacle is detected, the system decides whether to stop or change direction. Otherwise, normal operation continues.

Critical Decision Points

- Obstacle detection condition
- Decision between stopping or avoidance

Critical Interactions

- Real-time sensor processing
- Immediate control command execution

Connection to Requirements

- Supports safe navigation (SR-05)
- Prevents collisions (SR-08)

Architectural Element: Activity Diagram – Blocked Vehicle Notification

Architectural Element	Description
Activity Diagram	Blocked Vehicle Notification <pre>graph TD; Start(()) --> A[Mini PC monitors movement]; A --> B{Movement detected?}; B -- yes --> C[Continue Monitoring]; C --> B; B -- no --> D{Timeout reached?}; D -- yes --> E[Mini PC sends Alert to Cloud Backend]; D -- no --> F[Wait]; E --> G[Cloud triggers Notification Service]; G --> H[Operator receives Alert Message]; F --> H; H --> I{ }; I --> End((()));</pre>
Description	This diagram shows how the system detects a blocked vehicle condition. The Mini PC monitors movement, and if no movement is detected within a defined timeout, an alert is sent to the cloud backend, triggering a notification to the operator.

Critical Decision Points

- Movement detection vs timeout threshold

Critical Interactions

- Communication between Mini PC and cloud backend
- Notification delivery to operator

Connection to Requirements

- Supports monitoring and fault detection
- Improves system responsiveness

3. System Coordination & Monitoring

Architectural Element: Sequence Diagram – System Initialization Coordination

Architectural Element	Description
Sequence Diagram	
Description	The Operator represents the user who initiates the system startup. The Web Dashboard is the user interface used to send the initialization request and display system status. The Cloud Backend manages communication between the interface and the vehicle, coordinating initialization steps and data flow. The Mini PC acts as the main onboard controller, orchestrating the initialization of all subsystems. The Sensors (GPS, Camera, Obstacle) provide environmental perception and positioning capabilities. The Arduino handles low-level control and communication with hardware components. The Steering System represents the physical actuation layer used for movement validation and control checks. The Operational DB stores system data, logs, and initialization information. The process begins when the Operator starts the system through the Web Dashboard, which sends an initialization request to the Cloud Backend. The Cloud Backend forwards the request to the Mini PC, triggering the system initialization sequence. First, safety systems are initialized and verified through the Arduino and Steering System to ensure safe operation. Next, the sensors are initialized and calibrated, enabling accurate perception and positioning. The control system is then initialized, confirming that hardware components respond correctly. After that, navigation is configured by loading routes and obtaining GPS position data. The system then establishes communication with the Cloud Backend and initializes data services, including logging to the Operational DB. Once all subsystems report readiness, the Mini PC signals completion to the Cloud Backend, which updates the Web Dashboard. Finally, the Operator is informed that the system is ready and enters standby mode, prepared for operation.

Critical Interactions

- Synchronized startup across subsystems
- Dependency between safety and control systems

Connection to Requirements

- Ensures system readiness before operation

- Supports diagnostics and configuration

Architectural Element: Sequence Diagram – Autonomous Transport Execution

Architectural Element	Description
Sequence Diagram	<pre>sequenceDiagram actor Operator participant WD as Web Dashboard participant CB as Cloud Backend participant MPC as Mini PC participant AR as Arduino participant SS as Steering System Operator->>WD: Send transport command WD->>CB: REST API (task request) CB->>MPC: Commands / Updates MPC->>AR: Control commands AR->>SS: PWM signals SS-->>AR: Status feedback AR-->>MPC: Execution status MPC->>CB: Telemetry / Logs CB-->>WD: Status update WD-->>Operator: </pre>
Description	The Operator represents the human user who initiates transport tasks. The Web Dashboard is the user interface through which commands are sent and system status is visualized. The Cloud Backend acts as an intermediary that processes requests, manages communication, and stores operational data. The Mini PC is the main controller located on the vehicle, responsible for executing high-level logic and coordinating subsystems. The Arduino handles low-level control by converting commands into hardware actions. The Steering System represents the physical actuation layer that executes movement through PWM signals and provides feedback on execution status. The process begins when the Operator sends a transport command through the Web Dashboard. This request is transmitted to the Cloud Backend via a REST API, which then forwards the command to the Mini PC as control instructions. The Mini PC processes the request and sends control commands to the Arduino, which translates them into PWM signals for the Steering System to perform the required movement. The Steering System executes the action and returns status feedback to the Arduino, which propagates execution status back to the Mini PC. The Mini PC then sends telemetry and logs to the Cloud Backend, allowing the system to track performance and state. Finally, the Cloud Backend updates the Web Dashboard, providing the Operator with real-time status information about the transport operation.

Connection to Requirements: SR-01, SR-05, SR-07

Architectural Element: Sequence Diagram – Obstacle Detection & Avoidance

Architectural Element	Description
-----------------------	-------------

Sequence Diagram	<pre> sequenceDiagram participant Sensors participant Mini PC participant Safety Module participant Arduino participant Steering System Sensors->>Mini PC: Obstacle data Mini PC->>Safety Module: Analyze data alt [Obstacle detected] Safety Module->>Mini PC: Stop/Avoid decision Mini PC->>Arduino: Stop or change direction Arduino->>Steering System: Apply stop/turn else [No obstacle] Safety Module->>Mini PC: Continue movement Mini PC->>Steering System: Maintain motion end </pre>
Description	The Sensors represent the perception layer responsible for detecting obstacles and sending environmental data. The Mini PC analyzes incoming sensor data and coordinates system responses. The Safety Module evaluates risks and determines whether the vehicle should stop or change direction. The Arduino translates decisions into control signals, and the Steering System executes movement commands. The process begins when sensors detect an obstacle and send data to the Mini PC. The system evaluates the situation and either stops, changes direction, or continues movement.

Connection to Requirements: SR-09

Architectural Element: Sequence Diagram – Blocked Vehicle Notification

Architectural Element	Description
Sequence Diagram	<pre> sequenceDiagram participant Mini PC participant Cloud Backend participant Notification Service/ UI participant Operator Mini PC->>Mini PC: Detect no movement (timeout) Mini PC->>Cloud Backend: Send alert Cloud Backend->>Notification Service/ UI: Trigger notification Notification Service/ UI->>Operator: Alert message </pre>
Description	The Mini PC monitors vehicle movement and detects abnormal situations such as lack of motion. When a timeout condition is reached, it sends an alert to the Cloud Backend. The Cloud processes the alert and triggers a notification through the Notification Service, which informs the Operator.

Connection to Requirements: SR-14

Architectural Element: Sequence Diagram – Emergency Stop

Architectural Element	Description
-----------------------	-------------

Sequence Diagram	<pre> sequenceDiagram participant Operator participant UI as Emergency Button / UI participant PC as Mini PC participant Arduino participant SS as Steering System Operator->>UI: Press stop activate UI UI->>PC: Emergency signal deactivate UI activate PC PC->>Arduino: Immediate stop command deactivate PC activate Arduino Arduino->>SS: Cut motor signal deactivate Arduino deactivate SS </pre>
Description	The Operator triggers an emergency stop through a physical button or UI. The signal is sent to the Mini PC, which immediately issues a stop command to the Arduino. The Arduino cuts motor signals, and the Steering System halts the vehicle instantly.

Connection to Requirements: SR-10

Architectural Element: Sequence Diagram – Incremental Movement

Architectural Element	Description
Sequence Diagram	<pre> sequenceDiagram participant Worker participant CI as Control Interface participant PC as Mini PC participant Arduino participant SS as Steering System Worker->>CI: Press forward button activate CI CI->>PC: Increment command deactivate CI activate PC PC->>Arduino: Move forward small distance deactivate PC activate Arduino Arduino->>SS: Execute movement deactivate Arduino activate SS SS-->>Arduino: Movement complete deactivate SS deactivate Arduino </pre>
Description	The Worker initiates movement using a control interface. The Mini PC interprets the command and sends movement instructions to the Arduino. The Arduino converts the command into control signals, and the Steering System executes a small movement step. A confirmation is returned after execution.

4. System State Models

The state diagrams describe the dynamic behavior of each major system component, showing transitions between operational states and responses to events

Architectural Element: State Diagram – Mini PC (Main Controller)

Architectural Element	Description
-----------------------	-------------

State Diagram	<pre>stateDiagram-v2 [*] --> Booting state Booting { Load OS and services Initialize communication Initialize sensor interfaces } Booting --> SelfCheck : startup complete state SelfCheck { Verify GPS, sensors, camera Verify Arduino link } SelfCheck --> Ready : all checks passed state Ready { Waiting for route/task } Ready --> MissionPlanning : command received from Cloud state MissionPlanning { Validate task Build/confirm route } MissionPlanning --> Ready : task cancelled MissionPlanning --> AutonomousDriving : route valid state AutonomousDriving { Process GPS/RTK Process obstacle data Send control commands to Arduino } AutonomousDriving --> Ready : destination reached AutonomousDriving --> MissionComplete : destination reached state MissionComplete { Send completion status Upload final telemetry } MissionComplete --> Ready : reset successful AutonomousDriving --> ObstacleHandling : obstacle detected state ObstacleHandling { Stop or slow vehicle Evaluate safe continuation } ObstacleHandling --> AutonomousDriving : path clear ObstacleHandling --> RemoteOverride : operator intervention required state RemoteOverride { Accept remote/manual command } RemoteOverride --> AutonomousDriving : autonomy resumed RemoteOverride --> Fault : control conflict / failure AutonomousDriving --> Fault : critical error ObstacleHandling --> Fault : route invalid / no localization state Fault { Stop command issued Publish alert to cloud } Fault --> [*] : shutdown Fault --> Ready : component failure detected</pre>
Description	This diagram illustrates the lifecycle of the Mini PC controller, including booting, self-check, ready state, mission planning, autonomous driving, obstacle handling, remote override, and fault states. It shows transitions based on events such as command reception, obstacle detection, and system failures.

Architectural Element: State Diagram – Arduino (Low-Level Control Unit)

Architectural Element	Description
-----------------------	-------------

State Diagram	The diagram illustrates the Arduino control lifecycle. It begins with a start node leading to the PowerOn state, which performs 'MCU reset' and 'Initialize PWM / I/O'. From PowerOn, an 'init OK' transition leads to the Standby state ('Waiting for control command'). A 'valid command received' transition leads to the ExecutingCommand state ('Interpret command', 'Generate PWM signal'). From ExecutingCommand, a 'command completed' transition returns to Standby, while an 'invalid command / safety fault' transition leads to the EmergencyStop state ('Cut actuation output', 'Hold safe state'). From Standby, a 'steering command' transition leads to the SteeringActive state ('Apply steering PWM'). From SteeringActive, a 'target reached' transition returns to Standby, and an 'emergency stop signal' transition leads to EmergencyStop. From EmergencyStop, a 'power off' transition leads to an end node. Additionally, a 'reset from Mini PC / operator' transition from ExecutingCommand and an 'init failed' transition from PowerOn both lead back to Standby. A 'safety signal received' transition also leads from Standby to EmergencyStop.
Description	This diagram shows the Arduino control lifecycle, including initialization, standby, command execution, steering activation, and emergency stop states. It demonstrates how low-level commands are processed and how safety signals override normal operation.

Architectural Element: State Diagram – Cloud Backend

Architectural Element	Description
State Diagram	The diagram represents the cloud backend behavior. It starts at an Idle state ('Await API request or telemetry'). A 'telemetry packet received' transition leads to the ReceivingTelemetry state ('Validate payload', 'Parse telemetry/log data'). From ReceivingTelemetry, an 'invalid packet discarded' transition returns to Idle, and a 'valid packet' transition leads to the StoringData state ('Write telemetry to DB'). From StoringData, a 'storage complete' transition returns to Idle, and an 'anomaly detected during processing' transition leads to the AlertProcessing state ('Register event', 'Notify dashboard'). From Idle, an 'operator command received' transition leads to the ProcessingCommand state ('Validate operator request', 'Convert to vehicle command'). From ProcessingCommand, a 'rejected / cancelled' transition returns to Idle, and a 'request accepted' transition leads to the DispatchingCommand state ('Send command to Mini PC'). From DispatchingCommand, a 'delivery timeout / failure' transition leads to AlertProcessing, and an 'acknowledged' transition returns to Idle. From AlertProcessing, a 'fault / alert event received' transition returns to Idle.
Description	This diagram represents the cloud backend behavior, including receiving telemetry, processing commands, storing data, and handling alerts. It shows how the system reacts to incoming requests and system events.

Architectural Element: State Diagram – Web Dashboard (UI)

Architectural Element	Description
State Diagram	<pre>stateDiagram-v2 [*] --> Disconnected state Disconnected { No active backend session } state Authenticating { Validate credentials/session } state Monitoring { Display status, telemetry, map } state ViewingAlert { Show warning / fault / event } state SendingCommand { Build and submit command } Disconnected --> Authenticating : user opens dashboard / login failed Authenticating --> Disconnected : session lost / logout Authenticating --> Monitoring : login successful Monitoring --> ViewingAlert : alert received ViewingAlert --> Monitoring : alert acknowledged Monitoring --> SendingCommand : operator issues command SendingCommand --> Monitoring : command acknowledged SendingCommand --> ViewingAlert : command failed / rejected ViewingAlert --> SendingCommand : operator responds with action</pre>
Description	This diagram illustrates the user interface behavior, including authentication, monitoring, alert viewing, and command submission. It shows how user interactions drive system operations.

Architectural Element: State Diagram – Steering System

Architectural Element	Description
State Diagram	<pre>stateDiagram-v2 [*] --> Neutral state Neutral { No active steering motion } state AutoSteering { Actuator follows Arduino signal } state ManualSteering { Operator directly controls steering } state LockedSafe { Steering held in safe state } Neutral --> AutoSteering : PWM command from Arduino AutoSteering --> Neutral : steering target reached AutoSteering --> ManualSteering : manual override engaged ManualSteering --> AutoSteering : autonomous mode restored ManualSteering --> LockedSafe : emergency stop / fault LockedSafe --> Neutral : manual input released LockedSafe --> Neutral : reset / recovery</pre>
Description	This diagram shows the steering system states, including neutral, auto steering, manual steering, and locked safe state. It illustrates transitions based on control commands, manual overrides, and safety events.

1. Component Special Characteristics Overview

This document presents the special characteristics analysis of the Autonomous Golf Cart System, organized by its major components. Each component contains critical characteristics that require particular attention during design, implementation, and operation. The analysis follows the system's layered architecture: User Layer, Cloud Layer, and Edge/Vehicle Layer.

2. Mini PC System (Software Systems Architect)

The Mini PC functions as the central intelligence and main decision-making component of the autonomous golf cart. As the brain of the vehicle, it receives data from all sensors, processes information in real time, and generates commands to control the vehicle. The following special characteristics are critical to its operation:

Special Characteristic	Description
Decision Logic Timing	Must maintain consistent sub-100ms response time to ensure real-time synchronization with physical components and immediate reaction to environmental changes such as obstacles or route deviations
System State Management	Critical for maintaining operational integrity during mode transitions between normal autonomous operation, manual control, and emergency stop modes
Sensor Fusion Accuracy	SensorFusion module must correctly merge GPS, ultrasonic, and camera data to support reliable obstacle detection and navigation decisions
Fault Tolerance Logic	Essential for handling partial system failures gracefully, allowing continued operation in degraded mode when appropriate, particularly when one sensor source becomes unavailable

3. Arduino Low-Level Controller (Robotics / Embedded Systems Engineer)

The Arduino controller represents the low-level control layer of the system. It receives high-level commands from the Mini PC and executes them in a stable and predictable way, forming a critical safety boundary between software decisions and physical actuation.

Special Characteristic	Description
Command Execution Reliability	Must translate high-level movement commands into correct motor control signals with zero ambiguity; the CommandParser must validate all commands before execution
Emergency Stop Response Time	Critical for immediate vehicle halt when safety conditions are triggered, preventing equipment damage or human injury in agricultural environments where people and machinery share space
Motor Control Precision	MotorController must maintain accurate speed and direction signals to enable consistent autonomous navigation and safe deceleration
Safety Module Continuity	SafetyModule must continuously monitor signals from the Mini PC and autonomously trigger emergency stop even if higher-level communication is interrupted

4. Web Dashboard — User Interface System (UI/UX Team Lead)

The Web Dashboard is the primary interaction point between the operator and the system, enabling real-time monitoring and remote command dispatch. In agricultural environments where operators may supervise multiple simultaneous processes, the following characteristics are critical:

Special Characteristic	Description
Alert Prioritization Algorithm	Critical for ensuring that high-priority safety and operational alerts are prominently displayed to operators, preventing them from missing crucial information such as obstacle detections or communication failures
Role-Based Access Control	Essential for maintaining system security and preventing unauthorized configuration changes that could compromise vehicle performance or safety features
Real-Time Status Synchronization	DashboardView must update vehicle position, speed, and operational status in real time to ensure operators always act on current system state
Command Transmission Reliability	ControlPanel must guarantee that operator commands (move, stop, manual override) reach the cloud backend and are confirmed received before UI feedback is displayed

5. Cloud Backend System (Data / Systems Engineer)

The Cloud Backend serves as the data management and communication layer, connecting the Web Dashboard with the Mini PC and providing centralized storage and processing. Its special characteristics govern system-wide data integrity and command reliability:

Special Characteristic	Description
------------------------	-------------

Real-Time Data Pipeline Latency	Must maintain sub-second telemetry transfer for position, battery, and operational metrics to support timely operator decision-making and accurate dashboarding
Command Dispatch Integrity	CommandDispatcher must guarantee ordered, non-duplicate delivery of operator commands to the vehicle even when multiple commands are submitted concurrently
Data Consistency Validation	DataManager must ensure reliable storage and retrieval of operational logs, supporting compliance verification, performance analysis, and system diagnostics
Authentication Security	APIGateway must validate all incoming requests to prevent unauthorized system access that could compromise vehicle operations or safety features

6. Sensor System (Computer Vision / Perception Engineer)

The Sensor System constitutes the perception layer of the autonomous vehicle, collecting environmental data and transmitting it to the Mini PC. The accuracy and reliability of this layer directly determines navigation quality and safety outcomes.

Special Characteristic	Description
GPS Positioning Accuracy	GPSTransceiver must provide high-accuracy coordinates with measurable precision metrics; critical for autonomous route following and geofencing
Obstacle Detection Reliability	UltrasonicSensors must detect obstacles with no acceptable false negatives; missed detections create direct collision risk in dynamic agricultural environments
Camera Processing Latency	CameraModule must capture and stream visual data within strict time constraints to support real-time environment monitoring and potential image-recognition features
Sensor Data Validation	SensorSystem must filter out false or corrupted readings from any sensor module before forwarding data to the Mini PC, preventing erroneous navigation or unnecessary emergency stops

7. Steering & Motor System (Mechanical Systems Engineer)

The Steering System converts electrical signals from the Arduino controller into physical vehicle movement, forming the final actuation layer of the system. Its characteristics directly affect vehicle maneuverability, path accuracy, and safe transitions between control modes.

Special Characteristic	Description
Movement Precision	SteeringMechanism must maintain accurate wheel rotation in response to control signals to ensure reliable path following across varied terrain conditions
PWM Signal Stability	MotorDriver must apply consistent PWM signals to prevent speed fluctuations that could cause unsafe vehicle behavior or navigation deviations
Manual-Autonomous Handoff Safety	ManualControl must enable smooth, safe transitions between autonomous and manual operation modes without abrupt movement or loss of control
Emergency Braking Response	System must achieve consistent sub-second stopping capability when emergency stop is triggered, preventing accidents in shared human-vehicle spaces

8. Rationale and Failure Mode Analysis

8.1 Mini PC — Decision Logic Timing

The Mini PC's decision logic timing is identified as a special characteristic due to its fundamental role in coordinating the entire autonomous operation. The system architecture requires consistent sub-100ms decision-making for safe navigation. Processing delays would cause the vehicle to miss obstacle avoidance windows or issue commands that are no longer valid for the vehicle's current position. This failure mode presents high severity risk.

Mitigation: Implement real-time processing prioritization, dedicated CPU resource allocation for the navigation and decision engines, and automated alerting when execution times approach the 80ms threshold.

8.2 Sensor System — Obstacle Detection Reliability

Failure Mode	Risk	Severity	Mitigation Strategy
--------------	------	----------	---------------------

Sensor blind spots or false negatives in ultrasonic detection	A human, animal, or obstacle in the vehicle's path goes undetected, resulting in collision	Critical — direct potential for injury or property damage	Implement redundant, overlapping sensor coverage using multiple detection technologies (ultrasonic + vision-based); require positive safety zone clearance before resuming motion; implement periodic automatic self-testing of sensor health
---	--	---	---

8.3 Arduino — Emergency Stop Response Time

The Arduino's emergency stop capability is a special characteristic because it operates as the last hardware-enforced safety boundary. If the SafetyModule fails to trigger immediately when a hazardous condition is detected, the vehicle's momentum could cause injury or equipment damage. This is especially critical in environments where human operators may walk near the vehicle.

Mitigation: Implement emergency stop as a hardware-level interrupt independent of software execution state. Regular functional testing of the stop circuit should be part of pre-operation checklists. Response time must not exceed 200ms from trigger to full motor halt.

8.4 Web Dashboard — Alert Prioritization Algorithm

Failure Mode	Risk	Severity	Mitigation Strategy
Critical safety alerts buried beneath routine operational notifications	Operator misses a proximity alert, communication failure, or emergency stop event	High — delayed response to safety events could cause harm	Implement tiered alert classification (critical/warning/informational); critical alerts must interrupt all other UI activity with persistent visual and audible signals; alerts must require explicit operator acknowledgment before dismissal

8.5 Cloud Backend — Command Dispatch Integrity

The CommandDispatcher must guarantee that vehicle commands are delivered in the correct order and are not duplicated. A command sent twice (e.g., 'start' followed by an erroneous duplicate) or delivered out of sequence could cause unpredictable vehicle behavior. This is especially important during high-latency network conditions.

Mitigation: Implement sequence-numbered command queuing with acknowledgment confirmation. Commands without confirmed delivery should be retried with exponential backoff, and conflicting or duplicate commands should be resolved by the APIGateway before forwarding.

9. System Requirements Refinement

Based on the analysis of failure modes and mitigation strategies, the following additions and modifications to system requirements are recommended:

9.1 New Requirements

Requirement ID	Description
SR-11 (Decision Logic Performance)	The Mini PC shall maintain decision logic execution times below 100ms for 99.99% of all navigation cycles, with automated monitoring and alerting when execution times exceed 80ms. Dedicated processing allocation shall prevent resource contention from impacting navigation timing.
SR-12 (Sensor Redundancy)	The Sensor System shall implement overlapping detection coverage using at least two independent sensing technologies for obstacle detection. The system shall perform automatic self-tests of all sensor modules at startup and at four-hour intervals during operation.
SR-13 (Emergency Stop Verification)	The Arduino Controller shall verify emergency stop circuit integrity at system initialization and log response time for each triggered stop event. Stop response time shall not exceed 200ms from trigger signal to full motor halt.
SR-14 (Command Integrity)	The Cloud Backend shall implement sequence-numbered command queuing with delivery confirmation. Duplicate or out-of-sequence commands shall be rejected, and unconfirmed commands shall be retried with a maximum of three attempts before operator notification.
SR-15 (Alert Classification)	The Web Dashboard shall implement a three-tier alert classification system (Critical, Warning, Informational). Critical alerts shall interrupt all other dashboard activity and require explicit operator acknowledgment. Alert logs shall be stored with timestamps for audit review.
SR-16 (System Diagnostics)	All critical special characteristics shall be subject to automated diagnostic verification during system initialization and at scheduled intervals during operation. Diagnostic results shall be logged with timestamp and verification status.

9.2 Modified Requirements

Req uire ment ID	Original	Modified
SR-08	Safety system shall respond to detected obstacle or human presence within 1 second	The safety system shall respond to detected human or obstacle presence within 200ms and include blind spot detection verification protocols to be executed at startup and every four hours. The system shall maintain safety response time logs for compliance verification.
SR-09	Vehicle shall follow predefined routes autonomously	Vehicle shall follow predefined routes autonomously with GPS positional accuracy verified to within $\pm 0.5\text{m}$. Route deviation exceeding 1m shall trigger an operator alert and automatic speed reduction until re-alignment is confirmed.
SR-10	Web dashboard shall display vehicle status	The Web Dashboard shall display vehicle position, speed, battery level, and active alerts in real time with maximum display latency of 2 seconds. Critical alerts shall persist until explicitly acknowledged by the operator.

10. Conclusion

The detailed analysis of special characteristics across all system components of the Autonomous Golf Cart reveals several critical areas requiring careful design attention and ongoing monitoring. The time-sensitive aspects of the system — particularly Mini PC decision logic timing, sensor data processing, and command dispatch latency — form the operational backbone of reliable autonomous behavior.

Safety features, especially obstacle detection and emergency stop response, require redundancy and rigorous validation protocols beyond basic functional requirements. The refined requirements address these concerns by adding specific performance targets, redundancy mandates, and verification procedures that will enhance overall system reliability.

The identification of potential failure modes highlights the need for robust self-monitoring, calibration verification, and automated diagnostics throughout all system layers. By implementing the refined requirements and focusing development effort on the identified special characteristics, the Autonomous Golf Cart system will achieve the reliability, safety, and operational performance required for deployment in real agricultural environments.